

SELF-HOSTED ACADEMY

Introduction à Rust et à la Sécurité Mémoire : Du Débutant à la Programmation Robuste

COURS ACADÉMIQUE COMPLET

Ce cours universitaire est conçu spécifiquement pour les personnes n'ayant aucune notion préalable de 'code sûr' (safe code) ou de gestion de la mémoire. Il introduit de manière progressive et pédagogique les concepts fondamentaux de la programmation, le système unique de propriété (ownership) de Rust, et explique comment ce langage élimine par construction les bugs informatiques les plus critiques sans sacrifier les performances.

Généré par SelfLearned • Technologie AI-Driven

MODULE 1

Les Fondations de la Programmation et le Problème de la Mémoire

Introduction à Rust et premier programme : Comprendre l'écosystème Cargo

1. Introduction Conceptuelle

Le "Pourquoi" avant le "Comment" : La genèse de Rust

Depuis les débuts de l'informatique moderne, le développement de logiciels système — les systèmes d'exploitation, les navigateurs web, les moteurs de base de données ou les systèmes embarqués — est dominé par les langages C et C++. Ces langages offrent un contrôle absolu sur le matériel et des performances maximales, car ils ne s'encombrent pas d'un *Ramasse-miettes* (ou *Garbage Collector*), un mécanisme automatique qui suspend périodiquement l'exécution du programme pour nettoyer la mémoire inutilisée.

Cependant, ce pouvoir absolu s'accompagne d'une responsabilité immense. En C et C++, c'est au développeur de gérer manuellement chaque octet de mémoire : l'allouer quand on en a besoin, et la libérer précisément au bon moment. Une simple omission, et c'est la fuite de mémoire (*memory leak*). Une libération trop précoce, et le programme tente d'accéder à une zone mémoire invalide, provoquant des plantages aléatoires ou, pire, des failles de sécurité critiques exploitables par des attaquants (comme les dépassements de tampon ou *buffer overflows*).

Selon les statistiques de Microsoft et de Google (pour le projet Chromium), environ 70 % de toutes les vulnérabilités de sécurité majeures sont dues à des problèmes de gestion de la mémoire.

C'est pour résoudre ce dilemme historique entre **performance** et **sécurité** que le langage Rust a été créé par Graydon Hoare chez Mozilla, puis propulsé par une communauté

mondiale. Rust repose sur une promesse révolutionnaire : garantir la sécurité de la mémoire à la compilation, sans aucun compromis sur les performances à l'exécution.

La philosophie de Rust : Les trois piliers

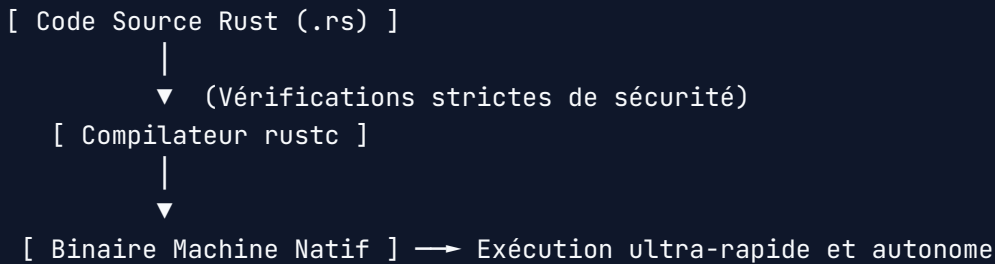
1. **Sécurité (Safety)** : Le compilateur Rust agit comme un garde-fou ultra-strict. Il analyse statiquement votre code lors de la compilation pour s'assurer qu'aucun bug de gestion mémoire (comme l'accès à de la mémoire libérée ou les conflits d'accès simultanés) ne puisse exister dans le programme final. Si le code compile, il est garanti sans comportement indéfini lié à la mémoire.
 2. **Performance (Speed)** : Rust n'utilise pas de ramasse-miettes. Il utilise un système innovant de propriété des données (que nous étudierons en détail dans le *Module 2*). Les performances de Rust sont ainsi équivalentes à celles du C et du C++.
 3. **Productivité (Productivity)** : Souvent, les langages système souffrent d'un outillage fragmenté et complexe. Rust prend le contre-pied de cette tradition en fournissant une suite d'outils intégrés de premier ordre, unifiée sous une seule commande : `cargo`.
-

2. Fondations Théoriques

Compilation vs Interprétation

Pour comprendre comment fonctionne Rust, il convient de distinguer deux grandes familles de langages de programmation :

- **Les langages interprétés (ex. Python, JavaScript)** : Le code source est lu et exécuté ligne par ligne à l'exécution par un autre programme appelé interpréteur. C'est flexible, mais lent.
- **Les langages compilés (ex. C, C++, Rust)** : Le code source est entièrement traduit à l'avance par un programme appelé **compilateur** en instructions machines natives (un fichier binaire exécutable, comme un `.exe` sous Windows ou un fichier ELF sous Linux). Une fois compilé, le programme s'exécute à la vitesse maximale du processeur, sans intermédiaire.



Le rôle du compilateur (`rustc`)

Le compilateur officiel de Rust s'appelle `rustc`. Contrairement aux compilateurs traditionnels qui se contentent de traduire le code et de signaler les erreurs de syntaxe de base, `rustc` effectue des analyses sémantiques extrêmement poussées.

Il applique des règles mathématiques strictes sur la durée de vie des variables et l'accès aux données. Si une règle est violée, `rustc` refuse de générer l'exécutable. Il produit alors des messages d'erreur d'une clarté inégalée dans l'industrie, expliquant précisément *pourquoi* le code est dangereux et *comment* le corriger.

L'architecture de l'écosystème Cargo

Bien qu'il soit possible d'appeler directement `rustc` pour compiler un fichier isolé, les projets réels utilisent Cargo. Cargo est à la fois le gestionnaire de paquets, le système de construction (*build system*) et l'orchestrateur de tests de Rust.

Un projet géré par Cargo adopte une structure de répertoires standardisée et immuable :

```
mon_projet/
├─ Cargo.toml
├─ Cargo.lock
└─ src/
    └─ main.rs
```

1. Le manifeste : `Cargo.toml`

Le fichier `Cargo.toml` (écrit en syntaxe TOML - *Tom's Obvious Minimal Language*) est la carte d'identité de votre projet. C'est ici que vous définissez le nom du projet, sa version, l'édition de Rust utilisée, ainsi que les bibliothèques externes (appelées *crates* en Rust) dont votre projet a besoin.

2. Le verrou de dépendances : `Cargo.lock`

Ce fichier est généré automatiquement par Cargo. Il enregistre les versions exactes de toutes les dépendances utilisées lors de la compilation réussie. Cela garantit la reproductibilité : toute personne qui compilera votre projet obtiendra exactement le même binaire, au bit près, peu importe quand ou sur quelle machine elle le fait.

3. Le répertoire `src/`

Ce dossier contient l'intégralité du code source de votre application. Par convention, le point d'entrée d'un programme exécutable est toujours le fichier `src/main.rs`.

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Installation de la chaîne d'outils Rust

La méthode officielle et recommandée pour installer Rust sur tous les systèmes d'exploitation est d'utiliser l'installateur `rustup`.

Sur Linux et macOS :

Ouvrez un terminal et exécutez la commande suivante :

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Suivez les instructions à l'écran (l'option par défaut `1` est recommandée). Une fois l'installation terminée, redémarrez votre terminal ou rechargez votre configuration avec :

```
source $HOME/.cargo/env
```

Sur Windows :

Téléchargez et exécutez l'installateur `rustup-init.exe` disponible sur le site officiel <https://rustup.rs>. Note : Rust nécessite les outils de compilation C++ de Microsoft Visual Studio. Si l'installateur vous le demande, acceptez l'installation de ces outils.

Vérification de l'installation :

Pour valider que Rust et Cargo sont correctement installés, tapez les commandes suivantes dans votre terminal :

```
rustc --version  
cargo --version
```

Vous devriez voir s'afficher les numéros de version actifs (par exemple : `rustc 1.75.0`).

Étape 2 : Création du premier projet avec Cargo

Nous allons utiliser Cargo pour générer automatiquement la structure de notre premier projet, que nous nommerons `hello_rust` .

Exécutez la commande suivante dans votre terminal :

```
cargo new hello_rust
```

Le terminal doit afficher :

```
Created binary (application) `hello_rust` package
```

Entrez dans le répertoire nouvellement créé :

```
cd hello_rust
```

Étape 3 : Exploration de la structure générée

Ouvrons et analysons le fichier `Cargo.toml` généré :

```
[package]
name = "hello_rust"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
```

- `[package]` : Introduit la section de configuration de notre programme.
- `name` : Le nom de notre exécutable.
- `version` : Version actuelle du projet, respectant le versionnage sémantique (SemVer).
- `edition` : Spécifie l'édition majeure du langage Rust (ici, l'édition 2021). Rust introduit des améliorations majeures via des éditions tous les trois ans environ, tout en garantissant une rétrocompatibilité totale.
- `[dependencies]` : Cette section est actuellement vide. C'est ici que nous listerions les bibliothèques externes nécessaires.

Étape 4 : Analyse syntaxique du "Hello, World!"

Ouvrez maintenant le fichier `src/main.rs` . Cargo a déjà généré le code minimal suivant :

```
fn main() {
    println!("Hello, world!");
}
```

Décomposons méticuleusement ce code ligne par ligne :

1. `fn main()` :
2. `fn` est le mot-clé réservé pour déclarer une fonction.
3. `main` est le nom de la fonction spéciale qui sert de point d'entrée unique à tout programme Rust. C'est cette fonction qui est appelée en premier lors du démarrage du programme.
4. Les parenthèses `()` indiquent que cette fonction ne prend aucun paramètre en entrée.
5. Les accolades `{ ... }` :

6. Elles délimitent le corps de la fonction `main`. Tout le code situé entre ces accolades sera exécuté séquentiellement.
 7. `println!("Hello, world!");` :
 8. `println!` est une macro Rust (et non une simple fonction). Nous la reconnaissons immédiatement grâce au point d'exclamation `!` qui suit son nom. Les macros en Rust sont des outils puissants qui génèrent du code lors de la compilation. Dans le cas de `println!`, elle analyse la chaîne de caractères à la compilation pour s'assurer que les arguments fournis sont valides et sécurisés.
 9. `"Hello, world!"` est une chaîne de caractères (littéral) passée en argument à la macro.
 10. Le point-virgule `;` est obligatoire. Il indique la fin de l'instruction courante. En Rust, l'omission d'un point-virgule change fondamentalement la signification de la ligne (ce point sera détaillé lors de l'étude des expressions dans le Module 1.2).
-

Étape 5 : Compilation et exécution du projet

Cargo simplifie drastiquement le cycle de développement grâce à trois commandes fondamentales à connaître par cœur.

Commande A : Vérifier le code sans compiler (`cargo check`)

Pendant que vous écrivez du code, vous voulez savoir rapidement si votre syntaxe est correcte sans attendre que le compilateur génère le binaire final (ce qui peut prendre du temps sur de gros projets).

Exécutez :

```
cargo check
```

Sortie attendue :

```
Checking hello_rust v0.1.0 (/chemin/vers/votre/projet/hello_rust)
Finished dev [unoptimized + debuginfo] target(s) in 0.15s
```

Le code est syntaxiquement correct et respecte les règles de sécurité de Rust.

Commande B : Compiler le projet (`cargo build`)

Pour générer l'exécutable de développement, exécutez :

```
cargo build
```

Sortie attendue :

```
Compiling hello_rust v0.1.0 (/chemin/vers/votre/projet/hello_rust)
Finished dev [unoptimized + debuginfo] target(s) in 0.42s
```

Cette commande a créé un dossier `target/debug/` contenant l'exécutable nommé `hello_rust` (ou `hello_rust.exe` sous Windows).

Commande C : Compiler et exécuter en une seule étape (`cargo run`)

C'est la commande la plus utilisée au quotidien par les développeurs Rust.

Exécutez :

```
cargo run
```

Sortie attendue :

```
Finished dev [unoptimized + debuginfo] target(s) in 0.01s
Running `target/debug/hello_rust`
Hello, world!
```

Cargo détecte si des modifications ont été apportées depuis la dernière compilation. Si ce n'est pas le cas, il saute l'étape de compilation et lance directement le binaire existant.

Commande D : Compiler pour la production (`cargo build --release`)

Par défaut, `cargo build` compile le code en mode Debug. Le binaire généré contient des informations de débogage et n'est pas optimisé, ce qui permet une compilation rapide mais des performances d'exécution réduites.

Lorsque votre code est prêt à être déployé, compilez-le avec :

```
cargo build --release
```

Sortie attendue :

```
Compiling hello_rust v0.1.0 (/chemin/vers/votre/projet/hello_rust)
Finished release [optimized] target(s) in 0.50s
```

Le compilateur applique alors des optimisations agressives (comme l'inlining de fonctions et la vectorisation). Le binaire résultant, situé dans `target/release/`, est extrêmement rapide et léger.

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Oublier le point d'exclamation `!` sur les macros

Un débutant habitué à d'autres langages a tendance à écrire :

```
println("Hello, world!"); // ERREUR !
```

Pourquoi c'est une erreur : `println` (sans `!`) serait interprété par Rust comme un appel de fonction classique. Or, il n'existe pas de fonction nommée `println` dans la bibliothèque standard, car la vérification du formatage textuel nécessite la puissance d'analyse d'une macro à la compilation.

Message d'erreur du compilateur :

```

error[E0423]: expected function, found macro `println`
  → src/main.rs:2:5
   |
2 |     println("Hello, world!");
   |     ^^^^^^^ not a function
   |
help: use `!` to invoke the macro
   |
2 |     println!("Hello, world!");
   |             +

```

Piège 2 : L'omission du point-virgule ;

En Rust, presque tout est une expression qui retourne une valeur. Oublier un point-virgule à la fin d'une instruction de type action (comme un affichage) indique au compilateur que vous tentez de retourner la valeur de cette ligne.

```

fn main() {
    println!("Hello, world!") // ERREUR ! Point-virgule manquant
}

```

Message d'erreur du compilateur :

```

error: expected `;`, found `}`
  → src/main.rs:2:30
   |
2 |     println!("Hello, world!")
   |                               ^ help: add `;` here
3 | }
   | - unexpected token

```

Bonnes Pratiques de l'Écosystème

1. Formater automatiquement son code (`cargo fmt`)

La communauté Rust est très stricte sur le style de code. Pour éviter les débats stériles sur l'emplacement des accolades ou des espaces, Rust fournit un outil de formatage officiel : `rustfmt` .

Pour formater automatiquement tous les fichiers de votre projet selon les standards officiels, exécutez simplement :

```
cargo fmt
```

2. Analyser son code avec le linter (`cargo clippy`)

Rust propose un outil d'analyse statique avancé nommé `clippy` . Il analyse votre code pour y détecter des inefficacités, des pratiques non idiomatiques ou des erreurs logiques subtiles qui compilent mais pourraient être améliorées.

Exécutez régulièrement :

```
cargo clippy
```

5. Synthèse Pédagogique

Résumé des commandes Cargo indispensables

Commande	Rôle	Quand l'utiliser ?
<code>cargo new</code> <code><nom></code>	Crée un nouveau projet Rust structuré.	Au début d'un nouveau projet.
<code>cargo check</code>	Vérifie la validité du code sans générer de binaire.	Constamment, pendant la phase d'écriture.
<code>cargo build</code>	Compile le projet en mode de développement (Debug).	Pour tester localement avec les outils de debug.
<code>cargo run</code>	Compile (si nécessaire) et exécute le programme.	Pour tester rapidement le comportement du code.
<code>cargo build --release</code>	Compile le projet avec des optimisations maximales.	Avant de distribuer ou de mesurer les performances.
<code>cargo fmt</code>	Formate proprement le code source.	Avant chaque commit ou partage de code.
<code>cargo clippy</code>	Analyse le code pour suggérer des améliorations.	Pour s'assurer de la qualité et de l'idiomaticité du code.

Le flux de travail standard du développeur Rust

```
[ Écriture du code ] → [ cargo check ] (Rapide, correction des erreurs)
                        | (Si OK)
                        ▼
[ Amélioration du code ] ← [ cargo run ] (Exécution et tests)
                        |
                        ▼ (Prêt pour la production)
[ cargo build --release ]
```

6. Exercice Pratique d'Application

Énoncé : Le Calculateur de Périmètre

L'objectif de cet exercice est de vous familiariser avec l'utilisation de Cargo, la structure d'un programme Rust, et l'utilisation de la macro `println!` pour afficher des variables et des résultats de calculs simples.

1. Créez un nouveau projet binaire Cargo nommé `calculateur_perimetre`.
 2. Dans le fichier `src/main.rs`, écrivez un programme qui :
 3. Déclare une variable pour la `longueur` d'un rectangle (valeur entière : `15`).
 4. Déclare une variable pour la `largeur` d'un rectangle (valeur entière : `6`).
 5. Calcule le périmètre de ce rectangle en appliquant la formule mathématique :
$$\text{\$Périmètre} = 2 \times (\text{Longueur} + \text{Largeur})$$
 6. Affiche un message de bienvenue chaleureux.
 7. Affiche les dimensions du rectangle ainsi que le résultat du calcul du périmètre de manière lisible.
 8. Utilisez `cargo fmt` pour vous assurer que le code est parfaitement formaté.
 9. Exécutez le programme avec `cargo run`.
-

Indices pour la résolution

- **Déclarer une variable** : En Rust, on déclare une variable à l'aide du mot-clé `let`. Par exemple : `let x = 5;`. (Nous étudierons les détails de la mutabilité et des types au Module 1.2).
 - **Afficher des variables avec `println!`** : Pour afficher la valeur d'une variable au sein d'un texte, on utilise des accolades `{}` comme "espace réservé" (*placeholder*). Exemple :

```
rust let score = 42; println!("Le score est de : {}", score);
```
 - **Opérateurs arithmétiques** : Rust utilise les opérateurs standards : `+` pour l'addition, `*` pour la multiplication. Les parenthèses `()` permettent de gérer les priorités d'opérations.
-

Correction Complète

1. Initialisation du projet

Dans votre terminal, exécutez :

```
cargo new calculateur_perimetre
cd calculateur_perimetre
```

2. Code source (`src/main.rs`)

Remplacez le contenu de `src/main.rs` par le code suivant :

```
fn main() {
    // Affichage du message de bienvenue
    println!("=====");
    println!("  Bienvenue dans le Calculateur Rust !  ");
    println!("=====");

    // Déclaration des dimensions du rectangle
    let longueur = 15;
    let largeur = 6;

    // Calcul du périmètre : 2 * (longueur + largeur)
    let perimetre = 2 * (longueur + largeur);

    // Affichage des informations et du résultat
    println!("Dimensions du rectangle :");
    println!(" - Longueur : {} cm", longueur);
    println!(" - Largeur  : {} cm", largeur);
    println!("-----");
    println!("Le périmètre calculé est de : {} cm", perimetre);
    println!("=====");
}
```

3. Formatage et exécution

Exécutez la commande de formatage pour nettoyer le code :

```
cargo fmt
```


Compilez et lancez le programme :

```
cargo run
```

4. Sortie attendue dans le terminal

```
=====
  Bienvenue dans le Calculateur Rust !
=====
Dimensions du rectangle :
- Longueur : 15 cm
- Largeur  : 6 cm
-----
Le périmètre calculé est de : 42 cm
=====
```


Exercices d'Application :

Introduction à Rust et premier programme : Comprendre l'écosystème Cargo

Exercice 1 : Le Convertisseur de Températures (Difficulté : Facile)

Énoncé

Pour vous échauffer avec la syntaxe de base de Rust et l'utilisation de Cargo, vous allez créer un programme utilitaire qui convertit une température de degrés Celsius (°C) en degrés Fahrenheit (°F).

1. Créez un nouveau projet binaire Cargo nommé `convertisseur_celsius`.
2. Dans le fichier `src/main.rs`, écrivez un programme qui :
3. Déclare une variable entière pour la température en Celsius (choisissez la valeur `20`).
4. Calcule la température équivalente en Fahrenheit en utilisant la formule de conversion entière :
$$\text{Fahrenheit} = \frac{\text{Celsius} \times 9}{5} + 32$$
5. Affiche un message clair présentant la température d'origine et le résultat de la conversion.
6. Formatez votre code avec `cargo fmt`.
7. Compilez et exécutez votre programme à l'aide de Cargo.

Indices

1. **Priorité des opérations** : En Rust, comme en mathématiques, la multiplication `*` et la division `/` sont prioritaires sur l'addition `+`. Utilisez des parenthèses pour structurer clairement votre calcul : `((celsius * 9) / 5) + 32`.
2. **Affichage multiple** : Vous pouvez afficher plusieurs variables dans un seul `println!` en ajoutant autant de paires d'accolades `{}` que de variables passées en paramètres.
3. **Exemple** : `println!("{}", Celsius vaut {} Fahrenheit", variable1, variable2);`

Correction Détaillée

Étape 1 : Création du projet

Ouvrez votre terminal et exécutez la commande suivante pour générer la structure du projet :

```
cargo new convertisseur_celsius
cd convertisseur_celsius
```

Étape 2 : Écriture du code (`src/main.rs`)

Ouvrez le fichier `src/main.rs` et remplacez son contenu par le code suivant :

```
fn main() {
    // 1. Déclaration de la température de départ en Celsius
    let celsius = 20;

    // 2. Calcul de la conversion en Fahrenheit (calcul sur des entiers)
    let fahrenheit = ((celsius * 9) / 5) + 32;

    // 3. Affichage du résultat dans le terminal
    println!("≡ CONVERTISSEUR DE TEMPÉRATURE ≡");
    println!("Température de départ : {}°C", celsius);
    println!("Température convertie : {}°F", fahrenheit);
    println!("=====");
}
```

Étape 3 : Formatage, vérification et exécution

Avant de lancer le programme, assurez-vous que le style respecte les standards de Rust :

```
cargo fmt
```

Vérifiez qu'il n'y a pas d'erreur de syntaxe :

```
cargo check
```

Enfin, exécutez le programme :

```
cargo run
```

Sortie attendue dans le terminal :

```
≡ CONVERTISSEUR DE TEMPÉRATURE ≡  
Température de départ : 20°C  
Température convertie : 68°F  
=====
```

Exercice 2 : La Chasse aux Bugs (Difficulté : Moyenne)

Énoncé

Un développeur junior vient de vous envoyer un script Rust qui refuse de compiler. Votre mission est de jouer le rôle du compilateur et de l'analyste de code pour identifier, comprendre et corriger les erreurs.

1. Créez un nouveau projet Cargo nommé `chasse_aux_bugs` .
2. Remplacez le contenu de `src/main.rs` par le code erroné suivant :

```
fn main() {  
    let base = 10  
    let hauteur = 5;  
  
    let aire = base * hauteur / 2;  
  
    println("L'aire du triangle est de : {} cm²", aire)  
}
```

1. Tentez de compiler le projet avec `cargo check` ou `cargo build` .
2. Analysez attentivement les messages d'erreur renvoyés par le compilateur `rustc` .
3. Corrigez le code afin qu'il compile parfaitement et affiche le bon résultat.

Indices

1. **Regardez la fin des lignes** : En Rust, chaque instruction qui effectue une action ou une assignation doit impérativement se terminer par un caractère spécifique pour indiquer sa fin au compilateur.
2. **Fonction vs Macro** : Portez une attention particulière à la syntaxe de l'outil d'affichage textuel. Est-ce une fonction classique ou un mécanisme de métaprogrammation propre à Rust ?

Correction Détaillée

Étape 1 : Analyse des erreurs du compilateur

Lorsque vous tentez de compiler le code initial avec `cargo check` , le compilateur Rust génère deux messages d'erreur extrêmement précis :

Erreur 1 :

```

error: expected `;`, found `let`
  → src/main.rs:3:5
   |
2 |     let base = 10
   |               ^ help: add `;` here
3 |     let hauteur = 5;
   |     ^^^ unexpected token

```

Explication : Rust signale qu'il manque un point-virgule `;` à la fin de la ligne 2 avant de pouvoir déclarer la variable suivante à la ligne 3.

Erreur 2 :

```

error[E0423]: expected function, found macro `println`
  → src/main.rs:7:5
   |
7 |     println("L'aire du triangle est de : {} cm²", aire)
   |     ^^^^^^^ not a function
   |
help: use `!` to invoke the macro
   |
7 |     println!("L'aire du triangle est de : {} cm²", aire);
   |                   +                                     +

```

Explication : Rust détecte que vous tentez d'appeler `println` comme s'il s'agissait d'une fonction classique. Il vous rappelle que `println!` est une macro et requiert un point d'exclamation `!`. De plus, il note qu'il manque également un point-virgule `;` à la fin de cette instruction.

Étape 2 : Code corrigé (`src/main.rs`)

Voici le code source corrigé et conforme aux attentes du compilateur :

```
fn main() {  
    // Correction 1 : Ajout du point-virgule manquant après le chiffre 10  
    let base = 10;  
    let hauteur = 5;  
  
    let aire = base * hauteur / 2;  
  
    // Correction 2 : Ajout du '!' pour appeler la macro et du ';' final  
    println!("L'aire du triangle est de : {} cm²", aire);  
}
```

Étape 3 : Validation

Exécutez `cargo run` pour valider la correction.

Sortie attendue dans le terminal :

```
L'aire du triangle est de : 25 cm²
```

Exercice 3 : Le Simulateur de Facturation Éco-Responsable (Difficulté : Avancée)

Énoncé

Vous devez concevoir un prototype d'application de facturation pour une coopérative agricole biologique. Ce programme doit calculer le montant total d'une commande de pommes bio en appliquant une taxe fixe et une réduction de fidélité, puis afficher un reçu propre et professionnel.

1. Créez un nouveau projet Cargo nommé `facture_bio`.
2. Votre programme doit déclarer les variables suivantes :
3. `prix_au_kilo` (valeur entière : 4 euros).
4. `quantite_kilos` (valeur entière : 12 kilos).
5. `taxe_transport` (valeur entière fixe : 5 euros).

6. `reduction_fidelite` (valeur entière fixe : 3 euros).
7. Effectuez les calculs suivants :
8. Le montant brut (prix au kilo multiplié par la quantité).
9. Le montant final net à payer (montant brut auquel on ajoute la taxe de transport et on soustrait la réduction de fidélité).
10. Affichez une facture structurée dans le terminal.
11. Utilisez l'outil d'analyse statique `cargo clippy` pour vérifier si votre code respecte les meilleures pratiques de Rust, puis formatez-le avec `cargo fmt`.

Indices

1. **Calculs par étapes** : Pour garder un code lisible, n'hésitez pas à créer des variables intermédiaires (comme `let montant_brut = ...;`) avant de calculer le montant net final.
2. **Mise en page textuelle** : Vous pouvez utiliser des sauts de ligne `\n` ou aligner vos affichages avec plusieurs macros `println!` successives pour dessiner les contours de votre facture (en utilisant des tirets `-` ou des astérisques `*`).

Correction Détaillée

Étape 1 : Initialisation

```
cargo new facture_bio
cd facture_bio
```

Étape 2 : Implémentation du code (`src/main.rs`)

Remplacez le contenu de `src/main.rs` par le code structuré suivant :

```

fn main() {
    // 1. Définition des paramètres de la commande
    let prix_au_kilo = 4;
    let quantite_kilos = 12;
    let taxe_transport = 5;
    let reduction_fidelite = 3;

    // 2. Calculs intermédiaires et finaux
    let montant_brut = prix_au_kilo * quantite_kilos;
    let montant_total = montant_brut + taxe_transport - reduction_fidelite;

    // 3. Affichage du reçu client
    println!("*****");
    println!("      COOPÉRATIVE AGRICOLE BIO      ");
    println!("      REÇU DE PAIEMENT                ");
    println!("*****");
    println!(" Détails des articles :");
    println!(" - Pommes bio : {} kg x {} €/kg", quantite_kilos, prix_au_kilo);
    println!("-----");
    println!(" Sous-total brut      : {} €", montant_brut);
    println!(" Frais de port        : +{} €", taxe_transport);
    println!(" Remise fidélité      : -{} €", reduction_fidelite);
    println!("-----");
    println!(" TOTAL À PAYER       : {} €", montant_total);
    println!("*****");
    println!("  Merci de soutenir le local & bio !  ");
    println!("*****");
}

```

Étape 3 : Analyse de qualité et formatage

Exécutez le linter de Rust pour vérifier la qualité de votre code :

```
cargo clippy
```

Note : Si votre code est propre, clippy ne renverra aucun avertissement et affichera simplement qu'il a terminé l'analyse.

Formatez le code pour aligner parfaitement les blocs :

```
cargo fmt
```

Étape 4 : Exécution

Lancez l'application :

```
cargo run
```

Sortie attendue dans le terminal :

```
*****
      COOPÉRATIVE AGRICOLE BIO
      REÇU DE PAIEMENT
*****
Détails des articles :
- Pommes bio : 12 kg x 4 €/kg
-----
Sous-total brut      : 48 €
Frais de port        : +5 €
Remise fidélité      : -3 €
-----
TOTAL À PAYER       : 50 €
*****
      Merci de soutenir le local & bio !
*****
```

Variables, mutabilité et types de données de base

1. Introduction Conceptuelle

Dans la majorité des langages de programmation traditionnels (comme C, C++, Java ou Python), les variables sont mutables par défaut. Cela signifie qu'une fois qu'une zone mémoire est allouée et nommée, n'importe quelle partie du programme ayant accès à cette variable peut en modifier la valeur à tout moment. Bien que cette approche semble intuitive, elle est à l'origine d'une classe majeure de bogues logiciels : les effets de bord imprévisibles et les concurrences critiques (data races).

Rust prend le contre-pied de cette philosophie en imposant l'immutabilité par défaut. Lorsque vous déclarez une variable en Rust, sa valeur est figée. Si vous tentez de la modifier, le compilateur refusera de générer le binaire.

Pourquoi ce choix ?

1. **Prévisibilité du code** : Un développeur (ou un compilateur) qui lit une fonction peut garantir qu'une variable immuable conservera la même valeur du début à la fin de la portée. Il n'est pas nécessaire de tracer mentalement toutes les lignes de code pour vérifier si la valeur a été altérée ailleurs.
2. **Optimisation du compilateur** : Le compilateur Rust (basé sur LLVM) peut optimiser agressivement le code machine s'il sait de source sûre que certaines données ne changeront jamais (par exemple, en plaçant la valeur directement dans les registres du processeur).
3. **Sécurité en programmation concurrente** : Si plusieurs threads (fils d'exécution) accèdent simultanément à une donnée en lecture seule, aucun conflit ne peut survenir. L'immutabilité élimine nativement le besoin de verrous complexes pour ces scénarios.

La mutabilité reste indispensable pour modéliser le monde réel (comme l'évolution d'un score dans un jeu vidéo ou le remplissage d'un panier d'achat). Rust permet donc la

mutabilité, mais elle doit être **explicite**. Ce choix conceptuel force le développeur à se poser la question systématique : *"Cette donnée a-t-elle réellement besoin de changer ?"*

2. Fondations Théoriques

2.1. Immuabilité vs Mutabilité

En Rust, la déclaration d'une variable se fait avec le mot-clé `let`. Par défaut, cette liaison est immuable. Pour la rendre mutable, on adjoint le mot-clé `mut`.

```
let x = 5;      // Immuable : x vaudra toujours 5
let mut y = 5; // Mutable : y peut être réassigné plus tard
```

2.2. Le Masquage (Shadowing)

Le masquage consiste à déclarer une nouvelle variable portant exactement le même nom qu'une variable existante dans la même portée. La seconde variable "masque" (shadows) la première.

```
let x = 5;
let x = x + 1; // x est maintenant masqué par un nouveau x qui vaut 6
```

Différence fondamentale entre Mutabilité et Masquage :

- **Mutabilité (`mut`)** : On modifie la valeur stockée dans la *même* boîte mémoire. Le type de la variable ne peut pas changer.
- **Masquage (`let`)** : On crée une *nouvelle* variable, dans un nouvel espace mémoire, en réutilisant le même nom. Cela permet notamment de **changer le type** de la variable tout en conservant un nom cohérent.

2.3. Constantes (`const`) vs Variables Immuables (`let`)

Bien qu'elles semblent similaires, les constantes et les variables immuables répondent à des règles différentes :

Caractéristique	Variable Immuable (<code>let</code>)	Constante (<code>const</code>)
Évaluation	Évaluée à l'exécution (Runtime)	Évaluée à la compilation (Compile-time)
Type	Optionnel (inféré par le compilateur)	Obligatoire (doit être explicité)
Portée	Locale au bloc de code	Globale ou locale
Mutabilité	Peut devenir mutable avec <code>mut</code>	Strictement immuable (jamais de <code>mut</code>)
Valeur	Peut être le résultat d'une fonction	Doit être une expression constante pure

```
const VITESSE_LUMIERE_M_S: u32 = 299_792_458; // Déclaration d'une constante
```

2.4. Types de Données Scalaires

Un type scalaire représente une valeur unique. Rust dispose de quatre types scalaires principaux.

A. Les Entiers (Integers)

Ils se divisent en deux catégories : signés (peuvent être négatifs, préfixés par `i`) et non signés (uniquement positifs ou nuls, préfixés par `u`).

Taille	Signé	Non signé
8 bits	<code>i8</code>	<code>u8</code>
16 bits	<code>i16</code>	<code>u16</code>
32 bits	<code>i32</code> (défaut)	<code>u32</code>
64 bits	<code>i64</code>	<code>u64</code>
128 bits	<code>i128</code>	<code>u128</code>
Architecture	<code>isize</code>	<code>usize</code>

Note sur `isize` et `usize` : Leur taille dépend de l'architecture de la machine cible (32 bits ou 64 bits). Ils sont principalement utilisés pour l'indexation des collections ou la mesure de la taille d'objets en mémoire.

B. Les Flottants (Floating-Point)

Rust gère les nombres à virgule flottante selon la norme IEEE-754 : * `f32` : Simple précision (32 bits). * `f64` : Double précision (64 bits, type par défaut car plus précis sur les processeurs modernes sans perte notable de performance).

C. Le Booléen (Boolean)

Le type `bool` ne peut prendre que deux valeurs : `true` ou `false`. Il occupe 1 octet en mémoire.

D. Le Caractère (Character)

Le type `char` représente un point de code Unicode. Contrairement à d'autres langages où un caractère fait 1 octet (ASCII), en Rust, un `char` fait 4 octets. Il peut représenter un caractère accentué, un idéogramme chinois, ou même un émoji.

```
let c: char = 'z';  
let z: char = 'z';  
let heart_eye_emoji: char = '🤩';
```

2.5. Types de Données Composés

Les types composés regroupent plusieurs valeurs dans un seul type.

A. Les Tuples

Un tuple est une collection de valeurs de types différents avec une taille fixe.

```
let mon_tuple: (i32, f64, u8) = (500, 6.4, 1);
```

Pour récupérer les valeurs d'un tuple, on utilise soit la déstructuration, soit l'accès direct par point (`.`) :

```
// Déstructuration  
let (x, y, z) = mon_tuple;  
  
// Accès direct  
let cinq_cents = mon_tuple.0;  
let un = mon_tuple.2;
```

B. Les Tableaux (Arrays)

Un tableau regroupe plusieurs valeurs du **même type** avec une **taille fixe** déterminée à la compilation.

Note de progression : Contrairement aux vecteurs (qui seront étudiés dans le Module 3), la taille d'un tableau ne peut ni augmenter ni diminuer.

```
let mon_tableau: [i32; 5] = [1, 2, 3, 4, 5];  
let premier_element = mon_tableau[0];
```


3. Implémentation Pratique Pas-à-Pas

Nous allons écrire un programme complet mettant en œuvre ces concepts. Créez un projet Cargo nommé `variables_et_types` :

```
cargo new variables_et_types  
cd variables_et_types
```

Remplacez le contenu du fichier `src/main.rs` par le code suivant :

```

// Déclaration d'une constante globale
const TAUX_CONVERSION_USD: f64 = 1.12;

fn main() {
    // --- 1. Immuabilité et Mutabilité ---
    let solde_initial = 100; // Immuable par défaut
    println!("Solde initial : {} EUR", solde_initial);

    // Tentative de modification (décommenter la ligne ci-dessous provoquera une erreur de
    // solde_initial = 120;

    let mut solde_courant = solde_initial; // Copie de la valeur, mutable
    solde_courant += 50;
    println!("Nouveau solde après dépôt : {} EUR", solde_courant);

    // --- 2. Le Masquage (Shadowing) ---
    let texte_espace = "   "; // Type: &str (chaîne de caractères)
    let texte_espace = texte_espace.len(); // Masquage : Nouveau type (usize)
    println!("Nombre d'espaces : {}", texte_espace);

    // --- 3. Types Scalaires ---
    let entier_signe: i8 = -120;
    let entier_non_signe: u32 = 4_294_967_295; // Utilisation de '_' pour la lisibilité
    let flottant: f32 = 3.14159;
    let est_actif: bool = true;
    let symbole: char = 'λ';

    println!("Scalaires : {}, {}, {}, {}, {}", entier_signe, entier_non_signe, flottant, es

    // --- 4. Types Composés ---
    // Tuple représentant un point géographique (Latitude, Longitude, Nom)
    let coordonnees: (f64, f64, char) = (48.8566, 2.3522, 'P');
    let (lat, lon, label) = coordonnees;
    println!("Coordonnées de '{}' : ({} , {})", label, lat, lon);

    // Tableau de températures sur une semaine (taille fixe de 7)
    let temperatures: [f32; 7] = [12.5, 14.0, 13.8, 11.2, 15.1, 16.0, 14.5];
    println!("Température du premier jour : {} °C", temperatures[0]);
    println!("Température du dernier jour : {} °C", temperatures[6]);

    // Conversion de devise en utilisant la constante globale
    let solde_usd = (solde_courant as f64) * TAUX_CONVERSION_USD;
    println!("Solde équivalent en USD : {:.2} $", solde_usd);
}

```

Compilation et exécution :

```
cargo run
```

Sortie attendue dans la console :

```
Solde initial : 100 EUR
Nouveau solde après dépôt : 150 EUR
Nombre d'espaces : 3
Scalaire : -120, 4294967295, 3.14159, true, λ
Coordonnées de 'P' : (48.8566, 2.3522)
Température du premier jour : 12.5 °C
Température du dernier jour : 14.5 °C
Solde équivalent en USD : 168.00 $
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Confondre Mutabilité et Masquage

Un développeur débutant peut penser que `let mut x` et le masquage de `x` sont équivalents. C'est faux.

```
// Erreur de type avec la mutabilité
let mut donnees = "123";
donnees = donnees.len(); // ERREUR DE COMPILATION : attendu &str, trouvé usize
```

Explication : La mutabilité permet de changer la *valeur*, pas le *type*. Le masquage permet de changer les deux car il recrée une variable.

Piège 2 : Le dépassement d'entier (Integer Overflow)

Que se passe-t-il si vous tentez de dépasser la valeur maximale d'un type entier ?

```
let mut nombre: u8 = 255;
nombre = nombre + 1;
```

- En mode Debug (`cargo run`) : Rust inclut des vérifications à l'exécution. Le programme va immédiatement s'arrêter en générant une erreur panique (*panic*).
- En mode Release (`cargo build --release`) : Pour des raisons de performance, Rust désactive ces vérifications. Le dépassement s'effectue selon le principe du complément à deux (wrapping) : `255 + 1` devient `0`.

Bonne pratique : Pour gérer explicitement les risques de dépassement, utilisez les méthodes dédiées de la bibliothèque standard : * `wrapping_add(val)` : force le bouclage (retour à 0). * `checked_add(val)` : retourne un type `Option` (étudié dans le Module 4). * `overflowing_add(val)` : retourne un tuple avec le résultat et un booléen indiquant s'il y a eu dépassement.

Piège 3 : L'accès hors limites dans un tableau

Tenter d'accéder à un index inexistant dans un tableau provoque un arrêt brutal du programme (*panic*) à l'exécution.

```
let tab = [1, 2, 3];
let element = tab[5]; // ERREUR : panic à l'exécution (ou erreur de compilation si l'index
```

Bonne pratique : Rust protège votre mémoire contre les accès illégaux (pas de faille de dépassement de tampon / *buffer overflow*). Cependant, pour éviter les plantages, privilégiez l'utilisation de structures dynamiques sécurisées ou de vérifications de limites.

5. Synthèse Pédagogique

Tableau Comparatif des Mécanismes de Liaison

Mécanisme	Syntaxe	Type modifiable ?	Valeur modifiable ?	Portée d'utilisation
Liaison Immuable	<code>let x = 5;</code>	Non	Non	Locale
Liaison Mutable	<code>let mut x = 5;</code>	Non	Oui	Locale
Masquage	<code>let x = 5; let x = "cinq";</code>	Oui	Oui (nouvelle var)	Locale
Constante	<code>const X: u32 = 5;</code>	Non	Non	Globale ou Locale

Points Clés à Retenir

1. Sécurité par défaut : Tout est immuable en Rust tant que vous n'avez pas écrit explicitement `mut`.
2. Typage statique et fort : Le compilateur doit connaître le type de chaque variable à la compilation. S'il ne peut pas l'inférer, vous devez l'annoter (`let x: u32 = 10;`).
3. Unicode natif : Le type `char` fait 4 octets et supporte l'ensemble des caractères mondiaux.
4. Zéro comportement indéfini : Rust préfère faire planter proprement votre programme (panic) plutôt que de vous laisser accéder à de la mémoire corrompue (dépassement d'index de tableau).

6. Exercice Pratique d'Application

Énoncé : "Le Système de Télémétrie d'un Drone"

Vous devez concevoir la logique de traitement des données de vol d'un drone de reconnaissance. Le programme doit manipuler plusieurs variables pour calculer l'état de l'appareil.

1. Déclarez une constante globale `GRAVITE` de type `f32` égale à `9.81`.
2. Dans la fonction `main`, déclarez les variables suivantes avec les types appropriés :
3. L'identifiant du drone : une chaîne de caractères immuable (ex: `"Alpha-1"`).
4. L'altitude actuelle : un entier mutable de 32 bits (initialisé à `150` mètres).
5. Les coordonnées GPS : un tuple immuable contenant la latitude (`f64`) et la longitude (`f64`).
6. Les 3 dernières vitesses enregistrées : un tableau de 3 flottants (`f32`) initialisé à `10.2`, `12.5`, et `11.8`.
7. Simulez une mise à jour des données de vol :
8. Le drone monte de `50` mètres (modifiez l'altitude).
9. Calculez la vitesse moyenne à partir du tableau de vitesses.
10. Utilisez le **masquage** pour convertir la vitesse moyenne (qui est un `f32`) en un entier arrondi (`i32`) afin de simplifier l'affichage sur l'écran de contrôle.
11. Affichez l'ensemble de ces informations de manière claire dans la console.

Indices pour la résolution

- Pour modifier l'altitude, n'oubliez pas le mot-clé permettant la mutabilité.
- La formule de la moyenne de 3 valeurs : `(val1 + val2 + val3) / 3.0`.
- Pour convertir un type numérique en un autre en Rust, on utilise l'opérateur `as` (ex: `let entier = mon_flottant as i32;`).

Correction Complète et Commentée

Voici le code source complet permettant de résoudre l'exercice :

```
// 1. Déclaration de la constante globale
const GRAVITE: f32 = 9.81;

fn main() {
    // 2. Initialisation des variables de télémétrie
    let identifiant_drone: &str = "Alpha-1";
    let mut altitude: i32 = 150; // Doit être mutable car l'altitude va changer
    let coordonnees_gps: (f64, f64) = (43.6047, 1.4442); // Toulouse, France
    let vitesses: [f32; 3] = [10.2, 12.5, 11.8];

    // Affichage de l'état initial
    println!("--- TÉLÉMÉTRIE INITIALE [{}] ---", identifiant_drone);
    println!("Altitude : {} m", altitude);
    println!("Position GPS : (Lat: {}, Lon: {})", coordonnees_gps.0, coordonnees_gps.1);
    println!("Constante de gravité appliquée : {} m/s²", GRAVITE);
    println!("-----");

    // 3. Simulation des modifications
    // A. Le drone prend de l'altitude
    altitude += 50;

    // B. Calcul de la vitesse moyenne (f32)
    let vitesse_moyenne: f32 = (vitesse[0] + vitesses[1] + vitesses[2]) / 3.0;

    // C. Masquage pour convertir le type de la vitesse moyenne en entier (i32)
    // Nous réutilisons le nom 'vitesse_moyenne' mais changeons son type grâce à 'let'
    let vitesse_moyenne: i32 = vitesse_moyenne as i32;

    // 4. Affichage des résultats mis à jour
    println!("\n--- MISE À JOUR DES DONNÉES ---");
    println!("Nouvelle Altitude : {} m", altitude);
    println!("Vitesse Moyenne Arrondie : {} km/h", vitesse_moyenne);
    println!("Statut de vol : Stable");
    println!("-----");
}
```

Analyse de la correction :

- **Mutabilité** : La variable `altitude` a été déclarée avec `let mut` car sa valeur a été incrémentée de 50 (`altitude += 50`).
- **Masquage** : La variable `vitesse_moyenne` est d'abord calculée en tant que `f32` pour conserver la précision du calcul. Ensuite, nous avons utilisé `let vitesse_moyenne: i32 = ...` pour la masquer. Cela évite de polluer l'espace de nommage avec des variables redondantes comme `vitesse_moyenne_float` et `vitesse_moyenne_int` .

- **Sécurité** : Le compilateur garantit qu'aucune autre partie du code ne pourra modifier accidentellement l'identifiant du drone ou les coordonnées GPS, assurant ainsi l'intégrité des données de vol critiques.

Exercices d'Application : Variables, mutabilité et types de données de base

Exercice 1 : Le Convertisseur de Devises Sécurisé (Difficulté : Facile)

Énoncé

Vous devez concevoir un petit module pour une application financière. Ce module doit gérer le solde d'un utilisateur, appliquer un bonus de fidélité, puis convertir ce solde dans une autre devise (le Yen japonais, JPY) sous forme d'un entier arrondi, car le Yen n'utilise pas de centimes.

1. Déclarez une constante globale `TAUX_EUR_TO_JPY` de type `f64` égale à `162.45`.
2. Dans la fonction `main` :
3. Déclarez un solde initial immuable `solde_initial` de `120.50` euros.
4. Déclarez un bonus de fidélité immuable `bonus` de `15.00` euros.
5. Déclarez une variable mutable `solde_courant` initialisée avec la valeur de `solde_initial`.
6. Ajoutez le `bonus` au `solde_courant`.
7. Calculez la valeur convertie en Yen (multiplication du `solde_courant` par le taux de conversion).
8. Utilisez le `masquage` (shadowing) pour convertir cette valeur en Yen (qui est un `f64`) en un entier non signé de 32 bits (`u32`) afin d'obtenir une valeur arrondie (tronquée).

9. Affichez le solde final en Euros et le solde converti en Yen avec leurs symboles respectifs (€ et ¥).

Indices

- Pour ajouter le bonus, utilisez l'opérateur d'assignation combiné `+=` .
- Pour convertir un type en un autre (par exemple de `f64` à `u32`), utilisez le mot-clé `as` : `let variable_entiere = variable_flottante as u32;` .
- Le symbole du Yen ¥ et de l'Euro € peuvent être affichés directement dans vos chaînes de caractères de formatage `println!("{}", solde_jpy)` .

Correction Détaillée

```
// 1. Déclaration de la constante globale pour le taux de change
const TAX_EUR_TO_JPY: f64 = 162.45;

fn main() {
    // 2. Déclaration des variables de base
    let solde_initial: f64 = 120.50;
    let bonus: f64 = 15.00;

    // Utilisation de 'mut' car le solde va être modifié par le bonus
    let mut solde_courant = solde_initial;
    solde_courant += bonus;

    // Calcul de la conversion en JPY (le résultat est temporairement un f64)
    let solde_jpy = solde_courant * TAX_EUR_TO_JPY;

    // Masquage (Shadowing) : on réutilise le nom 'solde_jpy' pour changer son type en u32
    let solde_jpy: u32 = solde_jpy as u32;

    // 3. Affichage des résultats
    println!("--- Relevé de Compte ---");
    println!("Solde initial : {:.2} €", solde_initial);
    println!("Bonus appliqué : {:.2} €", bonus);
    println!("Nouveau solde : {:.2} €", solde_courant);
    println!("Solde converti : {} ¥", solde_jpy);
}
```

Exercice 2 : Analyseur de Données d'une Station Météo (Difficulté : Moyenne)

Énoncé

Une station météo locale enregistre les températures d'une ville sur une semaine (7 jours). Vous devez écrire un programme qui stocke ces données, extrait des mesures spécifiques, calcule la variation de température et produit un rapport résumé sous forme de tuple.

1. Déclarez une constante `VILLE` contenant le nom de la ville étudiée (ex: `"Strasbourg"`).
2. Créez un tableau contenant les températures (de type `f32`) relevées du lundi au dimanche : `14.2` , `15.5` , `13.8` , `12.0` , `11.5` , `14.8` , `16.2` .
3. Calculez la température moyenne de la semaine. (Puisque nous n'avons pas encore vu les boucles, faites la somme manuellement en accédant aux index du tableau).
4. Calculez la différence de température (delta) entre le jour le plus chaud (dimanche) et le jour le plus froid (vendredi) en accédant directement aux éléments du tableau par leur index.
5. Stockez les résultats dans un tuple nommé `rapport` contenant :
6. Le nom de la ville (`&str`)
7. La température moyenne (`f32`)
8. Le delta de température (`f32`)
9. Affichez les informations du rapport en utilisant la déstructuration de tuple, puis en utilisant l'accès direct par point (`.`).

Indices

- Les index d'un tableau de taille 7 vont de `0` à `6` . Le vendredi correspond à l'index `4` et le dimanche à l'index `6` .
- La déstructuration d'un tuple se fait ainsi : `let (a, b, c) = mon_tuple; .`
- L'accès direct par point se fait ainsi : `mon_tuple.0` pour le premier élément.

Correction Détaillée

```
const VILLE: &str = "Strasbourg";

fn main() {
    // 1. Tableau des températures de la semaine (taille fixe de 7)
    let temperatures: [f32; 7] = [14.2, 15.5, 13.8, 12.0, 11.5, 14.8, 16.2];

    // 2. Calcul manuel de la moyenne
    let somme = temperatures[0]
        + temperatures[1]
        + temperatures[2]
        + temperatures[3]
        + temperatures[4]
        + temperatures[5]
        + temperatures[6];
    let moyenne = somme / 7.0;

    // 3. Calcul du delta entre le dimanche (index 6) et le vendredi (index 4)
    let temp_max = temperatures[6]; // 16.2
    let temp_min = temperatures[4]; // 11.5
    let delta = temp_max - temp_min;

    // 4. Création du tuple de rapport
    let rapport = (VILLE, moyenne, delta);

    // 5. Affichage via la déstructuration de tuple
    let (ville_nom, temp_moyenne, temp_delta) = rapport;
    println!("--- Rapport Météo (Déstructuration) ---");
    println!("Ville : {}", ville_nom);
    println!("Température moyenne : {:.1} °C", temp_moyenne);
    println!("Amplitude thermique : {:.1} °C", temp_delta);

    // 6. Affichage via l'accès direct par point (pour démonstration)
    println!("\n--- Rapport Météo (Accès direct) ---");
    println!("Ville cible : {}", rapport.0);
    println!("Moyenne calculée : {:.1} °C", rapport.1);
}
```

Exercice 3 : Gestionnaire d'Expérience de Jeu Rétro

(Difficulté : Avancée)

Énoncé

Dans les consoles de jeux vidéo rétro (comme la NES), la mémoire était très limitée. Les concepteurs utilisaient souvent des entiers non signés de 8 bits (`u8`) pour stocker les statistiques comme les points d'expérience (XP). Un `u8` ne peut pas dépasser la valeur `255` .

Vous devez concevoir un système de gestion d'XP qui simule un gain de points et gère de manière sécurisée le risque de dépassement de capacité (*integer overflow*), afin d'éviter que le jeu ne plante (panic) ou que l'XP du joueur ne retombe à zéro de façon injuste.

1. Déclarez une variable mutable `xp_actuelle` de type `u8` initialisée à `245` .
2. Déclarez une variable `xp_gagne` de type `u8` égale à `20` .
3. Si vous tentez de faire `xp_actuelle + xp_gagne` directement en mode Debug, le programme va planter. Pour éviter cela, utilisez la méthode de la bibliothèque standard `overflowing_add` .
4. La méthode `overflowing_add` renvoie un tuple contenant deux valeurs : `(resultat_calculé, a_depasse)` . Le premier élément est le résultat de l'addition (avec bouclage si dépassement), le second est un booléen valant `true` s'il y a eu dépassement.
5. Récupérez ce tuple en le déstructurant.
6. Affichez le résultat. Si un dépassement a eu lieu, affichez un message indiquant que le joueur a atteint le niveau maximum et forcez manuellement sa valeur d'XP à `255` (le maximum pour un `u8`).

Indices

- La syntaxe pour appeler la méthode de dépassement sur un entier est : `let (nouveau_score, a_depasse) = xp_actuelle.overflowing_add(xp_gagne);` .
- Bien que nous n'ayons pas formellement étudié les structures conditionnelles `if` dans ce module, elles s'utilisent comme dans la plupart des langages : `if`

`condition { ... } else { ... } . Utilisez-les pour plafonner l'XP à 255 si a_depasse est vrai.`

Correction Détaillée

```
fn main() {
    // 1. Initialisation de l'XP proche de la limite d'un u8 (max 255)
    let mut xp_actuelle: u8 = 245;
    let xp_gagne: u8 = 20;

    println!("XP Actuelle : {} / 255", xp_actuelle);
    println!("Points d'XP gagnés : {}", xp_gagne);

    // 2. Utilisation de overflowing_add pour éviter le crash (panic) du programme
    // Cette méthode retourne un tuple : (valeur_bouclée, bool_depassement)
    let (resultat, a_depasse) = xp_actuelle.overflowing_add(xp_gagne);

    // 3. Traitement du dépassement de capacité
    if a_depasse {
        println!("\n[ATTENTION] Dépassement de capacité détecté ! (Calcul de l'XP a été bouclé à 255)");
        println!("Félicitations, vous avez atteint le niveau maximum !");
        // On plafonne manuellement l'XP à la valeur maximale autorisée pour le type u8
        xp_actuelle = 255;
    } else {
        println!("\nGain d'XP validé sans dépassement.");
        xp_actuelle = resultat;
    }

    // 4. Affichage du statut final du joueur
    println!("Statut final - XP du joueur : {} / 255", xp_actuelle);
}
```

Qu'est-ce que la mémoire ?

Comprendre la Pile (Stack), le Tas (Heap) et les dangers du code non sécurisé

1. Introduction Conceptuelle

Pour écrire du code rapide, fiable et sécurisé, un développeur ne peut pas traiter la mémoire de l'ordinateur comme une boîte noire magique. Chaque variable créée, chaque fichier lu et chaque structure de données instanciée doit résider quelque part dans la mémoire vive (RAM) de la machine.

Le "Pourquoi" avant le "Comment"

Dans les langages de programmation, la gestion de la mémoire se divise historiquement en deux grandes familles : 1. La gestion manuelle (ex: C, C++) : Le programmeur demande explicitement de la mémoire au système d'exploitation et doit impérativement la libérer lui-même. C'est d'une efficacité redoutable, mais la moindre distraction humaine conduit à des failles de sécurité critiques ou à des plantages. 2. La gestion automatique par Garbage Collector (ex: Java, Python, Go) : Un programme d'arrière-plan surveille constamment la mémoire et nettoie ce qui n'est plus utilisé. C'est extrêmement sécurisé, mais cela consomme des ressources (CPU, RAM) et introduit des pauses imprévisibles (les fameux "GC pauses").

Rust propose une troisième voie révolutionnaire : la sécurité mémoire absolue, sans Garbage Collector, garantissant des performances maximales. Pour comprendre comment Rust réalise ce tour de force (que nous étudierons en détail dans le Module 2), il est indispensable de maîtriser les deux zones fondamentales de la mémoire : la Pile (Stack) et le Tas (Heap).

2. Fondations Théoriques

La mémoire allouée à un programme en cours d'exécution (un processus) est segmentée en plusieurs zones. Nous allons nous concentrer sur les deux plus importantes pour le développeur : la Pile et le Tas.

MÉMOIRE DU PROCESSUS			
LA PILE (STACK)		LE TAS (HEAP)	
- Taille fixe, connue à la comp.		- Taille dynamique	
- Accès ultra-rapide (LIFO)		- Accès plus lent	
- Gestion automatique par CPU		- Allocation manuelle	

2.1 La Pile (Stack)

La Pile fonctionne selon le principe LIFO (*Last In, First Out* - Dernier entré, premier sorti). On peut la comparer à une pile d'assiettes : vous ne pouvez ajouter ou retirer une assiette que par le haut.

- **Caractéristiques :**
- **Taille connue et fixe :** Toutes les données stockées sur la pile doivent avoir une taille connue et fixe dès la compilation.
- **Vitesse :** L'allocation est extrêmement rapide. Le processeur se contente de déplacer un pointeur de pile (*stack pointer*).
- **Cycle de vie :** Les données sont liées à la fonction qui les a créées. Dès que la fonction se termine, sa "tranche" de pile (appelée *Stack Frame* ou cadre de pile) est immédiatement et automatiquement libérée.

2.2 Le Tas (Heap)

Le Tas est un grand espace de mémoire désorganisé. On peut le comparer à un immense entrepôt de stockage.

- **Caractéristiques :**

- **Taille dynamique** : On y stocke des données dont la taille peut changer au cours de l'exécution, ou dont la taille est inconnue au moment de la compilation (par exemple, un texte saisi par l'utilisateur).
- **Vitesse** : L'allocation est plus lente. Le système d'exploitation doit chercher un espace libre assez grand pour accueillir les données, marquer cet espace comme "occupé", puis retourner l'adresse de cet emplacement.
- **Le Pointeur** : Pour retrouver ces données, on stocke l'adresse mémoire du Tas (un pointeur) sur la Pile. Le pointeur a une taille fixe (par exemple, 64 bits sur un système moderne), il peut donc loger sur la pile.

LA PILE (STACK)	LE TAS (HEAP)
+-----+	+-----+
Variable 'x' : 42	
+-----+	
Pointeur 'p' : 0x00FF8A	[1, 2, 3, 4, 5, 6]
+-----+	Données dynamiques
	+-----+

2.3 Les Dangers du Code Non Sécurisé

Lorsque la gestion de ces deux zones est mal maîtrisée (comme en C ou C++), quatre bogues majeurs et extrêmement dangereux surviennent :

1. **L'utilisation après libération (Use-After-Free)** : Un pointeur pointe vers une adresse sur le Tas. La mémoire à cette adresse est libérée, mais le programme tente de lire ou d'écrire à nouveau via ce pointeur. Un attaquant peut exploiter cela pour exécuter du code malveillant.
2. **La double libération (Double Free)** : Le programme tente de libérer deux fois la même zone de mémoire sur le Tas. Cela corrompt les structures internes de l'allocateur de mémoire et provoque des plantages ou des failles de sécurité.
3. **Le débordement de tampon (Buffer Overflow)** : Le programme écrit des données au-delà de la limite allouée (sur la Pile ou le Tas), écrasant les variables voisines ou les instructions de retour de fonction.
4. **La fuite de mémoire (Memory Leak)** : De la mémoire est allouée sur le Tas, mais le programme perd le pointeur associé sans l'avoir libérée. La mémoire reste marquée comme occupée, et le programme finit par consommer toute la RAM du système.

3. Implémentation Pratique Pas-à-Pas

Voyons comment Rust gère concrètement ces concepts. Nous allons utiliser le type `Box<T>`, qui est le moyen le plus simple en Rust de forcer l'allocation d'une donnée sur le Tas.

Étape 1 : Allocation sur la Pile

Par défaut, les types primitifs de taille fixe (comme étudiés dans le Module 1.2) sont alloués sur la pile.

```
fn main() {
    // Allocation sur la PILE
    let x: i32 = 42;
    let y: bool = true;

    println!("x vit sur la pile : {}", x);
    println!("y vit sur la pile : {}", y);
} // x et y sont retirés de la pile automatiquement ici
```

Étape 2 : Allocation sur le Tas avec `Box<T>`

Pour placer un entier sur le Tas, nous l'enveloppons dans une `Box`.

```
fn main() {
    // 'valeur_tas' est un pointeur stocké sur la PILE.
    // Il pointe vers la valeur 512 stockée sur le TAS.
    let valeur_tas: Box<i32> = Box::new(512);

    println!("La valeur pointée sur le tas est : {}", valeur_tas);
} // 'valeur_tas' sort de la portée (scope) ici.
// Rust libère AUTOMATIQUEMENT la mémoire sur le tas.
```

Étape 3 : Visualiser le nettoyage automatique (RAII)

Rust utilise un concept appelé RAII (*Resource Acquisition Is Initialization*). La mémoire est acquise lors de la création de la variable et libérée automatiquement dès que la variable "sort de la portée" (à l'accolade de fermeture `}`).

Pour le prouver, créons un exemple où nous contrôlons la portée avec un bloc de code `{ ... }` :

```
fn main() {
    println!("Début du programme.");

    {
        // Début d'une portée locale
        let pointeur_interne = Box::new(1024);
        println!("Pointeur interne créé : {}", pointeur_interne);
    } // ← 'pointeur_interne' sort de la portée ICI.
        // La mémoire sur le tas contenant 1024 est immédiatement libérée.

    // println!("{}", pointeur_interne);
    // ^ Si vous décommentez la ligne ci-dessus, le compilateur Rust
    // refusera de compiler car la variable n'existe plus !

    println!("Fin du programme.");
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Tenter de retourner une référence vers la Pile

Un grand classique des erreurs de segmentation en C est de retourner l'adresse d'une variable locale à la fin d'une fonction. En Rust, le compilateur bloque immédiatement cette tentative.

```
// CE CODE NE COMPILE PAS
fn creer_reference_dangereuse() -> &i32 {
    let valeur_locale = 42; // Allouée sur la pile de cette fonction
    &valeur_locale // On tente de retourner une référence (un pointeur) vers la pile
} // 'valeur_locale' est détruite ici ! La référence pointerait vers du vide.

fn main() {
    let _ref_invalide = creer_reference_dangereuse();
}
```

Message d'erreur du compilateur :

```
error[E0106]: missing lifetime specifier
  → src/main.rs:2:36
   |
2  | fn creer_reference_dangereuse() → &i32 {
   |                                     ^ expected named lifetime parameter
```

Note : Le concept de "lifetime" (durée de vie) sera étudié en détail dans le Module 5.1. Pour l'instant, retez que Rust empêche physiquement la création de pointeurs vers de la mémoire détruite.

Bonne Pratique : Privilégier la Pile par défaut

L'allocation sur le Tas a un coût en performances. N'utilisez `Box` (ou d'autres structures dynamiques comme `Vec` ou `String` que nous verrons dans le Module 3.3) que si : 1. Vous devez manipuler des données dont la taille est inconnue à la compilation. 2. Vous devez transférer la propriété d'une structure de données volumineuse sans la copier en mémoire.

5. Synthèse Pédagogique

Tableau Comparatif : Pile vs Tas

Critère	La Pile (Stack)	Le Tas (Heap)
Taille des données	Fixe, connue à la compilation.	Dynamique, inconnue à la compilation.
Vitesse d'accès	Ultra-rapide (accès direct).	Plus lente (indirection par pointeur).
Mécanisme d'allocation	Automatique par le CPU (LIFO).	Requête au système d'exploitation.
Libération	Immédiate à la fin de la fonction.	Gérée par Rust via la portée (RAII).
Types Rust typiques	<code>i32</code> , <code>f64</code> , <code>bool</code> , <code>char</code> , Tableaux fixes <code>[T; N]</code> .	<code>Box<T></code> , <code>Vec<T></code> , <code>String</code> .

Résumé des garanties de Rust

Grâce à son compilateur strict, Rust élimine à la compilation :

- * Les **Use-After-Free** : Impossible d'accéder à une variable hors de sa portée.
- * Les **Double Free** : Rust sait exactement quand libérer la mémoire (une seule fois, à la sortie de la portée).
- * Les **Buffer Overflows** : Les accès aux tableaux sont vérifiés à l'exécution (nous verrons cela dans le Module 2.3 sur les Slices).

6. Exercice Pratique d'Application

Énoncé

Le but de cet exercice est de concevoir un programme qui simule et compare l'allocation sur la pile et sur le tas, puis de corriger une tentative d'accès mémoire invalide.

1. Créez une fonction `allover_sur_pile()` qui déclare un tableau de 3 entiers sur la pile et l'affiche.
2. Créez une fonction `allover_sur_tas()` qui alloue ce même tableau sur le tas à l'aide d'une `Box`, puis le retourne au programme principal.
3. Dans la fonction `main`, récupérez le tableau alloué sur le tas, modifiez sa première valeur, puis affichez-le.

Indices

- Un tableau de taille fixe s'écrit sous la forme `[type; taille]`. Par exemple : `[i32; 3]`.
- Pour allouer un tableau sur le tas, utilisez `Box::new([valeur1, valeur2, valeur3])`.

Correction Complète

Voici le code source complet et fonctionnel que vous pouvez tester :

```

// 1. Allocation sur la pile
fn allouer_sur_pile() {
    // Ce tableau est stocké entièrement sur la pile de la fonction
    let tableau_pile: [i32; 3] = [10, 20, 30];
    println!("Tableau sur la pile : {:?}", tableau_pile);
} // 'tableau_pile' est détruit ici.

// 2. Allocation sur le tas et retour de la Box
fn allouer_sur_tas() → Box<[i32; 3]> {
    // Le tableau est créé sur le tas.
    // La Box (le pointeur) est sur la pile et est retournée.
    let tableau_tas = Box::new([100, 200, 300]);
    tableau_tas
} // La mémoire sur le tas n'est PAS libérée ici car nous avons retourné la Box

fn main() {
    println!("--- Début de l'exercice ---");

    // Appel de la fonction sur la pile
    allouer_sur_pile();

    // 3. Récupération et modification de la valeur sur le tas
    // Nous devons déclarer la variable comme mutable 'mut' pour pouvoir la modifier
    let mut mon_tableau_tas = allouer_sur_tas();

    println!("Tableau initial sur le tas : {:?}", mon_tableau_tas);

    // Modification de la première valeur
    mon_tableau_tas[0] = 999;

    println!("Tableau modifié sur le tas : {:?}", mon_tableau_tas);

    println!("--- Fin de l'exercice ---");
} // 'mon_tableau_tas' sort de la portée ici.
    // La mémoire sur le tas contenant [999, 200, 300] est proprement libérée.

```

Explications de la correction

- Ligne 11 : La fonction `allouer_sur_tas` retourne un type `Box<[i32; 3]>`. En retournant cette Box, nous transférons la propriété du pointeur au `main`. C'est ce qui évite que la mémoire du tas soit libérée prématurément.
- Ligne 23 : Nous utilisons le mot-clé `mut` (étudié dans le Module 1.2) car par défaut, toutes les variables en Rust sont immuables. Cela s'applique également aux données stockées sur le tas.

- **Sécurité** : À aucun moment nous n'avons eu besoin d'écrire `free()` ou `delete` . Rust a inséré le code de libération de mémoire de manière invisible à la ligne 31, garantissant l'absence totale de fuite de mémoire.

Exercices d'Application : Qu'est-ce que la mémoire ? Comprendre la Pile (Stack), le Tas (Heap) et les dangers du code non sécurisé

Exercice 1 : Visualisation de la Mémoire et Portée (RAII)

Énoncé

L'objectif de cet exercice est de comprendre précisément le cycle de vie des variables sur la Pile et sur le Tas en utilisant des blocs de portée (`{ ... }`).

Soit le programme Rust suivant :

```
fn main() {  
    let a = 10; // Variable A  
  
    {  
        let b = Box::new(20); // Variable B  
        let c = 30; // Variable C  
        println!("Dans le bloc : a = {}, b = {}, c = {}", a, b, c);  
    } // ← Point de contrôle X  
  
    // println!("Hors du bloc : b = {}", b); // Ligne commentée  
    let d = Box::new(40); // Variable D  
    println!("Fin du main : a = {}, d = {}", a, d);  
} // ← Point de contrôle Y
```

1. Analyse de la mémoire :

2. Au Point de contrôle X (juste avant l'accolade de fermeture du bloc interne), listez les variables présentes sur la Pile et celles présentes sur le Tas.
 3. Que se passe-t-il pour la mémoire au moment précis où l'on franchit le Point de contrôle X ?
 4. Cycle de vie :
 5. Si l'on décommente la ligne `println!("Hors du bloc : b = {}", b);`, que se passe-t-il à la compilation ? Pourquoi ?
 6. État final :
 7. Au Point de contrôle Y (juste avant la fin du `main`), quelles données sont encore en mémoire ?
-

Indices

1. *Indice 1* : Les variables déclarées avec `Box::new` possèdent un pointeur sur la Pile, mais la valeur réelle qu'elles enveloppent est stockée sur le Tas.
 2. *Indice 2* : Le concept de RAII stipule que lorsqu'une variable sort de sa portée (délimitée par `{}`), sa mémoire est immédiatement libérée. Cela s'applique à la fois à son espace sur la Pile et à la mémoire sur le Tas vers laquelle elle pointe.
-

Correction Détaillée

1. Analyse de la mémoire au Point de contrôle X

Juste avant la fermeture du bloc interne : * Sur la Pile (Stack) : * `a` : contient la valeur entière `10` (taille fixe). * `b` : contient l'adresse mémoire (pointeur) qui pointe vers le Tas (ex: `0x00AF12`). * `c` : contient la valeur entière `30` (taille fixe). * Sur le Tas (Heap) : * À l'adresse pointée par `b` (ex: `0x00AF12`), on trouve la valeur entière `20`.

Au franchissement du Point de contrôle X : Le bloc interne se termine. Les variables locales à ce bloc (`b` et `c`) sortent de la portée. * `c` est dépilée (libérée de la pile). * `b` (le pointeur) est dépilée. Puisque `b` possédait la propriété de la

donnée sur le tas via la `Box`, Rust libère immédiatement et automatiquement l'espace mémoire sur le Tas contenant la valeur `20`.

2. Décommenter la ligne interdite

Si on décommente la ligne `println!("Hors du bloc : b = {}", b);`, le compilateur Rust génère une erreur de compilation stricte :

```
error[E0425]: cannot find value `b` in this scope
```

Explication : `b` a été détruite au Point de contrôle X. Tenter d'y accéder en dehors de son bloc d'existence est impossible. En C/C++, cela aurait pu causer un bogue de type *Use-After-Free* (si le pointeur avait été conservé manuellement), mais Rust l'empêche dès la compilation.

3. État final au Point de contrôle Y

Juste avant la fin du `main` : * Sur la Pile (Stack) : * `a` (valeur `10`). * `d` (pointeur vers le Tas, ex: `0x00AF90`). * Sur le Tas (Heap) : * À l'adresse pointée par `d`, on trouve la valeur `40`.

Dès que la fonction `main` se termine (accolade finale), `a` et `d` sont retirées de la pile, et la mémoire du tas contenant `40` est libérée.

Exercice 2 : Résoudre le piège de la référence locale (Dangling Pointer)

Énoncé

Un développeur junior tente d'écrire une fonction qui configure un identifiant de session. Il souhaite créer cet identifiant dans une fonction dédiée et retourner une référence vers celui-ci pour éviter de copier la donnée.

Voici son code qui refuse obstinément de compiler :

```
// CE CODE NE COMPILE PAS
fn obtenir_identifiant_session() → &i32 {
    let id = 8192; // Alloué sur la pile de la fonction
    &id           // Retourne une référence vers 'id'
}

fn main() {
    let session_id = obtenir_identifiant_session();
    println!("ID de session : {}", session_id);
}
```

1. Expliquez précisément pourquoi le compilateur refuse ce code en vous basant sur le fonctionnement de la Pile (Stack). Quel bogue mémoire historique Rust est-il en train d'éviter ici ?
2. Proposez une première correction en modifiant la fonction pour qu'elle retourne directement la valeur (transfert de propriété sur la pile).
3. Proposez une seconde correction en allouant l'identifiant sur le Tas (Heap) à l'aide d'une `Box`, puis en retournant cette `Box`.

Indices

1. *Indice 1* : Lorsqu'une fonction se termine, son cadre de pile (*Stack Frame*) est entièrement détruit. Toute référence pointant vers cette zone devient un "pointeur suspendu" (*dangling pointer*).
 2. *Indice 2* : Pour retourner une `Box`, la signature de la fonction doit retourner le type `Box<i32>`.
-

Correction Détaillée

1. Explication théorique du problème

La variable `id` est une variable locale à la fonction `obtenir_identifiant_session`. Elle est donc allouée sur la Pile dans le cadre de pile de cette fonction.

Dès que la fonction se termine, son cadre de pile est détruit et la mémoire associée à `id` est marquée comme disponible pour de futures fonctions. Si Rust nous laissait retourner `&id` (une référence/un pointeur vers cette zone mémoire détruite), la variable `session_id` dans le `main` pointerait vers de la mémoire invalide.

C'est le bogue historique du **Pointeur Suspendu (Dangling Pointer)** ou **Use-After-Free**. Un attaquant pourrait exploiter cette adresse pour lire des données sensibles écrites par d'autres fonctions à cet emplacement de la pile.

2. Solution 1 : Retourner la valeur par copie/déplacement (Pile)

La méthode la plus simple pour les types de taille fixe (comme `i32`) est de retourner directement la valeur. La valeur est alors copiée dans le cadre de pile de la fonction appelante (`main`).

```
// Solution 1 : Retour par valeur
fn obtenir_identifiant_session() → i32 {
    let id = 8192;
    id // On retourne la valeur directement (copie sur la pile)
}

fn main() {
    let session_id = obtenir_identifiant_session();
    println!("ID de session (Solution 1) : {}", session_id);
}
```

3. Solution 2 : Allocation sur le Tas avec `Box`

Si la donnée était très volumineuse et que nous voulions éviter de la copier sur la pile, nous pourrions l'allouer sur le Tas. En retournant la `Box`, nous transférons la propriété du pointeur au `main`.

```
// Solution 2 : Allocation sur le Tas
fn obtenir_identifiant_session_tas() → Box<i32> {
    let id = Box::new(8192); // Alloué sur le Tas
    id // On retourne la Box (le pointeur est transféré)
}

fn main() {
    let session_id = obtenir_identifiant_session_tas();
    println!("ID de session (Solution 2) : {}", session_id);
} // La mémoire sur le Tas contenant 8192 est libérée proprement ici.
```

Exercice 3 : Structure Récursive et Nécessité du Tas

Énoncé

En Rust, toutes les structures de données doivent avoir une taille connue à la compilation. C'est pourquoi il est impossible de définir une structure qui se contient elle-même directement.

Soit le scénario suivant : nous voulons modéliser une liste chaînée ultra-simple de tâches à faire (une "To-Do List" où chaque tâche pointe vers la tâche suivante).

Voici notre première tentative de code :

```
// CE CODE NE COMPILE PAS
struct Tache {
    description: String,
    suivante: Option<Tache>, // Une tâche contient potentiellement la tâche suivante
}
```

Le compilateur rejette ce code avec l'erreur suivante :

```
error[E0072]: recursive type `Tache` has infinite size
```

1. Pourquoi le compilateur dit-il que cette structure a une "taille infinie" ?

2. Comment l'utilisation du Tas (Heap) via une `Box` permet-elle de résoudre ce problème de taille ?
 3. Corrigez la structure `Tache` et écrivez une fonction `main` qui crée une liste de deux tâches chaînées (ex: "Acheter le pain" qui pointe vers "Préparer le dîner").
-

Indices

1. *Indice 1* : Pour calculer la taille d'une structure sur la pile, le compilateur doit additionner la taille de tous ses champs. Si un champ `Tache` contient une autre `Tache`, qui elle-même contient une `Tache`, la taille devient théoriquement infinie.
 2. *Indice 2* : Un pointeur (`Box`) a toujours une taille fixe (8 octets sur une architecture 64 bits), peu importe la taille de la donnée vers laquelle il pointe sur le Tas.
 3. *Indice 3* : Le type du champ `suivante` doit devenir `Option<Box<Tache>>` .
-

Correction Détaillée

1. Pourquoi la taille est-elle "infinie" ?

Pour allouer une structure sur la Pile, Rust doit savoir exactement combien d'octets réserver dès la compilation. Si `Tache` contient directement une `Option<Tache>`, alors pour calculer la taille de `Tache`, il faut connaître la taille de `Option<Tache>`, qui requiert de connaître la taille de `Tache`, et ainsi de suite à l'infini. C'est une récursion infinie de types.

2. Comment la `Box` résout le problème

En encapsulant la tâche suivante dans une `Box` (`Box<Tache>`), on ne stocke plus la structure `Tache` directement à l'intérieur d'elle-même. À la place, on stocke un pointeur vers le Tas. Un pointeur a une taille fixe et connue (64 bits / 8 octets sur les processeurs modernes). Le compilateur peut désormais calculer précisément la taille de la structure sur la pile : `taille(String) + taille(Option de pointeur)` .

3. Code corrigé et implémentation

Voici le code complet et fonctionnel :

```
// Définition de la structure avec Box pour l'allocation sur le Tas
struct Tache {
    description: String,
    suivante: Option<Box<Tache>>, // Taille fixe grâce à la Box !
}

fn main() {
    // 1. On crée la deuxième tâche (la fin de la liste)
    let deuxieme_tache = Tache {
        description: String::from("Préparer le dîner"),
        suivante: None, // Pas de tâche suivante
    };

    // 2. On crée la première tâche qui pointe vers la deuxième
    // On doit placer 'deuxieme_tache' sur le tas avec Box::new
    let premiere_tache = Tache {
        description: String::from("Acheter le pain"),
        suivante: Some(Box::new(deuxieme_tache)),
    };

    // 3. Affichage pour vérifier la structure
    println!("Première tâche : {}", premiere_tache.description);

    // On inspecte la tâche suivante en "déballant" l'Option
    if let Some(ref tache_suivante) = premiere_tache.suivante {
        println!("Tâche suivante : {}", tache_suivante.description);
    }
} // Tout est nettoyé automatiquement ici :
// 'premiere_tache' sort de la portée, ce qui libère sa Box sur le tas,
// ce qui détruit 'deuxieme_tache' à son tour. Aucune fuite de mémoire
```

MODULE 2

Le Cœur de la Sécurité en Rust : L'Ownership (Propriété)

Les trois règles d'or de l'Ownership :

Qui possède quoi et pourquoi ?

1. Introduction Conceptuelle

Dans le développement logiciel, la gestion de la mémoire a historiquement été divisée en deux grandes écoles : 1. La gestion manuelle (C, C++) : Le développeur alloue et libère explicitement la mémoire (via `malloc` et `free` ou `new` et `delete`). Cette approche offre des performances maximales, mais expose le logiciel à des vulnérabilités critiques (doubles libérations, pointeurs pendants, fuites de mémoire) si une seule ligne de code est omise ou mal positionnée. 2. La gestion automatique par Garbage Collector (Java, Go, Python) : Un programme d'arrière-plan surveille l'exécution et libère la mémoire qui n'est plus utilisée. Bien que hautement sécurisée, cette méthode introduit un coût en performances (temps de processeur, pauses d'exécution appelées GC *pauses* et consommation mémoire accrue).

Rust introduit une troisième voie révolutionnaire : l'Ownership (la Propriété).

Le "Pourquoi" : La sécurité sans coût à l'exécution

L'Ownership est un système de règles vérifiées par le compilateur lors de la phase de compilation. Si ces règles ne sont pas respectées, le programme refuse de compiler.

Ce mécanisme garantit : * La sécurité mémoire absolue : Impossible d'accéder à de la mémoire libérée ou non initialisée. * L'absence de Garbage Collector : La mémoire est libérée dès qu'elle n'est plus nécessaire, de manière déterministe, sans aucun impact sur les performances d'exécution (*zero-cost abstraction*).

Analogie du monde réel

Imaginez un livre de bibliothèque unique et précieux. * Dans un système avec Garbage Collector, un agent de la bibliothèque suit en permanence chaque lecteur dans la ville pour vérifier s'il lit encore le livre. Dès que le lecteur s'endort ou pose le livre, l'agent le récupère.

C'est sûr, mais extrêmement lourd et intrusif. * Dans un système de gestion manuelle, le lecteur doit lui-même ramener le livre. S'il oublie, le livre est perdu à jamais (fuite mémoire). S'il le détruit par erreur mais qu'un autre lecteur tente de le lire à la même table, ce dernier lit du vide (pointeur pendante). * Le système d'Ownership de Rust impose qu'une seule personne à la fois détienne la clé de la boîte contenant le livre. Lorsque cette personne quitte la bibliothèque (sortie de portée), elle doit obligatoirement verrouiller la boîte et restituer la clé. Le compilateur est le bibliothécaire strict qui vérifie ce protocole avant même que quiconque ne puisse entrer.

2. Fondations Théoriques

Le système d'Ownership repose sur trois règles fondamentales, immuables et appliquées par le compilateur Rust.

Les Trois Règles d'Or

1. *Chaque valeur en Rust possède une variable appelée son propriétaire (owner).*
2. *Il ne peut y avoir qu'un seul propriétaire à la fois pour une valeur donnée.*
3. *Lorsque le propriétaire sort de la portée (scope), la valeur est automatiquement détruite.*

Règle 1 & 2 : Propriétaire unique et sémantique de déplacement (Move)

Pour comprendre pourquoi Rust impose un propriétaire unique, analysons le comportement de la mémoire lors d'une affectation.

Comme étudié dans le Module 1.3, les types dont la taille est connue à la compilation (comme les entiers `i32`) sont stockés sur la Pile (Stack). Les types dont la taille peut varier dynamiquement (comme `String`) stockent leurs données réelles sur le Tas (Heap), tandis qu'un pointeur, une longueur et une capacité sont stockés sur la Pile.

Le problème de la copie superficielle (Shallow Copy)

Considérons le code conceptuel suivant :

```
let s1 = String::from("hello");  
let s2 = s1;
```

Si Rust fonctionnait comme d'autres langages, copier `s1` dans `s2` copierait uniquement le pointeur situé sur la Pile. Nous aurions alors deux variables (`s1` et `s2`) pointant vers la même adresse mémoire sur le Tas :

```
      PILE (Stack)                TAS (Heap)  
s1 [ptr, len, cap] -----> [ 'h', 'e', 'l', 'l', 'o' ]  
                                ^  
s2 [ptr, len, cap] -----|
```

Le danger : Lorsque `s1` et `s2` sortent de leur portée, Rust tente de libérer la mémoire sur le Tas pour chacune d'elles. Cela provoquerait une **double libération de la mémoire** (*double free error*), une faille de sécurité majeure permettant la corruption de données ou des piratages.

La solution de Rust : Le Déplacement (Move)

Pour empêcher ce problème, la Règle 2 stipule qu'il ne peut y avoir qu'un seul propriétaire. Lors de l'affectation `let s2 = s1;`, Rust ne copie pas les données. Il **déplace** (*move*) la propriété de la mémoire de `s1` vers `s2`.

À partir de cet instant, `s1` est considérée comme **invalid** par le compilateur.

```
      PILE (Stack)                TAS (Heap)  
s1 [Invalide / Désactivé]  
  
s2 [ptr, len, cap] -----> [ 'h', 'e', 'l', 'l', 'o' ]
```

Si vous tentez d'utiliser `s1` après ce déplacement, le compilateur générera une erreur immédiate.

Règle 3 : La Portée (Scope) et la libération automatique (Drop)

La portée est la zone du code à l'intérieur de laquelle un élément est valide. En Rust, une portée est délimitée par des accolades `{}`.

```
{                                // s n'est pas valide ici, elle n'est pas encore déclarée
    let s = String::from("hello"); // s est valide à partir d'ici
    // On peut utiliser s
}                                // La portée se termine ici. s n'est plus valide.
```

Lorsque la variable `s` sort de la portée (à la fermeture de l'accolade `}`), Rust appelle automatiquement une fonction spéciale appelée `drop`.

La fonction `drop` est fournie par le protocole (trait) de destruction de Rust. Elle restitue immédiatement la mémoire du Tas occupée par `"hello"` au système d'exploitation. Ce mécanisme est analogue au paradigme RAI (Resource Acquisition Is Initialization) utilisé en C++.

3. Implémentation Pratique Pas-à-Pas

Scénario 1 : Observation du mécanisme de Move (Erreur de compilation)

Créons un programme pour observer comment le compilateur protège l'accès à la mémoire déplacée.

```
fn main() {
    // 1. Allocation d'une String sur le Tas. 's1' est le propriétaire.
    let s1 = String::from("Rust de haute sécurité");

    // 2. Transfert de propriété (Move) de 's1' vers 's2'.
    let s2 = s1;

    // 3. Tentative d'utilisation de la variable d'origine 's1'.
    // Le compilateur va bloquer cette ligne.
    println!("Contenu de s1 : {}", s1);
}
```

Analyse du message d'erreur du compilateur :

Si vous tentez de compiler ce code avec `cargo run`, le compilateur Rust (`rustc`) produit un rapport d'une précision exceptionnelle :

```
error[E0382]: borrow of moved value: `s1`
  → src/main.rs:9:36
   |
3 |     let s1 = String::from("Rust de haute sécurité");
   |         -- move occurs because `s1` has type `String`, which does not implement the `Copy` trait
4 |
5 |     let s2 = s1;
   |             -- value moved here
...
9 |     println!("Contenu de s1 : {}", s1);
   |                                   ^^ value borrowed here after move
```

Le compilateur nous explique clairement : 1. `s1` est de type `String`, qui n'implémente pas le comportement de copie automatique (`Copy`). 2. La valeur a été déplacée à la ligne 5 (`let s2 = s1;`). 3. Nous tentons d'utiliser la valeur déplacée à la ligne 9.

Scénario 2 : Données sur la Pile et sémantique de Copie (`Copy`)

Pourquoi le code suivant fonctionne-t-il sans erreur, alors qu'il semble identique au précédent ?

```
fn main() {
    let x = 42;
    let y = x;

    // Ceci est parfaitement valide !
    println!("x = {}, y = {}", x, y);
}
```

Explication :

Les types comme les entiers (`i32`, `u64`), les booléens (`bool`), et les nombres à virgule flottante (`f64`) ont une taille fixe connue à la compilation. Ils sont stockés intégralement sur la Pile.

Pour ces types, faire `let y = x;` réalise une copie bit à bit rapide et peu coûteuse de la valeur. Rust implémente pour eux un trait spécial nommé `Copy`. Contrairement aux types sur le Tas, la valeur d'origine n'est pas invalidée après l'affectation.

Scénario 3 : Ownership et Fonctions

Le passage d'arguments à une fonction suit exactement les mêmes règles d'Ownership que l'affectation à une variable. Passer une variable à une fonction va soit la **déplacer** (Move), soit la **copier** (Copy).

Voici un programme complet et fonctionnel illustrant ces flux de propriété :

```

fn main() {
    // 'ma_chaine' entre en portée
    let ma_chaine = String::from("Données Secrètes");

    // Le propriétaire 'ma_chaine' donne sa propriété à la fonction.
    // 'ma_chaine' devient invalide ici dans 'main'.
    prendre_propriete(ma_chaine);

    // println!("{}", ma_chaine); // ERREUR de compilation si décommenté !

    let mon_entier = 100;
    // 'mon_entier' est copié dans la fonction.
    // Nous pouvons continuer à l'utiliser après.
    faire_copie(mon_entier);
    println!("Dans main : l'entier {} est toujours disponible.", mon_entier);

    // Récupération de propriété depuis une fonction
    let chaine_recue = donner_propriete();
    println!("Chaine reçue de la fonction : {}", chaine_recue);
} // Ici, chaine_recue sort de la portée et est détruite.
   // mon_entier sort de la portée (rien de spécial ne se produit).

fn prendre_propriete(texte: String) {
    println!("La fonction a reçu : '{}'", texte);
} // Ici, 'texte' sort de la portée. Rust appelle `drop`.
   // La mémoire sur le Tas est libérée.

fn faire_copie(nombre: i32) {
    println!("La fonction a copié le nombre : {}", nombre);
} // Ici, 'nombre' sort de la portée. Rien ne se passe car il est sur la Pile.

fn donner_propriete() → String {
    let nouvelle_chaine = String::from("Propriété transférée");
    nouvelle_chaine // La propriété est retournée à l'appelant.
}

```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Tenter d'utiliser une variable après un appel de fonction

C'est l'erreur classique du débutant. On passe une structure de données à une fonction pour traitement, puis on tente de l'utiliser à nouveau.

```
fn traiter_configuration(config: String) {
    // Traitement...
}

fn main() {
    let config = String::from("Port: 8080");
    traiter_configuration(config);

    // Erreur ! L'ownership a été transféré à la fonction.
    // println!("Configuration traitée : {}", config);
}
```

Résolution sans les références (en utilisant `.clone()`) :

Si nous avons absolument besoin de conserver l'original, nous pouvons effectuer une copie profonde (Deep Copy) explicite de la mémoire sur le Tas en utilisant la méthode `.clone()`.

```
fn main() {
    let config = String::from("Port: 8080");
    // On passe un clone (duplication de la mémoire sur le Tas).
    traiter_configuration(config.clone());

    // Valide, car l'original 'config' n'a pas été déplacé !
    println!("Configuration d'origine : {}", config);
}
```

Attention : `.clone()` duplique les données sur le Tas. C'est une opération coûteuse en performances si la structure est volumineuse.

(Note : Nous verrons dans le Module 2.2 comment partager des données de manière optimale et sans copie grâce aux Références et à l'Emprunt).

Piège 2 : Le retour multiple fastidieux pour restituer l'Ownership

Si vous n'utilisez pas les clones, vous pourriez être tenté de renvoyer systématiquement la variable pour en redonner la propriété au contexte appelant :

```
fn calculer_longueur(s: String) → (String, usize) {
    let longueur = s.len();
    (s, longueur) // On renvoie la chaîne pour restituer l'ownership
}
```

Bien que fonctionnelle, cette syntaxe est lourde. Elle met en évidence le besoin fondamental du mécanisme d'Emprunt qui sera traité dans le prochain module.

5. Synthèse Pédagogique

Tableau Comparatif des Comportements Mémoire

Type de Donnée	Emplacement Mémoire	Comportement à l'affectation (<code>let a = b;</code>)	Coût CPU / RAM	Action lors de la sortie de portée
Types Primitifs (<code>i32</code> , <code>bool</code> , <code>f64</code>)	Pile (Stack)	Copy (Copie bit à bit automatique)	Négligeable	Libération immédiate de la pile
Types Dynamiques (<code>String</code> , <code>Vec<T></code>)	Tas (Heap) + Pointeurs sur la Pile	Move (Transfert de propriété, invalidation de la source)	Gratuit (seul le pointeur est copié)	Appel de <code>drop()</code> et libération de la mémoire sur le Tas
Types Clonnés (<code>s.clone()</code>)	Tas (Heap)	Copie profonde explicite	Élevé (allocation et copie physique)	Appel de <code>drop()</code> pour chaque instance indépendante

Les points clés à retenir :

- Rust élimine les bugs mémoire à la compilation grâce à un système de règles strictes de propriété.

- Un seul propriétaire à la fois : évite les conflits d'accès et les doubles libérations.
 - Le Move est le comportement par défaut pour tout type stocké sur le Tas.
 - La libération est déterministe : dès qu'une variable sort de sa portée `{ }`, sa mémoire est libérée via `drop`.
-

6. Exercice Pratique d'Application

Énoncé : Le Système de Journalisation de Sécurité

Vous devez concevoir un système de traitement de logs de sécurité pour un serveur d'université. Le programme doit charger un log, appliquer un filtre de sécurité, puis archiver le log.

Objectifs :

1. Écrire une fonction `generer_log() → String` qui crée et retourne un message de log (ex: `"ALERTE: Tentative d'intrusion détectée à 03:14"`).
2. Écrire une fonction `filtrer_sensible(log: String) → String` qui prend la propriété du log, remplace le mot `"intrusion"` par `"***REDACTED***"`, et retourne le log modifié.
3. Écrire une fonction `archiver_log(log: String)` qui consomme définitivement le log et l'affiche à l'écran avec la mention `[ARCHIVÉ]`.
4. Dans la fonction `main`, vous devez orchestrer ces appels. Vous devez également afficher le log original juste avant de le filtrer, et afficher le log filtré juste avant de l'archiver.

Contrainte temporaire : Vous n'avez pas encore étudié les références. Vous devez résoudre cet exercice uniquement en utilisant les concepts d'Ownership (Move, Clone, et retours de fonctions).

Indices pour la résolution

1. Pour remplacer une sous-chaine en Rust, vous pouvez utiliser la méthode `.replace("ancien", "nouveau")` sur un type `String`. Attention, cette méthode retourne une nouvelle `String`.
 2. Rappelez-vous que dès qu'une `String` est passée en paramètre à une fonction, elle est déplacée. Si vous souhaitez l'utiliser après l'appel, la fonction doit la retourner, ou vous devez lui passer un `.clone()`.
-

Correction Complète et Commentée

Voici la solution attendue et rigoureusement structurée :

```

// Fonction 1 : Génère un log et en donne la propriété à l'appelant
fn generer_log() → String {
    let log_original = String::from("ALERTE: Tentative d'intrusion détectée à 03:14");
    log_original // Transfert de propriété vers le main
}

// Fonction 2 : Consomme le log, le modifie et retourne la propriété du nouveau log
fn filtrer_sensible(log: String) → String {
    // .replace crée une nouvelle String sur le Tas
    let log_securise = log.replace("intrusion", "***REDACTED***");
    log_securise // Transfert de propriété du résultat vers l'appelant
}

// Fonction 3 : Consomme définitivement le log (fin de vie de la donnée)
fn archiver_log(log: String) {
    println!("[ARCHIVÉ] : {}", log);
} // Ici, 'log' sort de la portée. La mémoire est libérée via drop().

fn main() {
    // Étape A : Récupération du log initial
    let log_initial = generer_log();

    // Étape B : Nous voulons afficher le log initial ET le filtrer ensuite.
    // Si nous écrivons : let log_filtre = filtrer_sensible(log_initial);
    // Nous ne pourrions plus utiliser log_initial pour l'affichage !
    // Nous devons donc cloner log_initial pour le filtrage, ou l'afficher AVANT

    println!("--- ÉTAPE 1 : LOG BRUT REÇU ---");
    // On utilise log_initial.clone() pour garder l'original intact pour l'affichage
    // ou plus simplement, on l'affiche d'abord, puis on transfère sa propriété.
    println!("Log Original : {}", log_initial);

    // Étape C : Filtrage. On transfère la propriété de log_initial à la fonction
    // log_initial devient invalide après cette ligne.
    let log_filtre = filtrer_sensible(log_initial);

    // println!("{}", log_initial); // Compilerait en ERREUR si décommenté !

    // Étape D : Affichage du log filtré
    println!("\n--- ÉTAPE 2 : FILTRAGE DES DONNÉES SENSIBLES ---");
    println!("Log Filtré : {}", log_filtre);

    // Étape E : Archivage. On transfère la propriété de log_filtre.
    println!("\n--- ÉTAPE 3 : ARCHIVAGE FINAL ---");
    archiver_log(log_filtre);

    // log_filtre est maintenant détruit. Le programme se termine proprement
}

```

```
// sans aucune fuite mémoire et sans Garbage Collector.  
}
```

Analyse de la correction :

- Flux de propriété : Le log naît dans `generer_log`, voyage vers `main`, est transmis à `filtrer_sensible` qui le transforme et en produit un nouveau, qui retourne vers `main`, pour finir sa vie dans `archiver_log` où la mémoire est proprement libérée à la fermeture de l'accolade de la fonction.
- Sécurité garantie : À aucun moment il n'a été possible d'accéder à un log corrompu ou à de la mémoire libérée. Le compilateur a validé l'intégralité du cycle de vie des données.

Exercices d'Application : Les trois règles d'or de l'Ownership

Ce cahier d'exercices pratiques a pour objectif de consolider votre compréhension des mécanismes d'Ownership (propriété), de Move (déplacement), de Copy (copie) et de gestion de la portée (Scope) en Rust, sans utiliser les références (qui seront abordées au module suivant).

Exercice 1 : Le Diagnostic de Capteur (Niveau : Facile)

Énoncé

Dans une usine connectée, vous devez concevoir un module de diagnostic pour un capteur de température. Chaque capteur possède un **identifiant numérique** (type `i32`) et un **nom de modèle** (type `String`).

Le code suivant a été écrit par un stagiaire, mais il refuse obstinément de compiler. Votre tâche est d'analyser l'erreur, d'expliquer pourquoi elle se produit en vous basant sur les Règles d'Or de l'Ownership, puis de corriger le code de deux manières différentes.

```
fn main() {
    let id_capteur = 1042;
    let modele_capteur = String::from("TempSense-X1");

    // Envoi des données au système de log
    enregistrer_id(id_capteur);
    enregistrer_modele(modele_capteur);

    // Tentative de réutilisation des variables pour un second rapport
    println!("--- RAPPORT DE VÉRIFICATION ---");
    println!("ID Capteur : {}", id_capteur);
    println!("Modèle      : {}", modele_capteur);
}

fn enregistrer_id(id: i32) {
    println!("[LOG] ID enregistré : {}", id);
}

fn enregistrer_modele(modele: String) {
    println!("[LOG] Modèle enregistré : {}", modele);
}
```

Indices

1. Observez attentivement les types de `id_capteur` et `modele_capteur`. L'un d'eux est stocké sur la Pile et implémente le trait `Copy`, tandis que l'autre est stocké sur le Tas.
2. Que se passe-t-il pour la variable `modele_capteur` lorsqu'elle est passée en paramètre à la fonction `enregistrer_modele` ?

Correction Détaillée

1. Analyse de l'erreur

Le compilateur Rust refuse de compiler ce code et lève l'erreur suivante :
`error[E0382]: borrow of moved value: 'modele_capteur'` .

- Pourquoi `id_capteur` fonctionne ? `id_capteur` est un entier de type `i32` . Ce type a une taille fixe connue à la compilation et est stocké sur la Pile. Il implémente le trait `Copy` . Lors de l'appel `enregistrer_id(id_capteur)` , la valeur est copiée. L'original dans `main` reste donc parfaitement valide.

- Pourquoi `modele_capteur` échoue ? `modele_capteur` est une `String`, allouée sur le Tas. Elle n'implémente pas `Copy`. Lors de l'appel `enregistrer_modele(modele_capteur)`, la propriété de la chaîne est déplacée (Move) dans le paramètre `modele` de la fonction. À la fin de la fonction `enregistrer_modele`, la variable `modele` sort de la portée et la mémoire est libérée (`drop`). Tenter d'accéder à `modele_capteur` dans le `println!` du `main` revient à essayer d'accéder à de la mémoire déjà libérée, ce que Rust interdit formellement.

2. Solutions de correction

Solution A : Utilisation de `.clone()` (Duplication de la mémoire)

Cette solution est idéale si l'on souhaite que la fonction de log possède sa propre copie indépendante de la chaîne.

```
fn main() {
    let id_capteur = 1042;
    let modele_capteur = String::from("TempSense-X1");

    enregistrer_id(id_capteur);
    // On passe un clone. L'original 'modele_capteur' reste propriétaire sa chaîne.
    enregistrer_modele(modele_capteur.clone());

    println!("--- RAPPORT DE VÉRIFICATION ---");
    println!("ID Capteur : {}", id_capteur);
    println!("Modèle      : {}", modele_capteur); // Valide !
}

fn enregistrer_id(id: i32) {
    println!("[LOG] ID enregistré : {}", id);
}

fn enregistrer_modele(modele: String) {
    println!("[LOG] Modèle enregistré : {}", modele);
}
```

Solution B : Réorganisation du code (Sans coût de performance)

Si nous n'avons pas besoin de conserver la variable après le log, nous pouvons simplement déplacer l'affichage de vérification avant le transfert de propriété. Cela évite une allocation mémoire inutile sur le Tas via `.clone()`.

```
fn main() {
    let id_capteur = 1042;
    let modele_capteur = String::from("TempSense-X1");

    // On fait l'affichage de vérification d'abord, pendant que nous sommes propriétaire
    println!("--- RAPPORT DE VÉRIFICATION ---");
    println!("ID Capteur : {}", id_capteur);
    println!("Modèle      : {}", modele_capteur);

    // Ensuite, on transfère définitivement la propriété aux fonctions de log
    enregistrer_id(id_capteur);
    enregistrer_modele(modele_capteur);
} // Aucune libération de 'modele_capteur' n'a lieu ici dans main, car elle a déjà été

fn enregistrer_id(id: i32) {
    println!("[LOG] ID enregistré : {}", id);
}

fn enregistrer_modele(modele: String) {
    println!("[LOG] Modèle enregistré : {}", modele);
} // La mémoire de la String est libérée ici.
```

Exercice 2 : Le Pipeline de Traitement de Données (Niveau : Moyen)

Énoncé

Vous développez un système de nettoyage de données textuelles brutes issues d'un formulaire web. Le traitement doit suivre un pipeline strict à trois étapes : 1. **Saisie** : Récupération de la chaîne brute. 2. **Nettoyage** : Suppression des espaces superflus au début et à la fin (trim). 3. **Anonymisation** : Remplacement du mot "secret" par "[CONFIDENTIEL]".

Puisque vous ne pouvez pas utiliser de références, vous devez concevoir ce pipeline en transférant la propriété de la donnée d'une fonction à l'autre.

Consignes

1. Écrivez une fonction `nettoyer(donnee: String) → String` qui supprime les espaces. (Aide : la méthode `.trim()` sur un type `str` retourne une portion de texte. Vous pouvez la convertir en `String` avec `.to_string()`).
2. Écrivez une fonction `anonymiser(donnee: String) → String` qui remplace `"secret"` par `"[CONFIDENTIEL]"` à l'aide de la méthode `.replace()` .
3. Dans la fonction `main` , orchestrez le pipeline en enchaînant les appels de fonction de manière à ce que la variable finale contienne le résultat final, sans jamais dupliquer inutilement la mémoire avec `.clone()` .

```
fn main() {  
    let entree_brute = String::from("    Mon mot de passe secret    ");  
  
    // Votre code ici : Enchaîner les appels pour obtenir le résultat final  
  
    // Affichage attendu : "Résultat : Mon mot de passe [CONFIDENTIEL]"  
}
```

Indices

1. Pensez à la "Sémantique de déplacement avec retour". Une fonction peut prendre la propriété d'un paramètre, faire des calculs, et renvoyer la propriété d'une nouvelle valeur à l'appelant.
2. Vous pouvez réassigner une variable ou créer de nouvelles variables intermédiaires pour stocker la propriété retournée à chaque étape.

Correction Détaillée

Voici l'implémentation complète et optimisée du pipeline de traitement :

```

// Étape 1 : Nettoyage des espaces
fn nettoyer(donnee: String) → String {
    // .trim() retourne un &str, .to_string() alloue une nouvelle String sur le Tas
    let resultat = donnee.trim().to_string();
    resultat // Transfert de propriété du résultat vers l'appelant
} // La mémoire allouée à l'argument 'donnee' est libérée ici

// Étape 2 : Anonymisation des données sensibles
fn anonymiser(donnee: String) → String {
    // .replace crée et retourne une nouvelle String sur le Tas
    let resultat = donnee.replace("secret", "[CONFIDENTIEL]");
    resultat // Transfert de propriété du résultat vers l'appelant
} // La mémoire de l'argument 'donnee' est libérée ici

fn main() {
    let entree_brute = String::from("    Mon mot de passe secret    ");

    // Option A : Enchaînement par variables intermédiaires (Très lisible)
    let etape1 = nettoyer(entree_brute); // 'entree_brute' est déplacée et invalidée
    let etape2 = anonymiser(etape1);    // 'etape1' est déplacée et invalidée

    println!("Résultat Option A : '{}'", etape2);

    // Option B : Enchaînement direct (Plus compact, tout aussi performant)
    let entree_brute_2 = String::from("    Mon mot de passe secret    ");
    let resultat_direct = anonymiser(nettoyer(entree_brute_2));

    println!("Résultat Option B : '{}'", resultat_direct);
}

```

Exercice 3 : Le Cycle de Vie d'un Coffre-Fort (Niveau : Difficile)

Énoncé

Pour comprendre précisément la Règle d'Or n°3 (la destruction automatique via `drop` lors de la sortie de portée), vous allez concevoir un simulateur de coffre-fort numérique.

Nous voulons suivre à la trace le moment exact où les clés d'accès (représentées par des `String`) sont créées, déplacées, et détruites en mémoire.

Analysez le code ci-dessous, puis répondez aux questions suivantes avant de regarder la correction : 1. À la ligne 15, la variable `cle_temporaire` est-elle encore accessible ? Pourquoi ? 2. À la ligne 18, la variable `cle_principale` est-elle encore accessible ? 3. Listez précisément l'ordre de destruction (les appels à `drop`) de toutes les chaînes de caractères créées dans ce programme.

```
fn main() { // ← Ligne 1
    let cle_principale = String::from("CLE_A_992"); // ← Ligne 2

    { // ← Début du Scope Interne (Ligne 4)
        let cle_temporaire = String::from("CLE_TEMP_001"); // ← Ligne 5
        println!("Dans le scope interne : {}", cle_temporaire); // ← Ligne 6

        let cle_transferee = cle_principale; // ← Ligne 8
        println!("Clé transférée : {}", cle_transferee); // ← Ligne 9
    } // ← Fin du Scope Interne (Ligne 10)

    // println!("Clé temporaire : {}", cle_temporaire); // ← Ligne 12

    // println!("Clé principale : {}", cle_principale); // ← Ligne 14

    let cle_secours = String::from("CLE_SECOURS_888"); // ← Ligne 16
    println!("Fin du programme."); // ← Ligne 17
} // ← Ligne 18
```

Indices

1. Dessinez les accolades `{}` sur une feuille et tracez des lignes pour délimiter la "durée de vie" de chaque variable.
2. Attention au transfert de propriété à la ligne 8 : qui possède la valeur `"CLE_A_992"` après cette ligne ? Où cette nouvelle variable est-elle déclarée ?

Correction Détaillée

Réponses aux questions théoriques :

1. Accessibilité de `cle_temporaire` à la ligne 12 : Non. La variable `cle_temporaire` a été déclarée à l'intérieur du scope interne (délimité par les

accolades des lignes 4 et 10). Dès que l'exécution franchit la ligne 10, ce scope se termine. Rust appelle automatiquement `drop` sur `cle_temporaire`. Tenter de l'utiliser à la ligne 12 provoque une erreur de compilation.

2. Accessibilité de `cle_principale` à la ligne 14 : Non. Bien que `cle_principale` ait été déclarée dans le scope du `main`, sa propriété a été déplacée vers `cle_transferee` à la ligne 8. `cle_principale` est donc devenue invalide à partir de la ligne 8.
3. Ordre exact de destruction des données sur le Tas :
4. 1ère destruction (Ligne 10) : La valeur `"CLE_TEMP_001"` (portée par `cle_temporaire`) est détruite car sa variable sort de sa portée.
5. 2ème destruction (Ligne 10) : La valeur `"CLE_A_992"` (portée par `cle_transferee`) est détruite. En effet, `cle_transferee` a été déclarée dans le scope interne. Lorsque ce scope se termine à la ligne 10, la variable est détruite, entraînant avec elle la valeur dont elle avait acquis la propriété à la ligne 8.
6. 3ème destruction (Ligne 18) : La valeur `"CLE_SECOURS_888"` (portée par `cle_secours`) est détruite à la fin de la fonction `main`.

Code de démonstration complet avec commentaires de traçabilité :

Voici le code que vous pouvez exécuter. Les commentaires indiquent précisément ce que fait le compilateur en arrière-plan :

```

fn main() {
    // 1. Allocation de "CLE_A_992" sur le Tas. Propriétaire : cle_principale.
    let cle_principale = String::from("CLE_A_992");

    {
        // 2. Allocation de "CLE_TEMP_001" sur le Tas. Propriétaire : cle_temporaire.
        let cle_temporaire = String::from("CLE_TEMP_001");
        println!("Dans le scope interne : {}", cle_temporaire);

        // 3. Transfert de propriété (Move).
        // Le propriétaire de "CLE_A_992" est désormais cle_transferee.
        // cle_principale est désactivée.
        let cle_transferee = cle_principale;
        println!("Clé transférée : {}", cle_transferee);

    } // ← FIN DU SCOPE INTERNE.
    // Rust appelle drop(cle_transferee) → Libération de "CLE_A_992" sur le Tas.
    // Rust appelle drop(cle_temporaire) → Libération de "CLE_TEMP_001" sur le Tas.

    // 4. Allocation de "CLE_SECOURS_888" sur le Tas. Propriétaire : cle_secours.
    let cle_secours = String::from("CLE_SECOURS_888");

    println!("Fin du programme.");
} // ← FIN DU MAIN.
// Rust appelle drop(cle_secours) → Libération de "CLE_SECOURS_888" sur le Tas.

```

L'Emprunt (Borrowing) et les Références : Partager des données sans les copier

1. Introduction Conceptuelle

Dans le module précédent (Module 2.1), nous avons découvert les trois règles d'or de l'Ownership (la propriété). Nous avons vu que lorsqu'une variable est passée à une fonction, sa propriété est transférée (on parle de Move). Si la fonction appelante souhaite réutiliser cette donnée, la fonction appelée doit lui restituer la propriété en la renvoyant explicitement (par exemple, via un tuple).

Cette approche, bien que d'une sécurité absolue pour la mémoire, s'avère extrêmement lourde et fastidieuse à l'usage. Imaginez devoir écrire ce type de code à chaque fois que vous souhaitez simplement lire une information :

```
fn main() {
    let s1 = String::from("Bonjour");

    // Nous devons récupérer s1 car la fonction s'en approprie
    let (s2, longueur) = calculer_longueur(s1);

    println!("La longueur de '{}' est de {}.", s2, longueur);
}

fn calculer_longueur(s: String) → (String, usize) {
    let longueur = s.len();
    (s, longueur) // Restitution manuelle de la propriété
}
```

Ce mode de transfert permanent est l'équivalent de donner définitivement votre livre à un ami qui souhaite simplement y lire une phrase, pour qu'il vous le redonne ensuite.

C'est ici qu'intervient le concept d'Emprunt (*Borrowing*), matérialisé par les Références. Au lieu de transférer la propriété d'une ressource, Rust nous permet de prêter un accès à cette ressource. L'emprunteur peut lire ou modifier la donnée, mais la propriété reste fermement entre les mains de la variable d'origine. Lorsque l'emprunt se termine, la ressource n'est pas détruite, car elle appartient toujours à son propriétaire initial.

2. Fondations Théoriques

Qu'est-ce qu'une Référence ?

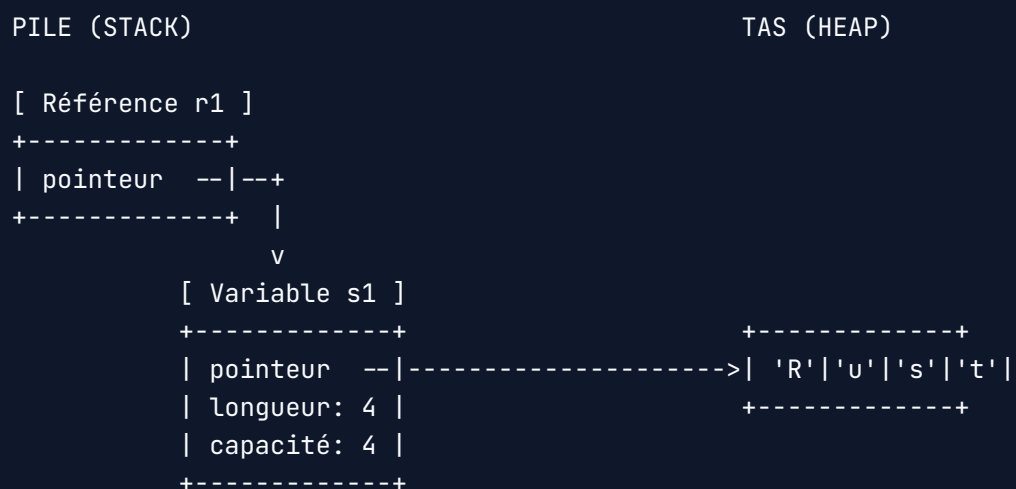
Une référence est un pointeur garanti comme valide, qui pointe vers l'adresse mémoire d'une autre variable contenant la donnée réelle. En Rust, on crée une référence en utilisant l'opérateur esperluette (`&`).

Représentation en mémoire

Considérons une variable `s1` de type `String` stockée sur la Pile (Stack) et pointant vers le Tas (Heap), et une référence `r1` pointant vers `s1` :

```
let s1 = String::from("Rust");
let r1 = &s1;
```

Voici comment ces éléments sont organisés en mémoire :



La référence `r1` ne possède pas la donnée. Elle pointe simplement vers la structure de contrôle de `s1` sur la pile. Comme `r1` n'est pas propriétaire, la donnée sur le tas ne sera pas libérée lorsque `r1` sortira de son scope.

Les deux types de Références

Rust propose deux manières d'emprunter une donnée, chacune répondant à des besoins précis :

1. La Référence Immuable (`&T`) : Permet de lire la donnée, mais pas de la modifier. Vous pouvez créer autant de références immuables que vous le souhaitez simultanément.
2. La Référence Mutable (`&mut T`) : Permet de lire et de modifier la donnée. L'accès est exclusif.

La Règle d'Or du Borrow Checker : L'Aliasing XOR la Mutabilité

Pour garantir l'absence totale de "Data Races" (situations de compétition sur les données) et de comportements indéfinis, le compilateur Rust (via son outil, le *Borrow Checker*) applique une règle stricte et mathématique :

*À tout moment, pour une ressource donnée, vous pouvez avoir : * Soit un nombre illimité de références immuables (`&T`). * Soit exactement une unique référence mutable (`&mut T`).*

Mais jamais les deux en même temps dans le même scope actif.

Cette règle peut se résumer par la formule : **Aliasing XOR Mutabilité** (Partage OU Modification, mais jamais les deux).

Pourquoi cette règle est-elle vitale ?

Si vous pouviez avoir une référence immuable (qui s'attend à ce que la donnée ne change pas) et une référence mutable (qui modifie la donnée) en même temps, la référence immuable pourrait soudainement pointer vers des données corrompues ou modifiées de manière inattendue. En interdisant cette coexistence, Rust élimine par construction toute une classe de bugs de concurrence et de corruption mémoire.

3. Implémentation Pratique Pas-à-Pas

Étape 1 : L'Emprunt Immuable (Lecture seule)

Voyons comment réécrire notre premier exemple de manière élégante grâce aux références immuables.

```
fn main() {
    let s1 = String::from("Bonjour");

    // Nous passons une référence (&s1) au lieu de transférer la propriété
    let longueur = calculer_longueur(&s1);

    // s1 est toujours valide ici ! Nous pouvons encore l'utiliser.
    println!("La longueur de '{}' est de {}.", s1, longueur);
}

// La fonction accepte un paramètre de type &String (référence vers un String)
fn calculer_longueur(s: &String) → usize {
    s.len() // Nous pouvons lire la donnée via la référence
} // Ici, s sort du scope, mais comme il n'a pas la propriété, rien n'est libéré
```

Étape 2 : L'Emprunt Mutable (Modification)

Si nous voulons qu'une fonction modifie une donnée empruntée, nous devons utiliser une référence mutable `&mut`. Pour cela, la variable d'origine doit elle-même être déclarée comme mutable avec le mot-clé `mut`.

```
fn main() {
    // 1. La variable d'origine doit être mutable
    let mut s = String::from("Bonjour");

    // 2. Nous passons une référence mutable (&mut s)
    ajouter_un_nom(&mut s);

    println!("Résultat : {}", s); // Affiche "Bonjour tout le monde"
}

// 3. La fonction accepte une référence mutable (&mut String)
fn ajouter_un_nom(texte: &mut String) {
    texte.push_str(" tout le monde"); // Modification de la donnée d'origine
}
```

Étape 3 : Visualiser le refus de compilation (Le Borrow Checker en action)

Tentons d'enfreindre la règle d'or en mélangeant emprunts immuables et mutables.

```
fn main() {
    let mut score = 10;

    let ref_immuable1 = &score; // Premier emprunt immuable (OK)
    let ref_immuable2 = &score; // Deuxième emprunt immuable (OK)

    // Tentative d'emprunt mutable alors que des emprunts immuables sont actifs
    let ref_mutable = &mut score;

    // Utilisation des références
    println!("Scores : {} et {}", ref_immuable1, ref_immuable2);
    ref_mutable += 5;
}
```

Si vous tentez de compiler ce code, le compilateur Rust refusera catégoriquement et affichera un message d'erreur extrêmement détaillé :

```

error[E0502]: cannot borrow `score` as mutable because it is also borrowed as immutable
  → src/main.rs:8:23
   |
5 |     let ref_immuable1 = &score; // Premier emprunt immuable
   |                           ----- immutable borrow occurs here
...
8 |     let ref_mutable = &mut score;
   |                       ^^^^^^^^^^ mutable borrow occurs here
9 |
10|     println!("Scores : {} et {}", ref_immuable1, ref_immuable2);
   |                                   ----- immutable borrow later used here

```

Le compilateur nous explique clairement que nous ne pouvons pas créer `ref_mutable` parce que `ref_immuable1` est encore utilisé plus bas dans le `println!`.

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Le scope d'une référence (NLL - Non-Lexical Lifetimes)

Depuis les versions modernes de Rust, le cycle de vie d'une référence ne s'arrête pas forcément à la fin du bloc de code (accolade fermante `}`), mais à sa dernière utilisation effective. C'est ce qu'on appelle les *Non-Lexical Lifetimes* (NLL).

Le code suivant est parfaitement valide :

```

fn main() {
    let mut s = String::from("Hello");

    let r1 = &s; // Emprunt immuable commence
    println!("{}", r1); // Dernière utilisation de r1. L'emprunt immuable s'arrête ICI.

    let r2 = &mut s; // OK : Aucun autre emprunt n'est actif à ce moment-là.
    r2.push_str(" World");
    println!("{}", r2);
}

```


Piège 2 : Les Références Pendantes (Dangling References)

Une référence pendante est un pointeur qui pointe vers une adresse mémoire dont le contenu a été libéré. En C ou C++, c'est une source majeure de failles de sécurité. En Rust, le compilateur garantit qu'il est impossible de créer une référence pendante.

Tentons d'en créer une :

```
fn main() {
    let reference_vers_rien = creer_reference_pendante();
}

fn creer_reference_pendante() → &String { // Retourne une référence vers un String
    let s = String::from("Fantôme"); // s est créé localement dans la fonction

    &s // Nous retournons une référence vers s
} // Ici, s sort du scope et est détruit (mémoire libérée).
    // La référence retournée pointerait vers de la mémoire invalide !
```

Le compilateur bloque immédiatement ce code avec l'erreur suivante :

```
error[E0106]: missing lifetime specifier
  → src/main.rs:5:33
   |
5 | fn creer_reference_pendante() → &String {
   |                                ^ expected named lifetime parameter
```

Note : Le concept de "durée de vie" (Lifetime) soulevé par cette erreur sera étudié en détail dans le Module 5.1. Pour l'instant, reprenez simplement que Rust vous empêche physiquement de faire pointer une référence vers une donnée qui va disparaître.

Pour corriger ce problème, il faut retourner la `String` directement (transfert de propriété) plutôt qu'une référence :

```
fn solution_correcte() → String {
    let s = String::from("Valide");
    s // Propriété transférée à l'appelant, pas de destruction.
}
```

5. Synthèse Pédagogique

Tableau Comparatif des Accès Mémoire

Concept	Syntaxe	Propriétaire ?	Modifiable ?	Nombre d'accès simultanés
Ownership (Move)	<code>T</code>	Oui	Oui (si <code>mut</code>)	1 seul propriétaire
Emprunt Immuable	<code>&T</code>	Non	Non	Illimité
Emprunt Mutable	<code>&mut T</code>	Non	Oui	Strictement 1 seul

Les Règles d'or à retenir

1. Pas de pointeurs nuls : Une référence en Rust pointe toujours vers une valeur valide.
2. Aliasing XOR Mutabilité : Soit plusieurs lecteurs, soit un seul rédacteur. Jamais les deux en même temps.
3. Le propriétaire décide : Une référence ne peut pas vivre plus longtemps que le propriétaire de la donnée vers laquelle elle pointe.

6. Exercice Pratique d'Application

Énoncé : Le Gestionnaire de File d'Attente Sécurisé

Vous devez concevoir un système de gestion de file d'attente pour un guichet administratif. Les clients sont représentés par des chaînes de caractères (`String`).

Vous devez implémenter trois fonctions en respectant scrupuleusement les règles d'emprunt de Rust :

1. `afficher_file` : Reçoit la file d'attente et affiche la liste des clients dans l'ordre sans détruire ni modifier la file.
2. `ajouter_client` : Reçoit la file d'attente et le nom d'un nouveau client, puis l'ajoute à la fin de la file.
3. `traiter_client` : Reçoit la file d'attente, retire le premier client de la file (s'il y en a un) et affiche son nom pour indiquer qu'il est pris en charge.

Indices

- Pour représenter la file d'attente, utilisez un vecteur de chaînes de caractères : `Vec<String>` .
 - Pour ajouter un élément à un vecteur, utilisez la méthode `.push(valeur)` .
 - Pour retirer le premier élément d'un vecteur, vous pouvez utiliser la méthode `.remove(0)` . Attention, cette opération modifie le vecteur !
 - Réfléchissez bien au type de référence (`&` ou `&mut`) requis pour chaque fonction selon qu'elle lit ou modifie le vecteur.
-

Correction Complète et Détaillée

Voici le code complet et fonctionnel qui résout l'exercice :

```

// Fonction 1 : Lecture seule. Nous utilisons un emprunt immuable (&Vec<String>)
// car nous voulons juste lire les données sans les modifier ni en prendre la propriété.
fn afficher_file(file: &Vec<String>) {
    println!("\n--- État actuel de la file d'attente ---");
    if file.is_empty() {
        println!("La file est vide.");
    } else {
        for (index, client) in file.iter().enumerate() {
            println!("{}", index + 1, client);
        }
    }
    println!("-----");
}

// Fonction 2 : Modification. Nous utilisons un emprunt mutable (&mut Vec<String>)
// car nous allons ajouter un élément au vecteur.
// Le paramètre `nom` est transféré (String) car le vecteur va s'en approprier.
fn ajouter_client(file: &mut Vec<String>, nom: String) {
    println!("→ Ajout de {} à la file d'attente.", nom);
    file.push(nom);
}

// Fonction 3 : Modification. Nous utilisons un emprunt mutable (&mut Vec<String>)
// car nous allons retirer un élément du vecteur.
fn traiter_client(file: &mut Vec<String>) {
    if file.is_empty() {
        println!("Impossible de traiter un client : la file est vide !");
    } else {
        // .remove(0) extrait le premier élément et décale les autres.
        // Cela nécessite un accès exclusif en écriture.
        let client_traite = file.remove(0);
        println!("[Guichet] Appel du client : {} !", client_traite);
    }
}

fn main() {
    // Le vecteur d'origine doit être mutable pour permettre les emprunts mutables
    let mut file_attente: Vec<String> = Vec::new();

    // 1. Ajout de clients
    // Nous passons une référence mutable à la file, et créons des Strings propriétaires
    ajouter_client(&mut file_attente, String::from("Alice"));
    ajouter_client(&mut file_attente, String::from("Bob"));
    ajouter_client(&mut file_attente, String::from("Charlie"));

    // 2. Affichage de la file (Emprunt immuable)
    afficher_file(&file_attente);
}

```

```

// 3. Traitement du premier client (Emprunt mutable)
traiter_client(&mut file_attente);

// 4. Nouvel affichage pour vérifier le changement
afficher_file(&file_attente);

// 5. Traitement des clients restants
traiter_client(&mut file_attente);
traiter_client(&mut file_attente);

// 6. Tentative sur une file vide
traiter_client(&mut file_attente);

// Affichage final
afficher_file(&file_attente);
}

```

Analyse pédagogique de la correction :

- Dans `main`, `file_attente` est déclarée avec `let mut` car elle va subir des modifications.
- Lors de l'appel à `ajouter_client(&mut file_attente, ...)`, le compilateur s'assure qu'aucune autre partie du code ne lit ou n'écrit dans `file_attente` à cet instant précis.
- Dans `afficher_file(&file_attente)`, nous passons une référence simple. Nous pourrions tout à fait appeler cette fonction plusieurs fois en parallèle si nous étions dans un contexte multi-threadé (ce qui sera étudié dans le Module 5.3), car la lecture partagée est totalement sûre.

Exercices d'Application : L'Emprunt (Borrowing) et les Références

Ce cahier d'exercices pratiques a pour but de consolider votre compréhension des concepts d'emprunt immuable (`&T`), d'emprunt mutable (`&mut T`) et des règles de validation du *Borrow Checker* de Rust. Les exercices sont classés par ordre de difficulté progressive.

Exercice 1 : Le Correcteur de Texte (Facile)

Énoncé

Vous devez concevoir un outil d'analyse et de correction de texte ultra-léger. Pour éviter de copier inutilement des chaînes de caractères en mémoire, vous devez utiliser exclusivement des références.

Implémentez deux fonctions : 1. `compter_mots` : Reçoit une référence immuable vers une `String` et retourne le nombre de mots qu'elle contient (on considère que les mots sont séparés par des espaces). 2. `rendre_poli` : Reçoit une référence mutable vers une `String`. Si la chaîne ne se termine pas par un point (`.`), la fonction doit ajouter ce point à la fin de la chaîne.

Dans votre fonction `main`, vous devrez initialiser une phrase, afficher son nombre de mots, la modifier avec `rendre_poli`, puis afficher le résultat final.

Indices

- Pour compter les mots, vous pouvez utiliser la méthode `.split_whitespace()` sur une chaîne, puis appeler `.count()` sur l'itérateur obtenu.
- Pour vérifier la fin d'une chaîne, utilisez la méthode `.ends_with('.')`.

- Pour ajouter un caractère à une `String`, utilisez la méthode `.push('.')`.

Correction Détaillée

```
// 1. Emprunt immuable (&String) car nous lisons uniquement la donnée
fn compter_mots(texte: &String) → usize {
    // split_whitespace() crée un itérateur sur les sous-parties du texte
    texte.split_whitespace().count()
}

// 2. Emprunt mutable (&mut String) car nous modifions la donnée d'origine
fn rendre_poli(texte: &mut String) {
    if !texte.ends_with('.') {
        texte.push('.'); // Modification directe en mémoire
    }
}

fn main() {
    // La variable d'origine doit être déclarée 'mut' pour pouvoir être empruntée mutable
    let mut phrase = String::from("Apprendre le Rust est passionnant");

    // Premier emprunt : Immuable.
    // Nous passons une référence pour que 'compter_mots' ne prenne pas la propriété de
    let nb_mots = compter_mots(&phrase);
    println!("La phrase contient {} mots.", nb_mots);

    // Deuxième emprunt : Mutable.
    // Nous passons une référence exclusive pour permettre la modification
    rendre_poli(&mut phrase);

    // Après l'exécution des fonctions, 'phrase' est toujours valide et a été modifiée.
    println!("Phrase corrigée : \"{}\\"", phrase);
}
```

Explications pédagogiques : * `compter_mots(&phrase)` : Nous prêtons la phrase en lecture seule. Une fois la fonction terminée, l'emprunt prend fin. * `rendre_poli(&mut phrase)` : Nous prêtons la phrase en écriture. Le compilateur s'assure qu'aucune autre référence (immuable ou mutable) n'est active à ce moment précis, évitant tout conflit d'accès.

Exercice 2 : Le Détecteur de Conflits (Moyen)

Énoncé

Un développeur junior a écrit le code suivant pour gérer l'état d'un livre dans une bibliothèque numérique. Cependant, le compilateur Rust refuse catégoriquement de compiler ce programme.

```
struct Livre {
    titre: String,
    disponible: bool,
}

fn main() {
    let mut livre = Livre {
        titre: String::from("Le Petit Prince"),
        disponible: true,
    };

    // Le développeur veut garder une référence vers le titre
    let ref_titre = &livre.titre

    // Il veut également marquer le livre comme emprunté (non disponible)
    let ref_livre_mut = &mut livre;
    ref_livre_mut.disponible = false;

    // Il affiche enfin le titre en utilisant la référence créée plus haut
    println!("Le livre '{}' a été emprunté.", ref_titre);
}
```

1. Identifiez et expliquez précisément pourquoi le compilateur refuse ce code (en faisant référence à la règle d'or du *Borrow Checker*).
2. Proposez une solution pour corriger ce code sans cloner le titre ni supprimer l'affichage final, en tirant parti du concept de *Non-Lexical Lifetimes* (NLL) ou en réorganisant les instructions.

Indices

- Relisez la règle d'or : *Aliasing XOR Mutabilité*.

- Regardez à quel endroit précis `ref_titre` (emprunt immuable) est utilisé pour la dernière fois, et où `ref_livre_mut` (emprunt mutable) commence à être utilisé.
-

Correction Détaillée

1. Analyse de l'erreur

Le code ne compile pas car il enfreint la règle d'or du partage des données : on ne peut pas avoir un emprunt mutable en même temps qu'un emprunt immuable sur la même ressource.

- À la ligne `let ref_titre = &livre.titre;`, un emprunt immuable est créé sur une partie de `livre`.
- À la ligne `let ref_livre_mut = &mut livre;`, un emprunt mutable est créé sur l'intégralité de `livre`.
- À la ligne `println!(..., ref_titre);`, l'emprunt immuable `ref_titre` est utilisé.

Puisque l'emprunt mutable `ref_livre_mut` intervient *pendant* que l'emprunt immuable `ref_titre` est encore actif (car utilisé plus bas), le compilateur bloque le programme pour éviter que la structure ne soit modifiée pendant qu'on lit l'un de ses champs.

2. Code corrigé (Solution)

Pour résoudre ce problème, il suffit de déplacer l'affichage (qui utilise l'emprunt immuable) avant la création de l'emprunt mutable. Grâce aux *Non-Lexical Lifetimes* (NLL), le compilateur comprendra que l'emprunt immuable se termine dès que la variable n'est plus lue.

```

struct Livre {
    titre: String,
    disponible: bool,
}

fn main() {
    let mut livre = Livre {
        titre: String::from("Le Petit Prince"),
        disponible: true,
    };

    // 1. Emprunt immuable créé
    let ref_titre = &livre.titre

    // 2. Utilisation de l'emprunt immuable.
    // C'est sa DERNIÈRE utilisation active. L'emprunt immuable s'arrête
    println!("Le livre '{}' va être emprunté.", ref_titre);

    // 3. Emprunt mutable créé.
    // C'est parfaitement valide car plus aucun autre emprunt n'est actif sur 'livre'.
    let ref_livre_mut = &mut livre;
    ref_livre_mut.disponible = false;

    println!("Statut de disponibilité mis à jour.");
}

```

Exercice 3 : L'Inventaire de Magasin (Difficile)

Énoncé

Vous développez un système de gestion de stock pour un magasin de jeux de société. Chaque produit est représenté par la structure suivante :

```

struct Produit {
    nom: String,
    prix: f64,
    quantite: u32,
}

```

Vous devez écrire un programme performant qui évite toute copie de structure. Implémentez les trois fonctions suivantes :

1. `calculer_valeur_stock` : Reçoit une référence immuable vers un vecteur de produits (`&Vec<Produit>`) et retourne la valeur totale du stock (somme de `prix * quantite` pour chaque produit).
2. `appliquer_remise` : Reçoit une référence mutable vers un vecteur de produits (`&mut Vec<Produit>`) et un pourcentage de remise (ex: `10.0` pour 10%). Elle doit réduire le prix de chaque produit du stock de ce pourcentage.
3. `trouver_produit_le_plus_cher` : Reçoit une référence immuable vers un vecteur de produits (`&Vec<Produit>`) et retourne une option contenant une référence immuable vers le produit le plus cher (`Option<&Produit>`). Si le vecteur est vide, retournez `None` .

Indices

- Pour parcourir un vecteur par référence immuable, utilisez `for produit in inventaire` .
 - Pour parcourir un vecteur par référence mutable afin de modifier ses éléments, utilisez `for produit in inventaire.iter_mut()` .
 - Pour trouver le produit le plus cher, vous pouvez déclarer une variable temporaire `let mut plus_cher: Option<&Produit> = None;` et comparer les prix lors du parcours du vecteur.
-

Correction Détaillée

```

#[derive(Debug)]
struct Produit {
    nom: String,
    prix: f64,
    quantite: u32,
}

// 1. Calcul de la valeur totale (Lecture seule → &Vec<Produit>)
fn calculer_valeur_stock(inventaire: &Vec<Produit>) → f64 {
    let mut total = 0.0;
    for produit in inventaire {
        total += produit.prix * (produit.quantite as f64);
    }
    total
}

// 2. Application d'une remise (Modification → &mut Vec<Produit>)
fn appliquer_remise(inventaire: &mut Vec<Produit>, pourcentage: f64) {
    // iter_mut() permet d'obtenir des références mutables sur chaque élément du vecteur
    for produit in inventaire.iter_mut() {
        produit.prix -= produit.prix * (pourcentage / 100.0);
    }
}

// 3. Recherche du produit le plus cher (Retourne une référence liée à la durée de vie du produit)
fn trouver_produit_le_plus_cher(inventaire: &Vec<Produit>) → Option<&Produit> {
    if inventaire.is_empty() {
        return None;
    }

    let mut produit_max = &inventaire[0]; // On commence par le premier élément

    for produit in inventaire {
        if produit.prix > produit_max.prix {
            produit_max = produit; // On met à jour la référence vers le produit le plus cher
        }
    }

    Some(produit_max)
}

fn main() {
    let mut stock = vec![
        Produit { nom: String::from("Dixit"), prix: 30.0, quantite: 5 },
        Produit { nom: String::from("Catan"), prix: 40.0, quantite: 3 },
        Produit { nom: String::from("7 Wonders"), prix: 45.0, quantite: 2 },
    ];
}

```

```

// Étape 1 : Calcul et affichage de la valeur initiale du stock
let valeur_initiale = calculer_valeur_stock(&stock);
println!("Valeur totale initiale du stock : {:.2} €", valeur_initiale);

// Étape 2 : Recherche du produit le plus cher
if let Some(cher) = trouver_produit_le_plus_cher(&stock) {
    println!("Le produit le plus cher est : {} ({:.2} €)", cher.nom, cher.prix);
}

// Étape 3 : Application d'une remise de 10%
println!("\n--- Application d'une remise de 10% ---");
appliquer_remise(&mut stock, 10.0);

// Étape 4 : Recalcul après remise
let valeur_apres_remise = calculer_valeur_stock(&stock);
println!("Nouvelle valeur totale du stock : {:.2} €", valeur_apres_remise);

if let Some(cher) = trouver_produit_le_plus_cher(&stock) {
    println!("Le produit le plus cher après remise : {} ({:.2} €)", cher.nom, cher.prix);
}
}

```

Pourquoi ce code est-il robuste et performant ?

1. **Zéro allocation superflue** : À aucun moment nous ne dupliquons les structures `Produit` ou les chaînes de caractères `nom`. Tout se fait par manipulation d'adresses mémoires (pointeurs sûrs).
2. **Signature de fonction propre** : La fonction `trouver_produit_le_plus_cher` renvoie un `Option<&Produit>`. Cette référence est garantie par Rust comme pointant vers un produit existant à l'intérieur du vecteur. Si le vecteur venait à être détruit, le compilateur interdirait l'accès à cette référence.

Les Tranches (Slices) : Garantir des accès mémoire sécurisés et sans débordement

1. Introduction Conceptuelle

Le problème de la référence partielle

Dans les modules précédents, nous avons appris à manipuler des variables entières et à prêter des données grâce aux références (`&T`). Cependant, un problème classique de la programmation système subsiste : comment faire référence à une sous-partie contiguë d'une collection sans la copier et sans risquer de désynchronisation mémoire ?

Imaginons que nous ayons une chaîne de caractères ou un tableau d'entiers. Si nous voulons isoler une portion de cette structure (par exemple, le premier mot d'une phrase), la méthode traditionnelle dans des langages comme le C consiste à stocker deux variables distinctes : un pointeur vers l'élément de départ et un entier représentant la taille de la sous-partie.

Cette approche souffre d'un défaut de sécurité majeur : l'absence de garantie de cohérence. Si la collection d'origine est modifiée, vidée ou libérée en mémoire, notre pointeur et notre taille deviennent obsolètes. Nous nous retrouvons alors avec un pointeur pendan (*dangling pointer*) ou un risque majeur de débordement de tampon (*buffer overflow*) si nous tentons d'accéder à cette mémoire devenue invalide.

La solution de Rust : Les Tranches (Slices)

Rust résout ce problème par le concept de Tranche (*Slice*). Une tranche est une vue sur une séquence contiguë d'éléments dans une collection. Contrairement à une collection entière, une tranche ne possède pas ses données (elle n'a pas l'*ownership*). Elle se contente de prêter une fenêtre d'accès sur des données existantes.

Grâce au *Borrow Checker* (étudié au cours du module précédent), Rust garantit qu'une tranche ne peut jamais survivre à la collection qu'elle référence, éliminant ainsi par construction toute possibilité de référence pendante ou d'accès hors limites.

2. Fondations Théoriques

Qu'est-ce qu'un "Pointeur Gras" (Fat Pointer) ?

Sur le plan de la représentation mémoire, une référence classique vers un type de taille connue (comme `&i32`) est un pointeur simple : une adresse mémoire de 64 bits (sur les architectures modernes).

Une référence sur une tranche (notée `&[T]` pour des éléments de type `T`, ou `&str` pour des chaînes de caractères) est ce que l'on appelle un **pointeur gras** (*fat pointer*). Elle occupe le double de l'espace d'un pointeur standard car elle contient deux informations cruciales :

1. Un pointeur vers le premier élément de la tranche.
2. La longueur de la tranche (le nombre d'éléments qu'elle contient).

Schéma de la structure en mémoire d'une tranche

Prenons un tableau d'entiers stocké sur la pile et une tranche référençant les éléments du milieu :

```
Tableau d'origine : `let array: [i32; 5] = [10, 20, 30, 40, 50];`  
Tranche créée      : `let slice: &[i32] = &array[1..4];`
```

Mémoire (Pile / Stack) :

```
[array] (Tableau complet)  
Adresse : 0x7ffee1a0  
+-----+-----+-----+-----+-----+  
|  10  |  20  |  30  |  40  |  50  |  
+-----+-----+-----+-----+-----+  
Index: 0      1      2      3      4  
          ^  
          |  
[slice] (Pointeur gras)  
+-----+-----+-----+-----+-----+  
| Pointeur: 0x7ffee1a4 | Longueur: 3      |  
+-----+-----+-----+-----+-----+
```

La sécurité par la vérification des bornes (Bounds Checking)

En Rust, l'accès aux éléments d'une tranche via l'indexation (ex: `slice[index]`) est protégé par une vérification stricte à l'exécution (*runtime bounds checking*).

Si votre programme tente d'accéder à un index supérieur ou égal à la longueur stockée dans le pointeur gras, Rust déclenche immédiatement une panique contrôlée (`panic!`) plutôt que de permettre un accès mémoire arbitraire. Cela empêche les failles de sécurité de type dépassement de tampon, très courantes en C et C++.

Intégration avec les règles d'emprunt

Une tranche étant une référence (`&[T]`), elle est soumise aux règles d'or de l'emprunt : * Vous pouvez avoir autant de tranches immuables (`&[T]`) que vous le souhaitez sur une même collection. * Si une tranche immuable est active, la collection d'origine ne peut pas être modifiée (pas d'emprunt mutable simultané).

3. Implémentation Pratique Pas-à-Pas

3.1 Les tranches de chaînes de caractères (`&str`)

Le type `&str` est une tranche de chaîne de caractères. C'est une référence pointant vers une séquence de données UTF-8 valides.

Voici comment créer et manipuler des tranches de chaînes de caractères :

```
fn main() {
    // Une String allouée sur le tas
    let message = String::from("Bonjour tout le monde");

    // Création de tranches à l'aide de la syntaxe d'intervalle [début..fin]
    // Note : la borne supérieure est exclusive
    let bonjour: &str = &message[0..7]; // Contient "Bonjour"
    let tout: &str = &message[8..12];   // Contient "tout"

    println!("Premier mot : '{}'", bonjour);
    println!("Deuxième mot : '{}'", tout);

    // Syntaxes raccourcies :
    let debut: &str = &message[..7]; // Équivalent à [0..7]
    let fin: &str = &message[13..];  // Va de l'index 13 jusqu'à la fin
    let tout_le_message: &str = &message[..]; // Référence la chaîne entière

    println!("Début : '{}'", debut);
    println!("Fin : '{}'", fin);
    println!("Complet : '{}'", tout_le_message);
}
```

3.2 Les tranches de tableaux (`&[T]`)

Le même concept s'applique aux tableaux de données numériques ou complexes.

```
fn main() {
    let temperatures: [f64; 7] = [12.5, 14.0, 15.2, 18.8, 19.1, 16.5, 13.0];

    // Création d'une tranche sur les températures du milieu de semaine
    let milieu_semaine: &[f64] = &temperatures[2..5]; // Éléments aux index 2, 3 et 4

    println!("Températures du milieu de semaine : {:?}", milieu_semaine);
    println!("Nombre de jours ciblés : {}", milieu_semaine.len());

    // Accès sécurisé aux éléments de la tranche
    if !milieu_semaine.is_empty() {
        println!("Première température de la tranche : {}", milieu_semaine[0]);
    }
}
```

3.3 Utilisation des tranches dans les signatures de fonctions

L'une des meilleures pratiques en Rust consiste à utiliser des tranches comme paramètres de fonction plutôt que des références à des collections spécifiques.

Par exemple, préférez toujours `&str` à `&String` dans vos fonctions. Cela permet à votre fonction d'accepter à la fois des `String`, des littéraux de chaînes (qui sont déjà des `&str`), et d'autres tranches de chaînes.

```
// Mauvaise pratique : limite l'usage aux objets String uniquement
fn inspecter_string(s: &String) {
    println!("Longueur : {}", s.len());
}

// Excellente pratique : accepte les String, les &str et les sous-parties
fn inspecter_tranche(s: &str) {
    println!("Longueur de la tranche : {}", s.len());
}

fn main() {
    let s_allouee = String::from("Rust");
    let s_litteral = "Langage";

    // inspecter_string(&s_litteral); // ERREUR DE COMPILATION !

    inspecter_tranche(&s_allouee); // Fonctionne (coercition de String en &str)
    inspecter_tranche(s_litteral); // Fonctionne
    inspecter_tranche(&s_allouee[1..3]); // Fonctionne sur une sous-partie ("us")
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Le découpage au milieu d'un caractère UTF-8

En Rust, les chaînes de caractères (`String` et `&str`) sont encodées en UTF-8. Cela signifie qu'un caractère (un point de code Unicode) peut occuper entre 1 et 4 octets en mémoire.

Si vous tentez de créer une tranche en utilisant des index d'octets qui coupent un caractère multi-octets en deux, Rust provoquera une panique immédiate à l'exécution.

```
fn main() {
    // Le caractère 'é' occupe 2 octets en UTF-8
    let mot = String::from("café");

    // Indexation mémoire :
    // c : index 0 (1 octet)
    // a : index 1 (1 octet)
    // f : index 2 (1 octet)
    // é : index 3 et 4 (2 octets)

    // Ce code compile mais va PANQUER à l'exécution !
    // On tente de couper au milieu du 'é' (index 4)
    let tranche_invalide = &mot[0..4];
    println!("{}", tranche_invalide);
}
```

Solution : Pour manipuler des textes contenant des caractères non-ASCII de manière sûre, utilisez des itérateurs de caractères (`.chars()`) ou assurez-vous que vos indices proviennent de méthodes de recherche sûres comme `.find()`.

Piège 2 : Tenter de modifier la collection d'origine pendant qu'une tranche est active

Le *Borrow Checker* empêche toute modification de la structure de données sous-jacente tant qu'une tranche immuable pointe dessus.

```
fn main() {
    let mut liste = String::from("Hello World");

    let tranche = &liste[0..5]; // Emprunt immuable de 'liste'

    liste.clear(); // ERREUR DE COMPILATION !
    // Impossible de modifier 'liste' car elle est actuellement prêtée via 'tranche'

    println!("La tranche est : {}", tranche);
}
```

Bonne pratique : L'accès sécurisé avec `.get()`

Plutôt que d'utiliser l'indexation directe `slice[index]` qui peut faire planter votre programme si l'index est hors limites, utilisez la méthode `.get()`. Elle renvoie une

`Option<T>` (concept qui sera approfondi dans le Module 4), permettant de gérer proprement l'absence de valeur sans interrompre le programme.

```
fn main() {
    let tableau = [1, 2, 3];
    let tranche = &tableau[..];

    // Accès risqué :
    // let element = tranche[5]; // Panique !

    // Accès sécurisé :
    match tranche.get(5) {
        Some(valeur) => println!("Valeur trouvée : {}", valeur),
        None => println!("Index hors limites, mais le programme continue en toute sécurité"),
    }
}
```

5. Synthèse Pédagogique

Concept	Type de données	Propriété (Ownership)	Représentation en mémoire	Sécurité
Tableau / String	<code>[T; N]</code> / <code>String</code>	Oui	Données contiguës (Pile ou Tas)	Totale (gérée par l'owner)
Référence simple	<code>&T</code>	Non	Pointeur simple (adresse seule)	Garantie par le cycle de vie
Tranche (Slice)	<code>&[T]</code> / <code>&str</code>	Non	Pointeur gras (adresse + longueur)	Vérification des bornes à l'exécution

Points clés à retenir :

1. Une tranche est une **vue immuable ou mutable** sur une séquence contiguë d'éléments.
2. Elle ne copie pas les données, elle les référence via un **pointeur gras** (adresse + taille).

3. Elle garantit la sécurité mémoire en interdisant toute modification de la collection source pendant sa durée de vie.
 4. Elle prévient les failles de sécurité grâce au Bounds Checking systématique à l'exécution.
-

6. Exercice Pratique d'Application

Énoncé : Le Parseur de Protocole Réseau

Dans le cadre du développement d'un serveur réseau ultra-sécurisé, vous devez écrire un parseur de paquets. Les paquets reçus respectent le format strict suivant :

```
"ID:<identifiant>;DATA:<données_utiles>;CRC:<somme_contrôle>"
```

Votre objectif est d'écrire une fonction hautement performante et sécurisée qui extrait uniquement la partie `<données_utiles>` sous forme de tranche (`&str`), sans effectuer la moindre allocation mémoire sur le tas (pas de `String` , pas de duplication de données).

Spécifications de la fonction :

- Nom : `extraire_donnees`
 - Signature : `fn extraire_donnees(paquet: &str) → &str`
 - Comportement :
 - Trouver la position de `"DATA:"` .
 - Trouver la position du point-virgule `";"` qui suit immédiatement ces données.
 - Retourner la tranche correspondante.
 - Si le format est invalide ou si les balises sont absentes, retourner une tranche vide `""` .
-

Indices pour la résolution

1. Utilisez la méthode `paquet.find("DATA:")` pour localiser le début de la balise de données. Cette méthode renvoie un `Option<usize>` .
2. Si `"DATA:"` est trouvé à l'index `i` , le contenu commence exactement à `i + 5` (la longueur de `"DATA:"`).

3. Utilisez ensuite la recherche à partir d'un sous-ensemble pour trouver le `";"` de fin :
`paquet[debut..].find(';')`.
 4. Prenez garde à ne pas faire de découpage hors limites si les caractères de contrôle sont manquants.
-

Correction Complète et Commentée

Voici l'implémentation complète et rigoureuse de l'exercice :

```

/// Extrait de manière sécurisée et sans allocation les données d'un paquet réseau.
fn extraire_donnees(paquet: &str) → &str {
    // 1. Recherche de la balise de début "DATA:"
    let index_balise = match paquet.find("DATA:") {
        Some(idx) ⇒ idx,
        None ⇒ return "", // Balise absente, on retourne une tranche vide
    };

    // Le contenu utile commence juste après "DATA:" (qui fait 5 caractères ASCII/octets)
    let index_debut_donnees = index_balise + 5;

    // Sécurité : on s'assure que l'index de début ne dépasse pas la taille du paquet
    if index_debut_donnees > paquet.len() {
        return "";
    }

    // 2. Recherche du point-virgule de fin à partir de l'index de début
    let reste_du_paquet = &paquet[index_debut_donnees..];
    let index_fin_relative = match reste_du_paquet.find(';') {
        Some(idx) ⇒ idx,
        None ⇒ return "", // Pas de point-virgule de fin, format invalide
    };

    // Calcul de l'index de fin absolu dans le paquet d'origine
    let index_fin_donnees = index_debut_donnees + index_fin_relative;

    // 3. Retour de la tranche de manière totalement sécurisée
    &paquet[index_debut_donnees..index_fin_donnees]
}

fn main() {
    // Cas nominal
    let paquet_valide = "ID:1024;DATA:temperature_moteur_95C;CRC:45A2";
    let donnees = extraire_donnees(paquet_valide);
    println!("Données extraites : '{}'", donnees);
    assert_eq!(donnees, "temperature_moteur_95C");

    // Cas d'erreur : balise DATA manquante
    let paquet_invalide_1 = "ID:1024;CRC:45A2";
    let donnees_vide_1 = extraire_donnees(paquet_invalide_1);
    println!("Test invalide 1 (sans DATA) : '{}'", donnees_vide_1);
    assert_eq!(donnees_vide_1, "");

    // Cas d'erreur : point-virgule de fin manquant
    let paquet_invalide_2 = "ID:1024;DATA:donnees_corrompues";
    let donnees_vide_2 = extraire_donnees(paquet_invalide_2);
    println!("Test invalide 2 (sans ';') : '{}'", donnees_vide_2);
    assert_eq!(donnees_vide_2, "");
}

```

```
println!("\nTous les tests de sécurité du parseur sont passés avec succès !")  
}
```


Exercices d'Application : Les Tranches (Slices) : Garantir des accès mémoire sécurisés et sans débordement

Exercice 1 : Analyseur de Télémétrie (Accès sécurisé et Fenêtre Glissante)

Énoncé

Dans le cadre du développement d'un logiciel embarqué pour un drone, vous recevez un flux continu de mesures d'altitude sous la forme d'un tableau d'éléments `f64`. Pour analyser la stabilité du vol, vous devez extraire des sous-parties de ces données (des fenêtres temporelles) afin de calculer des moyennes locales.

Cependant, le système est soumis à des contraintes de sécurité strictes : le programme ne doit **jamais paniquer**, même si l'opérateur demande une fenêtre de calcul qui dépasse les bornes physiques du tableau de données.

Votre objectif est d'écrire une fonction `extraire_fenetre` qui : 1. Prend en paramètre une tranche immuable de données de télémétrie (`&[f64]`), un index de départ (`debut`), et une taille de fenêtre désirée (`taille`). 2. Renvoie une option contenant la tranche demandée (`Some(&[f64])`) si la fenêtre est entièrement valide et comprise dans les limites. 3. Renvoie `None` de manière sécurisée si la fenêtre dépasse les limites du tableau ou si un débordement d'index se produit.

Indices

1. Pour éviter les paniques liées à l'indexation directe (comme `tranche[debut..fin]`), utilisez la méthode sécurisée `.get()` combinée avec un intervalle (un *Range*).
2. Attention aux débordements d'entiers (*overflow*) si `debut + taille` dépasse la valeur maximale d'un `usize` . Utilisez la méthode `debut.checked_add(taille)` pour sécuriser ce calcul.

Correction Détaillée

```
/// Extrait de manière sécurisée une sous-tranche de télémétrie.
/// Garantit l'absence totale de panique à l'exécution.
fn extraire_fenetre(donnees: &[f64], debut: usize, taille: usize) → Option<&[f64]> {
    // 1. Calcul sécurisé de l'index de fin pour éviter les débordements
    let fin = debut.checked_add(taille)?;

    // 2. Utilisation de .get() avec un Range pour obtenir une Option<&[f64]>
    // Si l'intervalle est hors limites, .get() renvoie naturellement None
    donnees.get(debut..fin)
}

fn main() {
    let telemetrie = [120.5, 122.1, 121.8, 123.0, 125.4, 124.2, 120.1];

    // Cas nominal : extraction d'une fenêtre valide au milieu
    match extraire_fenetre(&telemetrie, 2, 3) {
        Some(fenetre) => {
            println!("Fenêtre extraite : {:?}", fenetre);
            assert_eq!(fenetre, &[121.8, 123.0, 125.4]);
        }
        None => println!("Erreur inattendue dans le cas nominal."),
    }

    // Cas limite : la fenêtre dépasse de la fin du tableau
    let hors_limites = extraire_fenetre(&telemetrie, 5, 4);
    println!("Tentative hors limites (fin) : {:?}", hors_limites);
    assert!(hors_limites.is_none());

    // Cas limite : index de départ hors limites
    let depart_invalide = extraire_fenetre(&telemetrie, 10, 2);
    println!("Tentative départ invalide : {:?}", depart_invalide);
    assert!(depart_invalide.is_none());

    // Cas extrême : risque d'overflow sur usize
    let overflow = extraire_fenetre(&telemetrie, usize::MAX, 5);
    println!("Tentative avec risque d'overflow : {:?}", overflow);
    assert!(overflow.is_none());
}
```

Exercice 2 : Troncature Sécurisée de Chaînes UTF-8

Énoncé

Vous développez un composant de chat pour une application de messagerie. Pour limiter la bande passante, le serveur exige que les messages soient tronqués à un nombre maximum de octets (par exemple, 10 octets).

Comme vu dans le cours, les chaînes de caractères en Rust (`&str`) sont encodées en UTF-8, où un caractère peut occuper de 1 à 4 octets. Si vous coupez brutalement une tranche à un index d'octet arbitraire (par exemple `&message[..10]`), le programme risque de paniquer si cet index tombe au milieu d'un caractère multi-octets (comme un accent `é` ou un émoji `👉`).

Écrivez une fonction `tronquer_message` qui prend en paramètre un message (`&str`) et une limite de taille en octets (`max_octets`). Elle doit retourner une tranche immuable (`&str`) du message d'origine, raccourcie de manière à :

1. Ne pas dépasser `max_octets` .
2. S'arrêter exactement à la frontière d'un caractère UTF-8 valide pour éviter toute panique.
3. Ne réaliser aucune allocation mémoire sur le tas (pas de `String`).

Indices

1. Utilisez la méthode `message.is_char_boundary(index)` pour vérifier si un index donné correspond à un début ou une fin de caractère UTF-8 valide.
2. Si `max_octets` est supérieur ou égal à la longueur du message, vous pouvez retourner le message entier.
3. Sinon, commencez à l'index `max_octets` et décrémente cet index de 1 en 1 jusqu'à trouver une frontière de caractère valide (`is_char_boundary` renvoie `true`). Utilisez ensuite cet index sûr pour découper la tranche.

Correction Détaillée

```

/// Tronque un message à un nombre maximum d'octets sans jamais couper un caractère UTF-8
/// Ne produit aucune allocation sur le tas.
fn tronquer_message(message: &str, max_octets: usize) → &str {
    // Si le message est plus court que la limite, on le retourne en entier
    if max_octets ≥ message.len() {
        return message;
    }

    // On commence à la limite demandée et on recule jusqu'à trouver une frontière valide
    let mut index_securise = max_octets;
    while !message.is_char_boundary(index_securise) {
        // On recule d'un octet.
        // C'est mathématiquement sûr de décrémenter car l'index 0 est toujours une frontière valide
        index_securise -= 1;
    }

    // On retourne la tranche découpée de manière sécurisée
    &message[..index_securise]
}

fn main() {
    // Cas 1 : Caractères ASCII simples (1 caractère = 1 octet)
    let msg_ascii = "Hello World"; // 11 octets
    let coupe_ascii = tronquer_message(msg_ascii, 5);
    println!("ASCII tronqué : '{}'", coupe_ascii);
    assert_eq!(coupe_ascii, "Hello");

    // Cas 2 : Caractères accentués (ex: 'é' prend 2 octets en UTF-8)
    // "café" → 'c'(1), 'a'(1), 'f'(1), 'é'(2) = 5 octets au total
    let msg_accent = "café";
    // Si on coupe à 4 octets, on tombe au milieu du 'é'.
    let coupe_accent = tronquer_message(msg_accent, 4);
    println!("Accenté tronqué (limite 4) : '{}'", coupe_accent);
    // Le programme doit reculer à l'index 3 pour ne pas casser le 'é'
    assert_eq!(coupe_accent, "caf");

    // Cas 3 : Émojis (ex: '🦀' prend 4 octets en UTF-8)
    // "Rust 🦀" → 'R'(1), 'u'(1), 's'(1), 't'(1), ' '(1), '🦀'(4) = 9 octets au total
    let msg_emoji = "Rust 🦀";
    // Si on tente de couper à 7 octets (au milieu du crabe)
    let coupe_emoji = tronquer_message(msg_emoji, 7);
    println!("Émoji tronqué (limite 7) : '{}'", coupe_emoji);
    // Doit reculer à l'index 5 (juste après l'espace)
    assert_eq!(coupe_emoji, "Rust ");
}

```

Exercice 3 : Rotation de Tampon In-Place (Tranches Mutables)

Énoncé

Dans les systèmes de traitement de signal ou de réseau, on utilise souvent des tampons d'écriture circulaires. Lors de la réorganisation de ces tampons, il est fréquent de devoir intervertir deux parties d'un tableau.

Pour des raisons de performance, vous devez réaliser cette opération **in-place** (sans allouer de tableau temporaire) en manipulant des **tranches mutables** (`&mut [T]`).

Écrivez une fonction `pivoter_tampon` qui : 1. Prend en paramètre une tranche mutable d'entiers `tampon: &mut [i32]` et un index de `pivot` . 2. Divise virtuellement le tableau en deux parties au niveau du pivot : la partie gauche `[..pivot]` et la partie droite `[pivot..]` . 3. Intervertit ces deux parties pour que la partie droite se retrouve au début et la partie gauche à la fin.

Exemple : * Entrée : `[1, 2, 3, 4, 5]` avec un pivot à `2` (séparation entre `[1, 2]` et `[3, 4, 5]`). * Sortie attendue : `[3, 4, 5, 1, 2]` .

Indices

1. Pour diviser une tranche mutable en deux sous-tranches mutables distinctes sans violer les règles d'emprunt de Rust, utilisez la méthode dédiée `.split_at_mut(index)` .
2. Une astuce algorithmique élégante pour pivoter un tableau sans mémoire supplémentaire (in-place) consiste à utiliser des inversions de tranches (méthode `.reverse()`) :
3. Inversez la première partie.
4. Inversez la deuxième partie.
5. Inversez l'intégralité du tableau. (*Essayez de simuler cette logique sur papier avec `[1, 2]` et `[3, 4, 5]` !*)

Correction Détaillée

```

/// Pivote un tampon d'entiers in-place autour d'un pivot donné.
/// Utilise des tranches mutables et ne réalise aucune allocation mémoire
fn pivoter_tampon(tampon: &mut [i32], pivot: usize) {
    let taille = tampon.len();

    // Sécurité : si le pivot est hors limites ou inutile, on ne fait rien
    if pivot == 0 || pivot ≥ taille {
        return;
    }

    // 1. Division de la tranche mutable unique en deux tranches mutables adjacentes.
    // Le Borrow Checker autorise cela car split_at_mut garantit mathématiquement
    // que les deux tranches ne se chevauchent pas en mémoire.
    let (gauche, droite) = tampon.split_at_mut(pivot);

    // 2. Application de l'algorithme des trois inversions :
    // Étape A : Inverser la partie gauche
    // [1, 2] devient [2, 1]
    gauche.reverse();

    // Étape B : Inverser la partie droite
    // [3, 4, 5] devient [5, 4, 3]
    droite.reverse();

    // À ce stade, le tableau complet ressemble à : [2, 1, 5, 4, 3]

    // Étape C : Inverser la totalité du tableau
    // [2, 1, 5, 4, 3] inversé devient [3, 4, 5, 1, 2]
    tampon.reverse();
}

fn main() {
    let mut mon_tampon = [1, 2, 3, 4, 5];

    println!("Tampon d'origine : {:?}", mon_tampon);

    // Pivot à l'index 2 (éléments [1, 2] et [3, 4, 5])
    pivoter_tampon(&mut mon_tampon, 2);

    println!("Tampon pivoté : {:?}", mon_tampon);
    assert_eq!(mon_tampon, [3, 4, 5, 1, 2]);

    // Test de sécurité avec un pivot hors limites (ne doit pas paniquer)
    let mut autre_tampon = [10, 20, 30];
    pivoter_tampon(&mut autre_tampon, 10);
    assert_eq!(autre_tampon, [10, 20, 30]); // Inchangé
}

```

MODULE 3

Structurer ses Données et Éviter les Comportements Indéfinis

Structures (Structs) : Organiser ses données proprement

Dans les modules précédents, nous avons exploré les types de données primitifs (comme les entiers et les booléens) ainsi que les types composés de base (comme les tuples et les tableaux). Bien que ces outils soient indispensables, ils montrent rapidement leurs limites lorsqu'il s'agit de modéliser des concepts complexes du monde réel.

Ce module est dédié aux structures (`structs`), le mécanisme fondamental en Rust pour créer des types de données personnalisés, encapsuler des états cohérents et associer du comportement à ces données.

1. Introduction Conceptuelle : Le "Pourquoi" avant le "Comment"

Jusqu'à présent, pour représenter un point dans un espace bidimensionnel, vous auriez pu utiliser un tuple :

```
let point: (i32, i32) = (10, 20);
```

Cette approche pose plusieurs problèmes majeurs : 1. Perte de sémantique : Rien n'indique que la première valeur représente l'abscisse (x) et la seconde l'ordonnée (y), si ce n'est une convention arbitraire décidée par le développeur. 2. Absence de typage fort : Un tuple `(i32, i32)` représentant une coordonnée géographique pourrait accidentellement être passé à une fonction qui attend un tuple `(i32, i32)` représentant une fraction (numérateur, dénominateur). Le compilateur ne verrait aucune erreur, ce qui ouvre la porte à des bugs logiques subtils. 3. Évolutivité fastidieuse : Si vous décidez d'ajouter une troisième dimension (z), vous devez modifier la signature de toutes les fonctions manipulant ce tuple.

Les structures résolvent ces problèmes en permettant de nommer non seulement le type global, mais aussi chacun de ses composants (appelés **champs** ou *fields*). Elles permettent de regrouper des données hétérogènes sous une même entité logique, garantissant ainsi la cohérence et la clarté du code.

2. Fondations Théoriques

Rust propose trois grandes catégories de structures, chacune répondant à un besoin architectural précis.

A. Les trois types de structures

1. Les structures classiques (à champs nommés)

C'est la forme la plus courante. Chaque donnée possède un nom explicite et un type défini.

```
struct Utilisateur {  
    pseudo: String,  
    email: String,  
    nombre_de_connexions: u64,  
    actif: bool,  
}
```

2. Les structures tuples (Tuple Structs)

Elles sont utiles lorsque vous souhaitez donner un nom global à un tuple pour renforcer la sécurité du typage, mais que le nommage individuel des champs n'apporte aucune valeur ajoutée.

```
struct Point3D(f64, f64, f64);  
struct Couleur(u8, u8, u8);
```

Ici, `Point3D` et `Couleur` sont deux types distincts pour le compilateur, bien qu'ils contiennent tous deux trois valeurs numériques. Impossible de les confondre par inadvertance.

3. Les structures unitaires (Unit-Like Structs)

Ces structures ne contiennent aucun champ. Elles se comportent de manière similaire au type unitaire `()`.

```
struct ValideurDeDonnees;
```

Note : Elles sont principalement utilisées pour implémenter des comportements (via les traits, qui seront étudiés dans le Module 5) sur des types qui n'ont pas besoin de stocker d'état interne.

B. Représentation en mémoire, alignement et optimisation

Contrairement à d'autres langages de bas niveau comme le C, Rust ne garantit pas par défaut que les champs d'une structure seront disposés en mémoire dans leur ordre de déclaration.

Le compilateur Rust optimise la disposition des données en mémoire (*struct layout*) afin de minimiser le remplissage (*padding*) requis par les contraintes d'alignement du processeur.

Exemple d'optimisation de layout :

Considérons la structure suivante :

```
struct Exemple {  
    a: u8,    // 1 octet  
    b: u32,   // 4 octets (doit être aligné sur une adresse multiple de 4)  
    c: u8,    // 1 octet  
}
```

- En C (sans optimisation) : La structure occuperait 12 octets en mémoire à cause du rembourrage inséré pour aligner `b` et préserver l'alignement global.
- En Rust : Le compilateur peut réordonner les champs en interne (par exemple : `b` puis `a` puis `c`) pour que la structure ne prenne que 8 octets en mémoire.

[Représentation optimisée par Rust]

b (u32)	a (u8)	c (u8)	Padding (2 octets)
4 octets	1 octet	1 octet	Inutilisé

Total = 8 octets (au lieu de 12)

C. Propriété (Ownership) et Mutabilité des structures

Les règles d'or de l'ownership (étudiées dans le Module 2) s'appliquent rigoureusement aux structures :

1. **Propriété des champs** : Une structure est propriétaire de ses champs. Si un champ contient un type qui n'implémente pas le trait `Copy` (comme `String`), la structure devient l'unique propriétaire de cette donnée.
2. **Mutabilité globale** : En Rust, la mutabilité est une propriété de la *liaison de variable* (la variable déclarée avec `let mut`), et non de la structure elle-même. Il est impossible de déclarer certains champs mutables et d'autres immuables au sein de la même définition de structure.

```
struct Compteur {  
    valeur: u32,  
}  
  
let mut c1 = Compteur { valeur: 0 };  
c1.valeur = 5; // Autorisé car c1 est déclarée avec 'let mut'  
  
let c2 = Compteur { valeur: 0 };  
// c2.valeur = 5; // ERREUR de compilation : c2 est immuable
```

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Déclaration et Instanciation de base

Déclarons une structure représentant un livre dans une bibliothèque.

```

struct Livre {
    titre: String,
    auteur: String,
    pages: u32,
    disponible: bool,
}

fn main() {
    // Instanciation d'une structure immuable
    let mon_livre = Livre {
        titre: String::from("Le Petit Prince"),
        auteur: String::from("Antoine de Saint-Exupéry"),
        pages: 96,
        disponible: true,
    };

    // Accès aux champs via la notation pointée
    println!("Livre : '{}' par {}", mon_livre.titre, mon_livre.auteur);
}

```

Étape 2 : Raccourci d'initialisation des champs (Field Init Shorthand)

Lorsque les variables locales portent exactement le même nom que les champs de la structure, Rust permet de simplifier l'écriture :

```

fn creer_livre(titre: String, auteur: String, pages: u32) → Livre {
    // Au lieu de 'titre: titre', on écrit simplement 'titre'
    Livre {
        titre,
        auteur,
        pages,
        disponible: true,
    }
}

```

Étape 3 : Syntaxe de mise à jour de structure (Struct Update Syntax)

Il est fréquent de vouloir créer une nouvelle instance d'une structure en copiant la majorité des valeurs d'une instance existante. Rust propose l'opérateur `..` pour réaliser cela élégamment.

```
fn main() {
    let livre_original = Livre {
        titre: String::from("L'Étranger"),
        auteur: String::from("Albert Camus"),
        pages: 123,
        disponible: true,
    };

    // Création d'une copie avec modification d'un seul champ
    let livre_emprunte = Livre {
        disponible: false,
        ..livre_original // Récupère tous les autres champs de livre_original
    };

    // ATTENTION : livre_original ne peut plus être utilisé entièrement ici !
    // Pourquoi ? Parce que les champs de type String (titre, auteur) n'implémentent pas Copy
    // Ils ont été DÉPLACÉS (moved) dans livre_emprunte.

    // println!("{}", livre_original.titre); // ERREUR de compilation !
    println!("Le livre original n'est plus accessible car ses Strings ont été déplacées.");
}
```

Étape 4 : Implémentation de méthodes et fonctions associées (`impl`)

Pour associer du comportement à nos structures, nous utilisons le bloc `impl` (implémentation).

- **Fonctions associées** : Elles ne prennent pas `self` en paramètre. Elles sont souvent utilisées comme constructeurs (par convention nommés `new`). On les appelle avec l'opérateur de résolution de portée `::` .
- **Méthodes** : Elles prennent `self` , `&self` ou `&mut self` comme premier paramètre. On les appelle avec la notation pointée `.` .

```

struct Rectangle {
    largeur: f64,
    hauteur: f64,
}

impl Rectangle {
    // Fonction associée (Constructeur)
    fn new(largeur: f64, hauteur: f64) → Rectangle {
        Rectangle { largeur, hauteur }
    }

    // Méthode en lecture seule (emprunt immuable)
    fn calculer_aire(&self) → f64 {
        self.largeur * self.hauteur
    }

    // Méthode en écriture (emprunt mutable)
    fn redimensionner(&mut self, nouvelle_largeur: f64, nouvelle_hauteur: f64) {
        self.largeur = nouvelle_largeur;
        self.hauteur = nouvelle_hauteur;
    }
}

fn main() {
    // Appel de la fonction associée
    let mut rect = Rectangle::new(10.0, 5.0);

    // Appel de la méthode de lecture
    println!("Aire initiale : {} px²", rect.calculer_aire());

    // Appel de la méthode de modification
    rect.redimensionner(20.0, 10.0);
    println!("Nouvelle aire : {} px²", rect.calculer_aire());
}

```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Le déplacement partiel (Partial Move)

Si vous accédez à un champ non-`Copy` d'une structure et que vous le déplacez hors de la structure, celle-ci devient partiellement détruite. Vous ne pouvez plus utiliser la structure dans son ensemble, sauf si vous réinitialisez le champ déplacé.

```

struct Utilisateur {
    pseudo: String,
    actif: bool,
}

fn main() {
    let u = Utilisateur {
        pseudo: String::from("Alice"),
        actif: true,
    };

    let nom = u.pseudo; // Le String est déplacé ici. u.pseudo n'est plus valide

    // println!("Utilisateur : {:?}", u); // ERREUR : utilisation d'une valeur partiellement
    println!("Le pseudo extrait est : {}", nom);
}

```

Piège 2 : Tenter de rendre des champs mutables individuellement

La syntaxe suivante est invalide et rejetée par le compilateur :

```

// CODE INCORRECT - NE COMPILE PAS
struct ErreurDeSyntaxe {
    mut champ_mutable: i32, // Syntaxe interdite
    champ_immuable: i32,
}

```

Solution : Si vous avez besoin de mutabilité interne fine sans rendre toute la structure mutable, vous devrez utiliser des patterns avancés comme la "mutabilité intérieure" (`RefCell`, `Mutex`), qui seront abordés dans les modules avancés (Module 5). Pour l'instant, considérez que la mutabilité s'applique à l'instance complète.

Bonne Pratique 1 : Dériver le trait `Debug` pour l'affichage

Par défaut, vous ne pouvez pas afficher une structure avec `println!("{}", ma_struct)`. Pour faciliter le débogage, ajoutez toujours l'attribut `#[derive(Debug)]` au-dessus de vos structures. Cela vous permettra de les inspecter avec le formateur `{:?}` ou `{:#?}` (affichage joli).

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    println!("Debug simple : {:?}", p);
    println!("Debug élégant : {:#?}", p);
}
```

5. Synthèse Pédagogique

Type de Structure	Syntaxe de Déclaration	Cas d'usage principal	Exemple d'instanciation
Classique	<code>struct Nom { f1: T1, f2: T2 }</code>	Modélisation de concepts complexes avec données nommées.	<code>let n = Nom { f1: v1, f2: v2 };</code>
Tuple	<code>struct Nom(T1, T2);</code>	Création de types distincts (<i>Newtype pattern</i>) sans surcharge de noms.	<code>let t = Nom(v1, v2);</code>
Unitaire	<code>struct Nom;</code>	Implémentation de comportements (traits) sans état interne.	<code>let u = Nom;</code>

Règles clés à retenir :

1. Pas de mutabilité partielle : Une structure est mutable dans son ensemble ou immuable dans son ensemble.
2. Mise à jour de structure (`..`) : Attention aux transferts de propriété (*moves*) des champs non- `Copy` .

3. Méthodes vs Fonctions associées : Les méthodes prennent `self` (sous forme d'emprunt `&self` ou `&mut self`), les fonctions associées ne le prennent pas.
-

6. Exercice Pratique d'Application

Énoncé : Gestionnaire de Stock de Magasin

Vous devez concevoir un système de gestion de stock simplifié pour un magasin de jeux vidéo.

1. Créez une structure nommée `JeuVideo` contenant les champs suivants :
2. `titre` : un `String`
3. `prix` : un `f64`
4. `quantite_stock` : un `u32`
5. Implémentez un bloc `impl` pour `JeuVideo` contenant :
6. Une fonction associée `nouveau(titre: &str, prix: f64, quantite_stock: u32) → JeuVideo` servant de constructeur.
7. Une méthode `afficher_details(&self)` qui affiche proprement les informations du jeu.
8. Une méthode `appliquer_remise(&mut self, pourcentage: f64)` qui réduit le prix du jeu du pourcentage indiqué.
9. Une méthode `vendre(&mut self, quantite: u32) → bool` qui diminue la quantité en stock de la quantité vendue si le stock est suffisant, et retourne `true`. Si le stock est insuffisant, elle ne modifie rien et retourne `false`.

Indices de résolution

- Pour convertir un `&str` en `String` dans le constructeur, utilisez la méthode `.to_string()` ou `String::from()`.
 - Pour appliquer une remise de 20%, le nouveau prix est calculé par : $\text{prix} \times (1.0 - (\text{pourcentage} / 100.0))$.
-

Solution Complète et Commentée

Voici l'implémentation attendue. Prenez le temps de la lire et de comprendre chaque ligne avant de la tester dans votre environnement de développement.

```

#[derive(Debug)] // Permet d'afficher la structure pour le débogage
struct JeuVideo {
    titre: String,
    prix: f64,
    quantite_stock: u32,
}

impl JeuVideo {
    // 1. Constructeur (Fonction associée)
    fn nouveau(titre: &str, prix: f64, quantite_stock: u32) → JeuVideo {
        JeuVideo {
            titre: titre.to_string(), // Conversion de &str en String appartenant à la pile
            prix,
            quantite_stock,
        }
    }

    // 2. Méthode d'affichage (Emprunt immuable car on ne fait que lire)
    fn afficher_details(&self) {
        println!("--- Fiche Produit ---");
        println!("Titre      : {}", self.titre);
        println!("Prix       : {:.2} €", self.prix);
        println!("En Stock   : {} unités", self.quantite_stock);
        println!("-----");
    }

    // 3. Méthode de modification du prix (Emprunt mutable)
    fn appliquer_remise(&mut self, pourcentage: f64) {
        if pourcentage > 0.0 && pourcentage ≤ 100.0 {
            let reduction = self.prix * (pourcentage / 100.0);
            self.prix -= reduction;
            println!("Une remise de {}% a été appliquée sur '{}'.", pourcentage, self.titre);
        } else {
            println!("Erreur : Pourcentage de remise invalide.");
        }
    }

    // 4. Méthode de vente (Emprunt mutable car on modifie la quantité en stock)
    fn vendre(&mut self, quantite: u32) → bool {
        if self.quantite_stock ≥ quantite {
            self.quantite_stock -= quantite;
            println!("Vente réussie : {} exemplaire(s) de '{}' vendu(s).", quantite, self.titre);
            true
        } else {
            println!("Vente échouée : Stock insuffisant pour '{}' (Demandé: {}, Disponible: {})",
                    self.titre, quantite, self.quantite_stock);
            false
        }
    }
}

```

```

    }
}

fn main() {
    // Initialisation d'un jeu vidéo mutable
    let mut jeu = JeuVideo::nouveau("The Legend of Zelda: Breath of the Wild", 59.99, 10)

    // Affichage initial
    jeu.afficher_details();

    // Tentative de vente réussie
    let vente1 = jeu.vendre(3);
    println!("Statut de la vente : {}", vente1);
    println!("Nouveau stock : {} unités\n", jeu.quantite_stock);

    // Application d'une promotion de 20%
    jeu.appliquer_remise(20.0);
    jeu.afficher_details();

    // Tentative de vente supérieure au stock disponible
    let vente2 = jeu.vendre(15);
    println!("Statut de la vente : {}\n", vente2);

    // Vérification finale de l'état de la structure via le format Debug
    println!("État final de la structure (Debug) :");
    println!("{}", jeu);
}

```


Exercices d'Application : Structures (Structs) : Organiser ses données proprement

Ce cahier d'exercices pratiques a pour but de consolider vos connaissances sur les structures en Rust. Vous progresserez de la manipulation de types simples (Tuple Structs) vers la gestion de la propriété (*ownership*) lors de la copie de structures, pour finir par la conception d'un système complet orienté objet (méthodes et mutabilité).

Exercice 1 : Sécuriser les Mesures avec les Tuple Structs (Niveau : Facile)

Énoncé

Dans les applications d'ingénierie, confondre les unités de mesure peut mener à des catastrophes (comme la perte de la sonde spatiale *Mars Climate Orbiter* due à une confusion entre les unités métriques et impériales).

Vous devez concevoir un système de conversion de distance ultra-sécurisé en utilisant le pattern `Tuple Struct` (ou `Newtype`).

1. Créez deux structures tuples :
2. `Metre` qui enveloppe un `f64` .
3. `Pied` qui enveloppe un `f64` .
4. Implémentez une fonction classique (non associée) nommée `convertir_en_pieds` qui prend en paramètre une instance de `Metre` et retourne une instance de `Pied` .

5. Règle de conversion : $1 \text{ mètre} = 3.28084 \text{ pieds}$.
6. Dans votre fonction `main` :
7. Instanciez une distance de `10.0` mètres.
8. Convertissez-la en pieds à l'aide de votre fonction.
9. Affichez proprement le résultat.
10. Tentez de passer directement un `f64` brut à votre fonction de conversion pour observer comment le compilateur Rust protège votre code.

Indices

1. Pour accéder à la valeur brute (`f64`) contenue dans une structure tuple, utilisez la notation pointée avec l'index de position : `ma_structure.0` .
 2. Pour l'affichage, vous pouvez utiliser la déstructuration ou l'accès direct par index.
-

Correction Détaillée

```
// Définition des structures tuples pour typer fortement nos unités
struct Metre(f64);
struct Pied(f64);

// Fonction de conversion prenant un type strict 'Metre' et retournant un 'Pied'
fn convertir_en_pieds(m: Metre) → Pied {
    // On accède à la valeur f64 interne via .0
    let valeur_en_pieds = m.0 * 3.28084;
    Pied(valeur_en_pieds)
}

fn main() {
    // 1. Instanciation correcte
    let distance_metres = Metre(10.0);

    // 2. Conversion sécurisée
    let distance_pieds = convertir_en_pieds(distance_metres);

    // 3. Affichage du résultat
    println!("10.0 mètres équivalent à {:.4} pieds.", distance_pieds.0);

    // --- SÉCURITÉ DU TYPAGE ---
    // Si vous décommentez la ligne ci-dessous, le code refusera de compiler :
    // let erreur = convertir_en_pieds(10.0);
    // Erreur : type attendu `Metre`, trouvé `f64`

    // De même, impossible de confondre un Pied et un Metre :
    // let autre_erreur = convertir_en_pieds(distance_pieds);
    // Erreur : type attendu `Metre`, trouvé `Pied`
}
```

Exercice 2 : Clonage et Transfert de Propriété (Niveau : Moyen)

Énoncé

Cet exercice vise à comprendre le comportement de l'opérateur de mise à jour de structure (`..`) face aux types de données qui n'implémentent pas le trait `Copy` (comme `String`).

1. Déclarez la structure suivante représentant un profil utilisateur : `rust struct ProfilUtilisateur { pseudo: String, email: String, niveau: u32, actif: bool, }`
2. Dans la fonction `main` , créez une instance nommée `user1` avec les données de votre choix.
3. Créez une seconde instance `user2` en utilisant la syntaxe de mise à jour de structure (`..user1`). `user2` doit avoir un nouvel `email` (ex: `"nouveau@email.com"`), mais conserver toutes les autres informations de `user1` .
4. Essayez d'afficher le `pseudo` de `user1` après la création de `user2` . Que se passe-t-il ? Pourquoi ?
5. Corrigez le code afin que `user1` reste pleinement utilisable après la création de `user2` .

Indices

1. Le champ `pseudo` est de type `String` (qui n'est pas `Copy`). Lors de l'utilisation de `..user1` , la propriété de `user1.pseudo` est transférée (*moved*) à `user2` .
 2. Pour éviter ce transfert de propriété, vous devez copier explicitement la donnée non- `Copy` en utilisant la méthode `.clone()` .
-

Correction Détaillée

Étape 1 : Comprendre l'erreur de compilation (Scénario de départ)

Si vous tentez d'écrire ceci :

```
struct ProfilUtilisateur {
    pseudo: String,
    email: String,
    niveau: u32,
    actif: bool,
}

fn main() {
    let user1 = ProfilUtilisateur {
        pseudo: String::from("Alice"),
        email: String::from("alice@rust.org"),
        niveau: 42,
        actif: true,
    };

    // Utilisation de la syntaxe de mise à jour
    let user2 = ProfilUtilisateur {
        email: String::from("nouvel_alice@rust.org"),
        ..user1 // Déplace 'pseudo' de user1 vers user2 !
    };

    // ERREUR DE COMPILATION ICI :
    // println!("Pseudo de user1 : {}", user1.pseudo);
}
```

Explication : Le compilateur refuse de compiler car `user1.pseudo` a été déplacé dans `user2`. `user1` est désormais partiellement détruit.

Étape 2 : Résolution du problème

Pour corriger cela, nous devons cloner explicitement le champ `pseudo` afin que `user1` conserve sa propre version en mémoire.

```

#[derive(Debug)] // Pour pouvoir afficher nos structures facilement
struct ProfilUtilisateur {
    pseudo: String,
    email: String,
    niveau: u32,
    actif: bool,
}

fn main() {
    let user1 = ProfilUtilisateur {
        pseudo: String::from("Alice"),
        email: String::from("alice@rust.org"),
        niveau: 42,
        actif: true,
    };

    // Pour éviter le transfert de propriété de 'pseudo',
    // nous l'instancions explicitement avec un clone de celui de user1.
    let user2 = ProfilUtilisateur {
        pseudo: user1.pseudo.clone(), // On duplique la String en mémoire
        email: String::from("nouvel_alice@rust.org"),
        ..user1 // Les champs restants (niveau, actif) sont Copy, donc copyés sans move
    };

    // Maintenant, les deux structures sont parfaitement valides et utilisables !
    println!("User 1 : {} (Email: {})", user1.pseudo, user1.email);
    println!("User 2 : {} (Email: {})", user2.pseudo, user2.email);

    // Preuve de la mutabilité indépendante
    println!("\nStructure complète User 1 : {:?}", user1);
    println!("Structure complète User 2 : {:?}", user2);
}

```

Exercice 3 : Simulateur de Compte Épargne (Niveau : Difficile)

Énoncé

Vous devez concevoir un module de gestion financière modélisant un **Compte Épargne**.

1. Créez une structure `CompteEpargne` avec les champs suivants :
2. `titulaire` : `String`
3. `solde` : `f64`
4. `taux_interet` : `f64` (représenté sous forme décimale, ex: `0.02` pour 2%)
5. Implémentez un bloc `impl` contenant les fonctionnalités suivantes :
6. Une fonction associée (constructeur) `nouveau(titulaire: &str, depot_initial: f64, taux: f64) → CompteEpargne` .
7. Une méthode `deposer(&mut self, montant: f64)` qui ajoute de l'argent au solde.
8. Une méthode `retirer(&mut self, montant: f64) → Result<f64, String>` :
 - Si le solde est suffisant, elle déduit le montant et retourne le solde retiré enveloppé dans `Ok` .
 - Si le solde est insuffisant, elle ne modifie rien et retourne un message d'erreur enveloppé dans `Err` .
9. Une méthode `appliquer_interets(&mut self)` : calcule les intérêts sur le solde actuel et les ajoute directement au solde ($\text{solde} = \text{solde} \times \text{taux_interet}$).
10. Une méthode `afficher_solde(&self)` : affiche un résumé du compte (Nom du titulaire, solde actuel et taux).
11. Écrivez un scénario dans votre fonction `main` pour tester l'ensemble de ces fonctionnalités (création, dépôt, retrait réussi, retrait refusé, application des intérêts et affichages).

Indices

1. Pensez à utiliser `&mut self` lorsque la méthode doit modifier l'état interne de la structure (comme le solde), et `&self` lorsqu'elle doit simplement le lire.
 2. Pour le constructeur, convertissez le paramètre `titulaire` de type `&str` en `String` à l'aide de `.to_string()`.
-

Correction Détaillée

```

#[derive(Debug)]
struct CompteEpargne {
    titulaire: String,
    solde: f64,
    taux_interet: f64,
}

impl CompteEpargne {
    // 1. Constructeur
    fn nouveau(titulaire: &str, depot_initial: f64, taux: f64) → CompteEpargne {
        CompteEpargne {
            titulaire: titulaire.to_string(),
            solde: depot_initial,
            taux_interet: taux,
        }
    }

    // 2. Méthode de dépôt (Modification de l'état → &mut self)
    fn deposer(&mut self, montant: f64) {
        if montant > 0.0 {
            self.solde += montant;
            println!("Dépôt de {:.2} € effectué avec succès.", montant);
        } else {
            println!("Erreur : Le montant du dépôt doit être positif.");
        }
    }

    // 3. Méthode de retrait avec vérification de provision
    fn retirer(&mut self, montant: f64) → Result<f64, String> {
        if montant ≤ 0.0 {
            return Err(String::from("Le montant du retrait doit être positif."));
        }

        if self.solde ≥ montant {
            self.solde -= montant;
            Ok(montant)
        } else {
            Err(format!(
                "Fonds insuffisants. Solde actuel : {:.2} €, Retrait demandé : {:.2} €",
                self.solde, montant
            ))
        }
    }

    // 4. Méthode d'application des intérêts capitalisés
    fn appliquer_interets(&mut self) {
        let interets = self.solde * self.taux_interet;
        self.solde += interets;
    }
}

```

```

        println!(
            "Intérêts appliqués (+{:.2} € au taux de {:.1}%).",
            interets,
            self.taux_interet * 100.0
        );
    }

    // 5. Méthode de consultation (Lecture seule → &self)
    fn afficher_solde(&self) {
        println!("=====");
        println!("Titulaire du compte : {}", self.titulaire);
        println!("Solde actuel      : {:.2} €", self.solde);
        println!("Taux d'intérêt      : {:.2}%", self.taux_interet * 100.);
        println!("=====");
    }
}

fn main() {
    // Initialisation du compte (doit être mutable pour pouvoir faire des transactions)
    let mut mon_compte = CompteEpargne::nouveau("Jean Dupont", 1000.0, 0.05); // Taux à 5%

    // Affichage de départ
    mon_compte.afficher_solde();

    // Dépôt d'argent
    mon_compte.deposer(500.0);
    mon_compte.afficher_solde();

    // Tentative de retrait autorisé
    match mon_compte.retirer(300.0) {
        Ok(montant) => println!("Retrait de {:.2} € validé.", montant),
        Err(e) => println!("Erreur transactionnelle : {}", e),
    }
    mon_compte.afficher_solde();

    // Tentative de retrait refusé (fonds insuffisants)
    match mon_compte.retirer(2000.0) {
        Ok(montant) => println!("Retrait de {:.2} € validé.", montant),
        Err(e) => println!("Erreur transactionnelle : {}", e), // Ce cas n'a pas été exécuté
    }

    // Application des intérêts annuels
    mon_compte.appliquer_interets();
    mon_compte.afficher_solde();
}

```

Énumérations (Enums) et Pattern Matching : Le contrôle de flux infailible

1. Introduction Conceptuelle

Le problème des architectures d'états traditionnelles

Dans la majorité des langages de programmation traditionnels (comme le C, le C++ ou le Java pré-14), la gestion des états mutuellement exclusifs repose sur des mécanismes fragiles. Prenons l'exemple d'une connexion réseau qui peut être dans l'un des quatre états suivants : `Déconnecté` , `EnCoursDeConnexion` , `Connecté` , ou `Erreur` .

En C, un développeur représenterait généralement cela à l'aide d'une constante entière ou d'une énumération simple :

```
// Exemple de conception fragile en C
typedef enum {
    STATE_DISCONNECTED,
    STATE_CONNECTING,
    STATE_CONNECTED,
    STATE_ERROR
} ConnectionState;

struct Connection {
    ConnectionState state;
    char* ip_address;      // Valide uniquement si CONNECTED ou CONNECTING
    int error_code;        // Valide uniquement si STATE_ERROR
};
```

Cette approche souffre de failles de sécurité et de conception majeures : 1. Accès incohérent aux données : Rien n'empêche un développeur d'accéder à `error_code` alors que l'état est `STATE_CONNECTED` . Cela peut retourner une valeur résiduelle de la mémoire (comportement indéfini ou bug logique). 2. Oubli de cas (Switch incomplet) : Si l'on ajoute

un état `STATE_RECONNECTING`, le compilateur C n'obligera pas le développeur à mettre à jour tous les blocs `switch` du programme. Le comportement par défaut (`default`) sera exécuté, masquant potentiellement des bugs critiques. 3. Pointeurs nuls : L'adresse IP peut être un pointeur nul (`NULL`) dans l'état `STATE_DISCONNECTED`, obligeant à des vérifications manuelles constantes et sujettes à l'erreur.

La philosophie de Rust : Rendre les états invalides impossibles à représenter

Rust résout ce problème à la racine en combinant deux concepts puissants : les Énumérations avec données associées (souvent appelées types de données algébriques) et le Pattern Matching (filtrage par motif) garanti par le compilateur.

En Rust, une énumération ne liste pas seulement des étiquettes numériques ; elle permet d'associer des structures de données spécifiques et distinctes à chaque variante. Si une donnée n'a de sens que dans un état précis, elle est physiquement encapsulée dans cet état. Il est impossible d'y accéder si l'on n'est pas dans cet état.

2. Fondations Théoriques

Les Types de Données Algébriques (ADT)

Pour comprendre la puissance des énumérations en Rust, il faut faire un bref détour par la théorie des types. Les structures de données en Rust se divisent principalement en deux catégories d'ADT :

1. **Les Types Produits (Product Types)** : Ce sont les structures (`struct`) et les tuples.
L'ensemble des valeurs possibles d'une structure est le produit cartésien des ensembles de valeurs de ses champs.
$$\text{Taille de l'espace d'état} = \text{Taille}(\text{Champ}_1) \times \text{Taille}(\text{Champ}_2)$$
2. **Les Types Sommes (Sum Types)** : Ce sont les énumérations (`enum`). Une instance d'une énumération ne peut contenir qu'une seule variante à la fois. L'ensemble des valeurs possibles est la somme des ensembles de valeurs de chaque variante.
$$\text{Taille de l'espace d'état} = \text{Taille}(\text{Variante}_1) + \text{Taille}(\text{Variante}_2)$$

Cette distinction est cruciale pour la sécurité mémoire : les types sommes permettent de restreindre l'espace d'état au strict minimum nécessaire, éliminant ainsi les combinaisons de variables d'état invalides.

Représentation en mémoire (Layout)

Comment le compilateur Rust gère-t-il ces types sommes en mémoire sans perdre en performance ?

Une énumération Rust est stockée sous la forme d'un **tag** (ou discriminant) accompagné d'un **payload** (la charge utile contenant les données de la variante active).

Représentation mémoire théorique d'une Enum :

```
+-----+-----+
| Tag (Discriminant) | Payload (Espace de la variante la plus large) |
| (1 à 8 octets)    | (Partagé par toutes les variantes)           |
+-----+-----+
```

- Le Tag : Un petit entier généré par le compilateur (généralement 1 octet si l'énumération a moins de 256 variantes) qui indique quelle variante est actuellement active.
- Le Payload : La taille de cet espace est égale à la taille de la variante la plus grande. Rust réutilise la même zone mémoire pour toutes les variantes (similaire à une **union** en C/C++), garantissant une utilisation optimale de la mémoire.

Optimisation de la niche (Null Pointer Optimization)

Rust applique une optimisation remarquable appelée *l'optimisation de la niche*. Si une variante contient un type qui ne peut pas être nul (comme une référence **&T** ou un type pointeur non nul), Rust utilise la valeur **0x0** (le pointeur nul) pour représenter une autre variante sans données (comme **None** dans l'énumération **Option** qui sera étudiée au Module 4). Dans ce cas, la taille de l'énumération est strictement identique à la taille de la référence, et le coût du tag est réduit à zéro.

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Déclaration d'une énumération simple et complexe

Créons un module de gestion de messages pour une application de messagerie.

```
// Une énumération représentant différents types d'actions utilisateur
enum Message {
    // Variante sans donnée (similaire à un enum traditionnel)
    Quit,
    // Variante contenant des données anonymes (Tuple Variant)
    Move { x: i32, y: i32 },
    // Variante contenant une chaîne de caractères dynamique
    Write(String),
    // Variante contenant une couleur au format RGB (trois u8)
    ChangeColor(u8, u8, u8),
}
```

Étape 2 : Le contrôle de flux avec `match`

L'expression `match` est l'outil principal pour consommer une énumération. Contrairement au `switch` d'autres langages, `match` en Rust est une **expression** (elle retourne une valeur) et est **exhaustive** (le compilateur refuse la compilation si un cas est omis).

```
fn designer_action(message: Message) {
    match message {
        Message::Quit => {
            println!("L'utilisateur souhaite quitter.");
        }
        // Destructuration des champs nommés de la variante Move
        Message::Move { x, y } => {
            println!("Déplacement vers la position : x = {}, y = {}", x, y);
        }
        // Récupération de la String par propriété (ownership)
        Message::Write(texte) => {
            println!("Message écrit : {}", texte);
        }
        // Destructuration d'un tuple de données
        Message::ChangeColor(r, g, b) => {
            println!("Changement de couleur : Rouge={}, Vert={}, Bleu={}", r, g, b);
        }
    }
}
```

Étape 3 : Les Gardes de Filtrage (Match Guards)

Il est possible d'ajouter des conditions supplémentaires à une branche de `match` à l'aide d'un garde (`if`).

```
fn filtrer_coordonnees(message: Message) {
    match message {
        // Le garde "if x == y" ajoute une condition logique au filtrage
        Message::Move { x, y } if x == y => {
            println!("Déplacement diagonal parfait sur l'axe : {}", x);
        }
        Message::Move { x, y } => {
            println!("Déplacement standard : x = {}, y = {}", x, y);
        }
        // Utilisation du motif de liaison '_' pour ignorer les autres variantes
        _ => println!("Autre action non liée à un déplacement."),
    }
}
```

Étape 4 : La syntaxe concise `if let`

Lorsque vous ne souhaitez traiter qu'une seule variante spécifique d'une énumération et ignorer toutes les autres, l'utilisation de `match` peut s'avérer verbeuse. Rust propose la structure de contrôle `if let` pour ce cas d'usage précis.

```
fn verifier_si_ecriture(message: Message) {  
    // if let prend un motif à gauche et une expression à droite  
    if let Message::Write(contenu) = message {  
        println!("Traitement rapide du texte : {}", contenu);  
    } else {  
        println!("Le message n'est pas un texte.");  
    }  
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : L'abus du motif universel `_` (Wildcard)

Le motif `_` permet de capturer tous les cas non explicitement traités. Bien que pratique, son utilisation abusive détruit la sécurité apportée par l'exhaustivité du compilateur.

Exemple à éviter :

```
enum ModePaiement {  
    CarteBancaire,  
    Paypal,  
    Virement, // Ajouté plus tard  
}  
  
fn traiter_paiement(mode: ModePaiement) {  
    match mode {  
        ModePaiement::CarteBancaire => println!("Traitement CB"),  
        _ => println!("Autre mode de paiement"), // Dangereux !  
    }  
}
```

Si vous ajoutez `Virement` à l'énumération, le compilateur ne signalera aucune erreur car le `virement` sera capturé silencieusement par le `_`. *Bonne pratique* : Épelez explicitement toutes les variantes autant que possible. N'utilisez `_` que si l'ensemble des cas futurs doit réellement subir le même traitement par défaut.

Piège 2 : Le coût mémoire des variantes disproportionnées

Comme expliqué dans la section théorique, la taille d'une énumération en mémoire est calquée sur sa plus grande variante.

```
// Cette énumération pèse plus de 4096 octets en mémoire !
enum Fichier {
    Vide,
    PetitNom(String),
    ContenuBrut([u8; 4096]), // Cette variante force l'enum à être énorme
}
```

Chaque instance de `Fichier`, même si elle est `Fichier::Vide`, occupera plus de 4 Ko sur la pile (Stack). *Bonne pratique* : Si une variante contient une structure de données volumineuse, utilisez l'indirection (comme un pointeur intelligent `Box`, qui sera étudié en détail dans les modules suivants) pour stocker la donnée sur le tas (Heap) et ne conserver qu'un pointeur (8 octets) dans l'énumération.

5. Synthèse Pédagogique

Tableau Comparatif des Structures de Contrôle et de Données

Concept	Usage Principal	Avantage Majeur	Inconvénient / Limite
<code>Struct</code>	Regrouper des données hétérogènes (Produit)	Accès direct aux champs	Impossible de représenter l'exclusion mutuelle proprement
<code>Enum</code>	Représenter des états exclusifs (Somme)	Typage fort, sécurité mémoire	Nécessite un filtrage pour accéder aux données
<code>match</code>	Traitement exhaustif de tous les cas	Garantie de compilation sans oubli	Syntaxe parfois verbeuse pour un cas unique
<code>if let</code>	Traitement d'un cas unique ciblé	Code concis et lisible	Ignore silencieusement les autres variantes

Points Clés à Retenir

- Les énumérations Rust sont des **types sommes** capables de transporter des données hétérogènes.
- Le compilateur garantit l'exhaustivité du filtrage par motif (`match`). Aucun cas ne peut être omis.
- Les données associées à une variante ne sont accessibles que si le motif correspond, éliminant les accès mémoire invalides.
- L'optimisation de la mémoire garantit que l'empreinte d'une énumération est minimale (taille de la variante maximale + tag).

6. Exercice Pratique d'Application

Énoncé : Conception d'un automate de consigne automatique

Vous devez concevoir le logiciel de contrôle d'une consigne automatique de gare. Une consigne individuelle peut se trouver dans l'un des états suivants : 1. **Libre** (ne contient rien). 2. **Occupée** (contient un colis identifié par un code de suivi `String` et un poids `f64`). 3. **HorsService** (contient un motif de panne `String`).

Vous devez modéliser ces états à l'aide d'une énumération Rust, puis implémenter une fonction de transition de sécurité qui simule le dépôt d'un colis.

Spécifications :

- Créez l'énumération `EtatConsigne`.
- Créez une énumération `ResultatDepot` pour représenter l'issue de l'opération :
- `Succes` (contenant le nouvel état de la consigne).
- `ErreurRefusee` (si la consigne n'était pas libre ou était hors service, avec un message explicatif).
- Implémentez la fonction `deposer_colis(consigne_actuelle: EtatConsigne, code: String, poids: f64) → ResultatDepot`.

Indices pour la résolution

1. Rappelez-vous que pour extraire ou modifier l'état, vous devez utiliser un `match` sur `consigne_actuelle`.
2. Si la consigne est `Libre`, la transition doit retourner `ResultatDepot::Succes` contenant la consigne mise à jour en `Occupée` avec les données fournies.
3. Si la consigne est déjà occupée ou hors service, vous devez retourner `ResultatDepot::ErreurRefusee` avec une explication textuelle.

Correction Complète et Commentée

Voici l'implémentation complète, robuste et fonctionnelle du problème :


```

// Définition des états possibles de la consigne
#[derive(Debug, Clone)]
enum EtatConsigne {
    Libre,
    Occupee { code_suivi: String, poids: f64 },
    HorsService(String),
}

// Définition des résultats possibles d'une tentative de dépôt
#[derive(Debug)]
enum ResultatDepot {
    Succes(EtatConsigne),
    ErreurRefusee(String),
}

/// Tente de déposer un colis dans la consigne donnée.
/// Cette fonction consomme l'ancien état et retourne le nouvel état ou une erreur.
fn deposer_colis(consigne_actuelle: EtatConsigne, code: String, poids: f64) → ResultatDepot {
    match consigne_actuelle {
        // Cas nominal : la consigne est libre, on effectue la transition
        EtatConsigne::Libre ⇒ {
            let nouvel_etat = EtatConsigne::Occupee {
                code_suivi: code,
                poids,
            };
            ResultatDepot::Succes(nouvel_etat)
        }
        // Cas d'erreur : la consigne est déjà occupée, on refuse le dépôt
        EtatConsigne::Occupee { code_suivi, .. } ⇒ {
            ResultatDepot::ErreurRefusee(format!(
                "Opération rejetée : La consigne contient déjà le colis '{}'.",
                code_suivi
            ))
        }
        // Cas d'erreur : la consigne est en panne
        EtatConsigne::HorsService(raison) ⇒ {
            ResultatDepot::ErreurRefusee(format!(
                "Opération impossible : Consigne hors service pour la raison suivante : '{}'",
                raison
            ))
        }
    }
}

fn main() {
    // Test 1 : Dépôt réussi dans une consigne libre
    let consigne_a = EtatConsigne::Libre;
    println!("--- Test 1 : Consigne Initiale Libre ---");
}

```

```

match deposer_colis(consigne_a, String::from("FR-8892-XP"), 4.5) {
    ResultatDepot::Succes(nouvel_etat) => {
        println!("Dépôt réussi ! Nouvel état : {:?}", nouvel_etat);
    }
    ResultatDepot::ErreurRefusee(err) => {
        println!("Échec inattendu : {}", err);
    }
}

// Test 2 : Tentative de dépôt dans une consigne occupée
println!("\n--- Test 2 : Consigne Déjà Occupée ---");
let consigne_b = EtatConsigne::Occupee {
    code_suivi: String::from("DE-1120-LM"),
    poids: 12.8,
};
match deposer_colis(consigne_b, String::from("FR-8892-XP"), 2.0) {
    ResultatDepot::Succes(_) => {
        println!("Erreur de sécurité : Un colis a écrasé un autre colis !");
    }
    ResultatDepot::ErreurRefusee(err) => {
        println!("Refus sécurisé conforme : {}", err);
    }
}
}

```

Explications de la correction :

1. **Sécurité d'accès** : Dans le cas `EtatConsigne::Occupee`, nous utilisons la syntaxe `..` pour ignorer le champ `poids` dont nous n'avons pas besoin pour formuler notre message d'erreur.
2. **Garantie de l'état** : Grâce au système d'ownership de Rust, la fonction `deposer_colis` consomme l'ancien état (`consigne_actuelle`). Il est donc impossible pour l'appelant d'utiliser accidentellement l'ancien état après une transition réussie, éliminant ainsi les bugs de double allocation ou de corruption d'état.

Exercices d'Application :

Énumérations (Enums) et Pattern Matching

Ce cahier d'exercices pratiques a pour but de consolider vos compétences sur la modélisation d'états avec les énumérations Rust, la manipulation sécurisée des données via le filtrage par motif (`match` et `if let`), ainsi que l'optimisation de la mémoire associée à ces structures.

Exercice 1 : Contrôle d'Objets Connectés (Niveau Facile)

Énoncé

Dans le cadre du développement d'une passerelle pour maison connectée (Smart Home), vous devez concevoir un système capable de recevoir et de traiter des commandes pour des ampoules intelligentes.

Une commande d'ampoule peut prendre l'une des formes suivantes : 1. Allumer (sans paramètre). 2. Éteindre (sans paramètre). 3. Régler la luminosité (un pourcentage représenté par un entier `u8` de 0 à 100). 4. Changer la couleur (une couleur au format RGB représentée par trois valeurs `u8` : rouge, vert, bleu).

Objectifs :

1. Définissez l'énumération `CommandeAmpoule` modélisant ces quatre états.
2. Implémentez une fonction `executer_commande(commande: CommandeAmpoule)` qui utilise un bloc `match` pour afficher l'action en cours de réalisation (par exemple : "Luminosité réglée à 75%").

3. Afin de détecter rapidement les tentatives de pollution lumineuse nocturne, écrivez une fonction `intercepter_lumiere_rouge(commande: &CommandeAmpoule)` en utilisant la syntaxe concise `if let`. Cette fonction doit afficher un message d'alerte spécifique uniquement si la commande consiste à changer la couleur avec le canal rouge au maximum (`255`), indépendamment des valeurs de vert et de bleu.
-

Indices

1. Pour la variante de luminosité, utilisez un tuple ou un champ nommé contenant un `u8`.
 2. Pour la fonction `intercepter_lumiere_rouge`, vous devez passer une référence `&CommandeAmpoule` afin de ne pas consommer (déplacer l'ownership de) la commande.
 3. Dans le motif du `if let`, vous pouvez utiliser le caractère de substitution `_` pour ignorer les valeurs de vert et de bleu qui ne vous intéressent pas.
-

Correction Détaillée

```

// 1. Définition de l'énumération avec données associées
#[derive(Debug, Clone)]
enum CommandeAmpoule {
    Allumer,
    Eteindre,
    ReglerLuminosite(u8),          // Tuple variant
    ChangerCouleur(u8, u8, u8),    // Tuple variant (Rouge, Vert, Bleu)
}

// 2. Traitement exhaustif avec match
fn executer_commande(commande: CommandeAmpoule) {
    match commande {
        CommandeAmpoule::Allumer => {
            println!("Action : Allumage de l'ampoule.");
        }
        CommandeAmpoule::Eteindre => {
            println!("Action : Extinction de l'ampoule.");
        }
        CommandeAmpoule::ReglerLuminosite(pourcentage) => {
            // Optionnel : validation simple de la donnée
            if pourcentage > 100 {
                println!("Erreur : Luminosité impossible ({}% demandés, max 100%).", pourcentage);
            } else {
                println!("Action : Luminosité réglée à {}%.", pourcentage);
            }
        }
        CommandeAmpoule::ChangerCouleur(r, v, b) => {
            println!("Action : Couleur modifiée en RGB({}, {}, {})", r, v, b);
        }
    }
}

// 3. Interception ciblée avec if let et référence
fn interceptor_lumiere_rouge(commande: &CommandeAmpoule) {
    // On filtre uniquement sur la variante ChangerCouleur avec r = 255
    if let CommandeAmpoule::ChangerCouleur(255, _, _) = commande {
        println!("Alerte : Commande de lumière rouge maximale détectée !");
    }
}

fn main() {
    let cmd_on = CommandeAmpoule::Allumer;
    let cmd_dim = CommandeAmpoule::ReglerLuminosite(85);
    let cmd_rouge = CommandeAmpoule::ChangerCouleur(255, 0, 120);
    let cmd_bleu = CommandeAmpoule::ChangerCouleur(0, 0, 255);

    println!("--- Exécution des commandes ---");
    executer_commande(cmd_on);
}

```

```
executer_commande(cmd_dim);

println!("\n--- Phase d'interception ---");
intercepter_lumiere_rouge(&cmd_rouge); // Doit déclencher l'alerte
intercepter_lumiere_rouge(&cmd_bleu);  // Ne doit rien afficher
}
```

Exercice 2 : Système de Validation de Paiements (Niveau Moyen)

Énoncé

Une plateforme de commerce électronique accepte trois types de paiement : 1. **CarteBancaire** : contient un numéro de carte (`String`) et le solde disponible (`f64`). 2. **Crypto** : contient une adresse de portefeuille (`String`) et un booléen indiquant si la signature de transaction est valide (`bool`). 3. **CarteCadeau** : contient un code unique (`String`) et le solde restant (`f64`).

Vous devez concevoir un module de validation robuste qui vérifie si un paiement peut être validé pour un montant d'achat donné.

Objectifs :

1. Créez l'énumération `MoyenPaiement` .
2. Créez une énumération de sortie `StatutPaiement` ayant deux variantes :
3. `Accepte` (contenant le moyen de paiement mis à jour après déduction du montant).
4. `Refuse` (contenant une explication textuelle `String`).
5. Implémentez la fonction `valider_paiement(moyen: MoyenPaiement, montant: f64) → StatutPaiement` .
6. Règles de gestion (Gardes de filtrage obligatoires) :
 - Pour une `CarteBancaire` : Le paiement est accepté uniquement si le solde est supérieur ou égal au montant de l'achat.

- Pour la **Crypto** : Le paiement est accepté uniquement si la signature est valide (considérez que le portefeuille a toujours les fonds nécessaires pour simplifier).
 - Pour une **CarteCadeau** : Le paiement est accepté uniquement si le solde est supérieur ou égal au montant.
 - Tout autre cas doit être explicitement refusé avec un message clair (sans utiliser de wildcard `_` global afin de conserver la sécurité d'exhaustivité).
-

Indices

1. Utilisez des *Match Guards* (`if`) directement dans vos branches de `match` pour valider les soldes et les signatures.
 2. Pour mettre à jour le solde d'une carte bancaire ou d'une carte cadeau, vous devez reconstruire une nouvelle instance de la variante avec le solde diminué du montant, puis l'encapsuler dans `StatutPaiement::Accepte` .
 3. Évitez l'utilisation d'un unique bras `_ => ...` à la fin du `match`. Épelez les cas de refus (ex: solde insuffisant) pour chaque variante afin de garantir que si une nouvelle variante est ajoutée au code futur, le compilateur vous force à la traiter.
-

Correction Détaillée

```

#[derive(Debug, Clone)]
enum MoyenPaiement {
    CarteBancaire { numero: String, solde: f64 },
    Crypto { adresse: String, signature_valide: bool },
    CarteCadeau { code: String, solde: f64 },
}

#[derive(Debug)]
enum StatutPaiement {
    Accepte(MoyenPaiement),
    Refuse(String),
}

fn valider_paiement(moyen: MoyenPaiement, montant: f64) → StatutPaiement {
    match moyen {
        // Cas 1 : Carte Bancaire avec solde suffisant
        MoyenPaiement::CarteBancaire { numero, solde } if solde ≥ montant ⇒ {
            let nouveau_moyen = MoyenPaiement::CarteBancaire {
                numero,
                solde: solde - montant,
            };
            StatutPaiement::Accepte(nouveau_moyen)
        }
        // Cas 2 : Carte Bancaire avec solde insuffisant
        MoyenPaiement::CarteBancaire { numero: _, solde } ⇒ {
            StatutPaiement::Refuse(format!(
                "Refusé : Solde insuffisant sur la carte (disponible: {:.2}, requis: {:.2})",
                solde, montant
            ))
        }
        // Cas 3 : Crypto avec signature valide
        MoyenPaiement::Crypto { adresse, signature_valide: true } ⇒ {
            // On considère la transaction validée
            StatutPaiement::Accepte(MoyenPaiement::Crypto { adresse, signature_valide: true })
        }
        // Cas 4 : Crypto avec signature invalide
        MoyenPaiement::Crypto { adresse: _, signature_valide: false } ⇒ {
            StatutPaiement::Refuse(String::from("Refusé : Signature de transaction invalide"))
        }
        // Cas 5 : Carte Cadeau avec solde suffisant
        MoyenPaiement::CarteCadeau { code, solde } if solde ≥ montant ⇒ {
            let nouveau_moyen = MoyenPaiement::CarteCadeau {
                code,
                solde: solde - montant,
            };
            StatutPaiement::Accepte(nouveau_moyen)
        }
    }
}

```

```

    }
    // Cas 6 : Carte Cadeau avec solde insuffisant
    MoyenPaielement::CarteCadeau { code: _, solde } => {
        StatutPaielement::Refuse(format!(
            "Refusé : Solde de carte cadeau insuffisant ( {:.2} restant ",
            solde
        ))
    }
}

}

fn main() {
    let cb = MoyenPaielement::CarteBancaire {
        numero: String::from("4970-XXXX-XXXX-1234"),
        solde: 150.0,
    };

    let crypto_invalide = MoyenPaielement::Crypto {
        adresse: String::from("0x71C...397"),
        signature_valide: false,
    };

    println!("--- Tentative 1 : Achat de 50.00€ via Carte Bancaire ---");
    match valider_paiement(cb, 50.0) {
        StatutPaielement::Accepte(nouveau_moyen) => {
            println!("Paielement accepté ! Nouveau statut du compte : {:?}" , nouveau_moyen);
        }
        StatutPaielement::Refuse(err) => {
            println!("Échec : {}", err);
        }
    }

    println!("\n--- Tentative 2 : Achat de 10.00€ via Crypto non signée ---");
    match valider_paiement(crypto_invalide, 10.0) {
        StatutPaielement::Accepte(_) => println!("Succès inattendu !"),
        StatutPaielement::Refuse(err) => println!("Sécurité OK. Raison du refus : {}", err),
    }
}

```

Exercice 3 : Optimisation Mémoire d'un Système de Fichiers Virtuel (Niveau Difficile)

Énoncé

Dans un système d'exploitation embarqué, vous devez modéliser les nœuds d'un système de fichiers virtuel. Un nœud peut être : 1. Un `LienRapide` (qui contient simplement un identifiant numérique `u32`). 2. Un `FichierTexte` (qui contient le chemin sous forme de `String` et le contenu textuel brut). 3. Une `ImageBrute` (qui contient le chemin sous forme de `String` et une matrice brute de pixels représentée par un tableau fixe `[u8; 1024]`).

Problématique mémoire :

Si vous déclarez cette énumération de manière naïve, chaque instance de nœud (même un simple `LienRapide` de 4 octets) occupera plus de 1 Ko sur la pile (Stack) à cause de la taille de la variante `ImageBrute`.

Objectifs :

1. Déclarez une énumération naïve nommée `NoeudNaif` et observez sa taille en mémoire à l'aide de la fonction `std::mem::size_of`.
2. Concevez une version optimisée nommée `NoeudOptimise` en appliquant le principe d'indirection (en utilisant un pointeur intelligent `Box`) pour déporter la charge utile volumineuse de l'image sur le tas (Heap).
3. Implémentez une fonction `afficher_resume(noeud: &NoeudOptimise)` qui extrait et affiche le chemin ou l'identifiant du nœud, peu importe sa variante.

Indices

1. Pour mesurer la taille d'un type en octets : `std::mem::size_of::<MonType>()`.
2. Pour déplacer une donnée sur le tas, utilisez `Box::new(donnee)`. La taille d'un `Box` sur la pile est équivalente à la taille d'un pointeur (8 octets sur une architecture 64-bits).

3. Dans la variante optimisée, au lieu d'avoir `ImageBrute { chemin: String, pixels: [u8; 1024] }`, vous pouvez regrouper ces données dans une structure dédiée `DonneesImage` et stocker cette structure sous la forme `Box<DonneesImage>`.
-

Correction Détaillée

```

use std::mem::size_of;

// --- 1. Approche Naïve (Gourmande en mémoire) ---
#[allow(dead_code)]
enum NoeudNaif {
    LienRapide(u32),
    FichierTexte { chemin: String, contenu: String },
    ImageBrute { chemin: String, pixels: [u8; 1024] }, // Cette variante
}

// --- 2. Approche Optimisée (Utilisation de Box) ---

// Nous extrayons les données volumineuses dans une structure dédiée
#[derive(Debug)]
struct DonneesImage {
    chemin: String,
    pixels: [u8; 1024],
}

#[derive(Debug)]
enum NoeudOptimise {
    LienRapide(u32),
    FichierTexte { chemin: String, contenu: String },
    // L'image est maintenant stockée sur le tas (Heap).
    // La variante ne stocke plus qu'un pointeur Box (8 octets sur la Sta
    ImageBrute(Box<DonneesImage>),
}

// --- 3. Implémentation de la fonction d'affichage ---
fn afficher_resume(noeud: &NoeudOptimise) {
    match noeud {
        NoeudOptimise::LienRapide(id) => {
            println!("Type : Lien Rapide | ID : {}", id);
        }
        NoeudOptimise::FichierTexte { chemin, contenu } => {
            println!("Type : Fichier Texte | Chemin : {} (Taille : {} caractères)", che
        }
        // Destructuration à travers le Box
        NoeudOptimise::ImageBrute(donnees_image) => {
            println!(
                "Type : Image Brute | Chemin : {} (Taille de l'image : {} octets)",
                donnees_image.chemin,
                donnees_image.pixels.len()
            );
        }
    }
}

```



```

fn main() {
    // Comparaison des tailles en mémoire
    let taille_naive = size_of::<NoeudNaif>();
    let taille_optimisee = size_of::<NoeudOptimise>();

    println!("--- Analyse de l'empreinte mémoire ---");
    println!("Taille de NoeudNaif (sur la pile)      : {} octets", taille_naive);
    println!("Taille de NoeudOptimise (sur la pile) : {} octets", taille_optimisee);
    println!("Gain d'espace sur la pile                : {} %", (100 - (taille_naive - taille_optimisee) * 100 / taille_naive));

    println!("\n--- Utilisation du Noeud Optimisé ---");

    // Création d'un nœud image optimisé
    let image_sur_tas = Box::new(DonneesImage {
        chemin: String::from("/images/system_logo.raw"),
        pixels: [0u8; 1024], // Simulation de pixels vides
    });

    let noeud_image = NoeudOptimise::ImageBrute(image_sur_tas);
    let noeud_lien = NoeudOptimise::LienRapide(42);

    afficher_resume(&noeud_image);
    afficher_resume(&noeud_lien);
}

```

Analyse des résultats de l'exercice 3 :

- NoeudNaif pèse généralement autour de 1056 octets sur la pile. Cela s'explique par la taille de la variante `ImageBrute` qui doit pouvoir contenir le tableau de 1024 octets ainsi que la structure `String` (24 octets) et le tag de l'énumération.
- NoeudOptimise ne pèse plus que 32 octets sur la pile. Grâce à l'utilisation de `Box`, la charge utile de 1 Ko a été déplacée sur le tas. La pile reste propre, légère et rapide d'accès, évitant ainsi tout risque de dépassement de pile (*stack overflow*) lors de la manipulation de listes de nœuds.

Les Collections Standards (Vector, String) et leur gestion dynamique de la mémoire

1. Introduction Conceptuelle

Dans les chapitres précédents, nous avons exploré les types de données de base ainsi que les tableaux à taille fixe (`[T; N]` , étudiés au Module 1.2). Bien que ces structures soient extrêmement performantes grâce à leur allocation sur la Pile (*Stack*), elles souffrent d'une limitation majeure : leur taille doit être impérativement connue dès la compilation et ne peut plus jamais varier durant l'exécution du programme.

Or, la réalité du développement logiciel impose de manipuler des données dont le volume est par nature imprévisible : * Un fichier texte dont on ignore le nombre de lignes avant de l'ouvrir. * Les saisies successives d'un utilisateur au clavier. * Des paquets réseau reçus de manière asynchrone.

Pour répondre à ce besoin, Rust propose des collections dynamiques. Contrairement aux tableaux statiques, ces collections stockent leurs éléments sur le Tas (*Heap*, étudié au Module 1.3). Cela leur permet de croître ou de rétrécir dynamiquement au cours de l'exécution.

Le défi historique de cette flexibilité réside dans la gestion de la mémoire. Dans des langages comme le C, l'allocation dynamique manuelle (`malloc` / `free`) est une source intarissable de failles de sécurité (doubles libérations, fuites de mémoire, pointeurs ballants). Dans d'autres langages, un ramasse-miettes (*Garbage Collector*) gère ce fardeau au prix de pauses d'exécution imprévisibles et d'une surconsommation de ressources.

Rust résout cette équation grâce à son système d'Ownership (Propriété) (étudié au Module 2.1). Les collections standards comme `Vec<T>` et `String` agissent comme des gestionnaires sécurisés de la mémoire du Tas. Elles appliquent le principe de l'acquisition de ressources à l'initialisation (RAII) : dès que la collection sort de la portée (*scope*), la

mémoire allouée sur le Tas est automatiquement et instantanément libérée, sans aucun coût d'exécution caché.

2. Fondations Théoriques

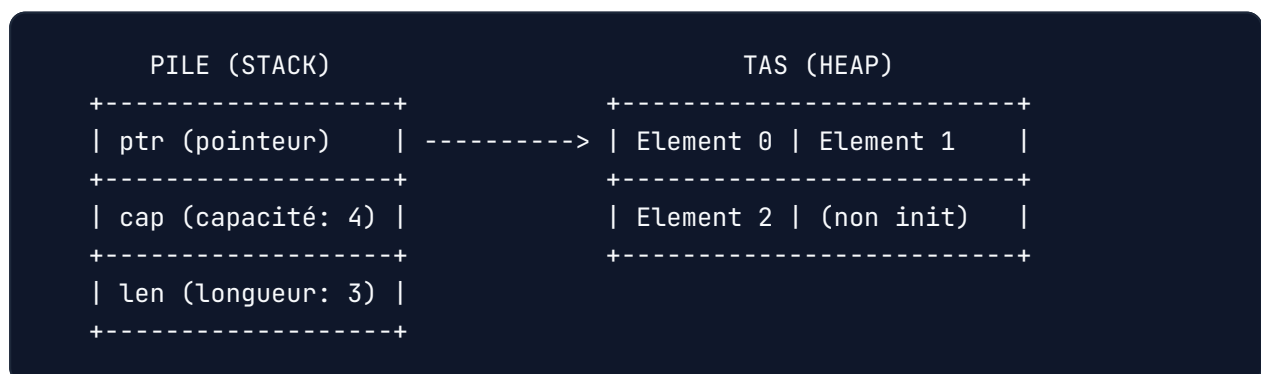
Pour comprendre l'efficacité et la sécurité des collections dynamiques en Rust, il est indispensable d'analyser leur structure interne sous le capot.

2.1 Anatomie d'un Vecteur (`Vec<T>`)

Un vecteur `Vec<T>` est une structure de données de taille fixe stockée sur la Pile, qui pointe vers un espace contigu de mémoire alloué sur le Tas.

Sur la Pile, un `Vec<T>` est représenté par exactement trois mots machine (soit 24 octets sur une architecture 64 bits) :

1. Le Pointeur (`ptr`) : L'adresse mémoire du premier élément stocké sur le Tas.
2. La Capacité (`cap`) : Le nombre total d'éléments que l'espace actuellement alloué sur le Tas peut contenir sans nécessiter de réallocation.
3. La Longueur (`len`) : Le nombre d'éléments actuellement initialisés et présents dans le vecteur.



2.2 Le Mécanisme de Réallocation

Que se passe-t-il lorsque l'on ajoute un élément à un vecteur alors que sa longueur est égale à sa capacité (`len == cap`) ?

1. **Allocation d'un nouveau bloc** : Rust demande à l'allocateur de mémoire un nouveau bloc sur le Tas, généralement deux fois plus grand que le précédent (stratégie de croissance géométrique).
2. **Copie/Déplacement** : Les éléments existants sont déplacés vers le nouvel emplacement.
3. **Désallocation** : L'ancien bloc de mémoire sur le Tas est libéré.
4. **Mise à jour des métadonnées** : Le pointeur du vecteur est mis à jour vers la nouvelle adresse, la capacité est doublée, et la longueur est incrémentée.

Cette stratégie de doublement garantit que l'insertion d'un élément s'effectue en un temps amorti constant, soit $\mathcal{O}(1)$.

2.3 Le Cas Particulier de `String`

En Rust, une chaîne de caractères `String` n'est rien d'autre qu'une enveloppe de protection (*wrapper*) autour d'un vecteur d'octets de type `Vecu8`.

La seule différence conceptuelle réside dans une garantie fondamentale : un `String` doit toujours contenir une séquence UTF-8 valide. Rust applique cette vérification de manière stricte lors de la création ou de la modification d'un `String`.

2.4 Sécurité Mémoire et Réallocation

Le système d'Ownership de Rust empêche un bug classique des langages système : le pointeur ballant post-réallocation.

Si un programme conserve une référence vers un élément situé à l'intérieur d'un vecteur, et que ce vecteur subit une réallocation, l'ancienne mémoire est libérée. La référence pointerait alors vers de la mémoire invalide. En Rust, le Borrow Checker (étudié au Module 2.2) détecte et interdit formellement cette situation à la compilation en empêchant toute modification du vecteur tant qu'une référence sur l'un de ses éléments est active.

3. Implémentation Pratique Pas-à-Pas

3.1 Manipulation des Vecteurs (`Vec<T>`)

Voici comment créer, modifier et inspecter des vecteurs de manière idiomatique.

```
fn main() {
    // 1. Création d'un vecteur vide. Rust exige la spécification du type si aucun élément
    let mut nombres: Vec<i32> = Vec::new();

    // 2. Ajout d'éléments (mutation)
    nombres.push(10);
    nombres.push(20);
    nombres.push(30);

    println!("Vecteur après ajouts : {:?}", nombres);
    println!("Longueur : {}, Capacité : {}", nombres.len(), nombres.capacity());

    // 3. Utilisation de la macro de commodité vec!
    let mut notes = vec![15, 18, 12];

    // 4. Retrait du dernier élément
    // pop() renvoie une Option<T> (étudiée en détail au Module 4.2)
    if let Some(derniere_note) = notes.pop() {
        println!("Note retirée : {}", derniere_note);
    }

    // 5. Accès sécurisé vs Accès direct
    // Accès direct par index (risque de panique si l'index est hors-limites)
    let premiere_note = notes[0];
    println!("Première note (direct) : {}", premiere_note);

    // Accès sécurisé via .get() qui renvoie une Option<&T>
    match notes.get(5) {
        Some(note) => println!("Note à l'index 5 : {}", note),
        None => println!("Aucune note à l'index 5 (Accès sécurisé sans plantage)"),
    }
}
```

3.2 Manipulation des Chaînes de Caractères (`String`)

Comprendre la distinction entre `String` (alloué sur le Tas, modifiable) et `&str` (tranche de chaîne de caractères, souvent stockée dans le binaire ou pointant vers un `String`, étudiée au Module 2.3).

```
fn main() {
    // 1. Création d'un String à partir d'une littérale de chaîne (&str)
    let mut message = String::from("Bonjour");

    // Alternative idiomatique :
    let mut message_alt = "Bonjour".to_string();

    // 2. Modification de la chaîne
    message.push(' ');          // Ajoute un seul caractère (char)
    message.push_str("à tous"); // Ajoute une tranche de chaîne (&str)

    println!("Message : {}", message);

    // 3. Pourquoi l'indexation directe s'avère impossible ?
    // Le code suivant NE compilera PAS :
    // let lettre = message[0];
    //
    // Explication : UTF-8 utilise une largeur variable (1 à 4 octets par caractère).
    // Indexer par octet pourrait couper un caractère Unicode en deux, produisant un état i

    // Comment parcourir correctement un String ?
    // Option A : Parcourir les caractères Unicode (sûr et logique)
    print!("Caractères : ");
    for c in message.chars() {
        print!("[{}] ", c);
    }
    println();

    // Option B : Parcourir les octets bruts (représentation sous-jacente)
    print!("Octets : ");
    for b in message.bytes() {
        print!("{:X} ", b);
    }
    println();
}
```

4. Pièges Fréquents et Bonnes Pratiques

4.1 L'Invalidation d'Itérateur (Le piège de la mutation concomitante)

C'est l'un des bugs les plus redoutables en C++. En Rust, le compilateur l'élimine purement et simplement.

Code incorrect (refusé par le compilateur) :

```
fn main() {
    let mut collection = vec![1, 2, 3];

    // On crée une référence de lecture (emprunt immuable) via l'itérateur
    for &valeur in &collection {
        if valeur == 2 {
            // TENTATIVE DE MUTATION : push requiert un emprunt mutable (&mut)
            collection.push(4);
        }
    }
}
```

Message d'erreur du compilateur :

```
error[E0502]: cannot borrow `collection` as mutable because it is also borrowed as immutable
   → src/main.rs:7:13
   |
5  |     for &valeur in &collection {
   |     ~~~~~
   |           |
   |           immutable borrow occurs here
   |           immutable borrow later used here
6  |         if valeur == 2 {
7  |             collection.push(4);
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

Pourquoi ce refus ? Si `push` déclenchait une réallocation mémoire, l'adresse de `collection` changerait sur le Tas. L'itérateur en cours d'exécution pointerait alors vers une zone mémoire libérée, provoquant un comportement indéfini (*Undefined Behavior*).

4.2 Optimisation des performances : L'allocation préventive

Chaque réallocation dynamique a un coût CPU (allocation, copie, libération). Si vous connaissez à l'avance le nombre approximatif d'éléments à stocker, utilisez `with_capacity`.

```
// Mauvaise pratique : Provoque de multiples réallocations sur le Tas
let mut v_lent = Vec::new();
for i in 0..1000 {
    v_lent.push(i); // Va réallouer environ 10 fois en interne
}

// Bonne pratique : Une seule allocation initiale sur le Tas
let mut v_rapide = Vec::with_capacity(1000);
for i in 0..1000 {
    v_rapide.push(i); // Zéro réallocation !
}
```

5. Synthèse Pédagogique

Collection	Emplacement des Métadonnées	Emplacement des Données	Croissance Dynamique	Garantie Clé
<code>Vec<T></code>	Pile (24 octets : <code>ptr</code> , <code>cap</code> , <code>len</code>)	Tas (contigu)	Oui (<code>push</code> / <code>pop</code>)	Accès contigu en mémoire ($\mathcal{O}(1)$ par index).
<code>String</code>	Pile (24 octets : <code>ptr</code> , <code>cap</code> , <code>len</code>)	Tas (contigu)	Oui (<code>push_str</code>)	Encodage UTF-8 valide garanti à tout instant.
<code>&[T]</code> (Slice)	Pile (16 octets : <code>ptr</code> , <code>len</code>)	Pile ou Tas	Non (Taille fixe)	Vue immuable ou mutable sur une portion contiguë.
<code>&str</code> (Slice)	Pile (16 octets : <code>ptr</code> , <code>len</code>)	Pile, Tas ou Segment Code	Non (Taille fixe)	Vue sur une séquence UTF-8 valide.

Points clés à retenir :

1. RAII automatique : Pas besoin de libérer manuellement la mémoire d'un `Vec` ou d'un `String` . Rust s'en charge dès que la variable sort de sa portée.
2. Capacité vs Longueur : La capacité est l'espace réservé ; la longueur est l'espace réellement utilisé.
3. Pas d'indexation directe sur `String` : L'indexation directe par entier est interdite pour éviter les erreurs de découpage UTF-8. Il faut utiliser `.chars()` ou `.bytes()` .

6. Exercice Pratique d'Application

Énoncé : Le Purificateur de Logs

Vous devez concevoir un utilitaire système chargé de nettoyer et de filtrer des fichiers de logs bruts. Chaque ligne de log est représentée par un `String`.

Objectifs :

1. Écrire une fonction `purifier_logs(logs_bruts: Vec<String>) → Vec<String>` qui :
 2. Ignore les lignes vides ou ne contenant que des espaces.
 3. Supprime les espaces superflus au début et à la fin de chaque ligne valide.
 4. Ne conserve que les logs qui commencent par la balise `"[ERROR]"` ou `"[WARNING]"`.
 5. Optimiser l'allocation mémoire de la collection de sortie en évitant les réallocations inutiles.
-

Indices pour la résolution

1. Pour nettoyer les espaces au début et à la fin d'une chaîne, utilisez la méthode `.trim()` disponible sur les tranches de chaînes.
 2. Pour vérifier le démarrage d'une chaîne, utilisez `.starts_with("motif")`.
 3. Pour optimiser la mémoire, initialisez le vecteur de sortie avec une capacité maximale égale à la taille du vecteur d'entrée.
-

Correction Complète et Commentée

Voici l'implémentation complète, robuste et optimisée de l'exercice.

```

/// Purifie une liste de logs bruts selon des critères de sécurité et de formatage.
fn purifier_logs(logs_bruts: Vec<String>) → Vec<String> {
    // Étape d'optimisation : Le vecteur de sortie aura au maximum la taille du vecteur d'entrée.
    // On pré-alloue donc cette capacité pour éviter toute réallocation sur le Tas.
    let mut logs_purifies = Vec::with_capacity(logs_bruts.len());

    for log in logs_bruts {
        // 1. Nettoyage des espaces blancs au début et à la fin.
        // .trim() renvoie une tranche de chaîne (&str) pointant à l'intérieur de 'log'.
        let log_nettoye: &str = log.trim();

        // 2. Vérification de vacuité
        if log_nettoye.is_empty() {
            continue; // On ignore les lignes vides
        }

        // 3. Filtrage par balises de sécurité
        if log_nettoye.starts_with("[ERROR]") || log_nettoye.starts_with("[WARNING]") {
            // On convertit la tranche &str nettoyée en un nouveau String alloué sur le Tas.
            logs_purifies.push(log_nettoye.to_string());
        }
    }

    // Optionnel : Libère la capacité excédentaire non utilisée sur le Tas
    logs_purifies.shrink_to_fit();

    logs_purifies
}

fn main() {
    // Simulation d'une entrée de logs bruts
    let entrees_brutes = vec![
        String::from(" [ERROR] Connexion perdue avec la base de données "),
        String::from(" "),
        String::from("[INFO] Démarrage du serveur sur le port 8080"),
        String::from("[WARNING] Utilisation CPU élevée à 85%"),
        String::from(" [ERROR] Échec d'authentification de l'utilisateur admin"),
    ];

    println!("--- Début de la purification des logs ---");

    let resultats = purifier_logs(entrees_brutes);

    // Affichage des résultats purifiés
    for (index, log) in resultats.iter().enumerate() {
        println!("Log épuré n°{} : {:?}", index + 1, log);
    }
}

```

```
// Vérification de l'optimisation de la capacité
println!("\nStatistiques mémoire du vecteur final :");
println!("Longueur : {}, Capacité : {}", resultats.len(), resultats.capacity());
}
```


Exercices d'Application : Les Collections Standards (Vector, String) et leur gestion dynamique de la mémoire

Ce cahier d'exercices pratiques a pour objectif de consolider vos connaissances sur la gestion de la mémoire dynamique en Rust, en mettant l'accent sur les vecteurs (`Vec<T>`), les chaînes de caractères (`String`), et les règles de sécurité imposées par le *Borrow Checker*.

Exercice 1 : L'Analyseur de Capacité et d'Accès Sécurisé

Énoncé

Dans cet exercice, vous allez concevoir un système de suivi de métriques pour un capteur de température. L'objectif est de comprendre précisément comment Rust gère la capacité mémoire d'un vecteur sur le Tas et comment accéder aux données de manière hautement sécurisée.

1. Créez un vecteur vide d'entiers (`i32`) nommé `temperatures` .
2. Ajoutez des valeurs de température une par une (par exemple : `21` , `22` , `20` , `24` , `25`). À chaque ajout, affichez :
3. La valeur ajoutée.
4. La longueur (`len`) du vecteur.
5. La capacité (`capacity`) du vecteur.

6. Écrivez une fonction sécurisée `obtenir_temperature(vec: &[i32], index: usize)` qui tente d'accéder à un élément. Si l'index existe, elle affiche la température. S'il n'existe pas, elle affiche un message d'erreur amical au lieu de faire planter (paniquer) le programme.
7. Optimisez la mémoire du vecteur après les insertions en utilisant la méthode appropriée pour libérer la capacité excédentaire, puis affichez la nouvelle capacité.

Indices

1. Utilisez les méthodes `.len()` et `.capacity()` sur votre vecteur.
2. Pour l'accès sécurisé, n'utilisez pas l'indexation directe `vec[index]` qui peut provoquer un crash (*panic*). Utilisez plutôt la méthode `.get()`, qui renvoie une `Option<T>`.
3. Pour libérer la mémoire inutilisée (lorsque la capacité est strictement supérieure à la longueur), documentez-vous sur la méthode `.shrink_to_fit()`.

Correction Détaillée

```
fn obtenir_temperature(vec: &[i32], index: usize) {
    // Utilisation de .get() pour un accès sécurisé qui retourne une Option
    match vec.get(index) {
        Some(temp) => println!("Température à l'index {} : {}°C", index, temp),
        None => println!("Erreur : Aucun enregistrement trouvé à l'index {}", index),
    }
}

fn main() {
    // 1. Création d'un vecteur vide
    let mut temperatures: Vec<i32> = Vec::new();
    println!("--- Phase d'insertion et observation de la mémoire ---");
    println!("Initiale → Longueur : {}, Capacité : {}\n", temperatures.len(), temperatures.capacity());

    let mesures = vec![21, 22, 20, 24, 25];

    // 2. Ajout dynamique et observation de la stratégie de doublement de capacité
    for temp in mesures {
        temperatures.push(temp);
        println!(
            "Ajout de {}°C → Longueur : {}, Capacité : {}",
            temp,
            temperatures.len(),
            temperatures.capacity()
        );
    }

    println!("\n--- Accès sécurisé aux données ---");
    // 3. Tests d'accès sécurisé
    obtenir_temperature(&temperatures, 2); // Index valide
    obtenir_temperature(&temperatures, 10); // Index hors-limites (ne planche pas !)

    println!("\n--- Optimisation de la mémoire ---");
    // 4. Libération de la capacité excédentaire
    println!("Avant shrink → Longueur : {}, Capacité : {}", temperatures.len(), temperatures.capacity());
    temperatures.shrink_to_fit();
    println!("Après shrink → Longueur : {}, Capacité : {}", temperatures.len(), temperatures.capacity());
}
```


Exercice 2 : Le Censeur Unicode (Manipulation de

`String`)

Énoncé

En Rust, manipuler des chaînes de caractères nécessite une grande rigueur en raison de l'encodage UTF-8. Cet exercice vous demande de créer un filtre de modération de texte (un "censeur") capable de traiter correctement les caractères multi-octets (comme les émojis ou les lettres accentuées).

1. Écrivez une fonction `analyser_chaine(texte: &str)` qui affiche :
2. Le nombre d'octets occupés par la chaîne en mémoire.
3. Le nombre réel de caractères Unicode.
4. Écrivez une fonction `censurer_caractere(texte: &str, cible: char, remplacement: char) → String` qui parcourt la chaîne caractère par caractère et remplace chaque occurrence du caractère `cible` par le caractère `remplacement` .
5. Testez votre programme avec une chaîne contenant des caractères spéciaux et des émojis (par exemple : "Café chaud ! ☕ et Crêpe 🍷").

Indices

1. La méthode `.len()` sur un `&str` ou un `String` renvoie la taille en octets, pas en caractères. Pour obtenir le nombre de caractères Unicode, utilisez `.chars().count()` .
2. Pour reconstruire une chaîne de caractères en remplaçant des éléments, vous pouvez initialiser un `String` vide avec `String::new()` (ou `String::with_capacity()`), boucler sur `.chars()` , et utiliser `.push(caractere)` .

Correction Détaillée

```
fn analyser_chaine(texte: &str) {
    let octets = texte.len();
    let caracteres = texte.chars().count();

    println!("Analyse de : \"{}\"", texte);
    println!(" - Taille en mémoire : {} octets", octets);
    println!(" - Nombre de caractères Unicode : {}", caracteres);
}

fn censurer_caractere(texte: &str, cible: char, remplacement: char) → String {
    // Optimisation : On pré-alloue de la mémoire.
    // Bien que la taille finale en octets puisse varier si les caractères cible/remplacement
    // n'ont pas la même taille UTF-8, la taille d'origine est une excellente estimation
    let mut resultat = String::with_capacity(texte.len());

    // Parcours sécurisé des caractères Unicode (et non des octets bruts)
    for c in texte.chars() {
        if c == cible {
            resultat.push(remplacement);
        } else {
            resultat.push(c);
        }
    }

    resultat
}

fn main() {
    let phrase = "Café chaud ! ☹️ et Crêpe 🍷";

    println!("--- 1. Analyse de la structure UTF-8 ---");
    analyser_chaine(phrase);
    // Notez que 'é' prend 2 octets, '☹️' prend 3 octets et '🍷' prend 4 octets en UTF-8

    println!("\n--- 2. Censure de caractères ---");
    // Remplacement du 'é' par un 'e' simple
    let phrase_censuree_1 = censurer_caractere(phrase, 'é', 'e');
    println!("Résultat 1 : {}", phrase_censuree_1);

    // Remplacement de l'emoji '☹️' par un emoji d'avertissement '⚠️'
    let phrase_censuree_2 = censurer_caractere(phrase, '☹️', '⚠️');
    println!("Résultat 2 : {}", phrase_censuree_2);
}
```

Exercice 3 : Le Duplicateur d'Alertes (Résoudre l'Invalidation d'Itérateur)

Énoncé

Vous travaillez sur un système de monitoring réseau. Vous disposez d'une liste de messages système stockés dans un `Vec<String>`. Vous souhaitez inspecter cette liste et, chaque fois que vous rencontrez un message contenant le mot `"CRITICAL"`, vous devez immédiatement insérer une copie de sécurité de ce message juste après dans le vecteur.

Le code naïf suivant ne compile pas à cause des règles d'Ownership et du problème d'invalidation d'itérateur :

```
// CE CODE NE COMPILE PAS
fn dupliquer_alertes_naif(alertes: &mut Vec<String>) {
    for alerte in alertes.iter() {
        if alerte.contains("CRITICAL") {
            // Erreur de compilation : impossible de modifier le vecteur
            // pendant qu'on l'emprunte pour l'itérer.
            alertes.push(alerte.clone());
        }
    }
}
```

Votre mission : 1. Expliquez pourquoi le compilateur refuse ce code (en lien avec la gestion de la mémoire et la réallocation). 2. Proposez une solution propre, compilable et optimisée en termes d'allocations mémoire en créant un nouveau vecteur de sortie.

Indices

1. Pourquoi le refus ? Si Rust autorisait l'ajout d'éléments (`push`) pendant l'itération, le vecteur pourrait dépasser sa capacité actuelle. Cela déclencherait une réallocation sur le Tas (déplacement des données vers une nouvelle adresse mémoire). L'itérateur pointerait alors vers l'ancienne zone mémoire libérée, provoquant un comportement indéfini (*Undefined Behavior*).

2. Pour la solution : Au lieu de modifier le vecteur en place pendant son parcours, créez un nouveau vecteur vide. Utilisez `Vec::with_capacity` pour limiter les réallocations mémoire lors du transfert et de la duplication des messages.

Correction Détaillée

1. Explication théorique

Le compilateur Rust applique strictement la règle : on ne peut pas modifier une collection (emprunt mutable) pendant qu'on la parcourt (emprunt immuable). Si le vecteur `alertes` devait se réallouer en mémoire sur le Tas lors du `push`, l'adresse mémoire de ses éléments changerait. L'itérateur interne, qui conserve un pointeur vers l'ancienne adresse, deviendrait un pointeur ballant (*dangling pointer*). Rust élimine ce risque dès la compilation.

2. Code corrigé et optimisé

```

/// Analyse les alertes et duplique celles contenant "CRITICAL" de manière sécurisée
fn dupliquer_alertes(alertes_brutes: Vec<String>) → Vec<String> {
    // Optimisation : On s'attend à ce que le vecteur final soit plus grand que l'original
    // On alloue une capacité initiale légèrement supérieure (ex: 1.5 fois la taille d'origine)
    // pour éviter de multiples réallocations sur le Tas.
    let capacite_estimee = (alertes_brutes.len() as f64 * 1.5) as usize;
    let mut alertes_traitees = Vec::with_capacity(capacite_estimee);

    // Consommation du vecteur d'entrée (parcours par transfert de propriété / ownership)
    for alerte in alertes_brutes {
        // Détection du mot clé
        if alerte.contains("CRITICAL") {
            // On clone l'alerte pour pouvoir l'insérer deux fois
            alertes_traitees.push(alerte.clone());
            alertes_traitees.push(alerte); // Pas besoin de cloner la deuxième fois (on a déjà la référence)
        } else {
            alertes_traitees.push(alerte);
        }
    }

    // Libération de la mémoire allouée en trop si le nombre de "CRITICAL" était faible
    alertes_traitees.shrink_to_fit();
    alertes_traitees
}

fn main() {
    let alertes_systeme = vec![
        String::from("[INFO] Système démarré"),
        String::from("[CRITICAL] Température CPU trop élevée !"),
        String::from("[WARNING] Espace disque faible (10%)"),
        String::from("[CRITICAL] Fuite de mémoire détectée dans le service web"),
    ];

    println!("--- Avant traitement ---");
    for alerte in &alertes_systeme {
        println!("{}", alerte);
    }

    // Appel de notre fonction de duplication sécurisée
    let alertes_finales = dupliquer_alertes(alertes_systeme);

    println!("\n--- Après traitement (Duplication des alertes critiques)");
    for alerte in &alertes_finales {
        println!("{}", alerte);
    }

    println!("\nStatistiques mémoire finales : {} éléments, {} de capacité",
        alertes_finales.len(),

```

```
        alertes_finales.capacity()  
    );  
}
```

MODULE 4

La Gestion des Erreurs et la Robustesse du Code

Erreurs irrécupérables (panic!) vs Erreurs récupérables (le type Result)

Dans les modules précédents, nous avons exploré comment Rust garantit la sécurité mémoire au niveau matériel grâce à l'ownership (Module 2) et comment structurer des données complexes (Module 3). Cependant, un logiciel robuste ne se contente pas de gérer correctement sa mémoire en situation nominale : il doit également réagir de manière prévisible et sécurisée face à l'imprévu.

La gestion des erreurs est l'un des piliers majeurs de la philosophie de Rust. Contrairement à de nombreux langages de programmation qui reposent sur un mécanisme unique d'exceptions, Rust sépare fondamentalement les erreurs en deux catégories distinctes : les erreurs irrécupérables et les erreurs récupérables.

Ce chapitre détaille cette dualité conceptuelle, analyse les mécanismes internes de ces deux approches et fournit les clés méthodologiques pour concevoir des applications hautement résilientes.

1. Introduction Conceptuelle

Le problème des exceptions traditionnelles

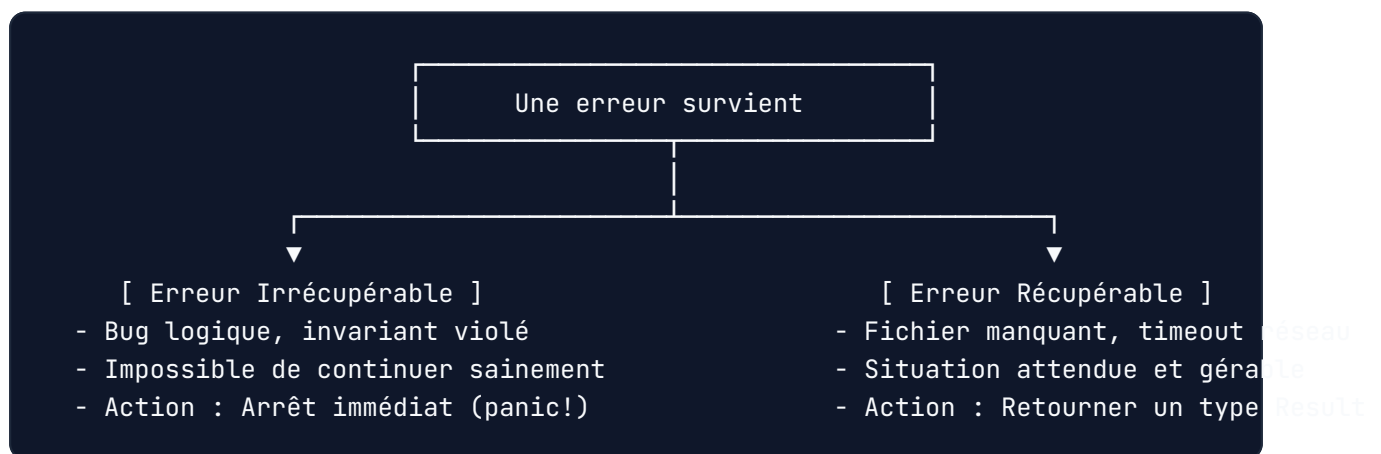
Dans des langages comme Java, C++ ou Python, la gestion des erreurs repose principalement sur la levée et la capture d'exceptions (`try-catch` ou `try-except`). Bien que populaire, cette approche présente plusieurs inconvénients majeurs du point de vue de la sécurité et de la clarté du code :

1. **Invisibilité à la signature des fonctions** : Rien dans la signature d'une fonction classique ne force un développeur à savoir qu'une exception peut être levée, à moins de lire la documentation ou le code source de la fonction appelée.

2. **Rupture invisible du flux d'exécution** : Une exception remonte la pile d'appels de manière invisible, ce qui rend difficile le raisonnement sur l'état des variables et peut introduire des bugs subtils si des ressources ne sont pas correctement libérées.
3. **Surcharge cognitive et oublis** : Il est facile d'oublier de capturer une exception, ce qui mène à des plantages inattendus en production.

La philosophie de Rust : L'explicitabilité avant tout

Rust fait un choix radicalement différent. Il n'existe pas d'exceptions traditionnelles. À la place, Rust sépare les erreurs selon leur nature :



- **Les erreurs irrécupérables** : Ce sont des situations qui traduisent un bug logique dans le programme (ex. accès hors des limites d'un tableau, division par zéro). Le programme est dans un état corrompu et ne peut pas continuer à s'exécuter en toute sécurité. La seule réponse saine est l'arrêt immédiat du programme via le mécanisme de `panic!`.
- **Les erreurs récupérables** : Ce sont des situations où l'échec est une issue normale et prévisible de l'opération (ex. un fichier introuvable lors d'une ouverture, une déconnexion réseau temporaire). Le programme doit être capable de traiter cet échec et de poursuivre son exécution. Rust utilise ici un type de retour explicite : l'énumération `Result<T, E>`.

2. Fondations Théoriques

2.1 Le mécanisme de `panic!`

Lorsqu'une erreur irrécupérable se produit, Rust déclenche une panique (`panic`). Ce processus effectue plusieurs actions critiques pour préserver l'intégrité du système.

Unwinding (Dépilage) vs Aborting (Arrêt brutal)

Par défaut, lorsqu'une panique se produit, Rust commence l'`unwinding` (le dépilage). Le programme remonte la pile d'exécution (stack) et libère la mémoire de chaque fonction en appelant les destructeurs (`drop`) de toutes les variables locales encore actives. Cela garantit qu'aucune ressource (fichiers, sockets, mémoire du tas) n'est laissée dans un état instable.

Cependant, ce dépilage a un coût en taille de binaire et en performance. Dans les systèmes embarqués ou les applications à haute performance, il est possible de configurer Rust pour qu'il effectue un `abort` (arrêt immédiat). Le programme s'arrête alors instantanément sans nettoyer la pile, laissant le système d'exploitation récupérer les ressources.

Pour configurer l'arrêt immédiat en cas de panique, on modifie le fichier `Cargo.toml` :

```
[profile.release]
panic = "abort"
```

Les Backtraces

Pour diagnostiquer l'origine d'une panique, Rust permet d'afficher la pile d'appels (backtrace). Par défaut, pour des raisons de performance, les backtraces sont désactivées. On les active en définissant la variable d'environnement `RUST_BACKTRACE=1` dans le terminal avant de lancer l'exécutable.

2.2 L'énumération `Result<T, E>`

Pour les erreurs récupérables, Rust utilise la puissance de son système de types, et plus particulièrement des énumérations (étudiées au Module 3.2). Le type `Result` est défini dans la bibliothèque standard de la manière suivante :

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

Ce type utilise la généricité (qui sera approfondie dans les modules ultérieurs) : * `T` représente le type de la valeur retournée en cas de succès (`Ok`). * `E` représente le type de l'erreur retournée en cas d'échec (`Err`).

Pourquoi ce système est-il ultra-sécurisé ?

Contrairement aux langages où une fonction peut renvoyer une valeur ou lever une exception en secret, une fonction Rust qui peut échouer doit retourner un type `Result` .

Le compilateur Rust applique une règle stricte : vous ne pouvez pas accéder à la valeur de type `T` sans inspecter explicitement le `Result` . Si vous tentez d'ignorer le résultat d'une fonction retournant un `Result` , le compilateur générera un avertissement (*warning*) très sévère (`unused_must_use`). Cela élimine par construction le risque d'ignorer accidentellement une erreur.

3. Implémentation Pratique Pas-à-Pas

3.1 Déclencher et analyser une panique (`panic!`)

Commençons par observer le comportement d'une panique provoquée manuellement ou par une erreur d'exécution.

```
fn verifier_age(age: i32) {
    if age < 0 {
        // Déclenchement explicite d'une panique
        panic!("L'âge ne peut pas être négatif ! Valeur reçue : {}", age);
    }
    println!("Âge valide : {}", age);
}

fn main() {
    println!("Début du programme.");
    verifier_age(-5);
    println!("Cette ligne ne sera jamais exécutée.");
}
```

Sortie attendue dans la console :

```
Début du programme.
thread 'main' panicked at 'L'âge ne peut pas être négatif ! Valeur reçue : -5', src/main.rs
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

3.2 Manipuler `Result` avec le Pattern Matching

Le moyen le plus fondamental et le plus sûr de traiter un `Result` est d'utiliser le filtrage par motif (`match`), que nous avons étudié au Module 3.2.

Voici un exemple complet simulant la division de deux nombres réels, où la division par zéro est traitée comme une erreur récupérable.

```

// Définition d'un type d'erreur personnalisé sous forme d'énumération
#[derive(Debug, PartialEq)]
enum ErreurMathematique {
    DivisionParZero,
}

// La fonction retourne un Result.
// En cas de succès, elle contient un f64. En cas d'erreur, elle contient notre énumération
fn diviser(numerateur: f64, denominateur: f64) → Result<f64, ErreurMathematique> {
    if denominateur == 0.0 {
        Err(ErreurMathematique::DivisionParZero)
    } else {
        Ok(numerateur / denominateur)
    }
}

fn main() {
    let cas_succes = diviser(10.0, 2.0);
    let cas_erreur = diviser(5.0, 0.0);

    // Traitement du cas de succès
    match cas_succes {
        Ok(valeur) ⇒ println!("Succès ! Le résultat est : {}", valeur),
        Err(e) ⇒ println!("Erreur rencontrée : {:?}", e),
    }

    // Traitement du cas d'erreur
    match cas_erreur {
        Ok(valeur) ⇒ println!("Succès ! Le résultat est : {}", valeur),
        Err(ErreurMathematique::DivisionParZero) ⇒ {
            println!("Erreur : Impossible de diviser par zéro !");
        }
    }
}

```

3.3 Les méthodes utilitaires de `Result`

Écrire un bloc `match` complet peut parfois s'avérer verbeux. La bibliothèque standard de Rust fournit plusieurs méthodes pratiques pour manipuler les `Result` de manière plus concise.

1. `unwrap()` et `expect()` : Les raccourcis risqués

Ces méthodes extraient la valeur contenue dans `Ok`. Cependant, si le résultat est un `Err`, le programme panique immédiatement.

```
let resultat_ok: Result<i32, &str> = Ok(42);
let valeur = resultat_ok.unwrap(); // Retourne 42

let resultat_err: Result<i32, &str> = Err("Une erreur critique");
// resultat_err.unwrap(); // ! FERA PANIQUER LE PROGRAMME !

// expect() permet de personnaliser le message de panique
// resultat_err.expect("Le chargement de la configuration a échoué");
```

2. `unwrap_or()` : La valeur par défaut sécurisée

Cette méthode permet de fournir une valeur de repli en cas d'erreur, évitant ainsi toute panique.

```
let port_serveur: Result<u16, &str> = Err("Port invalide");
// Si erreur, on utilise le port par défaut 8080
let port = port_serveur.unwrap_or(8080);
println!("Connexion sur le port : {}", port); // Affiche 8080
```

3. `and_then()` : Enchaîner les opérations (Monades)

`and_then` permet de passer le contenu d'un `Ok` à une autre fonction qui retourne elle-même un `Result`. Si le premier `Result` est un `Err`, l'enchaînement s'arrête et l'erreur est propagée.

```

fn verifier_format(entree: &str) → Result<i32, &str> {
    entree.parse::<i32>().map_err(|_| "Ce n'est pas un nombre valide")
}

fn verifier_positif(nombre: i32) → Result<i32, &str> {
    if nombre ≥ 0 {
        Ok(nombre)
    } else {
        Err("Le nombre doit être positif")
    }
}

fn main() {
    let entree_utilisateur = "42";

    // Enchaînement des vérifications
    let resultat: Result<i32, &str> = verifier_format(entree_utilisateur)
        .and_then(verifier_positif);

    match resultat {
        Ok(n) ⇒ println!("Nombre validé : {}", n),
        Err(msg) ⇒ println!("Validation échouée : {}", msg),
    }
}

```

Note sur la propagation d'erreurs : Rust dispose également d'un opérateur extrêmement puissant, l'opérateur point d'interrogation (?), dédié à la propagation simplifiée des erreurs. Ce mécanisme fondamental fera l'objet d'une étude exhaustive et dédiée dans le Module 4.3.

4. Pièges Fréquents et Bonnes Pratiques

4.1 L'abus de `unwrap()` (L'anti-pattern absolu)

L'utilisation systématique de `.unwrap()` est l'erreur la plus fréquente chez les débutants en Rust. Elle réintroduit le risque de plantages soudains à l'exécution, annulant ainsi les garanties de robustesse offertes par le langage.

Règle d'or : * N'utilisez `unwrap()` que dans les tests unitaires, les prototypes rapides, ou lorsque vous pouvez prouver mathématiquement au compilateur que l'erreur est impossible (ex. parser une chaîne de caractères codée en dur dont vous êtes absolument sûr du format). * Dans le code de production, préférez toujours `match` , `unwrap_or()` , ou la propagation d'erreurs.

4.2 Quand utiliser `panic!` plutôt que `Result` ?

Situation	Choix recommandé	Justification
Erreur de saisie utilisateur	<code>Result</code>	L'utilisateur fait des erreurs fréquemment. Le programme doit lui demander de corriger sa saisie sans planter.
Échec de connexion réseau	<code>Result</code>	Le réseau est intrinsèquement instable. C'est un événement externe attendu.
Violation d'un invariant interne	<code>panic!</code>	Si l'état interne de votre structure de données est corrompu, continuer l'exécution pourrait corrompre la mémoire ou corrompre la base de données.
Échec d'un test unitaire	<code>panic!</code>	Un test doit échouer de manière visible et immédiate pour signaler un bug.
Ressource système critique manquante	<code>panic!</code> ou <code>Result</code>	Si l'application ne peut absolument pas démarrer sans un fichier de configuration central, une panique propre au démarrage est acceptable.

4.3 Impact sur les performances

Contrairement aux exceptions dans d'autres langages qui nécessitent la création d'une pile d'appels complexe et coûteuse dès qu'elles sont levées, le type `Result` de Rust n'a aucun coût caché.

Lorsqu'une fonction retourne un `Result` , elle retourne simplement une valeur d'énumération contenant soit la donnée, soit l'erreur. Si vous n'utilisez pas le mécanisme de

panique, la gestion des erreurs en Rust est aussi rapide qu'un simple retour de structure en C.

5. Synthèse Pédagogique

Caractéristique	Erreur Irrécupérable (<code>panic!</code>)	Erreur Récupérable (<code>Result<T, E></code>)
Concept	Bug logique, état impossible à gérer.	Événement attendu qui peut échouer.
Mécanisme technique	Dépilage de la pile (unwinding) ou arrêt immédiat (abort).	Retour d'une énumération (<code>Ok</code> ou <code>Err</code>).
Impact sur le programme	Arrêt immédiat du thread ou du processus.	Le programme continue, l'appelant gère l'erreur.
Outils de manipulation	Macro <code>panic!</code> , <code>unwrap()</code> , <code>expect()</code> .	<code>match</code> , <code>unwrap_or()</code> , combinateurs (<code>and_then</code>).
Usage type	Indice hors limites, division par zéro, assertions de test.	Fichier introuvable, erreur de parsing, timeout.

6. Exercice Pratique d'Application

Énoncé : Le Système de Contrôle de Température d'un Réacteur

Vous devez concevoir le logiciel de surveillance d'un réacteur chimique. Le système lit des chaînes de caractères brutes provenant de capteurs thermiques.

Une chaîne de capteur valide respecte le format suivant : `"ID_CAPTEUR:TEMPERATURE"` .
Exemple : `"sensor_01:45.2"` ou `"sensor_02:-12.5"` .

Vous devez écrire une fonction de traitement qui analyse cette chaîne et applique les règles de sécurité suivantes :

1. **Erreur de format (Récupérable)** : Si la chaîne ne contient pas de séparateur `:` ou si la température n'est pas un nombre valide, la fonction doit retourner une erreur de format.
2. **Température hors limites physiques (Récupérable)** : La température mesurée ne peut pas être inférieure au zéro absolu (-273.15°C). Si c'est le cas, la fonction doit retourner une erreur de valeur incohérente.
3. **Erreur critique de sécurité (Irrécupérable)** : Si l'identifiant du capteur est `"CRITICAL_FAIL"`, cela signifie que le matériel est en train de fondre. Le système doit immédiatement paniquer pour couper les vannes d'alimentation du réacteur.

Squelette de départ

```
#[derive(Debug, PartialEq)]
enum ErreurCapteur {
    FormatInvalide,
    TemperatureIncoherente,
}

struct Lecture {
    id: String,
    temperature: f64,
}

// À implémenter
fn analyser_lecture(donnee_brute: &str) → Result<Lecture, ErreurCapteur> {
    // Votre code ici
    todo!()
}
```

Indices pour la résolution

1. Utilisez la méthode `split_once(':')` disponible sur les chaînes de caractères (`&str`) pour séparer l'identifiant de la température.
2. Pour convertir la température de `&str` vers `f64`, utilisez la méthode `.parse::<f64>()`. Cette méthode retourne un `Result`, que vous pouvez inspecter ou convertir.

3. Utilisez le filtrage par motif (`match`) pour valider l'identifiant avant de faire d'autres vérifications.
-

Correction Complète et Commentée

Voici l'implémentation robuste et conforme aux standards de production de Rust :

```

#[derive(Debug, PartialEq)]
enum ErreurCapteur {
    FormatInvalide,
    TemperatureIncoherente,
}

#[derive(Debug, PartialEq)]
struct Lecture {
    id: String,
    temperature: f64,
}

/// Analyse la lecture brute d'un capteur et applique les règles de sécurité.
fn analyser_lecture(donnee_brute: &str) → Result<Lecture, ErreurCapteur> {
    // 1. Découpage de la chaîne sur le caractère ':'
    let (id_str, temp_str) = match donnee_brute.split_once(':') {
        Some((id, temp)) ⇒ (id, temp),
        None ⇒ return Err(ErreurCapteur::FormatInvalide),
    };

    // 2. Vérification de l'erreur critique de sécurité (Matériel en panne)
    if id_str == "CRITICAL_FAIL" {
        panic!("URGENCE ABSOLUE : Défaillance critique du capteur détectée ! Arrêt d'urgence");
    }

    // 3. Conversion de la température en f64
    let temp_valeur: f64 = match temp_str.trim().parse::<f64>() {
        Ok(valeur) ⇒ valeur,
        Err(_) ⇒ return Err(ErreurCapteur::FormatInvalide),
    };

    // 4. Validation physique de la température (Zéro absolu)
    if temp_valeur < -273.15 {
        return Err(ErreurCapteur::TemperatureIncoherente);
    }

    // 5. Tout est correct, on retourne la structure encapsulée dans Ok
    Ok(Lecture {
        id: id_str.to_string(),
        temperature: temp_valeur,
    })
}

fn main() {
    println!("--- Début des tests du système de surveillance ---");

    // Test 1 : Cas nominal de succès
    let lecture_valide = "sensor_01:23.8";

```

```

match analyser_lecture(lecture_valide) {
    Ok(lecture) => println!("Succès - Capteur: {}, Température: {}°C", lecture.id, lecture.temperature);
    Err(e) => println!("Échec inattendu : {:?}" , e),
}

// Test 2 : Erreur de format (pas de ':')
let lecture_incorrecte = "sensor_02_sans_separateur";
assert_eq!(analyser_lecture(lecture_incorrecte), Err(ErreurCapteur::FormatInvalide));
println!("Test Erreur de Format : Réussi (Erreur détectée proprement)");

// Test 3 : Erreur de température incohérente (sous le zéro absolu)
let lecture_impossible = "sensor_03:-300.0";
assert_eq!(analyser_lecture(lecture_impossible), Err(ErreurCapteur::TemperatureIncohérente));
println!("Test Température Incohérente : Réussi (Erreur détectée proprement)");

// Test 4 : Erreur de parsing de la température
let lecture_lettres = "sensor_04:chaud";
assert_eq!(analyser_lecture(lecture_lettres), Err(ErreurCapteur::FormatInvalide));
println!("Test Parsing Température : Réussi (Erreur détectée proprement)");

// Test 5 : Cas critique (Déclenchement de la panique)
println!("\nTentative de traitement d'un signal critique (devrait paniquer)");
let signal_critique = "CRITICAL_FAIL:999.9";
analyser_lecture(signal_critique);

// Cette ligne ne sera jamais atteinte
println!("Ce message ne doit pas s'afficher.");
}

```


Exercices d'Application : Erreurs irrécupérables (panic!) vs Erreurs récupérables (le type Result)

Ce cahier d'exercices pratiques a pour objectif de consolider vos compétences sur la gestion des erreurs en Rust. Vous progresserez à travers trois exercices de difficulté croissante : de la manipulation simple de `Result` et des valeurs par défaut, jusqu'à la mise en place d'une architecture robuste alternant intelligemment entre `panic!` et `Result`.

Exercice 1 : Le Configurateur de Serveur (unwrap_or et Result)

Niveau : Facile

Énoncé

Dans cet exercice, vous devez concevoir un module de configuration pour un serveur web. Le programme doit lire une chaîne de caractères représentant la configuration du port d'écoute.

Le format attendu est strictement `"PORT=valeur"`.

Vous devez écrire une fonction `extraire_port` qui analyse cette chaîne et gère les cas d'erreurs suivants de manière récupérable : 1. **Format invalide** : La chaîne ne commence pas par `"PORT="` ou ne contient pas de valeur après le signe `=`. 2. **Port invalide** : La valeur fournie n'est pas un entier valide ou dépasse les limites d'un port réseau standard (les ports valides vont de `1` à `65535`).

Dans votre fonction `main`, vous simulerez la lecture de plusieurs configurations et utiliserez la méthode `unwrap_or` pour garantir que le serveur démarre toujours sur le port par défaut `8080` en cas d'erreur de configuration, sans jamais faire planter l'application.

Squelette de départ

```
#[derive(Debug, PartialEq)]
enum ErreurConfig {
    FormatInvalide,
    PortInvalide,
}

fn extraire_port(config: &str) → Result<u16, ErreurConfig> {
    // À implémenter
    todo!()
}

fn main() {
    let configs_a_tester = vec![
        "PORT=443",
        "PORT=80a",
        "PORT=70000",
        "IP=127.0.0.1",
        "PORT=",
    ];

    let port_par_defaut = 8080;

    for config in configs_a_tester {
        // Appeler extraire_port et utiliser unwrap_or pour obtenir un port valide
        // Afficher le port final qui sera utilisé par le serveur
    }
}
```

Indices

1. Pour vérifier si une chaîne commence par un motif et extraire le reste, vous pouvez utiliser la méthode `strip_prefix("PORT=")` qui renvoie un `Option<&str>`.

2. Pour convertir la chaîne extraite en nombre entier, utilisez `.parse::<u32>()` (ou directement `u16`). Si le parsing échoue ou si la valeur convertie sort de l'intervalle `[1, 65535]` , retournez une erreur appropriée.
-

Correction Détaillée

```

#[derive(Debug, PartialEq)]
enum ErreurConfig {
    FormatInvalide,
    PortInvalide,
}

/// Analyse une chaîne de configuration pour en extraire le port réseau.
fn extraire_port(config: &str) → Result<u16, ErreurConfig> {
    // 1. Extraction du préfixe "PORT="
    let valeur_str = match config.strip_prefix("PORT=") {
        Some(valeur) ⇒ valeur,
        None ⇒ return Err(ErreurConfig::FormatInvalide),
    };

    // Si la valeur après "PORT=" est vide
    if valeur_str.is_empty() {
        return Err(ErreurConfig::FormatInvalide);
    }

    // 2. Parsing de la valeur en entier (u32 pour éviter le débordement
    let port_u32: u32 = match valeur_str.parse::<u32>() {
        Ok(valeur) ⇒ valeur,
        Err(_) ⇒ return Err(ErreurConfig::PortInvalide),
    };

    // 3. Validation de la plage du port (1 - 65535)
    if port_u32 ≥ 1 && port_u32 ≤ 65535 {
        Ok(port_u32 as u16)
    } else {
        Err(ErreurConfig::PortInvalide)
    }
}

fn main() {
    let configs_a_tester = vec![
        "PORT=443",
        "PORT=80a",
        "PORT=70000",
        "IP=127.0.0.1",
        "PORT=",
    ];

    let port_par_defaut = 8080;

    println!("--- Initialisation du Serveur ---");
    for config in configs_a_tester {
        let resultat = extraire_port(config);
    }
}

```

```

        // Utilisation de unwrap_or pour appliquer la valeur de repli
        let port_final = resultat.unwrap_or(port_par_defaut);

        println!(
            "Entrée: {:<15} → Résultat brut: {:<25} → Port appliqué: {}
            config,
            format!("{:?}", resultat),
            port_final
        );
    }
}

```

Exercice 2 : Validation de Formulaire d'Inscription (Enchaînement avec `and_then`)

Niveau : Moyen

Énoncé

Vous développez le système d'inscription d'une plateforme en ligne. Avant d'enregistrer un utilisateur en base de données, vous devez valider ses informations à travers une série de filtres.

Vous devez modéliser ce processus à l'aide de fonctions retournant des `Result` et les enchaîner de manière fluide grâce à la méthode `and_then` .

Voici les règles de validation : 1. Validation du Pseudo : Le pseudo ne doit pas être vide et doit faire moins de 10 caractères. 2. Validation de l'Email : L'email doit impérativement contenir le caractère `@` .

Vous devez implémenter : * Une structure `Utilisateur` contenant les champs `pseudo` et `email` . * Les fonctions de validation individuelles. * Une fonction globale `creer_utilisateur(pseudo: &str, email: &str) → Result<Utilisateur, &'static str>` qui enchaîne ces validations. Si toutes les étapes réussissent, elle retourne l'utilisateur créé.

Squelette de départ

```
#[derive(Debug, PartialEq)]
struct Utilisateur {
    pseudo: String,
    email: String,
}

fn valider_pseudo(pseudo: &str) → Result<&str, &'static str> {
    // À implémenter
    todo!()
}

fn valider_email(email: &str) → Result<&str, &'static str> {
    // À implémenter
    todo!()
}

fn creer_utilisateur(pseudo: &str, email: &str) → Result<Utilisateur, &'static str> {
    // À implémenter en utilisant `and_then`
    todo!()
}

fn main() {
    let test_1 = creer_utilisateur("Alice", "alice@rust.org");
    let test_2 = creer_utilisateur("", "bob@rust.org");
    let test_3 = creer_utilisateur("PseudoTropLong", "test@test.com");
    let test_4 = creer_utilisateur("Charlie", "charlie_at_rust.org");

    println!("Test 1 (Valide) : {:?}", test_1);
    println!("Test 2 (Pseudo vide) : {:?}", test_2);
    println!("Test 3 (Pseudo long) : {:?}", test_3);
    println!("Test 4 (Email invalide) : {:?}", test_4);
}
```

Indices

1. La méthode `and_then` prend une fermeture (closure) en paramètre. Elle s'applique sur un `Result` et, si ce dernier est `Ok(valeur)`, elle passe cette `valeur` à la fonction suivante.
2. Pour `creer_utilisateur`, vous pouvez valider le pseudo, enchaîner avec `and_then` pour valider l'email, et enfin construire la structure `Utilisateur` dans un bloc `Ok` final.

Correction Détaillée

```

#[derive(Debug, PartialEq)]
struct Utilisateur {
    pseudo: String,
    email: String,
}

/// Valide la longueur du pseudonyme.
fn valider_pseudo(pseudo: &str) → Result<&str, &'static str> {
    if pseudo.is_empty() {
        Err("Le pseudo ne peut pas être vide")
    } else if pseudo.len() > 10 {
        Err("Le pseudo est trop long (max 10 caractères)")
    } else {
        Ok(pseudo)
    }
}

/// Valide la présence d'un caractère '@' dans l'email.
fn valider_email(email: &str) → Result<&str, &'static str> {
    if email.contains('@') {
        Ok(email)
    } else {
        Err("L'adresse email est invalide (manque '@')")
    }
}

/// Enchaîne les validations et construit l'utilisateur si tout est valide
fn creer_utilisateur(pseudo: &str, email: &str) → Result<Utilisateur, &'static str> {
    // Enchaînement fonctionnel avec and_then
    valider_pseudo(pseudo).and_then(|pseudo_valide| {
        valider_email(email).map(|email_valide| {
            Utilisateur {
                pseudo: pseudo_valide.to_string(),
                email: email_valide.to_string(),
            }
        })
    })
}

fn main() {
    let test_1 = creer_utilisateur("Alice", "alice@rust.org");
    let test_2 = creer_utilisateur("", "bob@rust.org");
    let test_3 = creer_utilisateur("PseudoTropLong", "test@test.com");
    let test_4 = creer_utilisateur("Charlie", "charlie_at_rust.org");

    // Assertions pour valider le comportement du code
    assert!(test_1.is_ok());
    assert_eq!(test_2, Err("Le pseudo ne peut pas être vide"));
}

```



```
assert_eq!(test_3, Err("Le pseudo est trop long (max 10 caractères)"));
assert_eq!(test_4, Err("L'adresse email est invalide (manque '@')"));

println!("Tous les tests de validation ont réussi !");
println!("Utilisateur créé avec succès : {:?}", test_1.unwrap());
}
```

Exercice 3 : Le Système Bancaire (panic! vs Result)

Niveau : Difficile / Synthèse

Énoncé

Vous devez concevoir le moteur d'exécution des transactions d'une banque. Ce système doit faire la distinction absolue entre :

- * Les erreurs opérationnelles (récupérables) : Un client tente de retirer plus d'argent qu'il n'en possède, ou demande un retrait négatif. Ces actions doivent retourner un `Result::Err` pour que l'application puisse en informer l'utilisateur sans s'arrêter.
- * Les violations d'invariants système (irré récupérables) : Le solde d'un compte devient mathématiquement impossible (valeur `NaN` - Not a Number), ou l'identifiant interne de la banque est corrompu (chaîne vide). Ces situations indiquent un bug logique majeur ou une corruption mémoire. Le système doit immédiatement déclencher un `panic!` pour empêcher toute corruption de la base de données globale.

Spécifications de l'implémentation

1. Créez une structure `CompteBancaire` avec un `id (String)` et un `solde (f64)`.
2. Implémentez la méthode `retirer(&mut self, montant: f64) → Result<f64, &'static str>` :
3. Si le montant est négatif ou nul, retourner une erreur.
4. Si le solde est insuffisant, retourner une erreur.
5. Si le solde actuel ou le montant est `NaN` (utilisez la méthode `.is_nan()`), déclenchez un `panic!`.

6. Sinon, déduisez le montant et retournez le nouveau solde dans `Ok` .
7. Implémentez une méthode de maintenance `auditer_coherence(&self)` :
8. Si l'identifiant `id` est vide ou si le solde est `NaN` , déclenchez un `panic!` avec un message explicite.

Squelette de départ

```
struct CompteBancaire {
    id: String,
    solde: f64,
}

impl CompteBancaire {
    fn nouveau(id: &str, solde_initial: f64) → Self {
        // Pensez à vérifier la cohérence dès la création !
        todo!()
    }

    fn retirer(&mut self, montant: f64) → Result<f64, &'static str> {
        todo!()
    }

    fn auditer_coherence(&self) {
        todo!()
    }
}

fn main() {
    // 1. Test des cas récupérables
    let mut compte = CompteBancaire::nouveau("FR76-1234", 500.0);

    // Tenter de retirer trop d'argent
    // Tenter de retirer un montant négatif

    // 2. Test d'un cas irrécupérable (décommenter pour observer la panique)
    // let mut compte_corrompu = CompteBancaire::nouveau("", 100.0); // D
```

paniquer

Indices

1. Pour tester si un nombre flottant est invalide (`NaN`), utilisez la méthode intégrée de Rust : `valeur.is_nan()` .

2. Rappelez-vous la règle d'or : un solde insuffisant est une erreur utilisateur normale (`Result`), tandis qu'un identifiant vide ou une valeur `NaN` est une anomalie système qui ne devrait jamais se produire dans un code correct (`panic!`).
-

Correction Détaillée

```

#[derive(Debug)]
struct CompteBancaire {
    id: String,
    solde: f64,
}

impl CompteBancaire {
    /// Crée un nouveau compte bancaire avec validation stricte des invariants système.
    fn nouveau(id: &str, solde_initial: f64) → Self {
        if id.trim().is_empty() {
            panic!("ERREUR SYSTÈME : Tentative de création d'un compte avec un identifiant vide.");
        }
        if solde_initial.is_nan() {
            panic!("ERREUR SYSTÈME : Le solde initial ne peut pas être NaN.");
        }

        CompteBancaire {
            id: id.to_string(),
            solde: solde_initial,
        }
    }

    /// Effectue un retrait. Retourne une erreur récupérable si l'opération est invalide,
    /// mais panique si le système est dans un état incohérent.
    fn retirer(&mut self, montant: f64) → Result<f64, &'static str> {
        // 1. Détection des corruptions système (Irrécupérable)
        if self.solde.is_nan() || montant.is_nan() {
            panic!(
                "CRITICAL: Valeur NaN détectée lors du retrait sur le compte {} (ID: {}, Solde: {})",
                self.id, self.solde, montant
            );
        }

        // 2. Détection des erreurs d'utilisation (Récupérable)
        if montant ≤ 0.0 {
            return Err("Le montant du retrait doit être strictement positif.");
        }

        if montant > self.solde {
            return Err("Solde insuffisant pour effectuer cette opération.");
        }

        // 3. Application de l'état nominal
        self.solde -= montant;
        Ok(self.solde)
    }

    /// Vérifie la validité des invariants de la structure.

```

```

    /// Doit être appelé périodiquement ou lors des audits de sécurité.
    fn auditer_coherence(&self) {
        if self.id.is_empty() {
            panic!("CORRUPTION DONNÉES : Identifiant de compte manquant !");
        }
        if self.solde.is_nan() {
            panic!("CORRUPTION DONNÉES : Solde corrompu (NaN) sur le compte {} !", self.id);
        }
    }
}

fn main() {
    println!("--- Initialisation du système bancaire ---");

    // Création d'un compte valide
    let mut compte = CompteBancaire::nouveau("FR-89234", 1000.0);
    println!("Compte créé avec succès : {:?}", compte);

    // --- CAS RÉCUPÉRABLES ---
    println!("\n--- Phase de test des opérations clients ---");

    // Retrait excessif
    match compte.retirer(1500.0) {
        Ok(nouveau_solde) => println!("Retrait réussi ! Nouveau solde : {:?}", nouveau_solde),
        Err(e) => println!("Refus opérationnel (Attendu) : {:?}", e),
    }

    // Retrait négatif
    match compte.retirer(-50.0) {
        Ok(nouveau_solde) => println!("Retrait réussi ! Nouveau solde : {:?}", nouveau_solde),
        Err(e) => println!("Refus opérationnel (Attendu) : {:?}", e),
    }

    // Retrait valide
    match compte.retirer(200.0) {
        Ok(nouveau_solde) => println!("Retrait réussi (Nominal) ! Nouveau solde : {:?}", nouveau_solde),
        Err(e) => println!("Échec inattendu : {:?}", e),
    }

    // Audit de routine
    compte.auditer_coherence();
    println!("Audit de cohérence : OK.");

    // --- CAS IRRÉCUPÉRABLES (PANIC) ---
    // Pour tester la panique système, décommentez l'une des lignes ci-dessous :

    println!("\n--- Déclenchement d'une panique de sécurité ---");

    // Déclencheur 1 : Création d'un compte invalide

```

```
// let _compte_anonyme = CompteBancaire::nouveau("", 500.0);

// Déclencheur 2 : Injection d'une valeur corrompue (NaN)
let montant_corrompu = f64::NaN;
let _ = compte.retirer(montant_corrompu);
}
```

L'absence de valeur : Dire adieu au pointeur nul avec le type Option

1. Introduction Conceptuelle

Le problème historique : L'erreur à un milliard de dollars

En 1965, l'informaticien britannique Tony Hoare a introduit la notion de référence nulle (`null`) dans le langage ALGOL W. Son objectif était simple : représenter l'absence de valeur. Quarante ans plus tard, lors d'une conférence en 2009, il a publiquement qualifié cette invention de "mon erreur à un milliard de dollars".

Dans la plupart des langages de programmation traditionnels (C, C++, Java, C#), une variable contenant une référence peut pointer vers une adresse mémoire valide, ou vers une adresse invalide appelée `null` (ou `nullptr` , `nil` , `None`).

Le problème fondamental de cette approche réside dans l'absence de distinction, au niveau du système de types, entre une référence qui pointe toujours vers une donnée valide et une référence qui peut être nulle. Le compilateur autorise l'accès aux membres de n'importe quelle référence. Si le développeur oublie de vérifier manuellement si la référence est nulle avant de l'utiliser (déréférencement), le programme lève une exception à l'exécution (comme la célèbre `NullPointerException` en Java) ou, pire encore en C/C++, provoque un comportement indéfini (*Undefined Behavior*) ou un plantage brutal (*Segmentation Fault*).

```
En C/Java/Python :  
[ Variable ] —> Peut pointer vers [ Donnée Réelle ] OU [ NULL ]  
Le compilateur vous laisse appeler des méthodes dessus sans vérification.  
Résultat : Plantage à l'exécution si la valeur est NULL.
```


La philosophie de Rust : Rendre l'absence explicite

Rust fait un choix radical et extrêmement sécurisant : le pointeur nul n'existe pas dans le code sûr (safe code). Une variable de type `String` contient toujours une chaîne de caractères valide. Une variable de type `i32` contient toujours un entier valide.

Pour représenter l'absence de valeur, Rust oblige le développeur à utiliser un type enveloppe (*wrapper*) explicite : le type énuméré `Option<T>`.

Si une valeur peut être absente, son type n'est plus `T`, mais `Option<T>`. Le compilateur refuse alors catégoriquement toute tentative d'utilisation directe de la valeur interne sans qu'une vérification explicite et exhaustive n'ait été effectuée au préalable. Le risque d'oublier de traiter le cas "vide" est ainsi réduit à zéro dès l'étape de compilation.

2. Fondations Théoriques

Définition de l'énumération `Option<T>`

Le type `Option<T>` est une énumération définie dans la bibliothèque standard de Rust. Sa structure est conceptuellement très simple :

```
pub enum Option<T> {  
    None,      // Représente l'absence de valeur  
    Some(T),   // Représente la présence d'une valeur de type T  
}
```

Note : Le symbole `<T>` représente un paramètre de type générique. Cela signifie que vous pouvez avoir une `Option<i32>`, une `Option<String>`, ou une `Option<User>`. Les concepts de généricité seront approfondis dans les modules suivants, mais retenez ici que `T` représente n'importe quel type de données.

Grâce au prélude de Rust (le code automatiquement importé dans chaque fichier), vous n'avez pas besoin d'importer explicitement `Option`, ni même d'écrire `Option::Some` ou `Option::None`. Vous pouvez directement utiliser les variantes `Some` et `None`.

Représentation en mémoire et Optimisation du Pointeur Nul (NPO)

On pourrait craindre que l'utilisation d'une énumération pour chaque valeur potentiellement absente n'introduise une pénalité de performance ou un surcoût mémoire (l'espace pour stocker le discriminant de l'énumération).

Heureusement, le compilateur Rust implémente une optimisation cruciale appelée l'Optimisation du Pointeur Nul (*Null Pointer Optimization* ou NPO).

Pour les types qui ne peuvent pas avoir une valeur binaire égale à `0` (comme les références `&T`, `&mut T`, ou les pointeurs intelligents comme `Box<T>`), Rust utilise la valeur `0` (le pointeur nul au niveau machine) pour représenter la variante `None`.

Type logique	Représentation en mémoire machine	Taille en mémoire
<code>&String</code>	Adresse mémoire valide (jamais 0)	8 octets (sur architecture 64-bit)
<code>Option<&String></code>	<code>0x0</code> pour <code>None</code> , ou l'adresse valide pour <code>Some</code>	8 octets (Zéro surcoût !)

Grâce à cette optimisation, `Option<&T>` a exactement la même taille en mémoire qu'un pointeur brut en C ou C++, tout en garantissant une sécurité absolue à la compilation. Rust offre ici une abstraction à coût nul (*zero-cost abstraction*).

3. Implémentation Pratique Pas-à-Pas

Découvrons comment manipuler concrètement le type `Option<T>` à travers différents mécanismes de filtrage et d'extraction.

Étape 1 : Déclarer et instancier des options

Pour déclarer une option, on utilise les variantes `Some` et `None`.

```
fn main() {
    // Une option contenant un entier
    let age_present: Option<i32> = Some(32);

    // Une option représentant l'absence d'entier
    // Ici, nous devons typer explicitement car le compilateur ne peut pas deviner le type
    let age_absent: Option<i32> = None;

    println!("Présent : {:?}", age_present);
    println!("Absent : {:?}", age_absent);
}
```

Étape 2 : Extraire la valeur de manière sécurisée

Méthode A : Le Pattern Matching (Le plus robuste)

Le moyen le plus fondamental pour lire la valeur dans une `Option` est d'utiliser l'expression `match`. Le compilateur vérifie que vous traitez obligatoirement les deux cas (`Some` et `None`).

```
fn saluer_utilisateur(nom_optionnel: Option<String>) {
    match nom_optionnel {
        Some(nom) => println!("Bonjour, {} !", nom),
        None => println!("Bonjour, visiteur anonyme !"),
    }
}
```

Méthode B : L'expression `if let` (Plus concise)

Si seul le cas où la valeur est présente vous intéresse, `if let` permet d'éviter la verbosité d'un `match` complet.

```
fn afficher_score(score_optionnel: Option<u32>) {
    // On n'exécute le bloc que si le score est présent (Some)
    if let Some(score) = score_optionnel {
        println!("Votre score est de : {} points", score);
    } else {
        println!("Aucun score enregistré.");
    }
}
```

Étape 3 : Les combinateurs de la bibliothèque standard

Extraire manuellement les valeurs avec `match` ou `if let` peut devenir répétitif. La structure `Option` propose de nombreuses méthodes utilitaires pour manipuler les données de manière fluide.

1. Fournir une valeur par défaut : `unwrap_or` et `unwrap_or_else`

- `unwrap_or(default)` : Retourne la valeur contenue ou une valeur par défaut immédiate.
- `unwrap_or_else(closure)` : Calcule la valeur par défaut de manière paresseuse (lazy) via une fonction ou une fermeture, ce qui est idéal si le calcul de la valeur par défaut est coûteux.

```
fn main() {
    let port_defini: Option<u16> = None;

    // Utilisation d'une valeur par défaut immédiate
    let port_final = port_defini.unwrap_or(8080);
    println!("Port utilisé : {}", port_final); // Affiche 8080

    // Utilisation d'une valeur par défaut calculée à la demande
    let cle_api: Option<String> = None;
    let cle_finale = cle_api.unwrap_or_else(|| {
        println!("Génération d'une clé temporaire...");
        String::from("TEMP_KEY_123")
    });
}
```

2. Transformer la valeur interne : `map`

La méthode `map` permet d'appliquer une transformation sur la valeur contenue dans `Some`, tout en laissant `None` inchangé.

```
fn main() {
    let texte: Option<String> = Some(String::from("rust"));

    // Transforme l'Option<String> en Option<usize> (la longueur du texte)
    let longueur: Option<usize> = texte.map(|s| s.len());

    println!("Longueur : {:?}", longueur); // Affiche Some(4)
}
```

3. Enchaîner des opérations retournant des options : `and_then`

Aussi appelée *flat_map* dans d'autres langages, `and_then` est cruciale lorsque la fonction de transformation retourne elle-même une `Option`. Cela évite de se retrouver avec une option imbriquée de type `Option<Option<T>>`.

```
struct Compte {
    solde: Option<f64>,
}

fn obtenir_compte(id: u32) → Option<Compte> {
    if id == 42 {
        Some(Compte { solde: Some(1500.0) })
    } else {
        None
    }
}

fn main() {
    // Sans and_then, map produirait un Option<Option<f64>>
    let solde_imbrique: Option<Option<f64>> = obtenir_compte(42).map(|c| c.solde);

    // Avec and_then, le résultat est aplati en Option<f64>
    let solde_propre: Option<f64> = obtenir_compte(42).and_then(|c| c.solde);

    println!("Solde : {:?}", solde_propre); // Affiche Some(1500.0)
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : L'utilisation abusive de `.unwrap()`

La méthode `.unwrap()` extrait la valeur interne d'une `Option`. Cependant, si l'option est `None`, le programme panique et s'arrête immédiatement.

```
// À ÉVITER ABSOLUMENT DANS DU CODE DE PRODUCTION
let valeur: Option<i32> = None;
let x = valeur.unwrap(); // CRASH : thread 'main' panicked at 'called `Option::unwrap` on a None value'
```

- Bonne pratique : Réservez `.unwrap()` uniquement pour les tests unitaires ou lorsque vous pouvez prouver mathématiquement au compilateur que l'option ne peut pas être `None` à cet endroit précis du code.
- Alternative : Utilisez `.expect("Message d'erreur explicite")` à la place de `.unwrap()`. Bien qu'il provoque également une panique en cas de `None`, il permet de documenter l'intention et de faciliter le débogage grâce à un message personnalisé.

Piège 2 : Consommer l'Option par mégarde (Ownership)

Lorsque vous effectuez un `match` ou un `if let` sur une `Option<T>` où `T` n'implémente pas le trait `Copy` (comme `String`), Rust va tenter de déplacer (*move*) la valeur hors de l'option.

```
let option_nom = Some(String::from("Alice"));

// Ce match déplace la String hors de option_nom !
match option_nom {
    Some(nom) => println!("Nom : {}", nom),
    None => println!("Anonyme"),
}

// Erreur de compilation : option_nom a été consommée !
// println!("{:?}", option_nom);
```

- Bonne pratique : Utilisez la méthode `.as_ref()` pour transformer une `Option<T>` en `Option<&T>`. Cela permet d'emprunter la valeur interne sans déplacer la propriété.

```
let option_nom = Some(String::from("Alice"));

// On emprunte le contenu de l'option
match option_nom.as_ref() {
    Some(nom) => println!("Nom emprunté : {}", nom), // nom est de type &String
    None => println!("Anonyme"),
}

// Parfaitement valide, option_nom est toujours utilisable ici !
println!("Option toujours valide : {:?}", option_nom);
```

Piège 3 : Utiliser des valeurs sentinelles au lieu d'Option

Certains développeurs habitués au C ou au Go continuent d'utiliser des valeurs sentinelles pour indiquer l'absence de valeur (par exemple, retourner `-1` pour un index non trouvé, ou une chaîne vide `""` pour un utilisateur sans nom).

- **Bonne pratique** : Bannissez les valeurs sentinelles. Elles sont sources de bugs silencieux. Utilisez systématiquement `Option<T>`. Le compilateur vous remerciera en vous forçant à traiter correctement tous les cas de figure.
-

5. Synthèse Pédagogique

Tableau comparatif des méthodes d'accès à `Option<T>`

Méthode	Comportement si <code>Some(v)</code>	Comportement si <code>None</code>	Sécurité	Cas d'usage recommandé
<code>match</code>	Exécute la branche <code>Some(v)</code>	Exécute la branche <code>None</code>	Absolue	Logique complexe à deux branches
<code>if let</code>	Exécute le bloc avec <code>v</code>	Ignore ou exécute le <code>else</code>	Absolue	Traitement ciblé d'un seul cas
<code>unwrap()</code>	Retourne <code>v</code>	Panique (Crash)	Dangereux	Prototypes rapides, tests unitaires
<code>expect(msg)</code>	Retourne <code>v</code>	Panique avec message	Moyenne	Cas théoriquement impossibles
<code>unwrap_or(d)</code>	Retourne <code>v</code>	Retourne la valeur par défaut <code>d</code>	Absolue	Remplacement par valeur par défaut simple
<code>unwrap_or_else(f)</code>	Retourne <code>v</code>	Calcule et retourne <code>f()</code>	Absolue	Remplacement par valeur calculée complexe

Points clés à retenir

1. Pas de `null` : Rust élimine toute une classe de failles de sécurité et de plantages en n'implémentant pas le concept de pointeur nul dans son code sûr.
2. Explicite et Typé : L'absence de valeur est représentée par le type énuméré `Option<T>`.

3. Sécurité à la compilation : Le compilateur garantit que vous ne pouvez pas accéder à la valeur enveloppée sans avoir préalablement vérifié sa présence.
 4. Zéro surcoût : Grâce à l'optimisation du pointeur nul (NPO), l'utilisation d'un type `Option<T>` ne consomme pas plus de mémoire qu'un pointeur brut traditionnel.
-

6. Exercice Pratique d'Application

Énoncé : Gestionnaire de Profils Utilisateurs

Vous devez concevoir le moteur de recherche d'une base de données d'utilisateurs pour une plateforme universitaire.

Chaque utilisateur possède : * Un identifiant unique (`id` de type `u32`). * Un prénom (`prenom` de type `String`). * Un deuxième prénom optionnel (`deuxieme_prenom` de type `Option<String>`). * Un nom de famille (`nom` de type `String`).

Objectifs :

1. Définir la structure de données `Utilisateur` .
2. Implémenter une fonction de recherche `trouver_utilisateur_par_id` qui prend en paramètre une tranche (slice) d'utilisateurs et un ID, et retourne une `Option` contenant une référence vers l'utilisateur trouvé.
3. Implémenter une fonction `generer_nom_complet` qui prend une référence d'un `Utilisateur` et retourne son nom complet sous forme de `String` .
4. Si le deuxième prénom est présent, le format doit être : `"Prénom Deuxième-Prénom NOM"` .
5. Si le deuxième prénom est absent, le format doit être : `"Prénom NOM"` .
6. Le nom de famille doit être converti en majuscules (utilisez la méthode `.to_uppercase()`).

Indices pour la résolution

- Pour la fonction de recherche, vous pouvez itérer sur la tranche avec `.iter()` et utiliser la méthode `.find(...)` des itérateurs, ou écrire une boucle `for` classique.

- Dans `generer_nom_complet`, attention à l'ownership ! Le deuxième prénom étant une `Option<String>`, l'utilisation directe de la valeur dans un `match` ou un `if let` va tenter de déplacer la chaîne. Pensez à utiliser `.as_ref()` ou `.as_deref()` pour n'obtenir qu'un emprunt de la chaîne de caractères.
-

Solution Complète et Commentée

Voici l'implémentation complète et rigoureuse attendue :

```

// 1. Définition de la structure Utilisateur
#[derive(Debug, Clone)]
struct Utilisateur {
    id: u32,
    prenom: String,
    deuxieme_prenom: Option<String>,
    nom: String,
}

// 2. Fonction de recherche d'un utilisateur par son ID
// Nous retournons une Option contenant une référence (&Utilisateur) pour éviter de copier
fn trouver_utilisateur_par_id(utilisateurs: &[Utilisateur], id_recherche: u32) → Option<&Utilisateur> {
    for utilisateur in utilisateurs {
        if utilisateur.id == id_recherche {
            // On a trouvé l'utilisateur, on le retourne enveloppé dans Some
            return Some(utilisateur);
        }
    }
    // Si la boucle se termine sans trouver l'utilisateur, on retourne None
    None
}

// 3. Fonction de génération du nom complet
fn generer_nom_complet(utilisateur: &Utilisateur) → String {
    let nom_majuscule = utilisateur.nom.to_uppercase();

    // On utilise .as_ref() pour obtenir une Option<&String> au lieu de consommer l'Option<String>
    match utilisateur.deuxieme_prenom.as_ref() {
        Some(deuxieme) => {
            // Cas où le deuxième prénom est présent
            format!("{}", utilisateur.prenom, deuxieme, nom_majuscule)
        }
        None => {
            // Cas où le deuxième prénom est absent
            format!("{}", utilisateur.prenom, nom_majuscule)
        }
    }
}

fn main() {
    // Initialisation de notre base de données fictive
    let base_utilisateurs = vec![
        Utilisateur {
            id: 1,
            prenom: String::from("Marie"),
            deuxieme_prenom: Some(String::from("Thérèse")),
            nom: String::from("Dupont"),
        },
    ],
}

```

```

    Utilisateur {
        id: 2,
        prenom: String::from("Jean"),
        deuxieme_prenom: None,
        nom: String::from("Martin"),
    },
];

// --- Test de recherche 1 : Utilisateur existant avec deuxième prénom ---
println!("--- Recherche ID 1 ---");
match trouver_utilisateur_par_id(&base_utilisateurs, 1) {
    Some(user) => {
        let nom_complet = generer_nom_complet(user);
        println!("Utilisateur trouvé ! Nom complet : {}", nom_complet);
    }
    None => println!("Erreur : Utilisateur non trouvé."),
}

// --- Test de recherche 2 : Utilisateur existant sans deuxième prénom ---
println!("\n--- Recherche ID 2 ---");
if let Some(user) = trouver_utilisateur_par_id(&base_utilisateurs, 2) {
    let nom_complet = generer_nom_complet(user);
    println!("Utilisateur trouvé ! Nom complet : {}", nom_complet);
}

// --- Test de recherche 3 : Utilisateur inexistant ---
println!("\n--- Recherche ID 99 ---");
let resultat_recherche = trouver_utilisateur_par_id(&base_utilisateurs, 99);

// Utilisation d'un combinateur pour formater le résultat de manière concise
let message = resultat_recherche
    .map(|user| format!("Trouvé : {}", generer_nom_complet(user)))
    .unwrap_or_else(|| String::from("Utilisateur introuvable dans la base de données."))

println!("{}", message);
}

```


Exercices d'Application : L'absence de valeur : Dire adieu au pointeur nul avec le type Option

Ce cahier d'exercices pratiques a pour objectif de consolider votre compréhension du type `Option<T>` en Rust. À travers trois exercices de difficulté progressive, vous passerez de la manipulation de base à l'enchaînement complexe de méthodes (combinateurs), tout en apprenant à éviter les pièges liés à la propriété (*ownership*).

Exercice 1 : Configuration d'un Serveur Web (Difficulté : Facile)

Énoncé

Dans cet exercice, vous devez concevoir le système de chargement de la configuration d'un serveur web. Souvent, les utilisateurs ne spécifient pas toutes les options dans leur fichier de configuration. Votre programme doit être capable de combler les valeurs manquantes par des valeurs par défaut sécurisées.

Objectifs :

1. Définir une structure `ConfigurationServeur` contenant :
2. `hote : Option<String>` (ex: `Some("127.0.0.1".to_string())`)
3. `port : Option<u16>` (ex: `Some(8080)`)
4. `max_connexions : Option<u32>` (ex: `None`)

5. Implémenter une fonction `obtenir_port_effectif(config: &ConfigurationServeur) → u16` qui retourne le port spécifié, ou la valeur par défaut `80` si aucun port n'est fourni.
 6. Implémenter une fonction `afficher_statut(config: &ConfigurationServeur)` qui :
 7. Affiche `"Serveur configuré sur l'hôte : [nom_hote]"` si l'hôte est présent.
 8. Affiche `"Serveur configuré sur l'hôte par défaut : localhost"` si l'hôte est absent.
 9. Utilisez l'expression `if let` pour cette question.
-

Indices

- Pour `obtenir_port_effectif`, la méthode `.unwrap_or(...)` vue dans le cours est parfaitement adaptée car le port par défaut (`80`) est une valeur simple et immédiate.
 - Pour `afficher_statut`, attention à l'ownership ! L'hôte est une `Option<String>`. Si vous faites un `if let Some(h) = config.hote`, Rust va tenter de déplacer la `String` hors de la structure empruntée. Utilisez `.as_ref()` pour obtenir une `Option<&String>`.
-

Correction Détaillée


```

// 1. Définition de la structure
struct ConfigurationServeur {
    hote: Option<String>,
    port: Option<u16>,
    max_connexions: Option<u32>,
}

// 2. Fonction pour obtenir le port avec valeur par défaut
fn obtenir_port_effectif(config: &ConfigurationServeur) → u16 {
    // On utilise unwrap_or pour fournir la valeur par défaut 80
    // config.port est de type Option<u16>, qui implémente Copy, donc pas
    config.port.unwrap_or(80)
}

// 3. Fonction d'affichage du statut
fn afficher_statut(config: &ConfigurationServeur) {
    // .as_ref() est crucial ici pour ne pas déplacer la String hors de la structure
    if let Some(hote_present) = config.hote.as_ref() {
        println!("Serveur configuré sur l'hôte : {}", hote_present);
    } else {
        println!("Serveur configuré sur l'hôte par défaut : localhost");
    }
}

fn main() {
    // Test 1 : Configuration partielle
    let config_partielle = ConfigurationServeur {
        hote: None,
        port: Some(3000),
        max_connexions: None,
    };

    println!("--- Test Configuration Partielle ---");
    println!("Port effectif : {}", obtenir_port_effectif(&config_partielle));
    afficher_statut(&config_partielle);

    // Test 2 : Configuration complète
    let config_complete = ConfigurationServeur {
        hote: Some(String::from("192.168.1.1")),
        port: None, // Devrait utiliser 80
        max_connexions: Some(100),
    };

    println!("\n--- Test Configuration Complète ---");
    println!("Port effectif : {}", obtenir_port_effectif(&config_complete));
    afficher_statut(&config_complete);
}

```

Exercice 2 : Système de Codes Promo (Difficulté : Moyenne)

Énoncé

Vous développez le module de paiement d'un site e-commerce. Les clients peuvent appliquer un code promotionnel à leur panier.

Certains paniers ont un code promo, d'autres non. De plus, les codes promo saisis par les utilisateurs peuvent être valides (ils donnent droit à une réduction) ou invalides/expirés (ils ne donnent droit à rien).

Objectifs :

1. Définir une structure `Panier` contenant un identifiant `id: u32` et un code optionnel `code_promo: Option<String>`.
 2. Implémenter une fonction de simulation `recuperer_reduction(code: &str) → Option<f64>` qui retourne :
 3. `Some(15.0)` si le code est `"WINTER15"`
 4. `Some(50.0)` si le code est `"BLACKFRIDAY"`
 5. `None` pour tout autre code.
 6. Implémenter une fonction `calculer_reduction_panier(panier: &Panier) → Option<f64>` qui extrait le code promo du panier (s'il existe), interroge la fonction `recuperer_reduction` et retourne le pourcentage de réduction trouvé.
 7. **Contrainte :** Vous devez enchaîner les opérations sur l'option sans utiliser de `match` ou de `if let`, en utilisant uniquement les combinateurs d'options (`map`, `and_then`, `as_deref`, etc.).
-

Indices

- Pour passer d'une `Option<String>` à une `Option<&str>`, la méthode `.as_deref()` est extrêmement utile en Rust. Elle évite d'avoir à gérer des références complexes de `String`.
 - La fonction `recuperer_reduction` prend une référence de chaîne de caractères (`&str`) et retourne elle-même une `Option<f64>`. Si vous utilisez `.map(...)` pour l'appeler depuis une `Option<&str>`, vous obtiendrez un type imbriqué : `Option<Option<f64>>`. Quel combinateur permet d'aplatir ce résultat ?
-

Correction Détaillée

```

// 1. Définition de la structure Panier
struct Panier {
    id: u32,
    code_promo: Option<String>,
}

// 2. Fonction de simulation de base de données de codes promo
fn recuperer_reduction(code: &str) → Option<f64> {
    match code {
        "WINTER15" ⇒ Some(15.0),
        "BLACKFRIDAY" ⇒ Some(50.0),
        _ ⇒ None, // Code inconnu ou expiré
    }
}

// 3. Fonction de calcul de la réduction du panier
fn calculer_reduction_panier(panier: &Panier) → Option<f64> {
    panier.code_promo
        .as_deref() // Convertit Option<String> en Option<&str> par emprunt
        .and_then(|code| recuperer_reduction(code)) // Applique la fonction et aplatit
}

fn main() {
    let paniers = vec![
        Panier {
            id: 101,
            code_promo: Some(String::from("WINTER15")),
        },
        Panier {
            id: 102,
            code_promo: Some(String::from("CODE_INVALIDE")),
        },
        Panier {
            id: 103,
            code_promo: None,
        },
    ];

    for panier in paniers {
        print!("Panier #{id} : ", panier.id);
        match calculer_reduction_panier(&panier) {
            Some(reduction) ⇒ println!("Réduction de {}% appliquée !", reduction),
            None ⇒ println!("Aucune réduction appliquée."),
        }
    }
}

```

Exercice 3 : Analyseur de Capteurs Thermiques

(Difficulté : Difficile)

Énoncé

Dans l'industrie, les vieux systèmes logiciels utilisent souvent des "valeurs sentinelles" pour indiquer des erreurs. Par exemple, un capteur de température défaillant peut renvoyer la valeur `-999.0` pour signifier qu'il n'a pas pu lire la température.

Vous devez écrire un module Rust moderne qui nettoie ces données historiques en remplaçant la valeur sentinelle par une `Option<f64>` propre, puis calcule des statistiques de sécurité.

Objectifs :

1. Écrire une fonction `assainir_lecture(lecture_brute: f64) → Option<f64>` qui :
2. Retourne `None` si la lecture est égale à `-999.0` .
3. Retourne `Some(lecture_brute)` sinon.
4. Écrire une fonction `calculer_moyenne(lectures: &[f64]) → Option<f64>` qui :
5. Convertit chaque température brute en `Option<f64>` .
6. Filtre les valeurs pour ne garder que les mesures valides.
7. Calcule la moyenne de ces mesures valides.
8. Si aucune mesure valide n'est présente (ou si le tableau est vide), la fonction doit retourner `None` .

Indices

- Pour transformer un vecteur de valeurs brutes en valeurs assainies, vous pouvez utiliser `.iter()` combiné à `.map()` .
- Pour filtrer et extraire les valeurs contenues dans des `Option` au sein d'un itérateur, Rust propose la méthode magique `.filter_map(|x| x)` (ou

`.flatten()`). Elle élimine automatiquement les `None` et extrait les valeurs des `Some` .

- Attention aux divisions par zéro : si le nombre de mesures valides est égal à `0` , retournez immédiatement `None` .
-

Correction Détaillée


```

// 1. Fonction d'assainissement de la valeur sentinelle
fn assainir_lecture(lecture_brute: f64) → Option<f64> {
    if lecture_brute == -999.0 {
        None
    } else {
        Some(lecture_brute)
    }
}

// 2. Fonction de calcul de la moyenne des températures valides
fn calculer_moyenne(lectures: &[f64]) → Option<f64> {
    let mut somme = 0.0;
    let mut compteur = 0;

    // On parcourt les lectures brutes
    for &lecture in lectures {
        // On assainit la lecture courante
        if let Some(temperature_valide) = assainir_lecture(lecture) {
            somme += temperature_valide;
            compteur += 1;
        }
    }

    // Sécurité : éviter la division par zéro et gérer l'absence de données valides
    if compteur == 0 {
        None
    } else {
        Some(somme / (compteur as f64))
    }
}

// Alternative élégante et idiomatique utilisant les itérateurs de la bibliothèque standard
fn calculer_moyenne_fonctionnel(lectures: &[f64]) → Option<f64> {
    let lectures_valides: Vec<f64> = lectures.iter()
        .map(|&l| assainir_lecture(l))
        .flatten() // Élimine les None et extrait les valeurs des Some
        .collect();

    if lectures_valides.is_empty() {
        None
    } else {
        let somme: f64 = lectures_valides.iter().sum();
        Some(somme / (lectures_valides.len() as f64))
    }
}

fn main() {
    // Cas 1 : Données mixtes (valides et invalides)

```

```

let donnees_capteur_1 = vec![19.5, 20.0, -999.0, 21.5, -999.0, 19.0];

println!("--- Capteur 1 ---");
match calculer_moyenne(&donnees_capteur_1) {
    Some(moyenne) => println!("Moyenne des températures valides : {:.2}°C", moyenne),
    None => println!("Erreur : Aucune donnée valide disponible."),
}

// Cas 2 : Capteur totalement en panne (uniquement des valeurs sentinelles)
let donnees_capteur_2 = vec![-999.0, -999.0, -999.0];

println!("\n--- Capteur 2 ---");
match calculer_moyenne_fonctionnel(&donnees_capteur_2) {
    Some(moyenne) => println!("Moyenne : {:.2}°C", moyenne),
    None => println!("Alerte : Le capteur n'a renvoyé aucune donnée exploitable !"),
}
}

```

Propagation propre des erreurs avec l'opérateur '?'

1. Introduction Conceptuelle

Dans les chapitres précédents, nous avons découvert comment Rust modélise l'absence de valeur avec le type `Option<T>` (étudié dans le Module 4.2) et les erreurs récupérables avec le type `Result<T, E>` (étudié dans le Module 4.1). Nous avons appris à extraire les données de ces enums en utilisant le filtrage par motif (*pattern matching*) via l'instruction `match`.

Cependant, une question cruciale se pose rapidement lors de l'écriture de programmes réels : que faire lorsque l'on ne souhaite pas ou que l'on ne peut pas gérer l'erreur immédiatement au sein de la fonction courante ?

Bien souvent, la fonction qui rencontre une erreur n'a pas le contexte nécessaire pour décider de la marche à suivre (afficher un message à l'utilisateur, tenter une nouvelle fois, utiliser une valeur par défaut, ou arrêter proprement le programme). La responsabilité de la décision doit alors être déléguée à la fonction appelante. C'est ce que l'on appelle la propagation d'erreur.

Le problème de la verbosité (La "taxe" du boilerplate)

Si nous utilisons uniquement le filtrage par motif traditionnel pour propager chaque erreur, notre code devient rapidement illisible. Imaginez une séquence de trois opérations dépendantes, chacune pouvant échouer (par exemple : ouvrir un fichier, lire son contenu, puis analyser ce contenu pour le convertir en nombre).

Sans outil adapté, nous ferions face à une cascade de blocs `match` imbriqués, provoquant une dérive du code vers la droite (*rightward drift*) et noyant la logique métier sous une montagne de code de gestion d'erreur répétitif (*boilerplate*).

La solution : L'opérateur point d'interrogation (?)

Pour résoudre ce problème sans sacrifier la sécurité ni l'explicité, Rust propose un opérateur unique : l'opérateur `?`.

Positionné immédiatement après une expression retournant un `Result` ou une `Option`, l'opérateur `?` réduit drastiquement la verbosité. Il permet d'écrire un code linéaire, fluide et lisible, presque identique à celui des langages utilisant des exceptions traditionnelles (comme Java ou Python), tout en conservant la rigueur du typage statique de Rust et l'absence totale de levée d'exceptions invisibles à l'exécution.

2. Fondations Théoriques

Pour comprendre l'opérateur `?`, il faut démystifier ce qu'il fait sous le capot. Il ne s'agit pas de magie noire, mais d'un sucre syntaxique extrêmement puissant.

Le mécanisme de déballage et de retour anticipé

Lorsque vous écrivez :

```
let valeur = une_fonction_qui_peut_echouer()?;
```

Le compilateur Rust traduit cette ligne par une structure équivalente à celle-ci :

```
let valeur = match une_fonction_qui_peut_echouer() {  
    Ok(val) => val,  
    Err(err) => return Err(err), // Retour anticipé immédiat de la fonction parente  
};
```

Analyse de ce comportement :

1. Cas de succès (`Ok(val)`) : L'erreur n'a pas eu lieu. L'opérateur `?` "déballé" la valeur contenue dans le variant `Ok` et l'assigne à la variable `valeur`. Le programme continue son exécution normalement sur la ligne suivante.

2. Cas d'échec (`Err(err)`) : L'erreur s'est produite. L'opérateur `?` interrompt immédiatement l'exécution de la fonction courante et effectue un retour anticipé (*early return*) en renvoyant l'erreur `Err` enveloppée à la fonction appelante.

La contrainte de signature

Ce mécanisme de retour anticipé impose une règle absolue : vous ne pouvez utiliser l'opérateur `?` que dans une fonction dont le type de retour est compatible avec la valeur propagée.

- Si vous utilisez `?` sur un `Result<T, E>`, la fonction parente doit impérativement retourner un `Result<U, F>`, où le type d'erreur `E` peut être converti en `F`.
- Si vous utilisez `?` sur une `Option<T>`, la fonction parente doit impérativement retourner une `Option<U>`.

Attention : Vous ne pouvez pas, par défaut, mélanger l'utilisation de `?` sur une `Option` et un `Result` au sein de la même fonction sans conversion préalable.

La conversion automatique de types avec le trait `From`

Une des fonctionnalités les plus élégantes de l'opérateur `?` est sa capacité à convertir automatiquement le type d'erreur retourné.

La véritable expansion macroscopique de l'opérateur `?` sur un `Result` est en réalité la suivante :

```
let valeur = match une_fonction_qui_peut_echouer() {  
    Ok(val) => val,  
    Err(err) => return Err(std::convert::From::from(err)),  
};
```

L'appel à `From::from(err)` permet de convertir automatiquement l'erreur interceptée vers le type d'erreur attendu par la fonction parente.

(Note : Le concept de *Trait* sera étudié en profondeur dans le Module 5.2. Pour l'instant, reprenez simplement que le trait `From` définit comment transformer un type A en un type B).

Grâce à cela, si une fonction interne renvoie une erreur de type `std::io::Error`, mais que votre fonction globale renvoie une erreur personnalisée de type `MonErreurApplication`, l'opérateur `?` effectuera la conversion de manière totalement transparente, à condition que vous ayez défini comment convertir l'une en l'autre.

3. Implémentation Pratique Pas-à-Pas

Pour illustrer la transition d'une gestion manuelle à une gestion moderne et élégante avec `?`, nous allons concevoir un programme qui lit le premier mot d'un fichier texte.

Étape 1 : L'approche manuelle verbeuse (sans `?`)

Voici comment nous devrions écrire cette fonction en utilisant uniquement des structures `match`.

```

use std::fs::File;
use std::io::{self, Read};

fn lire_premier_mot_manuel() → Result<String, io::Error> {
    // 1. Tentative d'ouverture du fichier
    let mut fichier_result = File::open("pseudo_config.txt");

    let mut fichier = match fichier_result {
        Ok(f) ⇒ f,
        Err(e) ⇒ return Err(e), // Propagation manuelle
    };

    // 2. Tentative de lecture du contenu
    let mut contenu = String::new();
    match fichier.read_to_string(&mut contenu) {
        Ok(_) ⇒ {},
        Err(e) ⇒ return Err(e), // Propagation manuelle
    };

    // 3. Extraction du premier mot
    let premier_mot = match contenu.split_whitespace().next() {
        Some(mot) ⇒ mot.to_string(),
        None ⇒ String::new(), // Si vide, on retourne une chaîne vide
    };

    Ok(premier_mot)
}

```

Ce code fonctionne parfaitement, mais il est lourd. Nous répétons le motif de retour anticipé `Err(e) ⇒ return Err(e)` à chaque étape risquée.

Étape 2 : Simplification radicale avec l'opérateur `?`

Remplaçons maintenant ces blocs `match` par l'opérateur `?`.

```

use std::fs::File;
use std::io::{self, Read};

fn lire_premier_mot_elegant() → Result<String, io::Error> {
    // Si File::open échoue, l'erreur io::Error est immédiatement retournée.
    let mut fichier = File::open("pseudo_config.txt"?);

    let mut contenu = String::new();
    // Si read_to_string échoue, l'erreur io::Error est immédiatement retournée.
    fichier.read_to_string(&mut contenu)?;

    // Extraction du premier mot (utilisation de Option)
    let premier_mot = contenu
        .split_whitespace()
        .next()
        .unwrap_or("")
        .to_string();

    Ok(premier_mot)
}

```

Gain immédiat : Le code est passé de 25 lignes à seulement quelques lignes claires. La logique métier (ouvrir, lire, extraire) est immédiatement visible, débarrassée du bruit visuel de la gestion d'erreur.

Étape 3 : Le chaînage de méthodes

Puisque l'opérateur `?` renvoie la valeur déballée en cas de succès, il est possible de chaîner les appels de fonctions de manière extrêmement concise :

```

use std::fs::File;
use std::io::{self, Read};

fn lire_et_recuperer_tout() → Result<String, io::Error> {
    let mut contenu = String::new();
    // Chaînage direct de l'ouverture et de la lecture
    File::open("pseudo_config.txt")?.read_to_string(&mut contenu)?;
    Ok(contenu)
}

```


Étape 4 : Propagation avec des types d'erreurs multiples

Que se passe-t-il si une fonction peut générer plusieurs types d'erreurs différents ?

Imaginons que nous lisons une chaîne dans un fichier, puis que nous voulons la convertir en entier (`u32`). * `File::open` et `read_to_string` renvoient une `std::io::Error`. * `str::parse::<u32>` renvoie une `std::num::ParseIntError`.

Si nous essayons d'utiliser `?` sur les deux, quel doit être le type de retour de la fonction ?

Solution : Utiliser un objet de trait d'erreur dynamique (`Box<dyn std::error::Error>`)

Pour les fonctions générales ou les prototypes rapides, nous pouvons utiliser un pointeur intelligent `Box` (étudié dans le cadre des collections et de l'allocation sur le tas au Module 1.3 et 3.3) contenant n'importe quel type implémentant le trait standard `Error`.

```
use std::fs::File;
use std::io::Read;
use std::error::Error;

// Le type de retour accepte désormais n'importe quel type d'erreur standard
fn lire_nombre_depuis_fichier() → Result<u32, Box<dyn Error>> {
    let mut contenu = String::new();

    // Propage une std::io::Error, convertie automatiquement en Box<dyn Error>
    File::open("nombre.txt")?.read_to_string(&mut contenu)?;

    // Propage une ParseIntError, convertie automatiquement en Box<dyn Error>
    let nombre: u32 = contenu.trim().parse()?;

    Ok(nombre)
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Utiliser `?` dans la fonction `main` sans modifier son type de retour

Par défaut, la fonction `main` générée par Cargo a la signature suivante :

```
fn main() {
    // ...
}
```

Son type de retour implicite est le type unité `()`. Si vous tentez d'utiliser l'opérateur `?` à l'intérieur, le compilateur lèvera une erreur explicite :

```
error[E0277]: the `?` operator can only be used in a function that returns `Result` or `Option`
```

Résolution :

Rust autorise la fonction `main` à retourner un type `Result`. Vous devez modifier sa signature :

```
use std::error::Error;

fn main() → Result<(), Box<dyn Error>> {
    let nombre = lire_nombre_depuis_fichier()?;
    println!("Le nombre lu est : {}", nombre);

    // Tout s'est bien passé, on retourne Ok contenant l'unité
    Ok(())
}
```

Piège 2 : Mélanger `Option` et `Result` sans conversion

Si vous tentez d'utiliser `?` sur une `Option` dans une fonction qui retourne un `Result`, le compilateur refusera de compiler.

```
// CODE ERRONÉ - NE COMPILE PAS
fn recuperer_donnee() → Result<String, String> {
    let liste = vec!["Rust", "C++", "Zig"];
    // .get() retourne une Option, mais la fonction attend un Result
    let element = liste.get(5)?;
    Ok(element.to_string())
}
```

Résolution :

Vous devez explicitement convertir l'un vers l'autre. * Pour convertir une `Option<T>` en `Result<T, E>`, utilisez `.ok_or(erreur)` ou `.ok_or_else(|| erreur)` :

```
fn recuperer_donnee_valide() → Result<String, String> {
    let liste = vec!["Rust", "C++", "Zig"];

    // Conversion de l'Option en Result avec un message d'erreur personnalisé
    let element = liste.get(5).ok_or("Index hors limites !");

    Ok(element.to_string())
}
```

Piège 3 : Abuser de `unwrap()` au lieu de propager avec `?`

Il est tentant d'utiliser `.unwrap()` ou `.expect()` pour aller plus vite. Cependant, ces méthodes provoquent un arrêt brutal du programme (*panic*) en cas d'erreur.

- Règle d'or : Utilisez `?` pour propager les erreurs qui sont hors du contrôle direct de votre programme (fichiers manquants, entrées utilisateur invalides, défaillances réseau). Réservez `unwrap()` uniquement pour les cas où vous avez la certitude mathématique que l'erreur ne peut pas se produire, ou dans les tests unitaires.

5. Synthèse Pédagogique

Approche	Syntaxe	Comportement en cas d'erreur	Recommandation d'usage
Filtrage par motif (<code>match</code>)	<code>match resultat { ... }</code>	Permet de gérer localement l'erreur ou de la propager manuellement.	Idéal pour la gestion finale de l'erreur ou si un traitement spécifique est requis.
Panique forcée (<code>unwrap</code>)	<code>resultat.unwrap()</code>	Fait planter le programme immédiatement (<i>panic!</i>).	À proscrire en production, sauf certitude absolue ou prototypage rapide.
Propagation propre (<code>?</code>)	<code>resultat?</code>	Retourne immédiatement l'erreur à la fonction appelante.	Standard de l'industrie pour écrire du code propre, robuste et lisible.

Points clés à retenir :

1. L'opérateur `?` est un sucre syntaxique pour un retour anticipé (`return Err(...)`).
2. Il ne peut être utilisé que dans des fonctions qui retournent un type compatible (`Result` ou `Option`).
3. Il réalise des conversions de types d'erreurs automatiques grâce au trait standard `From` .
4. La fonction `main` peut retourner un `Result<(), Box<dyn Error>>` pour permettre l'usage de `?` au niveau global du programme.

6. Exercice Pratique d'Application

Énoncé : "Le Configulateur Sécurisé"

Vous devez concevoir un module de chargement de configuration pour un serveur réseau. Le programme doit lire un fichier nommé `port.txt` qui contient uniquement le numéro de port sur lequel le serveur doit démarrer (par exemple, la chaîne `"8080"`).

Votre objectif est d'écrire une fonction `charger_port` qui : 1. Ouvre le fichier `port.txt` . 2. Lit son contenu sous forme de chaîne de caractères. 3. Nettoie les espaces ou retours à la ligne superflus (avec `.trim()`). 4. Convertit (parse) cette chaîne en un entier de type `u16` . 5. Gère proprement toutes les erreurs possibles en les propageant à l'aide de l'opérateur `?` .

Puisque cette fonction peut échouer à cause d'une erreur d'entrée/sortie (fichier absent) ou d'une erreur de parsing (le fichier contient "abc" au lieu d'un nombre), vous utiliserez le type de retour `Result<u16, Box<dyn std::error::Error>>` .

Indices utiles :

- Pour ouvrir un fichier : `std::fs::File::open("port.txt")` (renvoie un `Result<File, std::io::Error>`).
 - Pour lire le contenu : créez une chaîne vide `let mut contenu = String::new();` puis utilisez `fichier.read_to_string(&mut contenu)` (renvoie un `Result<usize, std::io::Error>`).
 - Pour nettoyer la chaîne : `contenu.trim()` .
 - Pour convertir en entier : `contenu.trim().parse::<u16>()` (renvoie un `Result<u16, ParseIntError>`).
 - N'oubliez pas d'importer les modules nécessaires : `std::fs::File` , `std::io::Read` et `std::error::Error` .
-

Correction Complète et Commentée

Voici l'implémentation attendue. Vous pouvez copier ce code dans votre fichier `src/main.rs` pour l'exécuter.

```

use std::fs::File;
use std::io::Read;
use std::error::Error;

/// Tente de charger et de parser le port depuis le fichier "port.txt".
/// Propage proprement toutes les erreurs via l'opérateur `?`.
fn charger_port() → Result<u16, Box<dyn Error>> {
    // 1. Ouverture du fichier. Si le fichier n'existe pas, l'erreur est propagée immédiatement.
    let mut fichier = File::open("port.txt")?;

    // 2. Lecture du contenu du fichier dans une chaîne de caractères.
    let mut contenu = String::new();
    fichier.read_to_string(&mut contenu)?;

    // 3. Nettoyage des espaces/retours à la ligne et tentative de conversion en u16.
    // L'opérateur `?` convertira automatiquement une ParseIntError en Box<dyn Error>.
    let port: u16 = contenu.trim().parse()?;

    // 4. Tout s'est bien passé, on retourne le port enveloppé dans Ok.
    Ok(port)
}

fn main() {
    // Simulation de l'exécution
    println!("Tentative de chargement de la configuration...");

    match charger_port() {
        Ok(port) ⇒ {
            println!("Configuration chargée avec succès !");
            println!("Le serveur va démarrer sur le port : {}", port);
        }
        Err(erreur) ⇒ {
            // Ici, nous interceptons l'erreur finale et l'affichons proprement à l'utilisateur.
            eprintln!("Erreur critique lors du chargement de la configuration : {}", erreur);
            eprintln!("Détails : Veuillez vérifier que le fichier 'port.txt' existe et contient un port valide.");
        }
    }
}

```

Explications de la correction :

- Ligne 9 : `File::open("port.txt")?` -> Si le fichier `port.txt` est manquant, la fonction s'arrête immédiatement ici et renvoie l'erreur `std::io::Error` encapsulée dans le `Box` de retour.

- Ligne 13 : `fichier.read_to_string(&mut contenu)?` -> Si le fichier est verrouillé ou illisible, l'erreur de lecture est propagée de la même façon.
- Ligne 17 : `contenu.trim().parse()?` -> Si le fichier contenait `"8080\n"`, `.trim()` le transforme en `"8080"`. Si le fichier contenait `"texte_invalide"`, `.parse()` échoue et renvoie une `ParseIntError`, qui est immédiatement convertie et propagée.
- Ligne 26 : Dans le `main`, nous utilisons un `match` pour traiter le résultat final. C'est la bonne pratique : on propage avec `?` tout au long de la chaîne d'appels, et on gère l'erreur à la toute fin (ici, dans le point d'entrée du programme).

Exercices d'Application :

Propagation propre des erreurs avec l'opérateur '?'

Ce cahier d'exercices a pour objectif de consolider votre maîtrise de l'opérateur `?` en Rust. À travers trois exercices de difficulté progressive, vous apprendrez à convertir des `Option` en `Result`, à manipuler des types d'erreurs multiples et à structurer un programme complet avec un point d'entrée `main` sécurisé.

Exercice 1 : Récupération de Clé d'API (Option vers Result)

Difficulté : Facile

Énoncé

Dans les applications réelles, les configurations sont souvent stockées sous forme de paires clé-valeur. Vous devez concevoir une fonction qui recherche une clé d'API spécifique (`"API_KEY"`) dans une table de hachage (`HashMap`).

Cependant, la méthode `.get()` d'une `HashMap` renvoie une `Option<V>`. Votre objectif est d'écrire une fonction `recuperer_cle` qui : 1. Cherche la clé `"API_KEY"` dans la configuration fournie. 2. Si la clé est présente, renvoie sa valeur sous forme de `String`. 3. Si la clé est absente, convertit l'absence en erreur avec un message explicite (`"Clé d'API manquante dans la configuration"`) et propage cette erreur sous forme de `Result<String, String>`.

Indices

1. Pour convertir une `Option<T>` en `Result<T, E>`, utilisez la méthode `.ok_or(erreur)`.
 2. N'oubliez pas d'utiliser l'opérateur `?` après la conversion pour extraire la valeur ou propager l'erreur immédiatement.
 3. La signature de votre fonction devra ressembler à ceci : `rust fn recuperer_cle(config: &HashMap<String, String>) → Result<String, String>`
-

Correction Détaillée

```
use std::collections::HashMap;

/// Recherche la clé "API_KEY" et propage une erreur textuelle si elle est absente.
fn recuperer_cle(config: &HashMap<String, String>) → Result<String, String> {
    // 1. .get() renvoie Option<&String>
    // 2. .ok_or() convertit l'Option en Result<&String, &String>
    // 3. L'opérateur `?` déballe le &String ou retourne anticipativement une erreur
    let cle = config
        .get("API_KEY")
        .ok_or("Clé d'API manquante dans la configuration")?;

    // On convertit le &String en String pour correspondre au type de retour
    Ok(cle.clone())
}

fn main() {
    // Cas 1 : La configuration est correcte
    let mut config_valide = HashMap::new();
    config_valide.insert("API_KEY".to_string(), "super-secret-token-123".to_string());

    match recuperer_cle(&config_valide) {
        Ok(cle) ⇒ println!("Succès (Cas 1) : Clé trouvée = {}", cle),
        Err(e) ⇒ println!("Erreur (Cas 1) : {}", e),
    }

    // Cas 2 : La configuration est vide
    let config_vide = HashMap::new();
    match recuperer_cle(&config_vide) {
        Ok(cle) ⇒ println!("Succès (Cas 2) : Clé trouvée = {}", cle),
        Err(e) ⇒ println!("Erreur (Cas 2) : {}", e), // Devrait afficher une erreur
    }
}
```

Explications : * Nous utilisons `.ok_or("...")` pour transformer l'absence de valeur (`None`) en une erreur contenant notre message. * L'opérateur `?` appliqué juste après permet de sortir immédiatement de la fonction en renvoyant `Err("...")` si la clé n'existe pas. Si elle existe, il extrait le contenu du `Some`.

Exercice 2 : Analyseur de Profil Utilisateur (Types Mixtes)

Difficulté : Intermédiaire

Énoncé

Vous recevez des données brutes sous forme de chaînes de caractères au format CSV simplifié : `"Nom, Age, Identifiant"` . Par exemple : `"Alice, 30, 1045"` .

Vous devez écrire une fonction `parser_profil` qui prend cette chaîne en paramètre et tente de construire une structure `Utilisateur` . Cette opération peut échouer pour plusieurs raisons : * Il manque des champs dans la chaîne (ex: `"Alice, 30"`). * L'âge n'est pas un nombre valide (ex: `"Alice, trente, 1045"`). * L'identifiant n'est pas un nombre valide (ex: `"Alice, 30, id_invalide"`).

Puisque vous allez manipuler des erreurs de parsing d'entiers (`ParseIntError`) et des erreurs d'éléments manquants (que vous générerez vous-même), vous utiliserez le type de retour dynamique `Result<Utilisateur, Box<dyn std::error::Error>>` .

Indices

1. Utilisez la méthode `.split(',')` pour découper la chaîne. Cela vous donne un itérateur.
 2. Pour récupérer chaque élément de l'itérateur, utilisez `.next()` . Cette méthode renvoie une `Option<&str>` .
 3. Convertissez chaque `Option<&str>` en `Result<&str, &str>` avec `.ok_or("Champ manquant")?` pour extraire la valeur textuelle de manière sécurisée.
 4. Utilisez `.parse::<u8>()?` et `.parse::<u32>()?` pour convertir l'âge et l'identifiant. L'opérateur `?` se chargera de convertir automatiquement les erreurs de parsing en `Box<dyn Error>` .
-

Correction Détaillée

```

use std::error::Error;

#[derive(Debug)]
struct Utilisateur {
    nom: String,
    age: u8,
    id: u32,
}

/// Parse une ligne CSV pour créer un Utilisateur.
/// Propage n'importe quel type d'erreur via Box<dyn Error>.
fn parser_profil(ligne: &str) → Result<Utilisateur, Box<dyn Error>> {
    let mut segments = ligne.split(',');

    // Extraction et validation du nom
    let nom = segments
        .next()
        .ok_or("Nom manquant dans les données")?;

    // Extraction et validation de l'âge
    let age_str = segments
        .next()
        .ok_or("Âge manquant dans les données")?;
    // Conversion de l'âge (propage une ParseIntError si invalide)
    let age = age_str.trim().parse::<u8>()?;

    // Extraction et validation de l'identifiant
    let id_str = segments
        .next()
        .ok_or("Identifiant manquant dans les données")?;
    // Conversion de l'identifiant (propage une ParseIntError si invalide)
    let id = id_str.trim().parse::<u32>()?;

    // Tout est correct, on construit et retourne l'utilisateur
    Ok(Utilisateur {
        nom: nom.to_string(),
        age,
        id,
    })
}

fn main() {
    let profils = vec![
        "Alice,30,1045", // Valide
        "Bob,quarante,2001", // Âge invalide
        "Charlie,25", // Identifiant manquant
    ];
}

```

```

    for profil in profils {
        match parser_profil(profil) {
            Ok(u) => println!("Utilisateur créé avec succès : {:?}", u),
            Err(e) => eprintln!("Échec du parsing pour '{}': {}", profil, e),
        }
    }
}

```

Explications : * Combinaison des types : `.next()` renvoie une `Option`. Nous la transformons en `Result` avec `.ok_or()`, puis nous appliquons `?`. * Conversion automatique : Quand nous écrivons `parse::<u8>()?` , le compilateur Rust détecte que la fonction retourne un `Box<dyn Error>`. Il utilise automatiquement le trait `From` pour encapsuler la `ParseIntError` dans la `Box`. Notre code reste parfaitement propre et linéaire.

Exercice 3 : Le Sommet de la Chaîne (Intégration Complète avec `main`)

Difficulté : Avancée

Énoncé

Vous allez concevoir un utilitaire en ligne de commande complet appelé "Calculateur de Somme". Le programme doit lire un fichier texte (dont le chemin est fixe : `"nombres.txt"`). Ce fichier contient un nombre entier par ligne. Votre programme doit lire le fichier, parser chaque ligne en un entier `i32`, calculer la somme totale, et l'afficher.

Vous devez : 1. Écrire une fonction `calculer_somme_fichier` qui prend le chemin du fichier en paramètre, lit son contenu, fait la somme et la renvoie. 2. Modifier la fonction `main` pour qu'elle retourne elle-même un `Result<(), Box<dyn Error>>` afin de pouvoir utiliser l'opérateur `?` directement à la racine du programme. 3. Gérer proprement le cas où le fichier n'existe pas ou contient des lignes non numériques en affichant l'erreur finale.

Indices

1. Pour lire l'intégralité d'un fichier dans un `String` en une seule ligne :
`std::fs::read_to_string(chemin)? .`
 2. Pour parcourir les lignes d'une chaîne : utilisez la méthode `.lines()` .
 3. Dans la boucle de calcul, utilisez `?` sur chaque conversion `.parse::<i32>()? .` pour interrompre immédiatement le calcul global si une seule ligne est corrompue.
-

Correction Détaillée

Pour tester ce code localement, créez un fichier nommé `nombres.txt` à la racine de votre projet Cargo avec le contenu suivant :

```
10
25
-5
30
```

Voici le code complet de l'application :


```

use std::fs;
use std::error::Error;

/// Lit un fichier ligne par ligne, convertit chaque ligne en i32 et calc
fn calculer_somme_fichier(chemin: &str) → Result<i32, Box<dyn Error>> {
    // 1. Lecture du fichier. Si le fichier n'existe pas, l'erreur d'E/S
    let contenu = fs::read_to_string(chemin)?;

    let mut somme = 0;

    // 2. Parcours de chaque ligne du fichier
    for ligne in contenu.lines() {
        // On nettoie les espaces blancs éventuels
        let ligne_nettoyee = ligne.trim();

        // On ignore les lignes vides pour éviter des erreurs de parsing
        if ligne_nettoyee.is_empty() {
            continue;
        }

        // 3. Tentative de conversion de la ligne en entier.
        // Si une ligne est invalide (ex: "abc"), la fonction s'arrête et
        let nombre: i32 = ligne_nettoyee.parse()?;
        somme += nombre;
    }

    Ok(somme)
}

// Configuration de la fonction main pour propager les erreurs au système
fn main() → Result<(), Box<dyn Error>> {
    println!("Démarrage du calculateur de somme...");

    let chemin_fichier = "nombres.txt";

    // Utilisation de l'opérateur ? dans le main
    let total = calculer_somme_fichier(chemin_fichier)?;

    println!("La somme totale des nombres dans '{}' est : {}", chemin_fic
    // Tout s'est bien passé, on retourne Ok contenant l'unité ()
    Ok(())
}

```

Explications : * `main` avec `Result` : En modifiant la signature de `main` pour renvoyer `Result<(), Box<dyn Error>>`, nous pouvons utiliser `?` à la ligne 37. Si

`calculer_somme_fichier` échoue, le programme s'arrête proprement et Rust affiche automatiquement une description textuelle de l'erreur sur la console (avec un code de retour système non nul). * **Robustesse** : Ce programme gère de manière transparente l'absence du fichier, les problèmes de permissions de lecture, ainsi que les erreurs de formatage des données à l'intérieur du fichier, le tout en moins de 40 lignes de code lisibles et structurées.

Concepts Avancés de Sécurité : Lifetimes et Concurrency

Les Durées de Vie (Lifetimes) :

Empêcher définitivement les références pendantes

Dans les modules précédents, nous avons exploré le système de propriété (*ownership*) et d'emprunt (*borrowing*) de Rust. Nous avons vu comment le compilateur garantit qu'une ressource possède un unique propriétaire et comment nous pouvons prêter cette ressource via des références.

Cependant, un problème fondamental subsiste pour tout système de gestion mémoire statique : comment s'assurer qu'une référence ne pointe jamais vers une donnée qui a déjà été libérée en mémoire ?

C'est ici qu'interviennent les *durées de vie* (*lifetimes*). Ce concept, souvent perçu comme l'un des plus intimidants de Rust, est en réalité la formalisation mathématique et logique de la portée des variables. Ce module va démystifier les *lifetimes*, vous expliquer pourquoi elles existent, comment le compilateur les analyse, et comment les utiliser pour concevoir des programmes d'une robustesse absolue.

1. Introduction Conceptuelle : Le "Pourquoi" avant le "Comment"

Le danger absolu : La référence pendante (Dangling Reference)

Une référence pendante se produit lorsqu'un programme pointe vers une adresse mémoire qui a été libérée ou réallouée. En C ou en C++, c'est une source majeure de failles de sécurité (vulnérabilités de type *Use-After-Free*) et de plantages imprévisibles.

Imaginons le scénario suivant en pseudo-code :

```
Créer une référence appelée 'pointeur'
{
    Créer une variable 'donnee' contenant la valeur 42
    Faire pointer 'pointeur' vers 'donnee'
} // 'donnee' est détruite ici (fin de portée)

Lire la valeur pointée par 'pointeur' // DANGER : 'pointeur' pointe vers du vide
```

Si le système d'exploitation a réutilisé cette zone mémoire pour une autre application ou une autre variable, lire ou écrire via `pointeur` corrompra les données ou provoquera un plantage (*Segmentation Fault*).

L'analogie du contrat de location

Pour comprendre comment Rust résout ce problème, imaginez que vous signez un **contrat de sous-location** (la référence) pour un appartement. * Le propriétaire de l'immeuble a un **bail principal** qui expire le 31 décembre (la durée de vie de la donnée d'origine). * Si votre contrat de sous-location stipule que vous pouvez occuper l'appartement jusqu'au 15 janvier de l'année suivante, il y a une incohérence majeure : vous essayez de louer un espace qui n'appartiendra plus au bailleur principal. * Pour éviter cela, une règle logique s'impose : la **durée de la sous-location doit être strictement inférieure ou égale à la durée du bail principal**.

En Rust, le *Borrow Checker* (le vérificateur d'emprunts) est l'inspecteur rigoureux qui s'assure que vos contrats de location (références) n'outrepassent jamais la validité des baux principaux (les propriétaires des données).

2. Fondations Théoriques

Le Borrow Checker à l'œuvre : Visualisation des portées

Le compilateur Rust utilise un composant appelé le *Borrow Checker* pour comparer les portées des variables et s'assurer que toutes les références sont valides.

Examinons un code Rust qui tente de créer une référence pendante :

```
fn main() {
    let r;                // -----+-- Durée de vie 'a
                          //          |
    {                    //          |
        let x = 5;        // ---+   |-- Durée de vie 'b
        r = &x;           //      |   |
    }                    // ---+   |
                          //          |
    println!("r: {}", r); //          |
}                        // -----+
```

Si nous analysons ce code avec des notations de durées de vie : * `'a` représente la durée de vie de la variable `r`. Elle commence à sa déclaration et se termine à la fin de la fonction `main`. * `'b` représente la durée de vie de la variable `x`. Elle est confinée au bloc interne.

Au moment de l'affectation `r = &x`, le compilateur constate que `r` (qui doit vivre pendant `'a`) fait référence à une donnée qui ne vit que pendant `'b`. Or, `'b` est plus courte que `'a` (`'b < 'a`).

Le *Borrow Checker* rejette immédiatement ce programme à la compilation avec un message d'erreur explicite : `x does not live long enough`. Aucun binaire n'est généré, le bug est éliminé avant même d'avoir pu exister à l'exécution.

Qu'est-ce qu'une annotation de durée de vie (Lifetime Annotation) ?

Les annotations de durées de vie ne modifient pas la durée réelle de vie d'une variable. Elles ne font que décrire les relations entre les durées de vie de plusieurs références pour aider le compilateur à valider les emprunts.

La syntaxe utilise une apostrophe (`'`) suivie d'un nom en minuscules (généralement `'a`, `'b`, etc.) :

- `&i32` : Une référence simple.
- `&'a i32` : Une référence avec une durée de vie explicite nommée `'a`.
- `&'a mut i32` : Une référence mutable avec une durée de vie explicite nommée `'a`.

Les trois règles d'élision des durées de vie (Lifetime Elision)

Pourquoi n'écrivons-nous pas des `'a` partout dans notre code quotidien ? Parce que le compilateur Rust est capable d'analyser le code et d'insérer ces annotations de manière déterministe dans les cas les plus fréquents. C'est ce qu'on appelle l'élision des durées de vie.

Le compilateur applique trois règles strictes pour déterminer si les durées de vie peuvent être omises :

1. Première règle : Chaque paramètre d'entrée qui est une référence se voit attribuer sa propre durée de vie.
2. `fn foo(x: &i32)` devient `fn foo<'a>(x: &'a i32)`
3. `fn bar(x: &i32, y: &i32)` devient `fn bar<'a, 'b>(x: &'a i32, y: &'b i32)`
4. Deuxième règle : S'il y a exactement une seule référence en entrée, sa durée de vie est automatiquement attribuée à toutes les références de sortie (les valeurs de retour).
5. `fn foo(x: &i32) → &i32` devient `fn foo<'a>(x: &'a i32) → &'a i32`
6. Troisième règle : S'il y a plusieurs paramètres de référence en entrée, mais que l'un d'eux est `&self` ou `&mut self` (méthodes d'une structure), la durée de vie de `self` est attribuée à toutes les valeurs de retour.

Si, après l'application de ces règles, le compilateur ne parvient pas à déterminer la durée de vie des références de retour, il s'arrête et demande une annotation explicite.

La durée de vie spéciale : `'static`

La durée de vie `'static` est une durée de vie réservée qui signifie que la donnée référencée vit pendant toute la durée de l'exécution du programme.

Il existe deux situations courantes où `'static` intervient :

1. Les littéraux de chaîne de caractères : `rust let s: &'static str = "Je suis stocké directement dans le binaire !";` Ces chaînes sont encodées directement dans le segment de données en lecture seule de l'exécutable. Elles ne sont jamais désallouées.
2. Les variables globales (déclarées avec le mot-clé `static`).

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Le problème de la fonction de sélection

Tentons d'écrire une fonction qui prend deux tranches de chaînes de caractères (*string slices*) et retourne la plus longue des deux.

```
// Ce code NE compile PAS
fn obtenir_la_plus_longue(x: &str, y: &str) → &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let chaine1 = String::from("Rust");
    let chaine2 = "Programmation";

    let resultat = obtenir_la_plus_longue(&chaine1, chaine2);
    println!("La plus longue est : {}", resultat);
}
```

Si nous essayons de compiler ce code, Rust lève une erreur :

```
error[E0106]: missing lifetime specifier
  → src/main.rs:2:48
   |
2 | fn obtenir_la_plus_longue(x: &str, y: &str) → &str {
   |                                ----      ^ expected named lifetime parameter
```

Pourquoi le compilateur échoue-t-il ? En appliquant la règle d'élision n°1, le compilateur génère deux durées de vie distinctes pour les entrées : `x: &'a str` et `y: &'b str`. Cependant, la règle n°2 ne s'applique pas (il y a deux entrées, pas une seule), et la règle n°3 non plus. Le compilateur ne sait pas si la référence retournée proviendra de `x` (durée de vie `'a`) ou de `y` (durée de vie `'b`). Il refuse de compiler pour garantir la sécurité mémoire.

Étape 2 : Résolution avec les annotations génériques de durées de vie

Pour résoudre ce problème, nous devons déclarer un paramètre de durée de vie générique `'a` et l'associer à nos paramètres et à notre valeur de retour :

```
// Ce code compile parfaitement !
fn obtenir_la_plus_longue<'a>(x: &'a str, y: &'a str) → &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let chaine1 = String::from("Le langage Rust");
    {
        let chaine2 = String::from("C++");
        let resultat = obtenir_la_plus_longue(&chaine1, &chaine2);
        println!("La plus longue est : {}", resultat);
    } // chaine2 est détruite ici
}
```

Que signifie concrètement `'a` ici ? Nous indiquons au compilateur : *"La référence retournée par cette fonction vivra au moins aussi longtemps que la plus courte des durées de vie de `x` et de `y`."*

Si nous tentons d'utiliser le résultat en dehors de la portée de la plus courte des variables, le compilateur nous arrêtera :

```
fn main() {
    let chaine1 = String::from("Le langage Rust");
    let resultat;
    {
        let chaine2 = String::from("C++");
        // Erreur : chaine2 ne vit pas assez longtemps pour 'resultat'
        resultat = obtenir_la_plus_longue(&chaine1, &chaine2);
    }
    // println!("{}", resultat); // Si décommenté, provoque une erreur de compilation !
}
```

Étape 3 : Structurer des données contenant des références

Jusqu'à présent (notamment dans le Module 3), nous avons principalement manipulé des structures possédant leurs propres données (comme `String` ou `i32`).

Si vous souhaitez qu'une structure contienne une référence vers une donnée détenue par quelqu'un d'autre, vous devez spécifier une durée de vie.

```
// La structure ne peut pas survivre à la référence qu'elle contient dans 'partie'
struct ExtraitImportant<'a> {
    partie: &'a str,
}

fn main() {
    let roman = String::from("Appelez-moi Ismael. Il y a quelques années...");

    // On extrait la première phrase
    let premiere_phrase = roman
        .split('.')
        .next()
        .expect("Impossible de trouver un point");

    let extrait = ExtraitImportant {
        partie: premiere_phrase,
    };

    println!("Extrait retenu : {}", extrait.partie);
}
```

Ici, `ExtraitImportant<'a>` signifie que toute instance de cette structure ne peut pas vivre plus longtemps que la référence stockée dans son champ `partie`.

4. Pièges Fréquents et Bonnes Pratiques

Piège n°1 : Tenter de retourner une référence vers une variable locale

C'est l'erreur classique du débutant en Rust.

```
// ERREUR DE COMPILATION MAJEURE
fn creer_message<'a>() → &'a str {
    let message = String::from("Bonjour");
    &message // Erreur : message est détruit à la fin de la fonction !
}
```

Pourquoi cela échoue-t-il ? La variable `message` est créée sur la pile de la fonction `creer_message`. À la fin de la fonction, `message` sort de la portée et sa mémoire est libérée. Tenter de retourner une référence vers cette mémoire libérée est une violation de sécurité. Même en ajoutant des annotations de durée de vie, le compilateur refusera le code.

La solution : Transférer la propriété (*ownership*) au lieu de retourner une référence.

```
// Solution correcte : on retourne la donnée elle-même
fn creer_message() → String {
    String::from("Bonjour")
}
```

Piège n°2 : La sur-annotation inutile

Parfois, par peur du compilateur, les développeurs ajoutent des durées de vie partout, ce qui rend le code illisible et inutilement complexe.

```
// Mauvais style : annotations inutiles car couvertes par l'élision
fn afficher<'a>(s: &'a str) {
    println!("{}", s);
}

// Bon style : simple et propre
fn afficher(s: &str) {
    println!("{}", s);
}
```

Bonnes pratiques

1. Laissez faire le compilateur d'abord : N'écrivez pas d'annotations de durée de vie à moins que le compilateur ne vous le demande explicitement.

2. Favorisez la propriété dans les structures : Si vous débutez, préférez utiliser des types possédés (`String` , `Vec`) dans vos structures plutôt que des références. Cela simplifie grandement la gestion des types et évite de propager des paramètres de durée de vie dans tout votre code.
3. Comprenez le message d'erreur : Rust fournit des explications extrêmement détaillées. Si le compilateur vous dit `borrowed value does not live long enough` , dessinez sur un papier les portées de vos variables pour identifier laquelle s'arrête trop tôt.

5. Synthèse Pédagogique

Tableau comparatif : Gestion des références

Concept	Syntaxe	Rôle principal	Quand l'utiliser ?
Référence simple	<code>&T</code>	Accéder à une donnée sans la copier.	Par défaut pour la lecture de données.
Élision	<i>Invisible</i>	Simplifier l'écriture en omettant les annotations évidentes.	Géré automatiquement par le compilateur.
Lifetime explicite	<code>&'a T</code>	Lier la validité d'une référence à une autre.	Fonctions à multiples entrées/sorties de références, structures contenant des références.
Lifetime statique	<code>&'static T</code>	Indiquer que la donnée est valide pour toute la durée du programme.	Constantes, chaînes littérales, données globales.

Points clés à retenir

- Les durées de vie n'altèrent pas le cycle de vie des variables à l'exécution ; elles ne font que le décrire au compilateur.
- Le *Borrow Checker* garantit qu'aucune référence ne peut survivre à son propriétaire.

- Si une structure stocke une référence, elle doit obligatoirement déclarer un paramètre de durée de vie générique.
-

6. Exercice Pratique d'Application

Énoncé : Le Parseur de Logs Zéro-Copie (Zero-Copy Log Parser)

Dans les systèmes haute performance, on évite au maximum d'allouer de la mémoire sur le tas (*heap*). Nous allons concevoir un parseur de logs qui extrait des informations d'une ligne de log sans jamais copier les chaînes de caractères d'origine. Tout se fera par le biais de références partagées.

Une ligne de log est structurée ainsi : `"[NIVEAU] Message de log"` (par exemple : `"[ERROR] Connexion échouée"`).

Objectifs :

1. Définir une structure `MessageLog<'a>` contenant deux champs de type `&'a str` : `niveau` et `contenu`.
2. Écrire une fonction `parser_log<'a>(ligne: &'a str) → Option<MessageLog<'a>>` qui analyse la chaîne de caractères et extrait le niveau et le contenu.
3. Si la ligne n'est pas correctement formatée (absence de `[` ou `]`), retourner `None`.
4. Écrire un programme de test dans le `main` démontrant la sécurité mémoire de votre implémentation.

Indices pour la résolution

- Pour trouver la position d'un caractère, vous pouvez utiliser `ligne.find('[')` ou `ligne.find(']')`.
 - Utilisez le découpage de chaînes (*string slicing*) : `&ligne[debut..fin]`.
 - Rappelez-vous que la structure `MessageLog` ne peut pas vivre plus longtemps que la chaîne de caractères `ligne` d'origine que vous lui passez.
-

Correction Complète et Commentée

Voici l'implémentation complète, rigoureuse et commentée de l'exercice.

```

// 1. Définition de la structure avec sa durée de vie générique 'a
#[derive(Debug)]
struct MessageLog<'a> {
    niveau: &'a str,
    contenu: &'a str,
}

// 2. Fonction de parsing utilisant la même durée de vie 'a pour l'entrée et la sortie
fn parser_log<'a>(ligne: &'a str) → Option<MessageLog<'a>> {
    // Recherche des délimiteurs du niveau de log
    let index_debut_crochet = ligne.find('(')?;
    let index_fin_crochet = ligne.find('')?;

    // Sécurité : s'assurer que le crochet fermant est bien après le crochet ouvrant
    if index_fin_crochet ≤ index_debut_crochet {
        return None;
    }

    // Extraction du niveau (entre les crochets)
    let niveau = &ligne[index_debut_crochet + 1..index_fin_crochet];

    // Le contenu commence après le crochet fermant et un espace potentiel
    let reste = &ligne[index_fin_crochet + 1..];
    let contenu = reste.trim(); // Supprime les espaces superflus au début et à la fin

    // On retourne la structure contenant les références vers la chaîne d'origine
    Some(MessageLog { niveau, contenu })
}

fn main() {
    // Cas nominal : la chaîne d'origine vit dans la portée du main
    let journal = String::from("[ERROR] Échec de la connexion à la base de données");

    if let Some(log) = parser_log(&journal) {
        println!("Parsing réussi !");
        println!("Niveau : {}", log.niveau);
        println!("Contenu : {}", log.contenu);
    } else {
        println!("Format de log invalide.");
    }

    // Démonstration de la sécurité mémoire par le compilateur :
    // Si nous essayons de faire ceci :
    /*
    let log_invalide;
    {
        let journal_temporaire = String::from("[WARN] Mémoire presque saturée");
        log_invalide = parser_log(&journal_temporaire);
    }
    */
}

```

```

    } // journal_temporaire est désalloué ici

    // Le compilateur refusera de compiler la ligne ci-dessous si elle est décommentée !
    // println!("Log temporaire : {:?} ", log_invalide);
    */
}

```

Analyse de la correction

- **Zéro allocation** : À aucun moment nous n'avons appelé `to_string()` ou `String::from()` dans le parseur. Les champs `niveau` et `contenu` de `MessageLog` pointent directement vers des segments de la chaîne `journal` présente dans la mémoire du `main`.
- **Sécurité absolue** : Si vous décommentez le bloc de test à la fin du `main`, le compilateur détectera immédiatement que `log_invalide` contient des références vers `journal_temporaire` qui n'existe plus. Le programme refusera de compiler, éliminant ainsi tout risque de bug de type *Use-After-Free*.

Exercices d'Application : Les Durées de Vie (Lifetimes)

Ce cahier d'exercices pratiques a pour objectif de consolider votre compréhension des durées de vie (*lifetimes*) en Rust. À travers trois exercices de difficulté progressive, vous passerez de la manipulation de fonctions simples à la conception de structures de données complexes et optimisées sans copie mémoire.

Exercice 1 : Le Sélecteur de Message avec Repli (Difficulté : Facile)

Énoncé

Dans cet exercice, vous devez concevoir un utilitaire de traitement de texte pour un système de notification. Vous devez écrire une fonction nommée `choisir_message` qui compare un message personnalisé et un message de repli (*fallback*).

La fonction doit : 1. Prendre en paramètre deux références de chaînes de caractères (`&str`) : `perso` et `repli` . 2. Prendre un troisième paramètre, un motif de recherche de type `&str` nommé `motif` . 3. Si le message `perso` contient le `motif` , la fonction retourne `perso` . 4. Sinon, elle retourne le message de `repli` .

Contrainte clé : Le motif de recherche (`motif`) est une variable temporaire qui n'a pas besoin de vivre aussi longtemps que les messages ou que la valeur de retour. Vous devez annoter les durées de vie de manière à ce que le compilateur comprenne que seule la durée de vie de `perso` et `repli` est liée à la valeur de retour.

Indices

1. Utilisez la méthode `chaine.contains(motif)` pour vérifier si le motif est présent.
2. Demandez-vous : *Quelles sont les références qui peuvent potentiellement être retournées par la fonction ?* Ce sont ces références (et uniquement elles) qui doivent partager la même durée de vie générique `'a`.
3. Le paramètre `motif` n'est jamais retourné. Sa durée de vie peut donc être éladée (laissée au compilateur) ou posséder sa propre durée de vie indépendante `'b`.

Correction Détaillée

```
// Nous définissons une durée de vie 'a pour les données qui peuvent être retournées.
// Le paramètre `motif` n'a pas besoin de vivre aussi longtemps, nous le faisons
// bénéficier de l'élimination automatique (ou nous aurions pu lui donner une durée de vie
fn choisir_message<'a>(perso: &'a str, repli: &'a str, motif: &str) -> &'a str {
    if perso.contains(motif) {
        perso
    } else {
        repli
    }
}

fn main() {
    let message_important = String::from("ALERTE: Une tentative de connexion suspecte a
    let message_secours = "Notification standard : RAS.";

    let resultat;
    {
        // Le motif est une variable à durée de vie très courte (limitée à ce bloc)
        let declencheur = String::from("ALERTE");

        resultat = choisir_message(&message_important, message_secours, &declencheur);
        // 'declencheur' est détruit ici, mais 'resultat' reste valide car il est lié
        // à la durée de vie de 'message_important' et 'message_secours'.
    }

    println!("Résultat de la sélection : {}", resultat);
}
```

Explications pas-à-pas : * Signature : `choisir_message<'a>(perso: &'a str, repli: &'a str, motif: &str) → &'a str` En liant `perso` , `repli` et le type de retour avec `'a` , nous garantissons au compilateur que la référence retournée sera valide tant que `perso` et `repli` le sont. * Indépendance de `motif` : Puisque `motif` n'est pas annoté avec `'a` , le compilateur lui attribue une durée de vie anonyme distincte. Cela permet d'appeler la fonction avec un motif temporaire (comme `declencheur` dans le bloc interne du `main`) sans restreindre la validité du résultat obtenu.

Exercice 2 : Le Parseur de Profil Utilisateur "Zéro-Copie" (Difficulté : Moyenne)

Énoncé

Pour éviter des allocations mémoire inutiles sur le tas (*heap*), vous devez concevoir un parseur de profil utilisateur "zéro-copie". Les données brutes arrivent sous la forme d'une seule chaîne de caractères formatée ainsi : `"ID:1024|Nom:Alice|Role:Administrateur"` .

Objectifs :

1. Créez une structure `ProfilUtilisateur<'a>` contenant trois champs de type `&'a str : id , nom , et role` .
2. Écrivez une fonction `parser_profil<'a>(donnees: &'a str) → Option<ProfilUtilisateur<'a>>` .
3. Cette fonction doit découper la chaîne d'origine pour extraire les valeurs associées à chaque champ. Si un champ est manquant ou si le format est invalide, retournez `None` .
4. Écrivez un code de test dans le `main` qui prouve que la structure ne peut pas survivre à la chaîne de caractères source.

Indices

1. Vous pouvez utiliser la méthode `donnees.split('|')` pour séparer les paires clé-valeur.
2. Pour chaque paire (ex: `"Nom:Alice"`), vous pouvez utiliser `.split_once(':')` qui retourne un tuple `Some((cle, valeur))` bien pratique.
3. Rappelez-vous que la structure contenant des références doit déclarer sa durée de vie : `struct MonNom<'a> { champ: &'a str } .`

Correction Détaillée

```

// 1. Définition de la structure avec sa durée de vie générique
#[derive(Debug)]
struct ProfilUtilisateur<'a> {
    id: &'a str,
    nom: &'a str,
    role: &'a str,
}

// 2. Fonction de parsing zéro-copie
fn parser_profil<'a>(donnees: &'a str) → Option<ProfilUtilisateur<'a>> {
    let mut id = None;
    let mut nom = None;
    let mut role = None;

    // Découpage par le délimiteur '|'
    for segment in donnees.split('|') {
        // Découpage de chaque segment sur le premier ':'
        if let Some((cle, valeur)) = segment.split_once(':') {
            match cle {
                "ID" ⇒ id = Some(valeur),
                "Nom" ⇒ nom = Some(valeur),
                "Role" ⇒ role = Some(valeur),
                _ ⇒ {} // On ignore les clés inconnues
            }
        }
    }

    // On reconstruit le profil si tous les champs obligatoires sont présents
    Some(ProfilUtilisateur {
        id: id?,
        nom: nom?,
        role: role?,
    })
}

fn main() {
    // Cas nominal
    let donnees_brutes = String::from("ID:8842|Nom:Thomas|Role:Developpeur");

    let profil = parser_profil(&donnees_brutes);
    match profil {
        Some(p) ⇒ println!("Profil parsé avec succès : {:?}", p),
        None ⇒ println!("Échec du parsing : format invalide."),
    }

    // Démonstration de la sécurité mémoire (Vérification du Borrow Checker)
    /*
    let profil_invalide;
    */
}

```

```

{
    let donnees_temporaires = String::from("ID:99|Nom:Inconnu|Role:Inconnu");
    profil_invalide = parser_profil(&donnees_temporaires);
} // 'donnees_temporaires' est détruite ici.

// Si vous décommentez la ligne suivante, le compilateur refusera de compiler !
// println!("Profil temporaire : {:?}", profil_invalide);
*/
}

```

Explications pas-à-pas : * Zéro allocation : La structure `ProfilUtilisateur` ne contient aucun type `String`. Elle ne fait que "pointer" (via des références `&str`) vers des sous-parties de la chaîne `donnees_brutes` déjà présente en mémoire. * Propagation de la durée de vie : La signature `parser_profil<'a>(donnees: &'a str) → Option<ProfilUtilisateur<'a>>` indique au compilateur que le profil retourné est intimement lié à la chaîne `donnees`. Si la chaîne source est désallouée, le profil devient inutilisable, empêchant toute tentative d'accès à de la mémoire libérée.

Exercice 3 : L'Indexeur de Mots-Clés (Difficulté : Difficile)

Énoncé

Dans les moteurs de recherche, on indexe des documents textuels pour retrouver rapidement les mots. Vous allez concevoir un `Indexeur` qui stocke des références vers des mots uniques extraits de différents textes.

Objectifs :

1. Créez une structure `Indexeur<'a>` qui contient un vecteur de références de chaînes : `mots: Vec<&'a str>`.
2. Implémentez les méthodes suivantes pour `Indexeur<'a>` :
3. `new()` : Initialise un indexeur vide.

4. `ajouter_phrase(&mut self, phrase: &'a str)` : Découpe la phrase en mots (séparés par des espaces) et ajoute à l'index les mots qui ne s'y trouvent pas déjà.
5. `contient(&self, mot: &str) → bool` : Indique si le mot est présent dans l'index.
6. Le Piège à éviter : Faites attention à la signature de la méthode `ajouter_phrase` . La référence mutable sur l'indexeur (`&mut self`) ne doit pas être confondue ou fusionnée de force avec la durée de vie `'a` des données qu'il stocke.

Indices

1. Pour implémenter des méthodes sur une structure ayant une durée de vie, la syntaxe est : `impl<'a> Indexeur<'a> { ... }` .
2. Dans `ajouter_phrase(&mut self, phrase: &'a str)` , notez bien que `&mut self` n'a pas besoin de l'annotation `'a` . Si vous écriviez `&'a mut self` , vous bloqueriez l'indexeur en écriture pour toute sa durée de vie, le rendant inutilisable après le premier appel !
3. Utilisez `phrase.split_whitespace()` pour extraire les mots.

Correction Détaillée

```

// 1. Définition de la structure de l'indexeur
struct Indexeur<'a> {
    mots: Vec<&'a str>,
}

// 2. Implémentation des méthodes
impl<'a> Indexeur<'a> {
    // Constructeur
    fn new() → Self {
        Indexeur { mots: Vec::new() }
    }

    // Ajoute les mots d'une phrase s'ils ne sont pas déjà présents
    fn ajouter_phrase(&mut self, phrase: &'a str) {
        for mot in phrase.split_whitespace() {
            // Nettoyage basique de la ponctuation pour l'exemple
            let mot_nettoye = mot.trim_matches(|c: char| c.is_ascii_punctuation());

            if !mot_nettoye.is_empty() && !self.contient(mot_nettoye) {
                self.mots.push(mot_nettoye);
            }
        }
    }

    // Vérifie la présence d'un mot (recherche insensible à la casse)
    fn contient(&self, mot: &str) → bool {
        self.mots
            .iter()
            .any(|&m| m.eq_ignore_ascii_case(mot))
    }
}

fn main() {
    let mut indexeur = Indexeur::new();

    // Premier texte (durée de vie 'static car c'est un littéral)
    let texte_1 = "Rust est un langage moderne.";
    indexeur.ajouter_phrase(texte_1);

    {
        // Deuxième texte (durée de vie plus courte, limitée à ce bloc)
        let texte_2 = String::from("La sécurité mémoire est essentielle !");
        indexeur.ajouter_phrase(&texte_2);

        println!("L'index contient 'sécurité' ? {}", indexeur.contient("sécurité")); // true
        println!("L'index contient 'moderne' ? {}", indexeur.contient("moderne")); // true

        // Tout fonctionne parfaitement ici car 'texte_2' est encore en v
    }
}

```

```

}
// 'texte_2' est détruit ici.

// ATTENTION : Si nous essayons d'utiliser `indexeur` maintenant, le compilateur
// lèvera une erreur car l'indexeur contient des références vers 'texte_2' qui a été détruit.

// println!("Contenu de l'index : {:?}", indexeur.mots); // ← ERREUR DE COMPILATION
}

```

Explications pas-à-pas : * Distinction des durées de vie : Dans `fn ajouter_phrase(&mut self, phrase: &'a str)`, le paramètre `&mut self` est équivalent à `&'b mut Self` où `'b` est une durée de vie très courte (juste le temps de l'appel de la fonction). En revanche, `phrase` est liée à `'a`. Cela permet d'appeler `ajouter_phrase` plusieurs fois de suite avec des phrases différentes, tant que ces phrases vivent au moins aussi longtemps que l'indexeur lui-même. * Sécurité du Borrow Checker : Dès que `texte_2` sort de la portée, l'indexeur devient "contaminé" par une référence invalide. Le compilateur Rust suit cette trace de validité à la trace et interdit toute lecture ultérieure de `indexeur`, protégeant ainsi le programme contre un crash ou un comportement indéterminé.

Module 5.2 : Introduction aux Traits :

Définir des comportements sûrs et partagés

1. Introduction Conceptuelle

Dans le paradigme de la programmation orientée objet classique (comme en C++ ou en Java), le partage de comportements et le polymorphisme reposent majoritairement sur l'héritage de classes. Une classe fille hérite des attributs et des méthodes d'une classe mère. Bien que répandue, cette approche présente des failles structurelles bien connues des concepteurs de systèmes :

- * **Le couplage fort** : Toute modification de la classe mère peut casser silencieusement le comportement des classes filles (problème de la classe de base fragile).
- * **L'héritage multiple problématique** : Connu sous le nom de "problème du diamant", il introduit des ambiguïtés complexes lorsque deux classes mères définissent la même méthode.
- * **Le mélange des responsabilités** : L'héritage force souvent à partager à la fois la structure des données (les champs) et le comportement (les fonctions), ce qui nuit à la modularité.

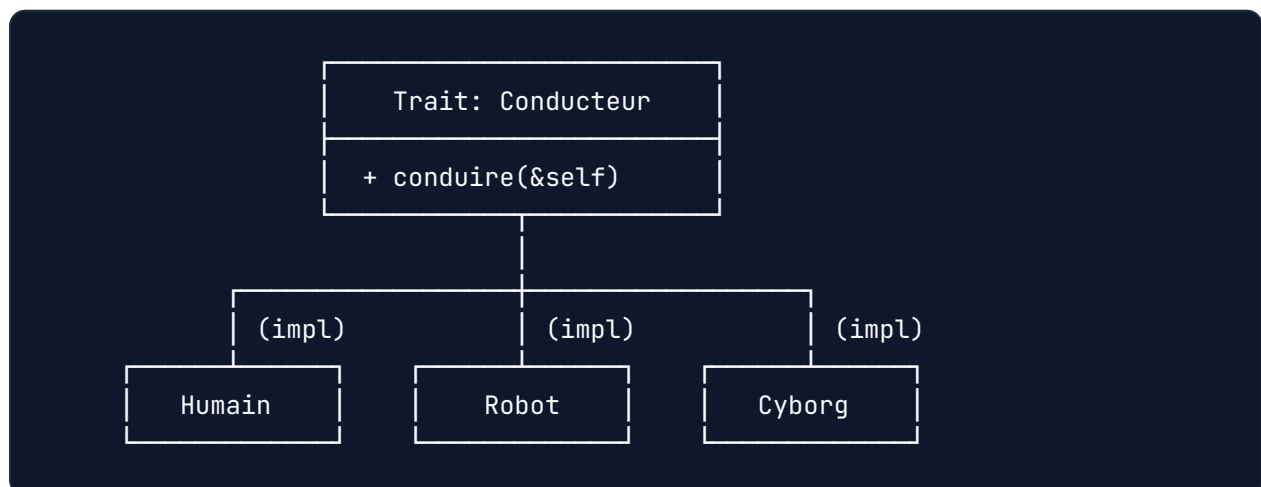
Rust fait un choix architectural radicalement différent : il n'y a pas d'héritage.

Pour résoudre le problème du partage de comportements de manière sûre et performante, Rust introduit le concept de Traits. Un trait est une définition abstraite d'un ensemble de méthodes qu'un type concret doit fournir pour satisfaire un certain rôle.

L'analogie du contrat

Visualisez un trait comme un contrat d'aptitude. * Considérons le contrat "Conducteur". Ce contrat stipule une exigence : "savoir conduire un véhicule". * Peu importe que vous soyez un `Humain`, un `Robot` ou un `Cyborg` (les types concrets). * Si votre type implémente le

contrat "Conducteur", le compilateur garantit que n'importe quelle partie du programme peut vous confier un véhicule en toute sécurité.



Du point de vue de la sécurité logicielle, les traits permettent d'appliquer le principe du polymorphisme ad-hoc et de la généricité contrainte. Le compilateur Rust vérifie à la compilation que toutes les promesses d'un trait sont tenues. Si un type prétend implémenter un trait mais omet une méthode, ou si une fonction exige un type possédant un comportement qu'il n'a pas, le code refuse tout simplement de compiler. Il n'y a aucune résolution hasardeuse à l'exécution, éliminant ainsi toute une classe de bugs applicatifs.

2. Fondations Théoriques

2.1. Définition et Implémentation d'un Trait

La syntaxe de base pour déclarer un trait utilise le mot-clé `trait`. Il contient uniquement les signatures des méthodes que les types implémentant ce trait devront définir.

```
// Déclaration du trait
pub trait Description {
    // Signature de la méthode : pas de corps, juste le nom, les arguments et le type de retour
    fn decrire(&self) → String;
}
```

Pour implémenter ce trait sur une structure de données (étudiée au [Module 3.1](#)), on utilise la syntaxe `impl [NomDuTrait] for [NomDuType]` :

```
pub struct Utilisateur {
    pub nom: String,
    pub age: u32,
}

// Implémentation du trait pour la structure Utilisateur
impl Description for Utilisateur {
    fn decrire(&self) → String {
        format!("Utilisateur : {} ({} ans)", self.nom, self.age)
    }
}
```

2.2. Les Méthodes par Défaut

Un trait peut fournir une implémentation par défaut pour certaines de ses méthodes. Les types qui implémentent ce trait peuvent alors choisir de conserver ce comportement par défaut ou de le surcharger (override).

```
pub trait Evalueur {
    fn obtenir_note(&self) → f64;

    // Méthode avec implémentation par défaut
    fn est_valide(&self) → bool {
        self.obtenir_note() ≥ 10.0
    }
}
```

Si un type implémente `Evalueur`, il est obligé de définir `obtenir_note(&self)`, mais il bénéficie automatiquement de `est_valide(&self)` sans effort supplémentaire.

2.3. Les Contraintes de Trait (Trait Bounds)

La véritable puissance des traits se révèle lorsqu'on les associe à la généricité. Si nous voulons écrire une fonction capable de traiter n'importe quel type, à condition que ce type possède un certain comportement, nous utilisons une **contrainte de trait** (*Trait Bound*).

Il existe plusieurs syntaxes pour exprimer ces contraintes.

La syntaxe compacte : `impl Trait`

Idéale pour les cas simples.

```
pub fn afficher_description(cible: &impl Description) {  
    println!("{}", cible.decrire());  
}
```

La syntaxe générique standard

Plus verbeuse, mais indispensable lorsque plusieurs paramètres doivent partager le même type générique.

```
pub fn comparer_descriptions<T: Description>(item1: &T, item2: &T) {  
    println!("1: {}\n2: {}", item1.decrire(), item2.decrire());  
}
```

La clause `where`

Recommandée pour maintenir la lisibilité lorsque les contraintes deviennent nombreuses ou complexes.

```
pub fn traiter_donnees<T, U>(source: &T, destination: &mut U)  
where  
    T: Description + Clone,  
    U: Description + std::fmt::Debug,  
{  
    // Corps de la fonction  
}
```

2.4. Le Polymorphisme en Rust : Dispatch Statique vs Dispatch Dynamique

Rust offre deux manières de résoudre les appels de méthodes sur des types polymorphes. Comprendre cette distinction est crucial pour maîtriser l'impact de votre code sur les performances de la mémoire et du processeur.

A. Le Dispatch Statique (Monomorphisation)

Par défaut, lorsque vous utilisez des génériques ou `impl Trait`, Rust utilise le dispatch statique.

Pendant la compilation, le compilateur analyse le code et génère une copie de la fonction générique pour chaque type concret utilisé. Ce processus s'appelle la monomorphisation.

- **Avantages** : Zéro coût à l'exécution (*Zero-cost abstraction*). Les appels de méthodes sont directs et le compilateur peut appliquer des optimisations agressives comme l'inlining (intégration directe du code de la méthode à l'endroit de l'appel).
- **Inconvénients** : Augmente la taille du binaire final (gonflement du code ou *code bloat*).

B. Le Dispatch Dynamique (Objets de Trait)

Parfois, le type exact d'une donnée ne peut pas être connu à la compilation (par exemple, un vecteur contenant des éléments de types différents mais implémentant tous le même trait). Dans ce cas, on utilise le dispatch dynamique via les objets de trait (*Trait Objects*), matérialisés par le mot-clé `dyn`.

Comme la taille d'un type dynamique est inconnue à la compilation (puisque'il peut s'agir de n'importe quelle structure), un objet de trait doit toujours être manipulé derrière un pointeur : `&dyn Trait` ou `Box<dyn Trait>`.

```
// Un vecteur contenant des objets de types différents implémentant Description
let elements: Vec<Box<dyn Description>> = vec![
    Box::new(Utilisateur { nom: String::from("Alice"), age: 30 }),
    // Imaginons un autre type implémentant aussi Description...
];
```

À l'exécution, Rust utilise une table de méthodes virtuelles (*vtable*) pour localiser la bonne fonction à appeler. Cela ajoute une légère surcharge (indirection de pointeur) et empêche l'inlining par le compilateur.

2.5. La Règle de Cohérence et l'Orphelinat (Orphan Rule)

Pour garantir la sécurité du système de types et éviter les conflits de définitions, Rust applique une règle stricte appelée la règle de l'orphelin (*Orphan Rule*) :

Vous ne pouvez implémenter un trait pour un type que si le trait ou le type (ou les deux) est défini dans votre propre crate (votre projet local).

- **Autorisé** : Implémenter votre trait local `Description` sur le type standard `Vec<T>` (le trait est local).
- **Autorisé** : Implémenter le trait standard `Display` sur votre structure locale `Utilisateur` (le type est local).
- **Interdit** : Implémenter le trait standard `Display` sur le type standard `Vec<T>` (ni le trait, ni le type ne sont locaux). Si cela était permis, deux bibliothèques différentes pourraient définir des implémentations conflictuelles pour le même type standard, rendant la compilation impossible ou le comportement du programme imprévisible.

3. Implémentation Pratique Pas-à-Pas

Nous allons concevoir un système de notification pour une plateforme de commerce électronique. Nous voulons envoyer des alertes via différents canaux (Email, SMS) tout en conservant un code d'envoi générique et hautement sécurisé.

Étape 1 : Définition des structures de données

Créons nos structures de base pour représenter un Email et un SMS.

```
pub struct Email {
    pub destinataire: String,
    pub sujet: String,
    pub corps: String,
}

pub struct Sms {
    pub numero: String,
    pub message: String,
}
```

Étape 2 : Définition du Trait `Notification`

Nous définissons le trait `Notification`. Nous y incluons une méthode obligatoire `envoyer` et une méthode avec implémentation par défaut `valider`.

```
pub trait Notification {  
    // Méthode à implémenter obligatoirement  
    fn envoyer(&self) → Result<(), String>;  
  
    // Méthode par défaut pour valider la cohérence des données avant envoi  
    fn valider(&self) → Result<(), String> {  
        // Par défaut, nous considérons que la notification est valide  
        Ok(())  
    }  
}
```

Étape 3 : Implémentation du Trait pour nos structures

Implémentons le trait pour `Email`. Nous allons surcharger la méthode `valider` pour vérifier la présence d'un caractère `@` dans l'adresse.

```
impl Notification for Email {  
    fn valider(&self) → Result<(), String> {  
        if self.destinataire.contains('@') {  
            Ok(())  
        } else {  
            Err(format!("Adresse email invalide : {}", self.destinataire))  
        }  
    }  
  
    fn envoyer(&self) → Result<(), String> {  
        // Validation préalable  
        self.valider()?;  
  
        // Simulation de l'envoi  
        println!(  
            "[EMAIL] Envoi à '{}' | Sujet : {}\nCorps : {}",  
            self.destinataire, self.sujet, self.corps  
        );  
        Ok(())  
    }  
}
```

Implémentons maintenant le trait pour `Sms`. Nous allons valider que le numéro n'est pas vide.

```
impl Notification for Sms {
    fn valider(&self) → Result<(), String> {
        if self.numero.is_empty() {
            Err(String::from("Le numéro de téléphone ne peut pas être vide"))
        } else {
            Ok(())
        }
    }

    fn envoyer(&self) → Result<(), String> {
        self.valider()?;

        println!("[SMS] Envoi au '{}' | Message : {}", self.numero, self.message);
        Ok(())
    }
}
```

Étape 4 : Écriture d'une fonction d'envoi générique (Dispatch Statique)

Cette fonction accepte n'importe quel type `T` qui implémente le trait `Notification`. Grâce à la monorphisation, aucun coût de performance n'est induit à l'exécution.

```
pub fn executer_envoi<T: Notification>(notif: &T) {
    match notif.envoyer() {
        Ok(_) ⇒ println!("Notification envoyée avec succès !"),
        Err(e) ⇒ println!("Erreur lors de l'envoi : {}", e),
    }
}
```

Étape 5 : Utilisation pratique dans la fonction `main`

Voici comment orchestrer l'ensemble de notre implémentation :

```
fn main() {
    let mon_email = Email {
        destinataire: String::from("etudiant@universite.fr"),
        sujet: String::from("Cours sur les Traits"),
        corps: String::from("Ne manquez pas ce chapitre crucial !"),
    };

    let mon_sms = Sms {
        numero: String::from("+33612345678"),
        message: String::from("Votre cours de Rust est disponible."),
    };

    // Utilisation du dispatch statique
    println!("--- Dispatch Statique ---");
    executer_envoi(&mon_email);
    executer_envoi(&mon_sms);

    // Utilisation du dispatch dynamique (Objets de Trait)
    // Utile si nous voulons stocker différentes notifications dans une même collection
    println!("\n--- Dispatch Dynamique ---");
    let liste_notifications: Vec<Box<dyn Notification>> = vec![
        Box::new(mon_email),
        Box::new(mon_sms),
    ];

    for notif in liste_notifications.iter() {
        // L'appel à envoyer() est résolu dynamiquement à l'exécution via la vtable
        if let Err(e) = notif.envoyer() {
            println!("Échec de l'envoi dynamique : {}", e);
        }
    }
}
```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Tenter de contourner la règle de l'orphelin (Orphan Rule)

Le problème : Vous souhaitez implémenter un trait externe (comme `Display` de la bibliothèque standard) sur un type externe (comme `Vec<T>`). Le compilateur refuse catégoriquement.

La solution : Le motif "Newtype" (Newtype Pattern). Emballez le type externe dans une structure locale (un tuple-struct à un seul élément). Cette structure étant locale à votre crate, vous êtes désormais autorisé à implémenter le trait externe sur celle-ci.

```
use std::fmt;

// Emballage de Vec<String> dans une structure locale
struct MaListeDeMots(Vec<String>);

// Nous pouvons maintenant implémenter Display pour notre structure locale
impl fmt::Display for MaListeDeMots {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) → fmt::Result {
        write!(f, "Liste : {}", self.0.join(", "))
    }
}
```

Piège 2 : Abuser de `dyn Trait` (Dispatch Dynamique)

Le problème : Utiliser systématiquement des objets de trait (`Box<dyn Trait>`) par habitude de la programmation orientée objet classique. Cela empêche de nombreuses optimisations du compilateur et ajoute une allocation sur le tas (Heap) via le `Box` .

La bonne pratique : Privilégiez toujours le dispatch statique (`impl Trait` ou les paramètres génériques `T: Trait`) par défaut. N'utilisez `dyn Trait` que lorsque vous avez un besoin réel de stocker des types hétérogènes dans une collection ou de retourner des types différents d'une fonction selon une condition d'exécution.

Piège 3 : Ignorer les durées de vie (Lifetimes) des Objets de Trait

Le problème : Lorsque vous créez une référence vers un objet de trait (`&dyn Trait`), le compilateur doit s'assurer que l'objet sous-jacent vit assez longtemps pour que la référence reste valide (concept étudié au [Module 5.1](#)). Par défaut, un objet de trait `dyn Trait` sans annotation de durée de vie est supposé avoir une durée de vie `'static` .

Si votre type concret contient des références avec une durée de vie plus courte, la compilation échouera avec un message obscur.

```
// Si la structure contient une référence, elle ne peut pas être 'static' par défaut
struct MessageTemporaire<'a> {
    contenu: &'a str,
}

impl<'a> Description for MessageTemporaire<'a> {
    fn decrire(&self) → String {
        self.contenu.to_string()
    }
}

// Erreur potentielle si le compilateur attend un `dyn Description + 'static`
// Solution : Spécifier explicitement la durée de vie de l'objet de trait
fn afficher_dynamique<'a>(objet: &'a dyn Description) {
    println!("{}", objet.decrire());
}
```

5. Synthèse Pédagogique

Dispatch Statique vs Dispatch Dynamique

Caractéristique	Dispatch Statique (<code>impl Trait</code> / <code>T: Trait</code>)	Dispatch Dynamique (<code>dyn Trait</code>)
Mécanisme	Monorphisation à la compilation.	Table de méthodes virtuelles (<code>vtable</code>) à l'exécution.
Performance	Maximale (pas d'indirection, optimisations/inlining possibles).	Légère surcharge due à l'indirection du pointeur.
Impact Mémoire	Augmentation de la taille du binaire (code dupliqué par type).	Binaire plus compact, mais nécessite souvent une allocation sur le tas (<code>Box</code>).
Flexibilité	Types homogènes uniquement lors de l'exécution d'une instance.	Permet des collections hétérogènes de types différents.

Points clés à retenir

1. Un **Trait** définit un ensemble de comportements (méthodes) sans imposer de structure de données interne.
 2. Les traits remplacent avantageusement l'héritage de classes en favorisant la composition et le couplage faible.
 3. La **monorphisation** permet d'utiliser le polymorphisme sans aucun coût de performance à l'exécution.
 4. La **règle de l'orphelin** protège l'écosystème Rust contre les collisions d'implémentations de traits.
 5. Le motif **Newtype** est la solution standard pour contourner la règle de l'orphelin de manière sécurisée.
 6. Les traits posent les fondations de concepts encore plus avancés, comme la concurrence sécurisée (avec les traits marqueurs **Send** et **Sync** que nous étudierons au [Module 5.3](#)).
-

6. Exercice Pratique d'Application

Sujet : Moteur de calcul pour formes géométriques 2D

Dans le cadre d'un logiciel de CAO (Conception Assistée par Ordinateur), vous devez concevoir un moteur géométrique capable de manipuler différentes formes bidimensionnelles.

Objectifs :

1. Définir un trait nommé **Forme** qui exige deux méthodes :
2. `aire(&self) → f64`
3. `perimetre(&self) → f64`
4. Créer deux structures :
5. **Cercle** (avec un champ `rayon: f64`)
6. **Rectangle** (avec des champs `largeur: f64` et `hauteur: f64`)

7. Implémenter le trait `Forme` pour ces deux structures.
8. Écrire une fonction générique `comparer_aires<T, U>(forme1: &T, forme2: &U)` utilisant le dispatch statique pour afficher laquelle des deux formes a la plus grande aire.
9. Écrire une fonction `calculer_total_perimetres(formes: &[Box<dyn Forme>]) → f64` utilisant le dispatch dynamique pour calculer la somme des périmètres d'une collection hétérogène de formes.

Indices :

- Pour la valeur de Pi, utilisez la constante standard `std::f64::consts::PI`.
 - La formule de l'aire d'un cercle est $\pi \times r^2$. Son périmètre est $2 \times \pi \times r$.
-

Correction Complète

Voici le code source complet et documenté répondant aux exigences de l'exercice.

```

use std::f64::consts::PI;

// 1. Définition du Trait Forme
pub trait Forme {
    fn aire(&self) → f64;
    fn perimetre(&self) → f64;
}

// 2. Définition des structures de données
pub struct Cercle {
    pub rayon: f64,
}

pub struct Rectangle {
    pub largeur: f64,
    pub hauteur: f64,
}

// 3. Implémentation du Trait Forme pour Cercle
impl Forme for Cercle {
    fn aire(&self) → f64 {
        PI * self.rayon * self.rayon
    }

    fn perimetre(&self) → f64 {
        2.0 * PI * self.rayon
    }
}

// Implémentation du Trait Forme pour Rectangle
impl Forme for Rectangle {
    fn aire(&self) → f64 {
        self.largeur * self.hauteur
    }

    fn perimetre(&self) → f64 {
        2.0 * (self.largeur + self.hauteur)
    }
}

// 4. Fonction générique avec Dispatch Statique (Trait Bounds)
// Nous utilisons deux types génériques différents (T et U) pour permettre
// de comparer deux formes de types concrets différents (ex: un Cercle et un Rectangle).
pub fn comparer_aires<T, U>(forme1: &T, forme2: &U)
where
    T: Forme,
    U: Forme,
{

```

```

let aire1 = forme1.aire();
let aire2 = forme2.aire();

println!("--- Comparaison d'aires ---");
println!("Aire de la première forme : {:.2}", aire1);
println!("Aire de la seconde forme   : {:.2}", aire2);

if aire1 > aire2 {
    println!("La première forme est plus grande.");
} else if aire2 > aire1 {
    println!("La seconde forme est plus grande.");
} else {
    println!("Les deux formes ont exactement la même aire.");
}
}

// 5. Fonction avec Dispatch Dynamique (Objets de Trait)
// Permet de traiter une liste contenant un mélange de Cercles et de Rectangles.
pub fn calculer_total_perimetres(formes: &[Box<dyn Forme>]) → f64 {
    let mut total = 0.0;
    for forme in formes {
        // L'appel à perimetre() est résolu dynamiquement à l'exécution
        total += forme.perimetre();
    }
    total
}

fn main() {
    // Instanciation de nos formes
    let mon_cercle = Cercle { rayon: 5.0 };           // Aire ~ 78.54, Périmètre ~ 31.42
    let mon_rectangle = Rectangle { largeur: 10.0, hauteur: 8.0 }; // Aire = 80.0, Périmètre = 36.0

    // Test du dispatch statique
    comparer_aires(&mon_cercle, &mon_rectangle);

    println!("\n--- Calcul dynamique sur collection hétérogène ---");
    // Création d'un vecteur contenant des objets de trait
    let catalogue_formes: Vec<Box<dyn Forme>> = vec![
        Box::new(mon_cercle),
        Box::new(mon_rectangle),
    ];

    let total_perimetres = calculer_total_perimetres(&catalogue_formes);
    println!("Somme totale des périmètres : {:.2}", total_perimetres);

    // Validation de la somme : 31.42 + 36.0 = 67.42
    assert!((total_perimetres - 67.4159).abs() < 0.001);
    println!("Validation réussie !");
}

```


Exercices d'Application :

Introduction aux Traits

Ce cahier d'exercices pratiques a pour objectif de consolider vos compétences sur la définition, l'implémentation et l'utilisation des Traits en Rust. À travers trois cas concrets, vous pratiquerez l'utilisation des méthodes par défaut, le contournement de la règle de l'orphelin (motif *Newtype*) et la manipulation de collections hétérogènes via le dispatch dynamique.

Exercice 1 : Système de Journalisation Flexible (Logger)

Énoncé

Dans le cadre du développement d'une application serveur, vous devez concevoir un système de journalisation (*logging*) capable d'envoyer des messages vers différentes destinations (la console ou un tampon mémoire pour les tests).

Objectifs :

1. Définir un trait `Log` qui exige la méthode suivante :
2. `fn ecrire(&mut self, message: &str);`
3. Ajouter deux méthodes par défaut au trait `Log` :
4. `fn log_info(&mut self, message: &str)` qui formate le message sous la forme `"[INFO] : [message]"` et l'envoie à `ecrire` .
5. `fn log_erreur(&mut self, message: &str)` qui formate le message sous la forme `"[ERREUR] : [message]"` et l'envoie à `ecrire` .

6. Créer une structure `ConsoleLogger` (sans champ) et implémenter le trait `Log` pour qu'elle affiche le message directement sur la sortie standard (`println!`).
7. Créer une structure `MemoryLogger` contenant un champ `historique`: `Vec<String>` et implémenter le trait `Log` pour qu'elle stocke les messages reçus dans son historique.
8. Écrire une fonction générique `enregistrer_evenements<T: Log>(logger: &mut T)` qui utilise le dispatch statique pour envoyer un message d'information et un message d'erreur.

Indices

- Pour formater une chaîne sans l'afficher directement, utilisez la macro `format!` .
- La signature des méthodes par défaut doit utiliser `&mut self` pour pouvoir appeler `ecrire(&mut self, ...)` .
- Dans `MemoryLogger` , pour ajouter un `&str` sous forme de `String` dans un `Vec<String>` , utilisez la méthode `.to_string()` ou `String::from()` .

Correction Détaillée

```

// 1. Définition du Trait Log avec méthodes par défaut
pub trait Log {
    // Méthode obligatoire à implémenter
    fn écrire(&mut self, message: &str);

    // 2. Méthodes par défaut
    fn log_info(&mut self, message: &str) {
        let message_format = format!("[INFO] : {}", message);
        self.écrire(&message_format);
    }

    fn log_erreur(&mut self, message: &str) {
        let message_format = format!("[ERREUR] : {}", message);
        self.écrire(&message_format);
    }
}

// 3. Implémentation de ConsoleLogger
pub struct ConsoleLogger;

impl Log for ConsoleLogger {
    fn écrire(&mut self, message: &str) {
        println!("{}", message);
    }
}

// 4. Implémentation de MemoryLogger
pub struct MemoryLogger {
    pub historique: Vec<String>,
}

impl Log for MemoryLogger {
    fn écrire(&mut self, message: &str) {
        self.historique.push(message.to_string());
    }
}

// 5. Fonction générique avec Dispatch Statique (Trait Bound)
pub fn enregistrer_evenements<T: Log>(logger: &mut T) {
    logger.log_info("Démarrage de l'application.");
    logger.log_erreur("Échec de connexion à la base de données.");
}

fn main() {
    println!("--- Test du ConsoleLogger ---");
    let mut console = ConsoleLogger;
    enregistrer_evenements(&mut console);
}

```



```
println!("\n--- Test du MemoryLogger ---");
let mut memoire = MemoryLogger { historique: Vec::new() };
enregistrer_evenements(&mut memoire);

// Vérification du contenu du MemoryLogger
println!("Contenu de l'historique en mémoire :");
for (i, log) in memoire.historique.iter().enumerate() {
    println!("{}", i + 1, log);
}

assert_eq!(memoire.historique.len(), 2);
assert_eq!(memoire.historique[0], "[INFO] : Démarrage de l'application");
}
```

Exercice 2 : Contourner la Règle de l'Orphelin (Le Motif Newtype)

Énoncé

Vous travaillez sur une application météorologique. Vous souhaitez afficher directement une liste de températures (représentée par un `Vec<f64>`) sur la console à l'aide de la macro `println!("{}", liste)`.

Cependant, vous vous heurtez à la règle de l'orphelin : vous ne pouvez pas implémenter le trait standard `std::fmt::Display` directement sur le type standard `Vec<T>`.

Objectifs :

1. Créer une structure locale nommée `Temperatures` en utilisant le motif Newtype pour envelopper un `Vec<f64>`.
2. Implémenter le trait standard `std::fmt::Display` pour votre structure `Temperatures`. Le format de sortie doit afficher chaque température séparée par une barre verticale (`|`) et suffixée par `°C`. Exemple attendu : `12.5°C | 18.0°C | 22.3°C`.

3. Tester votre implémentation dans la fonction `main` en instanciant `Temperatures` et en l'affichant avec `println!` .

Indices

- La syntaxe du motif Newtype est une structure de tuple à un seul élément :
`struct MonType(TypeExistant);` .
- Pour accéder aux données internes du Newtype à l'intérieur de l'implémentation, utilisez `self.0` .
- Pour implémenter `Display` , importez `std::fmt` et respectez la signature : `fn fmt(&self, f: &mut fmt::Formatter<'_>) → fmt::Result` .
- Vous pouvez utiliser la méthode `.iter().enumerate()` pour savoir si vous devez ajouter le séparateur `|` entre les éléments.

Correction Détaillée

```
use std::fmt;

// 1. Définition du Newtype enveloppant un Vec<f64>
struct Temperatures(Vec<f64>);

// 2. Implémentation du trait Display pour le Newtype
impl fmt::Display for Temperatures {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) → fmt::Result {
        // Accès au vecteur interne via self.0
        for (index, temp) in self.0.iter().enumerate() {
            if index > 0 {
                // Ajout du séparateur entre les éléments
                write!(f, " | ")?;
            }
            // Écriture de la température formatée
            write!(f, "{:.1}°C", temp)?;
        }
        Ok(())
    }
}

fn main() {
    // 3. Instanciation et affichage
    let releves = Temperatures(vec![14.2, 18.5, 22.0, 9.8]);

    println!("Relevés de la journée :");
    // Grâce à l'implémentation de Display, nous pouvons utiliser {}
    println!("{}", releves);

    // Validation du formatage attendu
    let chaine_formatée = format!("{}", releves);
    assert_eq!(chaine_formatée, "14.2°C | 18.5°C | 22.0°C | 9.8°C");
    println!("\nValidation du formatage réussie !");
}
```

Exercice 3 : Panier d'Achat Multi-Produits (Dispatch Dynamique)

Énoncé

Vous devez concevoir le moteur de calcul de facturation pour une plateforme d'e-commerce. Cette plateforme vend trois types d'articles : * Des **produits physiques** (soumis à une TVA de 20% et à des frais de port basés sur le poids). * Des **produits numériques** (soumis à une TVA de 5.5%, sans frais de port). * Des **services/abonnements** (facturés au taux horaire, soumis à une TVA de 20%, sans frais de port).

Puisque le panier d'un client peut contenir un mélange de ces trois types d'articles, vous devez utiliser le **dispatch dynamique** pour calculer le montant total.

Objectifs :

1. Définir un trait `Article` contenant trois méthodes :
2. `fn nom(&self) → &str;`
3. `fn prix_hors_taxe(&self) → f64;`
4. `fn calculer_tva(&self) → f64;`
5. Une méthode avec implémentation par défaut : `fn prix_ttc(&self) → f64` qui retourne la somme du prix hors taxe et de la TVA.
6. Créer les structures suivantes :
7. `ProduitPhysique` (champs : `nom: String` , `prix: f64` , `poids_kg: f64`). La TVA est de 20%. Les frais de port s'élèvent à 2.0 € par kilogramme (à rajouter au prix TTC final).
8. `ProduitNumerique` (champs : `nom: String` , `prix: f64`). La TVA est de 5.5%.
9. `Service` (champs : `nom: String` , `taux_horaire: f64` , `heures: f64`). Le prix hors taxe est égal au `taux_horaire * heures` . La TVA est de 20%.
10. Implémenter le trait `Article` pour ces trois structures. Note : Pour `ProduitPhysique` , surchargez la méthode `prix_ttc` pour y inclure les frais de port.

11. Créer une structure `Panier` contenant un vecteur d'objets de trait :
`Vec<Box<dyn Article>>` .
12. Implémenter une méthode `calculer_total(&self) → f64` sur `Panier` qui retourne la somme totale TTC de tous les articles du panier.

Indices

- Pour surcharger une méthode de trait (comme `prix_ttc` pour `ProduitPhysique`), il suffit de redéfinir la fonction dans le bloc `impl Article for ProduitPhysique` . Vous pouvez appeler les autres méthodes du trait (comme `self.prix_hors_taxe()`) à l'intérieur.
- N'oubliez pas que pour manipuler des objets de trait dans un vecteur, la syntaxe est `Vec<Box<dyn Article>>` .

Correction Détaillée

```

// 1. Définition du Trait Article
pub trait Article {
    fn nom(&self) → &str;
    fn prix_hors_taxe(&self) → f64;
    fn calculer_tva(&self) → f64;

    // Méthode par défaut
    fn prix_ttc(&self) → f64 {
        self.prix_hors_taxe() + self.calculer_tva()
    }
}

// 2. Définition des structures de données
pub struct ProduitPhysique {
    pub nom: String,
    pub prix: f64,
    pub poids_kg: f64,
}

pub struct ProduitNumerique {
    pub nom: String,
    pub prix: f64,
}

pub struct Service {
    pub nom: String,
    pub taux_horaire: f64,
    pub heures: f64,
}

// 3. Implémentation du Trait Article pour ProduitPhysique
impl Article for ProduitPhysique {
    fn nom(&self) → &str {
        &self.nom
    }

    fn prix_hors_taxe(&self) → f64 {
        self.prix
    }

    fn calculer_tva(&self) → f64 {
        self.prix * 0.20 // TVA 20%
    }

    // Surcharge de la méthode par défaut pour inclure les frais de port
    fn prix_ttc(&self) → f64 {
        let prix_base_ttc = self.prix_hors_taxe() + self.calculer_tva();
        let frais_port = self.poids_kg * 2.0;
    }
}

```

```

        prix_base_ttc + frais_port
    }
}

// Implémentation pour ProduitNumerique
impl Article for ProduitNumerique {
    fn nom(&self) → &str {
        &self.nom
    }

    fn prix_hors_taxe(&self) → f64 {
        self.prix
    }

    fn calculer_tva(&self) → f64 {
        self.prix * 0.055 // TVA 5.5%
    }
}

// Implémentation pour Service
impl Article for Service {
    fn nom(&self) → &str {
        &self.nom
    }

    fn prix_hors_taxe(&self) → f64 {
        self.taux_horaire * self.heures
    }

    fn fn_calculer_tva(&self) → f64 { // Note: alignement avec le trait
        self.prix_hors_taxe() * 0.20 // TVA 20%
    }
}

// Correction de la coquille de nommage pour Service
impl Article for Service {
    fn nom(&self) → &str {
        &self.nom
    }

    fn prix_hors_taxe(&self) → f64 {
        self.taux_horaire * self.heures
    }

    fn calculer_tva(&self) → f64 {
        self.prix_hors_taxe() * 0.20 // TVA 20%
    }
}

```



```

// 4. Définition de la structure Panier (Dispatch Dynamique)
pub struct Panier {
    pub articles: Vec<Box<dyn Article>>,
}

impl Panier {
    // 5. Calcul du total TTC cumulé
    pub fn calculer_total(&self) → f64 {
        let mut total = 0.0;
        for article in &self.articles {
            // L'appel à prix_ttc() est résolu dynamiquement à l'exécution
            total += article.prix_ttc();
        }
        total
    }
}

fn main() {
    // Instanciation de différents articles
    let livre = ProduitPhysique {
        nom: String::from("Livre de Rust en images"),
        prix: 30.0,          // TTC (20%) = 36.0 | Frais de port (1.5kg * 2) = 3.0 | Total = 39.0
        poids_kg: 1.5,
    };

    let ebook = ProduitNumerique {
        nom: String::from("Ebook Rust Avancé"),
        prix: 10.0,          // TTC (5.5%) = 10.55 | Total = 10.55
    };

    let formation = Service {
        nom: String::from("Consulting Rust"),
        taux_horaire: 100.0,
        heures: 2.0,         // HT = 200.0 | TTC (20%) = 240.0 | Total = 240.0
    };

    // Remplissage du panier hétérogène
    let mon_panier = Panier {
        articles: vec![
            Box::new(livre),
            Box::new(ebook),
            Box::new(formation),
        ],
    };

    // Affichage du détail des articles pour vérification
    println!("--- Détail du Panier ---");
    for art in &mon_panier.articles {
        println!(

```

```

        "- {} : HT = {:.2} € | TTC = {:.2} €",
        art.nom(),
        art.prix_hors_taxe(),
        art.prix_ttc()
    );
}

// Calcul du total
let total_panier = mon_panier.calculer_total();
println!("\nTotal Général du Panier : {:.2} €", total_panier);

// Validation : 39.0 + 10.55 + 240.0 = 289.55
assert!((total_panier - 289.55).abs() < 0.001);
println!("Validation du calcul du panier réussie !");
}

```

La Concurrency sans peur (Fearless Concurrency) : Partager des données entre threads en toute sécurité

1. Introduction Conceptuelle

Depuis l'avènement des processeurs multicœurs, la programmation concurrente et parallèle est devenue indispensable pour concevoir des applications hautement performantes. Cependant, elle est historiquement réputée pour être l'une des disciplines les plus complexes et sujettes aux bugs de l'ingénierie logicielle.

Dans des langages système traditionnels comme le C ou le C++, le partage de données entre plusieurs flux d'exécution (threads) s'apparente à un exercice de haute voltige sans filet. Les développeurs y sont constamment confrontés à des bogues redoutables : * **Les Data Races (Accès concurrents non synchronisés)** : Deux threads accèdent simultanément à la même zone mémoire, l'un d'eux au moins effectuant une écriture, sans aucune synchronisation. Cela provoque un comportement indéfini (*Undefined Behavior*), rendant le programme imprévisible et vulnérable. * **Les Deadlocks (Verrous mortels)** : Deux threads se bloquent mutuellement indéfiniment, chacun attendant une ressource verrouillée par l'autre. * **Les corruptions de mémoire** : Des pointeurs devenant invalides suite à la désallocation d'une ressource par un thread alors qu'un autre tente encore d'y accéder.

L'approche de Rust : La "Concurrency sans peur" (Fearless Concurrency)

Rust révolutionne cette discipline grâce à un paradigme novateur : rendre la concurrence sûre par construction, dès l'étape de compilation.

Le compilateur Rust utilise les règles d'or de l'*Ownership* (étudiées au *Module 2*) et son système de types pour interdire statiquement les accès mémoire concurrents non sécurisés. Si votre code concurrent contient une possibilité de *Data Race*, le code ne compilera pas.

L'expression *Fearless Concurrency* désigne cette confiance absolue qu'a le développeur Rust : vous pouvez refactoriser, paralléliser et optimiser votre code multi-threadé sans craindre de corrompre la mémoire ou d'introduire des bugs intermittents quasi impossibles à reproduire en production.

2. Fondations Théoriques

Pour comprendre comment Rust réalise ce tour de force, nous devons analyser les concepts de threads, la distinction entre les types de conflits, et les deux traits fondamentaux du langage : `Send` et `Sync`.

A. Threads système vs Vertuels

Rust utilise par défaut des **threads système** (parfois appelés threads 1:1), où chaque thread créé dans le langage correspond directement à un thread géré par le système d'exploitation. Cela garantit des performances maximales et un contrôle direct sur le matériel, sans le surcoût d'un runtime complexe.

B. Data Race vs Race Condition

Il est crucial de distinguer ces deux notions souvent confondues : 1. **Data Race (Concurrence de données)** : C'est une anomalie purement mémoire (accès simultané non synchronisé avec au moins une écriture). Rust garantit l'absence totale de Data Races en code sûr (*safe code*). 2. **Race Condition (Situation de compétition)** : C'est une anomalie logique où le résultat du programme dépend de l'ordre d'exécution des threads (par exemple, le Thread A termine avant le Thread B, modifiant le comportement attendu). Rust ne peut pas empêcher toutes les situations de compétition logique, mais il garantit que, quelle que soit l'issue, la mémoire restera intègre et saine.

C. Les Traits `Send` et `Sync` : Les piliers de la sécurité concurrentielle

La sécurité concurrentielle en Rust repose sur deux *traits marqueurs* (qui ne contiennent aucune méthode, mais servent d'indications pour le compilateur) :

- `Send` : Un type `T` est `Send` si la propriété d'une valeur de ce type peut être transférée à un autre thread. La grande majorité des types de base de Rust sont `Send` (comme `i32`, `String`, `Vec<T>` si `T` est `Send`).
- `Sync` : Un type `T` est `Sync` s'il est sécurisé de partager des références à ce type (`&T`) entre plusieurs threads. Autrement dit, `T` est `Sync` si et seulement si `&T` est `Send`.

L'exclusion mutuelle à la compilation

Si un type contient un pointeur non thread-safe (comme `Rc<T>`, le compteur de références non atomique étudié au Module 3), il n'implémente pas `Send` ni `Sync`. Si vous tentez de l'envoyer à un autre thread, le compilateur rejettera immédiatement votre programme.

D. Les outils de partage de données

1. `Rc<T>` vs `Arc<T>`

- `Rc<T>` (Reference Counted) : Conçu pour un usage mono-threadé. Ses opérations d'incrément/décément du compteur de références ne sont pas atomiques. Il n'est donc ni `Send` ni `Sync`.
- `Arc<T>` (Atomically Reference Counted) : Conçu pour le multi-threading. Il utilise des instructions CPU atomiques pour gérer le compteur de références. Il est donc `Send` et `Sync` (si `T` est `Send` et `Sync`), permettant un partage de propriété sécurisé entre threads, au prix d'un léger surcoût de performance par rapport à `Rc`.

2. `Mutex<T>` (Mutual Exclusion)

Pour modifier une donnée partagée via un `Arc`, nous devons garantir qu'un seul thread à la fois peut y accéder en écriture. C'est le rôle du `Mutex<T>`. En Rust, le `Mutex` encapsule la donnée qu'il protège (`Mutex<T>`). Pour accéder à la donnée, un thread doit verrouiller le mutex via la méthode `.lock()`. Ce verrouillage retourne un `Mutex Guard` qui implémente le trait `Deref` pour accéder à la donnée interne, et le trait `Drop` pour libérer automatiquement le verrou dès que la garde sort de la portée (concept RAII - Resource Acquisition Is Initialization).

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Création d'un Thread de base

Commençons par instancier un thread système simple à l'aide de la fonction `std::thread::spawn`.

```
use std::thread;
use std::time::Duration;

fn main() {
    // Création d'un nouveau thread
    let handle = thread::spawn(|| {
        for i in 1..5 {
            println!("[Thread Enfant] Message numéro {}", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    // Code exécuté par le thread principal (main)
    for i in 1..3 {
        println!("[Thread Principal] Message numéro {}", i);
        thread::sleep(Duration::from_millis(1));
    }

    // Attente de la fin du thread enfant pour éviter que le programme ne s'arrête prématurément
    handle.join().unwrap();
    println!("[Système] Tous les threads ont terminé.");
}
```

Étape 2 : Le mot-clé `move` avec les fermetures (closures)

Si un thread enfant doit utiliser une variable définie dans le thread parent, le compilateur exige que nous transférons explicitement la propriété de cette variable au thread enfant via le mot-clé `move`.

```

use std::thread;

fn main() {
    let message = String::from("Bonjour depuis le thread parent !");

    // L'utilisation de 'move' force la capture par valeur (transfert de propriété)
    let handle = thread::spawn(move || {
        println!("Thread enfant dit : {}", message);
    });

    // Ici, 'message' n'est plus accessible dans le thread principal.
    // Tenter d'écrire : println!("{}", message); provoquerait une erreur de compilation

    handle.join().unwrap();
}

```

Étape 3 : Passage de messages via des Canaux (Channels)

Rust implémente la célèbre philosophie de Go : *"Ne communiquez pas en partageant de la mémoire ; partagez de la mémoire en communiquant."* Pour cela, nous utilisons des canaux de type MPSC (Multi-Producer, Single-Consumer).

```

use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // Création du canal : tx (transmitter / émetteur), rx (receiver / récepteur)
    let (tx, rx) = mpsc::channel();

    // On clone l'émetteur pour permettre à un second thread d'envoyer aussi des messages
    let tx1 = tx.clone();

    // Premier thread producteur
    thread::spawn(move || {
        let messages = vec![
            String::from("Un"),
            String::from("Deux"),
        ];
        for msg in messages {
            tx1.send(msg).unwrap(); // Envoi du message
            thread::sleep(Duration::from_millis(100));
        }
    });

    // Deuxième thread producteur
    thread::spawn(move || {
        let messages = vec![
            String::from("Trois"),
            String::from("Quatre"),
        ];
        for msg in messages {
            tx.send(msg).unwrap(); // Envoi du message
            thread::sleep(Duration::from_millis(150));
        }
    });

    // Le thread principal consomme les messages reçus
    // L'itérateur bloque et attend des messages jusqu'à ce que tous les émetteurs soient d
    for received in rx {
        println!("Reçu : {}", received);
    }
}

```

Étape 4 : Partage d'état avec Arc et Mutex

Lorsque la communication par messages ne suffit pas et qu'un état partagé mutable est requis, nous combinons Arc et Mutex .


```

use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Initialisation d'un compteur partagé, protégé par un Mutex, enveloppé dans un Arc
    let compteur = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for i in 0..10 {
        // Incrémentation du compteur de références de l'Arc
        let compteur_clone = Arc::clone(&compteur);

        let handle = thread::spawn(move || {
            // Acquisition du verrou (lock).
            // Si un autre thread a paniqué en détenant le verrou, lock() échouera (Mutex est mortel).
            // L'appel à unwrap() propage la panique pour des raisons de sécurité.
            let mut donnee = compteur_clone.lock().unwrap();

            // Modification de la donnée interne par déréférencement
            *donnee += 1;

            println!("Thread {} a incrémenté le compteur à {}", i, *donnee);
            // Le verrou est automatiquement libéré ici car 'donnee' sort de la portée (Drop).
        });
        handles.push(handle);
    }

    // Attente de la fin de tous les threads
    for handle in handles {
        handle.join().unwrap();
    }

    // Affichage du résultat final
    println!("Valeur finale du compteur : {}", *compteur.lock().unwrap());
}

```

4. Pièges Fréquents et Bonnes Pratiques

A. Les Verrous Mortels (Deadlocks)

Un deadlock survient lorsqu'un thread tente d'acquérir un verrou alors qu'il en détient déjà un autre, créant une dépendance circulaire.

```
// EXEMPLE DE CODE PROVOQUANT UN DEADLOCK (À ÉVITER)
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let ressource_a = Arc::new(Mutex::new(1));
    let ressource_b = Arc::new(Mutex::new(2));

    let r_a_clone = Arc::clone(&ressource_a);
    let r_b_clone = Arc::clone(&ressource_b);

    let handle1 = thread::spawn(move || {
        let _guard_a = r_a_clone.lock().unwrap();
        thread::sleep(std::time::Duration::from_millis(10));
        let _guard_b = r_b_clone.lock().unwrap(); // Bloqué ici si le thread 2 a verrouillé
    });

    let r_a_clone2 = Arc::clone(&ressource_a);
    let r_b_clone2 = Arc::clone(&ressource_b);

    let handle2 = thread::spawn(move || {
        let _guard_b = r_b_clone2.lock().unwrap();
        thread::sleep(std::time::Duration::from_millis(10));
        let _guard_a = r_a_clone2.lock().unwrap(); // Bloqué ici si le thread 1 a verrouillé
    });

    handle1.join().unwrap();
    handle2.join().unwrap();
}
```

Bonne pratique : 1. Verrouillez toujours les ressources dans le même ordre à travers toute votre application. 2. Réduisez la portée des verrous en utilisant des blocs de code explicites `{ ... }` pour forcer la libération de la garde le plus tôt possible.

B. Mutex Empoisonnés (Poisoned Mutexes)

Si un thread panique alors qu'il détient le verrou d'un `Mutex`, le `Mutex` est marqué comme "empoisonné" (*poisoned*). Les autres threads tentant d'y accéder recevront une erreur lors de l'appel à `.lock()`. * Solution standard : Utiliser `.lock().unwrap()`. Si le mutex est empoisonné, le thread actuel panique également, ce qui est généralement préférable à la manipulation de données potentiellement corrompues par la panique précédente.

C. Le coût de l'atomicité

N'utilisez pas `Arc` si un simple `Rc` suffit (dans un contexte mono-threadé). Les opérations atomiques de `Arc` requièrent une synchronisation matérielle au niveau du processeur qui est plus coûteuse en cycles d'horloge.

5. Synthèse Pédagogique

Tableau Comparatif des Stratégies de Concurrency

Stratégie	Avantages	Inconvénients	Cas d'usage idéal
Passage de messages (Channels)	Pas de verrous, couplage faible, évite naturellement les deadlocks logiques.	Copie ou transfert de propriété requis, overhead de file d'attente.	Pipelines de données, architectures d'acteurs, tâches asynchrones indépendantes.
État partagé (<code>Arc<Mutex<T>></code>)	Accès direct en mémoire, idéal pour les structures de données complexes centralisées.	Risque de deadlocks, contention si trop de threads attendent le verrou.	Cache partagé en mémoire, compteurs globaux, mise à jour d'un état centralisé.

Les Règles d'Or de la Concurrency en Rust

1. **Le compilateur est votre allié** : Si le compilateur refuse votre code concurrent à cause d'un problème de durée de vie (*lifetime*) ou de trait `Send` / `Sync`, c'est qu'il vient de vous éviter un bug mémoire critique.
2. **Privilégiez l'immuabilité** : Si des données partagées n'ont pas besoin d'être modifiées, utilisez uniquement `Arc<T>` sans `Mutex<T>`. Plusieurs threads peuvent lire simultanément et en toute sécurité via des références immuables.
3. **Gardez les verrous courts** : Ne réalisez jamais d'opérations d'E/S (I/O) lentes ou de calculs lourds à l'intérieur d'une section critique protégée par un `Mutex`.

6. Exercice Pratique d'Application

Énoncé : Le Simulateur de Traitement d'Images Parallèle

Vous devez concevoir un programme qui simule le traitement parallèle d'un lot d'images.

Chaque "Image" est représentée par une structure contenant un identifiant (`id`) et un tableau dynamique d'entiers (`pixels`) représentant l'intensité lumineuse. Le traitement consiste à appliquer un filtre qui double la valeur de chaque pixel (simulant un traitement de luminosité).

Spécifications :

1. Définissez une structure `Image` contenant un `id: u32` et `pixels: Vec<u32>` .
2. Dans la fonction `main` , créez un vecteur contenant 3 images fictives.
3. Pour chaque image, lancez un thread enfant qui va appliquer le filtre (multiplier chaque pixel par 2).
4. Utilisez un canal de communication (`mpsc`) pour renvoyer l'image traitée au thread principal.
5. Le thread principal doit collecter toutes les images traitées et afficher le résultat final.

Indices

- Pensez à utiliser `move` lors de la création de chaque thread pour lui transférer la propriété de l'image et de l'émetteur du canal.
 - Pour envoyer plusieurs messages depuis différents threads, clonez l'émetteur (`tx`) pour chaque thread enfant.
 - N'oubliez pas de fermer ou de laisser sortir de la portée l'émetteur d'origine dans le thread principal, sinon la boucle de réception `for msg in rx` ne s'arrêtera jamais !
-

Solution Complète et Commentée

```

use std::sync::mpsc;
use std::thread;
use std::time::Duration;

// 1. Définition de la structure de données
#[derive(Debug)]
struct Image {
    id: u32,
    pixels: Vec<u32>,
}

fn main() {
    // 2. Création du lot d'images fictives
    let images = vec![
        Image { id: 1, pixels: vec![10, 20, 30] },
        Image { id: 2, pixels: vec![50, 100, 150] },
        Image { id: 3, pixels: vec![0, 12, 24] },
    ];

    // Création du canal de communication
    let (tx, rx) = mpsc::channel();

    println!("[Système] Début du traitement parallèle...");

    // 3. Lancement d'un thread par image
    for image in images {
        // Clonage de l'émetteur pour ce thread spécifique
        let tx_clone = tx.clone();

        thread::spawn(move || {
            println!(" → [Thread {}] Début du traitement...", image.id);

            // Simulation d'un calcul intensif
            thread::sleep(Duration::from_millis(500));

            // Application du filtre (on double la valeur des pixels)
            let pixels_traites: Vec<u32> = image.pixels.iter().map(|p| p * 2).collect();

            let image_traitee = Image {
                id: image.id,
                pixels: pixels_traites,
            };

            // Envoi de l'image traitée au thread principal
            tx_clone.send(image_traitee).unwrap();
            println!(" ← [Thread {}] Traitement terminé et envoyé.", image.id);
        });
    }
}

```

```

// Crucial : On doit détruire l'émetteur original possédé par le main.
// Sinon, le récepteur 'rx' attendra indéfiniment car au moins un émetteur (celui-ci) est toujours actif.
drop(tx);

// 4. Collecte des résultats par le thread principal
println!("[Système] Attente des résultats...");
for image_traitee in rx {
    println!(
        "[Résultat] Image ID: {} traitée avec succès ! Pixels : {:?}",
        image_traitee.id, image_traitee.pixels
    );
}

println!("[Système] Toutes les images ont été traitées avec succès.");
}

```


Exercices d'Application : La Concurrency sans peur (Fearless Concurrency)

Ce cahier d'exercices pratiques a pour but de consolider vos compétences sur le partage sécurisé de données entre threads en Rust. Les exercices sont classés par ordre de difficulté progressive.

Exercice 1 : Calcul de Somme Parallèle avec État Partagé (`Arc<Mutex<T>>`)

Énoncé

Dans cet exercice, vous devez concevoir un programme qui calcule la somme d'un grand tableau d'entiers en divisant le travail entre plusieurs threads.

Pour optimiser les performances et éviter la **contention de verrou** (le fait que plusieurs threads attendent constamment après le même `Mutex`), chaque thread devra d'abord calculer sa somme locale de manière indépendante. Une fois son calcul local terminé, chaque thread devra acquérir un verrou sur un compteur global partagé pour y ajouter son résultat.

Spécifications :

1. Créez un vecteur contenant les nombres de `1` à `100 000`.
2. Définissez un nombre de threads (par exemple, `4`).
3. Divisez le vecteur en morceaux (*chunks*) de taille égale, un pour chaque thread.

4. Créez un accumulateur global protégé par un `Mutex` et enveloppé dans un `Arc` .
5. Lancez les threads. Chaque thread doit calculer la somme de son morceau localement, puis ajouter ce résultat à l'accumulateur global.
6. Attendez la fin de tous les threads (`join`) et affichez la somme totale finale.

Indices

1. Utilisez la méthode `.chunks()` sur les slices Rust pour découper facilement votre vecteur : `let morceaux = donnees.chunks(taille_morceau);` .
2. Pour passer le morceau de vecteur au thread enfant sans problème de durée de vie (*lifetime*), vous pouvez convertir le morceau en un vecteur indépendant (`morceau.to_vec()`) avant de le transférer via le mot-clé `move` .
3. Veillez à ce que l'acquisition du verrou `Mutex` ne se fasse **qu'après** la boucle de calcul local du thread.

Correction Détaillée

```

use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // 1. Préparation des données
    let limites = 100_000;
    let donnees: Vec<u64> = (1..=limites).collect();

    // Définition du nombre de threads
    let nb_threads = 4;
    let taille_morceau = donnees.len() / nb_threads;

    // 2. Initialisation de l'accumulateur global partagé
    let somme_globale = Arc::new(Mutex::new(0u64));
    let mut handles = vec![];

    println!("[Système] Début du calcul parallèle sur {} threads...", nb_threads);

    // 3. Découpage et distribution du travail
    for (i, morceau) in donnees.chunks(taille_morceau).enumerate() {
        // Cloner l'Arc pour le thread enfant
        let somme_partagee = Arc::clone(&somme_globale);

        // Copie locale du morceau pour le transférer au thread en toute sécurité
        let donnees_locales = morceau.to_vec();

        let handle = thread::spawn(move || {
            // Calcul local : aucune synchronisation requise ici (très rapide !)
            let somme_locale: u64 = donnees_locales.iter().sum();
            println!(" → [Thread {}] Somme locale calculée : {}", i, somme_locale);

            // Section critique : mise à jour de l'état partagé
            // Le verrou n'est acquis que pour une opération ultra-rapide
            let mut garde = somme_partagee.lock().unwrap();
            *garde += somme_locale;
            // Le verrou est automatiquement libéré ici à la fin de la portée du thread
        });

        handles.push(handle);
    }

    // 4. Attente de la fin de tous les threads
    for handle in handles {
        handle.join().unwrap();
    }

    // 5. Affichage du résultat final
    let resultat_final = *somme_globale.lock().unwrap();

```

```
println!("[Système] Calcul terminé !");
println!("Somme totale calculée : {}", resultat_final);

// Vérification mathématique : (n * (n + 1)) / 2
let verification = (limites * (limites + 1)) / 2;
assert_eq!(resultat_final, verification);
println!("Vérification réussie : {} = {}", resultat_final, verification);
}
```

Exercice 2 : Système de Journalisation Centralisé (MPSC & Cycle de vie)

Énoncé

Dans les architectures de serveurs, il est courant d'avoir plusieurs threads de traitement (workers) qui génèrent des logs, et un seul thread dédié à l'écriture de ces logs (dans la console ou un fichier) afin d'éviter que les affichages ne s'entremêlent de façon chaotique.

Vous devez concevoir un système de journalisation asynchrone en utilisant un canal de communication multi-producteur, mono-consommateur (`mpsc`).

Spécifications :

1. Créez un canal `mpsc` acceptant des chaînes de caractères (`String`).
2. Lancez un thread "Logger" (le consommateur) qui écoute en continu sur le canal et affiche chaque message reçu avec un préfixe d'horodatage simulé (ex: `[LOG] : ...`).
3. Lancez 3 threads "Workers" (les producteurs). Chaque worker doit simuler une tâche en cours (avec un court `sleep`), puis envoyer un message de statut au Logger (ex: `[Worker 1] Étape A complétée`).
4. Le thread principal doit attendre que tous les workers aient terminé leur exécution.

5. Défi de cycle de vie : Assurez-vous que le thread "Logger" s'arrête proprement de lui-même une fois que tous les workers ont fini de travailler et qu'il n'y a plus aucun message à traiter.

Indices

1. Pour que le thread "Logger" s'arrête, la boucle `for message in rx` doit se terminer. Cette boucle ne se termine que lorsque tous les émetteurs (`tx`) associés au canal sont détruits (sortis de la portée ou explicitement supprimés avec `drop`).
2. Attention à l'émetteur original créé dans le thread principal : si vous ne le détruisez pas dans le thread principal après avoir lancé les workers, le récepteur attendra indéfiniment.

Correction Détaillée

```

use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // 1. Création du canal MPSC
    let (tx, rx) = mpsc::channel::<String>();

    // 2. Lancement du thread Logger (Consommateur)
    let logger_handle = thread::spawn(move || {
        println!("[Logger] Démarrage du thread de journalisation...");
        // La boucle s'arrêtera d'elle-même quand tous les 'tx' seront détruits
        for message in rx {
            println!("[LOG] {}", message);
        }
        println!("[Logger] Tous les émetteurs sont fermés. Arrêt du Logger");
    });

    // Vecteur pour stocker les handles des workers
    let mut worker_handles = vec![];

    // 3. Lancement de 3 threads Workers (Producteurs)
    for id in 1..=3 {
        // On clone l'émetteur pour chaque worker
        let tx_clone = tx.clone();

        let handle = thread::spawn(move || {
            tx_clone.send(format!("Worker {} : Initialisation", id)).unwrap();

            // Simulation d'un travail
            thread::sleep(Duration::from_millis(100 * id));
            tx_clone.send(format!("Worker {} : Étape 1 terminée", id)).unwrap();

            thread::sleep(Duration::from_millis(150 * id));
            tx_clone.send(format!("Worker {} : Tâche accomplie avec succès", id)).unwrap();

            // L'émetteur 'tx_clone' sort de la portée et est détruit ici
        });
        worker_handles.push(handle);
    }

    // 4. Étape cruciale : Détruire l'émetteur original possédé par le main
    // Si on oublie cette ligne, le logger restera bloqué en attente car 'tx' du main
    drop(tx);

    // 5. Attente de la fin des workers
    for handle in worker_handles {
        handle.join().unwrap();
    }
}

```



```

    }
    println!("[Système] Tous les workers ont terminé leur travail.");

    // 6. Attente de la fin du Logger
    // Le logger va consommer les derniers messages restants dans le canal
    logger_handle.join().unwrap();
    println!("[Système] Fin du programme.");
}

```

Exercice 3 : Éviter les Verrous Mortels (Deadlocks) dans un Système de Transactions (Avancé)

Énoncé

Dans cet exercice, nous allons simuler un système de transferts de fonds entre des comptes bancaires de manière concurrente. Chaque compte est représenté par une structure contenant un identifiant unique et un solde protégé par un `Mutex`.

Si le Thread 1 tente de transférer de l'argent du Compte A vers le Compte B (en verrouillant A puis B), et que simultanément le Thread 2 tente de transférer du Compte B vers le Compte A (en verrouillant B puis A), un Deadlock (verrou mortel) va se produire.

Votre tâche est d'implémenter une fonction de transfert sécurisée qui prévient activement les deadlocks en utilisant la technique de l'ordonnancement des verrous (*Lock Ordering*).

Spécifications :

1. Créez une structure `Compte` avec un `id: u32` et `solde: Mutex<i32>`.
2. Implémentez une fonction `transferer(source: &Compte, destination: &Compte, montant: i32)` :
3. Cette fonction doit acquérir les verrous des deux comptes.
4. Pour éviter les deadlocks, elle doit toujours verrouiller en premier le compte ayant le plus petit ID, quel que soit l'ordre des paramètres passés à la fonction.
5. Dans le `main`, créez deux comptes enveloppés dans des `Arc`.

6. Lancez deux threads effectuant des transactions croisées intensives (l'un fait A -> B, l'autre fait B -> A) dans une boucle de 1000 itérations.
7. Vérifiez qu'aucun deadlock ne se produit et que les soldes finaux sont corrects.

Indices

1. Comparez les identifiants : `if source.id < destination.id { ... } else { ... }`.
2. Dans chaque branche du `if`, verrouillez les mutex dans l'ordre établi par votre comparaison, puis effectuez le transfert de solde.
3. Utilisez des variables temporaires pour stocker les gardes de verrous (`_garde1` , `_garde2`) afin de maintenir les verrous actifs pendant toute la durée de la transaction locale.

Correction Détaillée

```

use std::sync::{Arc, Mutex};
use std::thread;

// 1. Définition de la structure de données
struct Compte {
    id: u32,
    solde: Mutex<i32>,
}

// 2. Fonction de transfert sécurisée contre les deadlocks
fn transferer(compte_a: &Compte, compte_b: &Compte, montant: i32) {
    // Si les IDs sont identiques, c'est le même compte (pas de transfert nécessaire)
    if compte_a.id == compte_b.id {
        return;
    }

    // Ordonnancement des verrous (Lock Ordering)
    // On verrouille toujours le compte avec le plus petit ID en premier.
    let (_garde_premier, _garde_second) = if compte_a.id < compte_b.id {
        // On acquiert le verrou de A, puis de B
        let g_a = compte_a.solde.lock().unwrap();
        let g_b = compte_b.solde.lock().unwrap();
        (g_a, g_b)
    } else {
        // On acquiert le verrou de B, puis de A
        let g_b = compte_b.solde.lock().unwrap();
        let g_a = compte_a.solde.lock().unwrap();
        (g_a, g_b)
    };

    // À ce stade, les deux verrous sont acquis en toute sécurité.
    // Grâce au destructeur (RAII), ils seront libérés à la sortie de cette fonction.

    // Accès aux données internes via déréréférencement des gardes de verrou mutables
    let mut solde_a = *_garde_premier;
    let mut solde_b = *_garde_second;

    if *solde_a ≥ montant {
        *solde_a -= montant;
        *solde_b += montant;
    }
}

fn main() {
    // 3. Initialisation des comptes enveloppés dans un Arc pour le partage
    let compte_1 = Arc::new(Compte {
        id: 1,
        solde: Mutex::new(1000),
    });

```

```

});

let compte_2 = Arc::new(Compte {
    id: 2,
    solde: Mutex::new(1000),
});

println!("[Système] Soldes initiaux : Compte 1 = 1000€, Compte 2 = 1000€");
println!("[Système] Lancement des transactions croisées concurrentes.");

// Clone des références pour les threads
let c1_t1 = Arc::clone(&compte_1);
let c2_t1 = Arc::clone(&compte_2);

// Thread 1 : Transfère de Compte 1 vers Compte 2 de manière répétée
let t1 = thread::spawn(move || {
    for _ in 0..10_000 {
        transferer(&c1_t1, &c2_t1, 10);
    }
});

let c1_t2 = Arc::clone(&compte_1);
let c2_t2 = Arc::clone(&compte_2);

// Thread 2 : Transfère de Compte 2 vers Compte 1 de manière répétée
let t2 = thread::spawn(move || {
    for _ in 0..10_000 {
        transferer(&c2_t2, &c1_t2, 10);
    }
});

// 4. Attente des threads
t1.join().unwrap();
t2.join().unwrap();

// 5. Affichage et vérification des résultats
let solde_final_1 = *compte_1.solde.lock().unwrap();
let solde_final_2 = *compte_2.solde.lock().unwrap();

println!("[Système] Toutes les transactions ont été exécutées avec succès !");
println!("Solde final Compte 1 : {}€", solde_final_1);
println!("Solde final Compte 2 : {}€", solde_final_2);
println!("Somme totale des comptes : {}€ (Attendu : 2000€)", solde_final_1 + solde_final_2);

assert_eq!(solde_final_1 + solde_final_2, 2000);
}

```

